Грамматики и автоматы

Ф. А. Новиков
fedornovikov51@gmail.com

Санкт-Петербург
2 марта 2026 года

Содержание

0. Введение в предмет

0.1. Что изучается в курсе

0.1.1. Формальные искусственные языки
0.1.2. Модели описания поведения
0.1.3. Методы алгоритмизации предметных областей

0.2. Архитектура курса

0.2.1. Цели и задачи курса
0.2.2. Структура курса
0.2.3. Используемые обозначения
0.2.4. Источники и благодарности

0.3. Математическое введение

0.3.1. Исчисление предикатов первого порядка
0.3.2. Операции над множествами и отношениями
0.3.3. Избранные понятия теории алгоритмов
0.3.4. Ориентированные графы и деревья

1. Языки и грамматики

1.1. Порождающие грамматики

1.1.1. Алфавит, слово и язык
1.1.2. Формальные грамматики
1.1.3. Перечислимость языков
1.1.4. Классификация грамматик и языков
1.1.5. Неукорачивающие грамматики
1.1.6. Контекстно-свободные грамматики
1.1.7. Линейные, регулярные и автоматные грамматики
1.1.8. Языки записи грамматик
1.1.9. Альтернативные средства задания синтаксиса

1.2. Регулярные выражения

1.2.1. Регулярные операции
1.2.2. Алгебра регулярных выражений
1.2.3. Языковые графы
1.2.4. Детерминированные и недетерминированные источники
1.2.5. Расширения языка регулярных выражений
1.2.6. Программирование в регулярных выражениях
1.2.7. Уравнения с регулярными коэффициентами
1.2.7. Уравнения с регулярными коэффициентами
1.2.8. Характеристика класса регулярных языков

1.3. Контекстно-свободные грамматики

1.3.1. Выводимость в контекстно-свободных грамматиках
1.3.2. Преобразования контекстно-свободных грамматик
1.3.3. Нормальная форма Хомского
1.3.4. Лемма о разрастании
1.3.4. Лемма о разрастании (Лемма Огдена)
1.3.5. Нормальная форма Грейбах
1.3.5. Нормальная форма Грейбах
1.3.6. Алгоритмические свойства грамматик
1.3.7. Специальные классы грамматик
1.3.8. Классификация контекстно-свободных грамматик

1.4. Методы синтаксического анализа

1.4.1. Постановка задачи синтаксического анализа
1.4.2. Нисходящий разбор
1.4.3. Восходящий разбор
1.4.4. Специальные методы синтаксическо-

го анализа

1.5. Методы определения семантики

1.5.1. Постановка задачи определения семантики
1.5.2. Атрибутные грамматики
1.5.3. Синтаксически управляемый перевод
1.5.4. Контекстные условия
1.5.5. Операционная семантика
1.5.6. Венский метод определения языков программирования
1.5.7. Денотационная семантика
1.5.8. Исчисления программ
1.5.9. Преобразователи предикатов

1.6. Трансляция языков программирования

2. Состояния и переходы

2.1. Конечные автоматы Мили и Мура

2.1.1. Автомат Мили
2.1.2. Задание автомата таблицей
2.1.3. Задание автомата графом переходов состояний
2.1.4. Полные и частичные автоматы
2.1.5. Начальное и заключительные состояния
2.1.6. Автоматное преобразование
2.1.7. Изоморфизм автоматов
2.1.8. Автомат Мура

2.2. Распознавание языков конечными автоматами

2.2.1. Автоматы-распознаватели
2.2.2. Распознаватели конечных языков
2.2.3. Распознаватели бесконечных языков
2.2.4. Построение автомата по грамматике

2.2.5. Синтез автоматов
2.2.6. Анализ автоматов
2.2.7. Множество языков, распознаваемых автоматами

2.3. Эквивалентность автоматов

2.3.1. Неотличимость состояний и автоматов
2.3.2. Прямое произведение автоматов
2.3.3. Минимальный конечный автомат
2.3.4. Алгоритм минимизации Мили
2.3.5. Эквивалентность автоматов Мура и Мили

2.4. Сети автоматов

2.4.1. Комбинационные и логические автоматы
2.4.2. Автоматные блок-схемы
2.4.3. Синхронные сети автоматов
2.4.4. Представление автомата сетью
2.4.5. Представление сети автоматом
2.4.6. Структурный синтез автоматов
2.4.7. Минимизация схем
2.4.8. Вложенные автоматы

2.5. Автоматы с памятью

2.5.1. Машины Тьюринга
2.5.2. Линейно-ограниченные автоматы
2.5.3. Автоматы с магазинной памятью
2.5.4. Сети Петри

2.6. Программирование автоматов

2.6.1. Интерпретаторы таблиц
2.6.2. Представление автомата в программном коде
2.6.3. Задача поиска вхождений
2.6.4. Функция постфикса
2.6.5. Автомат поиска вхождений
2.6.6. Построение функции переходов
2.6.7. Автоматное программирование
2.6.8. Взаимодействие автоматных объектов

3. Языки предметных областей

3.1. Языково-ориентированное программирование

3.1.1. Алгоритмизация предметных областей
3.1.3. Языковая классификация предметных областей
3.1.4. Предметная классификация языков
3.1.5. Модель использования и архитектура

3.2. Методы описания языков

3.2.1. Структура описания языка программирования
3.2.2. Средства описания лексики
3.2.3. Средства описания синтаксиса
3.2.4. Средства описания контекстных условий
3.2.2. Нормальные формы Бэкуса–Наура
3.2.3. Синтаксические диаграммы Вирта
3.2.4. Метамодель языка
3.2.5. Контекстные условия
3.2.6. Грамматики ван Вейнгаардена
3.2.X. ??? Нерассмотренные вопросы

3.3. Методы реализации языков

3.3.1. Определение семантики языка программирования
3.3.2. Исчисление программ
3.3.4. Денотационная семантика
3.3.5. Венский метод Оллонгрена
3.3.6. Автоматный метод Новиков / Тихонова

3.4. Применение языков предметной области

3.4.1 Классификация языков предметных областей
3.4.2. Языки трансформации
3.4.3. Командные интерпретаторы
3.4.4. Порождающее программирование
3.4.5. Автоматное программирование

0. Введение в предмет (1/3)

Устройства для обработки информации известны человечеству с незапамятных времён — абак, счёты, ... За исторически кратчайшее время компьютеры и информационные технологии стремительно прошли путь от экспериментальных академических экспонатов до устройств, которые составляют основу современной цивилизации.

Внедрение *цифровых* технологий неравномерно: при игре в городки применение компьютеров пока незначительно, но в бухгалтерских расчётах оно уже исключительно. В спортивных играх компьютеры пока могут помочь решить только некоторые частные задачи, а в бухгалтерии компьютеры уже решают *все* задачи, в том числе и те, на которые у людей может не хватать времени, терпения и сил. При этом ключевым фактором является степень алгоритмизации.

Алгоритмизацией предметной области называется разработка методов и программ, которые позволяют прикладным специалистам решать на компьютере все или большинство *типовых* задач данной предметной области. Известны различные методы алгоритмизации, начиная от традиционных баз данных и пакетов прикладных программ и заканчивая разработкой формальных языков предметных областей и трансформаций моделей.

0. Введение в предмет (2/3)

Для алгоритмизации предметной области необходимо решить три задачи:

1. определить *представление объектов* предметной области;
2. реализовать *операции* с этими объектами;
3. разработать методы решения типовых задач, как *систем правил* применения операций к объектам для получения значимого результата.

Важно понимать, что предметная область первична, а её алгоритмизация вторична, поэтому выделение объектов, операций и правил *субъективно*, и зависит от целей и условий каждой конкретной алгоритмизации, и алгоритмизация может быть более или менее успешной.

В настоящее время самым перспективным приёмом считается повышение *уровня абстракции* при выделении объектов, операций и правил. При этом практика показывает, что повышение уровня абстракции и учёт специализации предметных областей наилучшим образом закрепляются в форме *языка предметной области*, то есть формального языка, прямо оперирующего понятиями предметной области и позволяющего в удобной форме ставить и решать все или большинство типовых задач предметной области. Такой язык обычно называют DSL — Domain Specific Language.

0. Введение в предмет (3/3)

Примеры.

1. SQL — язык для работы с реляционными базами данных;
2. TeX — язык для вёрстки математических текстов, на котором подготовлены эти слайды;
3. ACN — алгебраическая шахматная нотация — набор обозначений и правил, позволяющий эффективно записывать шахматные позиции и партии.

Вообще говоря, DSL не обязательно имеет текстовую форму. Например, язык жестов пользователей смартфонов является DSL, но не имеет явно выраженной текстовой формы. Исторически сложилось так, что для текстовых языков были разработаны мощные математические методы формализации, которые теперь применяются и в других целях, а поэтому основное внимание в этом курсе уделяется именно текстовым языкам.

Название курса — «Грамматики и автоматы» — звучит немного неопределённо, но никакой тайны здесь нет. Речь идёт о двух основных математических конструкциях, используемых для алгоритмизации предметных областей на основе формальных языков.

0.1. Что изучается в курсе (1/2)

Предметом изучения в этом курсе являются формальные *языки предметных областей*, предназначенные для алгоритмизации предметных областей и автоматизированного решения прикладных задач с помощью компьютеров.

Для реализации языка предметной области необходимо решить три задачи:

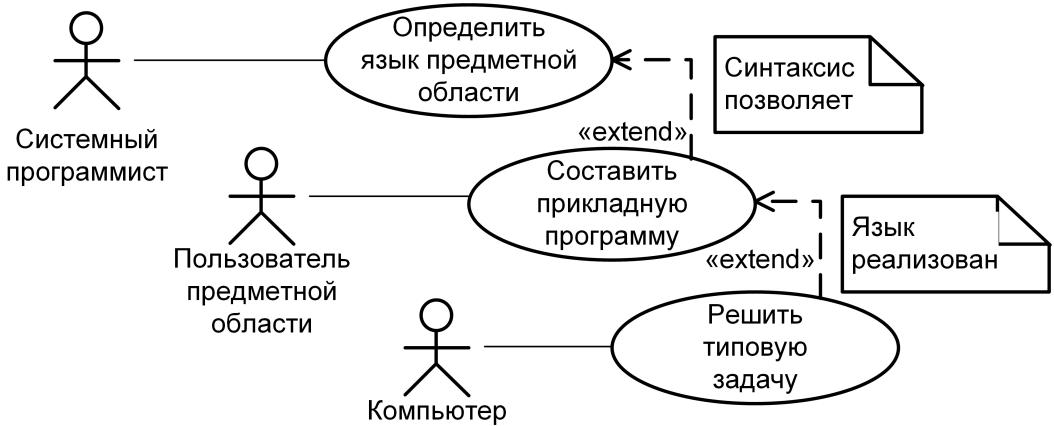
1. *описать систему обозначений языка* так, чтобы пользователь смог составить нужную программу на этом языке;
2. *реализовать алгоритм выполнения программ на языке* так, чтобы гарантированно получать результат;
3. *выработать регламенты применения языка* так, чтобы типовые задачи можно было решать в соответствии с требованиями предметной области.

Отступление. Есть и другие весьма интересные задачи, связанные с формальными языками предметных областей, которые, однако, не занимают заметного места в материалах этого курса.

Например, от системы обозначений языка можно потребовать, чтобы язык удобно было использовать для *обучения*. Программы на языке можно составлять не для выполнения, а для *публикации* алгоритмов. Поэтому для разработки эффективного языка предметной области, необходимо не только знать предметную область, но и иметь ясные требования к языку.

0.1. Что изучается в курсе (2/2)

Ниже приведена диаграмма использования языка предметной области.



В курсе рассматриваются известные методы определения языков предметной области и избранные приёмы эффективной реализации языков.

0.1.1. Формальные искусственные языки (1/3)

До конца позапрошлого века термин «язык» означал *естественный* язык, которым пользуются (или когда-то пользовались) люди для общения в повседневной жизни. Но в прошлом веке, особенно во второй его половине, резко возрос интерес к изучению формальных искусственных языков. В контексте данного курса самым важным классом таких языков выступают *языки программирования*. Для описания, анализа и синтеза подобных языков был разработан соответствующий математический аппарат и предложены разнообразные алгоритмы решения задач, связанных с языками.

В современной трактовке под языком понимается всякое средство общения (не обязательно только между людьми), состоящее из

1. *знаковой системы*, т. е. множества формальных знаков и правил комбинирования этих знаков;
2. *множества смыслов*, т. е. множества содержательных и значимых в данном общении элементов;
3. (частичного) *соответствия* между комбинациями знаков и смыслами, делающими осмысленными некоторые комбинации знаков.

0.1.1. Формальные искусственные языки (2/3)

Правила, описывающие множество знаков и способы их комбинирования, называются *синтаксисом* языка; правила, описывающие множество смыслов и соответствие между комбинациями знаков и смыслами, называются *семантикой* языка.

Замечание. В этих определениях мы (*пока!*) старательно избегаем использования терминов «текст» и «последовательность символов», имея в виду не только привычные текстовые языки программирования, но и другие знаковые системы, например, графические языки моделирования, или язык жестов пользователей смартфонов.

Очень многое зависит от назначения языка. Если формальный язык прост, в частности, если язык существенно конечен, то описание синтаксиса и семантики не требует изощренной математики — достаточно перечислить все допустимые фразы формального языка и для каждой фразы объяснить её семантику на естественном языке.

Пример. Пользоваться простейшими электронными часами с цифровой индикацией может любой ребёнок, знакомый с принятой системой измерения времени. Достаточно узнать, какую семантику имеют немногие кнопки на корпусе часов и уметь читать цифры на табло.

0.1.1. Формальные искусственные языки (3/3)

Далее рассматриваются более сложные знаковые системы, в которых множество комбинаций знаков велико или потенциально бесконечно, а множество смыслов и соответствие между знаками и смыслами не являются тривиальными.

Пример. Нотная запись позволяет создать бесконечно много различных комбинаций, причём извлечение смысла из нот, то есть воспроизведение музыки, отнюдь не является тривиальным процессом.

Описание семантики существенно зависит от типа объектов, которые описываются языком и поэтому оказывается различным для языков разных типов. Некоторые избранные способы описания семантики языков программирования и подобных формальных языков рассмотрены в этом курсе далее.

В области описания синтаксиса, напротив, удалось найти общие средства, которые практически не зависят от типа языка. Таких средств оказалось сравнительно немного, и в этом курсе содержится практически исчерпывающий обзор.

Особо следует оговорить правила практического применения формальных языков, то есть *прагматику*. В этой части пока нет общеизвестных математических средств описания, и мы ограничиваемся примерами, по возможности разнообразными.

0.1.2. Модели описания поведения

TODO Классификация моделей: потоки управления и потоки данных, сообщения, состояния / действия $\rightarrow$ Графы переходов состояний.
Синхронные и асинхронные модели,
параллельные и последовательные модели,
детерминированные и недетерминированные модели. ФАН
18.02.26 = пока несколько в стороне от темы курса. Чтобы приблизить, нужны два примера: IDEF и BPMN, взять учебный процесс по изучению курса грамматики и автоматы. Это DSL основанные на разных моделях поведения. IDEF = поток данных, BPMN = поток управления.

0.1.3. Методы алгоритмизации предметных областей

TODO Модель предметной области, язык предметной области, доверительный программный фонд, сильный пользователь, ...
Диаграмма деятельности с потоками данных и действующими лицами. ФАН
18.02.26 = пока несколько в стороне от темы курса. Чтобы приблизить, нужна перекличка с разделом 3.1. Здесь нужно замыкание:
алгоритмизация = язык $\times$ поведение.

0.2. Архитектура курса

Курс рассчитан на два семестра. В первом семестре больше лекций по теории и меньше практических занятий; первый семестр завершается экзаменом по теории. Во втором семестре меньше лекций и больше лабораторных работ; второй семестр и курс в целом завершается курсовым проектом.

В курсе параллельно изучаются два вида формализмов: (порождающие) грамматики и (конечные) автоматы. Эти формализмы родственны друг другу и имеют не только точки соприкосновения, но и взаимопроникновения. Однако с математической точки зрения эти формализмы, можно сказать, ортогональны: формальные грамматики тяготеют к *декларативным исчислениям*, в то время автоматы являются воплощением *императивных алгоритмов*.

Как грамматики, так и автоматы применяются в качестве основных инструментов при конструировании и реализации языков предметной области в настоящее время. Теория первого семестра общеизвестна и подкрепляется многими учебниками. Практика второго семестра во многом опирается на опыт автора и составляет отличительную особенность курса.

0.2.1. Цели и задачи курса

Цель курса — научить *эффективному* прикладному программированию. Автор на собственном опыте убедился, что конструирование и реализация подходящего языка предметной области является ключевым приёмом решения сугубо прикладных задач.

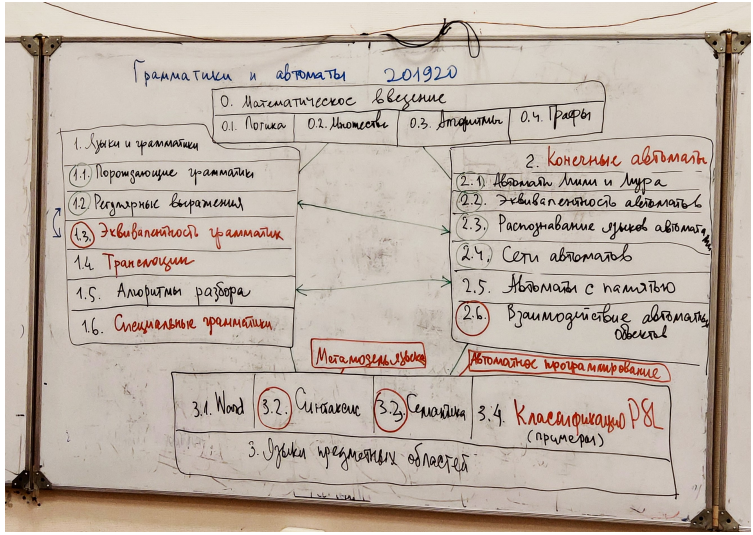
Из этой цели вытекают три задачи курса:

1) познакомить студентов с широким кругом классических и современных методов конструирования и описания *формальных языков предметной области*;
2) продемонстрировать преимущества и ограничения использования проблемно-ориентированных методов программирования в определённом контексте, и прежде всего *автоматного программирования*;
3) привить навыки планомерного использования подходящих *специализированных фреймворков* при реализации соответствующих программных компонентов.

Первая задача решается преимущественно в первом семестре на лекциях по теории и выполнении СНИРС (самостоятельная научно-исследовательская работа студента). Вторая и треья задачи решаются преимущественно во втором семестре за счет выполнения проекта по реализации некоторого языка предметной области.

0.2.2. Структура курса

TODO Структура существенно нелинейная — граф зависимостей = решётка. Диаграмма классов с зависимостями и с обобщениями. ФАН



0.2.3. Используемые обозначения

TODO Выделения в тексте: теоремы, алгоритмы, … , псевдокод **(или Python??? или Dijkstra???)**, используемые конструкции UML и онтологии. ФАН ответственный параграф.

0.2.4. Источники и благодарности

TODO Аннотированная библиография
Клавдиев, Лавров, Федорченко, ...
СТУДЕНТЫ за СЕМЬ ЛЕТ! ФАН

0.3. Математическое введение

Здесь конспективно, без доказательств, излагаются некоторые сведения из дискретной математики, которые далее считаются известными.

0.3.1. Исчисление предикатов первого порядка

TODO Язык исчисления на примерах словами, связанные и свободные вхождения, замкнутые и незамкнутые формулы, логический квадрат. ФАН

TODO Необходимо добавить определение формальной теории и повторить определение исчисления предикатов как пример, чтобы в главе 1 дать замечание, что грамматика суть исчисление. ФАН

0.3.2. Операции над множествами и отношениями

TODO Принадлежность, подмножества, бесконечные и счётные множества, теорема Кантора, операции, булеан, прямое произведение, бинарные отношения на, теорема о гомоморфизме, свойства отношений, замыкание и редукция отношений. ФАН

TODO ??? эквивалетность, факторизация, порядок, теорема о неподвижной точке функции. ФАН

0.3.3. Избранные понятия теории алгоритмов

TODO Алгоритмы на псевдокоде, машина Тьюринга, тезис Чёрча–Тьюринга, вычислимые и невычислимые функции, перечислимые и неперечислимые множества, разрешимые и полуразрешимые множества. ФАН

TODO Было бы желательно использовать новый псевдокод в качестве модели вычислимости, и доказывать алгоритмическую разрешимость на этой основе, а машину Тьюринга отправить в автоматы с памятью.

TODO Разные машины Тьюринга, Машина Поста, нормальный алгорифм Маркова
этот параграф нуждается в ревизии с учетом последующего материала. ФАН

0.3.4. Ориентированные графы и деревья (1/2)

TODO Графы, орграфы, корневые деревья. ФАН

TODO Нотация нагрузки на вершины и на рёбра. Представления графов и деревьев.

0.3.4. Ориентированные графы и деревья (2/2)

Ориентированное дерево — это орграф, в котором ровно один узел не имеет входящих дуг (это узел называется *корнем*), а все остальные узлы имеют ровно одну входящую дугу и при этом достижимы из корня. Если исходящие дуги в каждом узле линейно упорядочены, то дерево называется *упорядоченным*. Узлы, не имеющие исходящих дуг, называются *листьями*. Путь от корня к листу здесь назвается *ветвью*. Легко видеть, что в упорядоченном ордереве ветви, а значит, и листья линейно упорядочены. Упорядоченная последовательность листьев называется *кроной*. Последовательность узлов, для которой каждая ветвь содержит ровно один узел из этой последовательности называется *сечением*.

Здесь чаще используются *помеченные* упорядоченные ордеревья. В качестве пометок используются символы из заданных алфавитов. Пометки не обязательно различны.

TODO про изображение деревьв, пример

Диаграммы деревьев наглядны, но занимают много места. Здесь для записи помеченных упорядоченных ордеревьев используется следующая текстовая запись: в круглых скобках записывается пометка корня, а затем последовательность поддеревьев.

1. Языки и грамматики

Начинаем мы с самого заслуженного и до сих пор самого популярного средства описания синтаксиса формальных языков, имеющих текстовое представление — *теории формальных грамматик*.

Для особенно простых, но часто применяемых в программировании формальных языков, которые называются регулярными, имеется средство описания, которое называется *регулярные выражения*.

В этой главе приводятся основные определения, факты и алгоритмы из теории формальных грамматик, излагается техника регулярных выражений и рассматривается применение описанного аппарата для работы с языками программирования, в частности, с языками предметных областей.

1.1. Порождающие грамматики (1/3)

Первым рассматривается простейший случай, когда система знаков — это конечный алфавит символов, а способ комбинирования — это составление линейных последовательностей символов, то есть *текстов*.

Отступление. Исторически идеи порождающих грамматик возникли применительно к текстам, но в настоящее время развились до применения к изображениям, звукам, жестам и многим другим символам различной природы. Некоторые примеры рассматриваются в главе 3.

1.1. Порождающие грамматики (2/3)

№	Название параграфа	Ключевые термины и обозначения
1.1.1.	Алфавит, слово и язык	алфавит, буква, символ, слово, цепочка, строка, вхождение, пустое слово ε, вхождение, вырезка, длина слова, конкатенация, сцепление, степень, префикс, суффикс, пересечение слов, язык, лексикографический порядок
1.1.2.	Формальные грамматики	порождающая грамматика, нетерминал, переменная, терминал, константа, правило вывода, продукция, аксиома грамматики, вывод, непосредственный вывод, бесповторный вывод, порождаемый язык $\mathfrak{L}$, допуск слова, эквивалентность грамматик
1.1.3.	Перечислимость языков	
1.1.4.	Классификация грамматик и языков	Ноам Хомский, грамматика: с фразовой структурой, непосредственных составляющих, контекстно-зависимая, контекстно-свободная, регулярная, $\mathfrak{T}_i$

1.1. Порождающие грамматики (3/3)

№	Название параграфа	Ключевые термины и обозначения
1.1.5.	Неукорачивающие грамматики	разрешимый, полуразрешимый, легко разрешимый язык, распознающий алгоритм
1.1.6.	Контекстно-свободные грамматики	левосторонний вывод, правосторонний вывод, дерево разбора, дерево вывода, дерево порождения, неоднозначная грамматика
1.1.7.	Линейные, регулярные и автоматные грамматики	правило: начальное, заключительное, леволинейное, праволинейное, порождающие и недостижимые символы, цепное правило, ε-правило, S-правило, приведённая грамматика
1.1.8.	Языки записи грамматик	метасимвол, нормальная форма Бэкуса–Наура, металингвистическая переменная, металингвистическая формула
1.1.9.	Альтернативные средства задания синтаксиса	Никлаус Вирт, диаграмма, тривиальная диаграмма, элементарная диаграмма, язык разметки

1.1.1. Алфавит, слово и язык (1/9)

Пусть задано конечное множество $A = \{a_1, \ldots, a_n\}$, которое называется *алфавитом*. Элементы алфавита называются *буквами* или *символами*. Обычно отдельные символы обозначаются начальными строчными буквами латинского алфавита. Последовательность букв называется *словом* (в данном алфавите). Заметим, что в программировании слово часто называют *строкой*, а в теории формальных языков — *цепочкой*. Буквы в слове упорядочены (слева направо), и могут быть перенумерованы. Одна буква может входить в слово несколько раз, каждый отдельный экземпляр буквы в слове называется *вхождением* буквы в слово. Вхождения могут быть идентифицированы, например, номером буквы в слове относительно начала слова. Обычно слова обозначают начальными буквами греческого алфавита, а номера записывают в квадратных скобках.

Пример. Пусть $A = \{a, b\}$. Тогда $\alpha = aabb$ — слово, а $\alpha[3] = b$ — буква.

Замечание. Вхождения можно идентифицировать не номером, а смещением — это вопрос вкуса, который регулируется предварительным соглашением.

Количество вхождений букв в слово называется *длиной* слова: $|\alpha| = |a_1 \ldots a_k| = k$.

Пример. $|\text{Ура-а-а!}| = 8$.

Пустое слово обозначается ε, $|\varepsilon| = 0$.

1.1.1. Алфавит, слово и язык (2/9)

Можно рассматривать не только вхождения отдельных букв в слово, но и вхождения любых частей слова как отдельные слова. Операция взятия части слова называется *вырезкой*. Вырезку естественно также обозначить квадратными скобками.

Пример. самовар$[3..6] =$ мова.

Таким образом, определено отношение между словами «быть частью». Это отношение рефлексивно, транзитивно и антисимметрично, а потому является нестрогим частичным порядком, и обозначается здесь $\subset$.

Пример. луг $\subset$ слуга.

Замечание. Очевидно, что $\alpha \subset \beta \Longrightarrow |\alpha| \leqslant |\beta|$, но $\neg(|\alpha| \leqslant |\beta| \Longrightarrow \alpha \subset \beta)$!

Пример. гул $\not\subset$ слуга.

Для слов определена операция *конкатенации*, или *сцепления*. Обычно операция конкатенации никак не обозначается, а сцепляемые слова просто записываются подряд.

Пример. Конкатенация слов «пара» и «план» образует слово «параплан».

Сцепление можно применять многократно, таким образом определяется *степень* слова. Положим $\alpha^1 \stackrel{\text{Def}}{=} \alpha; \alpha^k \stackrel{\text{Def}}{=} \alpha\alpha^{k-1}; k > 1$. Можно определить нулевую степень слова: $\alpha^0 \stackrel{\text{Def}}{=} \varepsilon$.

Пример. $a^4 = aaaa; (ab)^3 = ababab.$

1.1.1. Алфавит, слово и язык (3/9)

Одни и те же буквы могут входить в разные слова. Это порождает отношения между словами, связанные с наличием или отсутствием общих частей.

Если $\alpha = \alpha_1\alpha_2$, то α_1 называется *началом*, или *префиксом*, слова α, а α_2 — *окончанием*, или *постфиксом*, слова α. Если при этом $\alpha_1 \neq \varepsilon \;\&\; \alpha_2 \neq \varepsilon$, то α_1 (соответственно, α_2) называется *собственным началом* (соответственно, *собственным окончанием*) слова α.

Пример. Слова «д» и «до» являются собственными префиксами, а слова «ом» и «м» являются собственными постфиксами слова «дом».

Замечание. Каждое слово является своим постфиксом и префиксом (несобственным!).

Слова α и β могут *пересекаться*. При пересечении либо первое слово является частью второго, то есть $\beta = \gamma\alpha\delta$, либо постфикс первого слова является префиксом второго, то есть $\alpha = \delta\gamma, \beta = \gamma\zeta$. Для пары слов α и β тройка слов γ, δ, ζ определяется однозначно, если выбирать *наибольшее* пересечение

Пример. Пусть $\alpha = abb$, $\beta = bbc$. Тогда можно взять $\gamma := b$, $\delta := ab$, $\zeta := bc$. Однако наибольшее пересечение даёт единственная тройка $\gamma := bb$, $\delta := a$, $\zeta := c$.

Таким образом, на множестве слов определена *операция пересечения*. Эта операция не коммутативна!

1.1.1. Алфавит, слово и язык (4/9)

Множество слов в алфавите A обозначается A^*. По определению $\varepsilon \notin A$, но $\varepsilon \in A^*$. Если пустое слово исключается из рассмотрения, то множество слов в алфавите A обозначается A^+.

Произвольное множество L слов в алфавите A называется *языком* в этом алфавите.

Примеры.

1. $A \subset A^*$ — язык, состоящий из однобуквенных слов;
2. $\{0,1\}^*$ — язык двоичных последовательностей;
3. $A^* \subset A^*$ — универсальный язык всех слов.

К любым языкам применимы операции над множествами: объединение, пересечение, разность и симметрическая разность. Поскольку алфавит языка известен, то определена и операция дополнения.

Отступление. Обычные операции наивной теории множеств играют сравнительно небольшую роль в теории языков, за исключением операции объединения.

Пусть есть два языка над одним алфавитом: $L_1 \subset A^*, L_2 \subset A^*$. Тогда операция сцепления слов индуцирует операцию сцепления языков: $L_1 L_2 \stackrel{\text{Def}}{=} \{\alpha\beta \mid \alpha \in L_1 \ \& \ \beta \in L_2\}$.

1.1.1. Алфавит, слово и язык (5/9)

Сцепление языков можно применять многократно, таким образом определяется *степень* языка. Пусть $L \subset A^*$. Положим $L^1 \stackrel{\text{Def}}{=} L; L^k \stackrel{\text{Def}}{=} LL^{k-1}; k > 1$. Можно определить нулевую степень языка: $L^0 \stackrel{\text{Def}}{=} \{\varepsilon\}$.

Но сам алфавит A — это язык, и значит определены степени этого языка A^k для любого натурального числа k. Из этого наблюдения и определений немедленно следует очевидная, но важная теорема.

Теорема. $A^k = \{\alpha \in A^* \mid |\alpha| = k\}$.

Замечание. Другими словами, A^k — это множество слов длины k в алфавите A.

Следствие. $|A^k| = |A|^k = n^k$.

Рассмотрим $\mathcal{A} = \{A^0, A^1, A^2, \dots\}$ — семейство классов эквивалентности по отношению равенства длин слов. Тогда

$$A^* = \bigcup_{k=0}^{\infty} A^k, \qquad A^+ = \bigcup_{k=1}^{\infty} A^k,$$

что вполне согласуется с предшествующими определениями.

1.1.1. Алфавит, слово и язык (6/9)

Множество A^* бесконечно, но счётно, то есть его можно занумеровать. Рассмотрим некоторые нумерации, которые особенно часто используются в других построениях.

Поскольку алфавит $A = \{a_1, \ldots, a_n\}$ конечен, его буквы можно перенумеровать (то есть *линейно упорядочить*) и представить алфавит словом $\alpha\beta_A \stackrel{\text{Def}}{=} a_1 \ldots a_n$, то есть конкатенацией всех букв в порядке возрастания номеров.

Отступление. Слово «алфавит» происходит от прочтения первых букв греческого алфавита: альфа, бета, ... Аналогичное происхождение имеет русское слово «азбука»: аз, буки, веди, ...

Обозначение $\alpha\beta_A$ допустимо читать «алфавит A».

Пример. Современная кириллица содержит 33 буквы, и для этих букв узаконен следующий порядок: $\alpha\beta_{\mathcal{K}} :=$ абвгдеёжзийклмнопрстуфхцчшщъыьэюя. Индекс $\mathcal{K}$ символизирует кириллицу.

Зафиксируем биекцию $A = \{a_1, \ldots, a_n\} \sim 1..n$ следующим образом.

func $n_A(a : A) : 1..n$
for i **from** 1 **to** n **do**
  if $\alpha\beta_A[i] = a$ **then return** i **end if**
end for

func $a_A(i : 1..n) : A$
return $\alpha\beta_A[i]$

1.1.1. Алфавит, слово и язык (7/9)

Нумерация букв индуцирует нумерацию слов. Пусть $\alpha \in A^k$. Положим

$$n_A(\alpha) := n_A(\alpha[k]) + n \cdot n_A(\alpha[k-1]) + \cdots + n^{k-1} \cdot n_A(\alpha[1]) = \sum_{i=1}^{k} n_A(\alpha[i]) n^{k-i}.$$

Пример. В этой нумерации для слов в кириллице имеем:

$n_{\mathcal{K}}(\text{а}) = 1$	$n_{\mathcal{K}}(\text{аа}) = 1 \cdot 33 + 1 = 34$	$n_{\mathcal{K}}(\text{ааа}) = 1 \cdot 33^2 + 1 \cdot 33 + 1 = 1123$
$n_{\mathcal{K}}(\text{б}) = 2$	$n_{\mathcal{K}}(\text{аб}) = 1 \cdot 33 + 2 = 35$	$n_{\mathcal{K}}(\text{ааб}) = 1 \cdot 33^2 + 1 \cdot 33 + 2 = 1124$
$\ldots$	$\ldots$	$\ldots$
$n_{\mathcal{K}}(\text{я}) = 33$	$n_{\mathcal{K}}(\text{яя}) = 33 \cdot 33 + 33 = 1122$	$n_{\mathcal{K}}(\text{яяя}) = 33 \cdot 33^2 + 33 \cdot 33 + 33 = 37059$

Теорема. $\forall A \ \ (A^* \sim \mathbb{N}_0)$.

Доказательство. Представленная нумерация реализует требуемую биекцию. Номер слова легко вычисляется известным алгоритмом позиционной системы счисления, представленным на следующем слайде слева. Обратный алгоритм определения слова по номеру чуть сложнее, поскольку особым образом требуется обрабатывать числа, кратные основанию n. Алгоритм представлен на следующем слайде справа. Для завершения доказательства остаётся положить $n_A(\varepsilon) := 0$.

1.1.1. Алфавит, слово и язык (8/9)

func $n_A(\alpha : A^*) : \mathbb{N}_0$
$i := 0$
for j **from** 1 **to** $|\alpha|$ **do**
  $i := n \cdot i + n_A(\alpha[j])$
end for
return i

func $a_A(i : \mathbb{N}_0) : A^*$
$\alpha := \varepsilon$
while $i > 0$ **do**
  if $i \bmod n = 0$ **then** $a := a_A(n); i := i - n$
  else $a := a_A(i \bmod n)$ **end if**
  $\alpha := a\alpha$; $i := i \operatorname{div} n$
end while
return α

Замечание. Предложенная нумерация обладает рядом приятных свойств.

Во-первых, нумерация монотонна по длине слова, то есть более длинные слова имеют номера больше, чем более короткие.

Во-вторых, нумерация локально взаимно-однозначна:
$A \sim 1..n$, $A^2 \sim (n+1)..(n^2+n)$ и вообще $\forall\, k \in \mathbb{N}\ \left(A^k \sim \frac{n^k-1}{n-1}..n\frac{n^k-1}{n-1}\right)$.

В-третьих, нумерация определяет естественный линейный порядок на множестве слов: $\alpha < \beta := n_A(\alpha) < n_A(\beta)$, что позволяет перечислить все слова в порядке возрастания.

1.1.1. Алфавит, слово и язык (9/9)

Часто на множестве слов используют *лексикографический порядок* (или *алфавитный*):

$$\alpha < \beta \stackrel{\text{Def}}{=} \beta = \alpha\gamma \;\&\; \gamma \neq \varepsilon \vee \exists j \leqslant |\alpha| \;\; (\alpha[j] < \beta[j] \;\&\; \forall i \in 1..(j-1) \;\; (\alpha[i] = \beta[i])) .$$

Лексикографический порядок линеен, так же как и введённый естественный порядок.

Замечание. Лексикографический порядок не позволяет построить генератор, который перечисляет *все* слова в возрастающем порядке.

Пример. В естественном порядке $1 < 2 < 11 < 12 < 21$, а в лексикографическом порядке $1 < 11 < 12 < 2 < 21$.

Отступление. Нумерация $n_A()$ очень похожа на обычную позиционную систему счисления с основанием n. Отличие в том, что в позиционных системах цифры берутся из диапазона $0..(n-1)$, а в предложенной системе цифры берутся из диапазона $1..n$. Наличие цифры 0 с её особыми арифметическими свойствами влечёт два важных следствия. С одной стороны, алгоритмы всех арифметических операций в обычной системе счисления оказываются проще, чем алгоритмы в системе без нуля, поэтому в компьютерах при вычислениях всегда применяются позиционные системы с нулём. С другой стороны, из-за нуля теряется однозначность соответствия между словами, составленными из цифр, и натуральными числами. Проблему создают *ведущие нули* — различные строки 0001 и 1 в позиционной системе соответствуют одному и тому же числу «один».

1.1.2. Формальные грамматики (1/4)

Если язык содержит только конечное множество слов, то их можно задать просто перечислением. Если же язык бесконечен, что чаще встречается, то необходимо использовать специальные средства для построения конечного описания бесконечного множества. Возникновение и развитие самого известного из таких средств связывают с Ноамом Хомским.

Формальная порождающая грамматика (далее просто грамматика) $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$ – это совокупность конечного алфавита M *нетерминальных* символов (то есть вспомогательных символов, не входящих в алфавит, над которым определяется язык), конечного алфавита Σ *терминальных* символов (то есть символов алфавита, над которым определяется язык, $\mathrm{M} \cap \Sigma = \varnothing$), конечного множества *правил вывода* (правила вывода называют также *правилами подстановки* или *продукциями*) $\Pi = \{\alpha \to \beta \mid \alpha, \beta \in (\mathrm{M} \cup \Sigma)^*\}$ и *основного символа* (или *аксиомы грамматики*) $S \in \mathrm{M}$.

Пример. Совокупность множеств
$\mathrm{M} := \{N\}$, $\Sigma := \{a, b\}$, $\Pi := \{N \to NN, N \to \varepsilon, N \to a, N \to b\}$
образует грамматику $G_0 = \langle \mathrm{M}, \Sigma, \Pi, N \rangle$.

1.1.2. Формальные грамматики (2/4)

Замечание. По традиции нетерминальные символы (или просто *нетерминалы*, иногда их также называют *переменными*) обозначают прописными латинскими буквами, терминальные символы (или просто *терминалы*) обозначают строчными латинскими буквами, а цепочки терминальных и нетерминальных символов — строчными греческими буквами. Было бы неудобно *множества* символов, то есть алфавиты, обозначать символами этих же алфавитов. Поэтому здесь приняты обозначения алфавитов и множества правил прописными греческими буквами, по первым буквам греческих слов $\mathrm{M}\varepsilon\tau\alpha\beta\lambda\eta\tau\eta$ (Переменная), $\Sigma\tau\alpha\theta o\rho o\varsigma$ (Постоянная), $\Pi\alpha\rho\alpha\gamma\omega\gamma\iota\kappa o\varsigma\ \nu o\mu o\varsigma$ (Производящее правило).

Поскольку все множества конечны, запись грамматики также конечна, однако конечная грамматика может порождать бесконечный язык. Основана на формализме *вывода*.

Говорят, что из цепочки γ *непосредственно выводима* цепочка δ в грамматике G (обозначение $\gamma \Rightarrow_G \delta$), если $\gamma = \gamma_1\alpha\gamma_2$, $\delta = \gamma_1\beta\gamma_2$ и $\alpha \to \beta \in \Pi$. Говорят, что из цепочки γ *выводима* цепочка δ в грамматике G, если существует *вывод*, т.е. последовательность цепочек, в которой каждая следующая выводима из предыдущей по одному из правил вывода:

$$\exists\, \sigma_0, \ldots, \sigma_n \ \left(\forall\, i \in 0..(n-1) \ \left(\sigma_i \Rightarrow_G \sigma_{i+1} \ \&\ \sigma_0 = \gamma \ \&\ \sigma_n = \delta\right)\right).$$

1.1.2. Формальные грамматики (3/4)

Число n (количество цепочек в последовательности, не считая первой) называется *длиной* вывода. Выводимость δ из γ обозначают $\gamma \Rightarrow_G^n \delta$, если нужно указать длину вывода или $\gamma \Rightarrow_G^* \delta$, если длина вывода не указывается. Если грамматика подразумевается, то индекс G опускают. Отметим, что $\Rightarrow_G$ является отношением на множестве цепочек. В таком случае отношение $\Rightarrow_G^*$ является транзитивным замыканием отношения $\Rightarrow_G$ на множестве цепочек.

Пример. Последовательность формул $NN, NNN, NNNN, aNNN, abNN, abbN, abba$ является выводом в грамматике G_0. Другими словами, $NN \Rightarrow_{G_0}^6 abba$.

Замечание. Таким образом, грамматика — это *формальная теория*, или же *исчисление*, в том строгом значении, которое имеют эти термины в математической логике. Теоремами этой теории являются терминальные цепочки, выводимые из аксиомы грамматики.

Пример. В грамматике G_0 предыдущего примера все слова из множества $\{a, b\}^*$, включая пустое слово, выводимы из начального символа грамматики. Другими словами, грамматика G_0 — это такая малоинтересная теория, в которой все формулы суть теоремы.

1.1.2. Формальные грамматики (4/4)

Вывод называется *бесповторным*, если все цепочки в нем различны.

Лемма. *Если существует вывод слова α в грамматике G, то существует бесповторный вывод слова α в грамматике G.*

Доказательство. Пусть $\sigma_0 = S, \sigma_1, \ldots, \sigma_i, \ldots, \sigma_j, \ldots \sigma_n = \alpha$ – вывод цепочки α в G, причём $\sigma_i = \sigma_j$. Очевидно, что $\sigma_0 = S, \sigma_1, \ldots, \sigma_i, \sigma_{j+1}, \ldots \sigma_n = \alpha$ также будет выводом цепочки α в G. □

Языком, порождаемым грамматикой G (обозначение $\mathfrak{L}(G)$) называется множество всех цепочек в терминальном алфавите, выводимых из начального символа грамматики:

$$\mathfrak{L}(G) \stackrel{\text{Def}}{=} \{\alpha \in \Sigma^* \mid S \Rightarrow_G^* \alpha\}.$$

Замечание. В традициях терминологии логических исчислений говорят, что слова языка $\mathfrak{L}(G)$ *допускаются* грамматикой G.

Один и тот же язык может быть порождён разными грамматиками. Говорят, что две грамматики G и G' *эквивалентны* (обозначение $G \sim G'$), если $\mathfrak{L}(G) = \mathfrak{L}(G')$.

Пример. Пусть $\mathrm{M} := \{N\}$, $\Sigma := \{a, b\}$, $\Pi' := \{N \to \varepsilon, N \to aN, N \to bN\}$, тогда грамматика ${G_0}' = \langle \mathrm{M}, \Sigma, \Pi', N \rangle$ эквивалентна G_0, то есть $\mathfrak{L}(G_0) = \mathfrak{L}({G_0}')$.

1.1.3. Перечислимость языков (1/3)

Язык, порождаемый всякой формальной грамматикой, является перечислимым множеством в смысле теории алгоритмов. Действительно, следующий алгоритм перечисляет множество слов языка $\mathfrak{L}(G)$, порождаемого грамматикой $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$.

TODO Первая часть эссе 1.1.3. Алгоритм на псевдокоде, с обоснованием.
Пример: двоичные последовательности, представляющие натуральные числа

1.1.3. Перечислимость языков (2/3)

Однако не все языки, как множества слов, являются перечислимыми. Существуют неперечислимые языки, просто потому, что множество алгоритмов счётно, а множество языков несчётно.

Если никаких ограничений на грамматики не накладывать, то формальная порождающая грамматика оказывается предельно общим механизмом описания языков, что видно из следующей теоремы.

Теорема. *Для любого перечислимого множества слов M существует такая формальная порождающая грамматика G, что $M = \mathfrak{L}(G)$.*

Доказательство.

TODO Вторая часть эссе 1.1.3. Есть доказательство на основе машины Тьюринга, но в грамматике 14 правил, очень тяжеловесно. Сократили до 11, файл прислан. Есть надежда на сокращение за счёт применения нормальных алгорифмов Маркова. □

1.1.3. Перечислимость языков (3/3)

Столь высокая степень общности ведёт к тому, что многие практически важные вопросы для грамматик общего вида оказываются очень трудоёмкими или даже алгоритмически неразрешимыми. Поэтому на практике применяются более узкие классы грамматик.

Общепринятая в настоящее время *классификация грамматик* была предложена Хомским[1].

[1] Ноам Хомский, р. 1928

Перечислимость языков (1/7)

Язык, порождаемый всякой формальной грамматикой, является перечислимым множеством в смысле теории алгоритмов. Действительно, следующий алгоритм перечисляет множество слов языка $\mathcal{L}(G)$, порождаемого грамматикой $G = \langle N, T, R, S \rangle$.

Вход: Грамматика $G = \langle N, T, R, S \rangle$
Выход: Множество всех терминальных строк, выводимых из S

$Q \leftarrow$ пустая очередь
$L \leftarrow$ пустое множество
PUSH(Q, S)
while $Q \neq \varnothing$ **do**
  $\alpha \leftarrow$ POP(Q) //извлечь первый элемент
  if $\forall c \in \alpha : c \in T$ **then**
    $L \leftarrow L \cup \{\alpha\}$
  else
    $A_1, A_2, \ldots, A_k$ – все нетерминалы в α
    for $i \leftarrow 1$ **to** k **do**
      $A \leftarrow A_i$
      for $\forall (A \to \beta) \in R$ **do**
        PUSH(Q, γ)
      end for
    end for
  end if

Перечислимость языков (2/7)

Обоснование. Используется поиск в ширину для перебора всех возможных выводов из начального символа S. Это гарантирует, что все слова, которые могут быть получены с помощью правил грамматики R, будут перечислены.
Поскольку алгоритм при каждом шаге применяет только правила из R к нетерминальным символам, каждая полученная строка соответствует допустимому выводу в грамматике G.
□

Пример.
Двоичные последовательности, представляющие десятичные числа. Количество символов 1 в цепочке есть представляемое этой цепочкой натуральное число.
$N = \{S\}$
$T = \{1\}$
$R = \{S \to 1, S \to 1S\}$

Перечислимость языков (3/7)

Пример. (Продолжение)

Первая итерация:

- $\alpha = S$
- 1S и 1 добавляется в Q

Вторая итерация:

- $\alpha = 1S$
- α не состоит только из терминалов
- 11S, 11 добавляется в Q
- $\alpha = 1$
- α состоит только из терминалов, добавляем в L.

Однако не все языки, как множества слов, являются перечислимыми. Существуют неперечислимые языки, просто потому, что множество алгоритмов счётно, а множество языков несчётно. Если никаких ограничений на грамматики не накладывать, то формальная порождающая грамматика оказывается предельно общим механизмом описания языков, что видно из следующей теоремы.

Перечислимость языков (4/7)

Теорема. *Для любого перечислимого множества слов M существует такая формальная порождающая грамматика G, что $M = \mathcal{L}(G)$*

Доказательство. Рассмотрим правила грамматики, с помощью которой будем доказывать теорему.

Пусть T – конечный алфавит терминалов, $\mathcal{A}$ – нормальный алгорифм Маркова (НАМ) перечисляющий $M \subseteq T^*$, $\Gamma \supseteq T$ – алфавит НАМ (включает терминалы и служебные символы), $\omega_0 \in \Gamma^*$ – начальное состояние НАМ ($\omega_0 = \varepsilon$), $P = \{r_1, r_2, \ldots, r_n\}$ – правила НАМ, где каждое r_i имеет вид $u_i \to v_i$ ($u_i, v_i \in \Gamma^*$)

Пусть $G = \langle N, T, R, S \rangle$:

- Терминалы: T
- Нетерминалы: $N = \{S, A, B, C\} \cup \Gamma_{\text{nt}}$, где $\Gamma_{\text{nt}} = \{\underline{a} \mid a \in \Gamma \setminus T\}$ – нетерминальные символы, соответствующие служебным символам НАМ.
- Аксиома грамматики: S.

□

Перечислимость языков (5/7)

Доказательство. (Продолжение)

Правила R:

1. Инициализация: $S \to A\omega_0 B$
   Здесь ω_0 записывается как цепочка символов из Γ
2. Преобразование для моделирования правил НАМ: $A \to AC$
   Символ C будет ходить по слову и применять правила НАМ.
3. Моделирование правил НАМ:
   $\forall(u_i \to v_i) \in P$, где $u_i, v_i \in \Gamma^*$:
   $$Cu_i \to v_i C$$
   Это позволяет C заменять вхождение u_i на v_i при движении слева направо.
4. Проход C через терминалы:
   $\forall a \in T$:
   $$Ca \to aC$$
5. Удаление C при достижении конца слова: $CB \to B$
6. Завершение вывода: $B \to \varepsilon, \quad A \to \varepsilon$

Перечислимость языков (6/7)

Доказательство. (Продолжение) Вывод в G имитирует работу НАМ: Начинаем с применения правила (1) $S \Rightarrow A\omega_0 B$. Правило (2) $A \to AC$ может применяться многократно, генерируя несколько копий C.

Каждая копия C движется вправо (правило (4) $CX \to YC$), применяя одно из правил НАМ при встрече с u_i, либо проскальзывая через терминалы.

Когда C достигает B, он исчезает: (5) $CB \to B$

Если после всех замен получено слово $\omega \in T^*$, то применется правило (6) $B \to \varepsilon$ и $A \to \varepsilon$, оставляя только ω

- Если $\omega \in M$:
  Существует последовательность применений правил НАМ, переводящая ω_0 в ω. В грамматике можно промоделировать ту же последовательность, выводя ω.
- Если $\omega \in \mathcal{L}(G)$:
  Тогда вывод в G соответствует некоторой корректной работе НАМ, значит, ω порождается НАМ, т.е. $\omega \in M$.

Таким образом, $\mathcal{L}(G) = M$.

Перечислимость языков (7/7)

Столь высокая степень общности ведёт к тому, что многие практически важные вопросы для грамматик общего вида оказываются очень трудоёмкими или даже алгоритмически неразрешимыми. Поэтому на практике применяются более узкие классы грамматик.

TODO Идейно это именно то, что нужно. Особенно нормальный алгориФм Маркова. Все обозначения неправильные, программы написаны ужасно, пример слишком слабый. В переработку.

TODO Замечания ФАН 21 января.
По алгоритмам: присваивание $:=$, а стрелочки как раз push и pop. Пустое множество $\varnothing$.
По обозначениям: кто такой Γ, обозначения для грамматик и языков другие.
По примерам: сделана «единичная» система, а требуется добавить двоичную. Вот она: $S \to 1 \mid S(0 \mid 1)$. Протокол выполнения алгоритма нужен, но это таблица = что на каждом шаге находится в каждой переменной.
По структуре: новые слайды должны быть вшиты в мои старые слайды и заканчиваться переходной фразой про классификацию. Плотность слайдов такая же, как везде.

1.1.4. Классификация грамматик и языков (1/5)

1. Грамматика *типа 0* (или грамматика *с фразовой структурой*) — это грамматика общего вида без ограничений на правила вывода. Класс таких грамматик обозначим $\mathfrak{T}_0$.
2. Грамматика *типа 1* (или грамматика *непосредственных составляющих* или *контекстно-зависимая*) — это грамматика, все правила которой имеют вид $\alpha A \beta \to \alpha \gamma \beta$, где $\alpha, \beta, \gamma \in (\mathrm{M} \cup \Sigma)^*$, $|\gamma| \geqslant 1$, $A \in \mathrm{M}$. Такое правило описывает замену нетерминального символа A на цепочку γ в определенном *контексте* (α, β), который не меняется при применении правила. Класс таких грамматик обозначим $\mathfrak{T}_1$.
3. Грамматика *типа 2* (или *контекстно-свободная* грамматика) — это грамматика, все правила которой имеют вид $A \to \alpha$, где $\alpha \in (\mathrm{M} \cup \Sigma)^*$, $A \in \mathrm{M}$. Такое правило описывает замену нетерминального символа A на цепочку α вне зависимости от контекста. Класс таких грамматик обозначим $\mathfrak{T}_2$.
4. Грамматика *типа 3* (или *регулярная* грамматика) — это грамматика, все правила которой имеют вид либо $A \to \alpha$, либо $A \to \alpha B$ (или $A \to B\alpha$), где $\alpha \in \Sigma^*$, $A, B \in \mathrm{M}$. В правой части правила регулярной грамматики может быть не более одного нетерминала. Класс таких грамматик обозначим $\mathfrak{T}_3$.

1.1.4. Классификация грамматик и языков (2/4)

Ясно, что $\forall\, i \in 1..3\ (G \in \mathfrak{T}_i \Longrightarrow G \in \mathfrak{T}_{i-1})$, поскольку всякое правило грамматики класса $\mathfrak{T}_i$ является частным случаем правила грамматики класса $\mathfrak{T}_{i-1}$.

Замечание. Исключением являются правила с пустой правой частью, но их можно устранить, см. п. 1.3.2.

Один и тот же язык может быть описан разными грамматиками, в том числе грамматиками разного типа. Принято относить язык к *наибольшему* типу из типов описывающих его грамматик. То есть если для языка L есть две описывающие его грамматики G' и G'', $L = \mathfrak{L}(G')\ \&\ L = \mathfrak{L}(G'')\ \&\ G' \in \mathfrak{T}_i\ \&\ G'' \in \mathfrak{T}_j\ \&\ i < j$, то язык L относится к типу $\mathfrak{T}_j$.

Пример. Рассмотрим грамматику $G = \langle\{S, N, D\}, \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, -, +\}, \{S \to N, S \to +N, S \to -N, N \to D, N \to ND, D \to 0, D \to 1, \ldots, D \to 9\}, S\rangle$, описывающую множество десятичных целых чисел со знаком. Эта грамматика относится к типу 2 (правило $N \to ND$ недопустимо в грамматике типа 3). Однако для того же самого языка можно предложить грамматику $G' = \langle\{S, N\}, \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, -, +\}, \{S \to N, S \to +N, S \to -N, N \to 0, N \to 1, \ldots, N \to 9, N \to 0N, N \to 1N, \ldots, N \to 9N\}, S\rangle$, которая является грамматикой типа 3. Таким образом, язык целых десятичных чисел со знаком является языком типа 3.

1.1.4. Классификация грамматик и языков (3/4)

Вопрос об определении типа конкретной грамматики решается очень просто — рассмотрением вида правил этой грамматики. К сожалению, вопрос об определении типа языка оказывается алгоритмически неразрешимым.

Теорема. *Следующая проблема является алгоритмически неразрешимой: дано* $L = \mathfrak{L}(G)$, $G \in \mathfrak{T}_i$, *определить, существует ли* G', *такая что* $M = \mathfrak{L}(G')$ & $G' \in \mathfrak{T}_j$ & $i < j$.

Доказательство. TODO Теорема Райса-Хиггинсона. Доказательство обязательно в обозначениях раздела 4.5 из Дискретной математики для программистов. Целесообразно добавить атом про проблему останова во Введении. Студенты, которые слушали курс Пастора по теории алгоритмов, могут помочь. □

1.1.4. Классификация грамматик и языков (4/4)

Отступление. Чем выше тип грамматики, порождающий язык, тем лучше — язык оказывается проще, его легче анализировать, описывать и применять. Поэтому авторы новых формальных языков стараются использовать такие грамматические конструкции, которые выражаются правилами регулярных грамматик, в крайнем случае контекстно-свободных грамматик. Однако иногда приходится иметь дело с унаследованными языками, не только естественными, но и искусственными, в частности, с ранними языками программирования. В таких случаях ради совместимости приходится жертвовать удобством.

Пример. В языке Algol-60 была впервые официально использована теория формальных грамматик для описания стандарта языка. При этом принятая грамматика не позволяла определить, к которому **if** относится **else** в условном операторе **if** B_1 **then if** B_2 **then** S_1 **else** S_2. Язык оказался существенно синтаксически неоднозначным.

Классификация грамматик и языков (4/5)

Вопрос об определении типа конкретной грамматики решается очень просто — рассмотрением вида правил этой грамматики. К сожалению, вопрос об определении типа языка оказывается алгоритмически неразрешимым.

Теорема. *Следующая проблема является алгоритмически неразрешимой: дано* $M = \mathcal{L}(G)$, $G \in T_i$, *определить, существует ли* G', *такая что* $M = \mathcal{L}(G') \ \& \ G' \in T_j \ \& \ i < j$

Доказательство. От противного. Предположим, что существует алгоритм $\mathcal{A}$, который для заданного языка $M = \mathcal{L}(G)$ и грамматики $G \in T_i$ определяет, существует ли грамматика $G' \in T_j$, $i < j$, такая что $\mathcal{L}(G') = M$. Поскольку M рекурсивно перечислим, существует грамматика $G \in T_0$, такая что $\mathcal{L}(G) = M$.

Пусть T – произвольная машина Тьюринга, ω – вход для этой машины. Построим язык M следующим образом:

$$M = \begin{cases} \{\omega\}, & \text{если } T \text{ останавливается на входе } \omega; \\ \varnothing, & \text{иначе.} \end{cases}$$

Заметим, что язык M рекурсивно перечислимый, так как мы можем симулировать работу T на ω.

Классификация грамматик и языков (5/5)

Доказательство. (Продолжение) Применим алгоритм из предположения A к языку M и грамматике $G \in T_0$, которая порождает M. Алгоритм A должен определить, существует ли контекстно-свободная грамматика $G' \in T_2$ (поскольку T_2 имеет более низкий тип по сравнению с T_0), такая что $\mathcal{L}(G') = M$.
Если алгоритм A определит, что такая грамматика G' существует, это будет означать, что M является контекстно-свободным языком. Однако, поскольку M построен на основе поведения машины Тьюринга T на входе ω, возможность определить, является ли M контекстно-свободным, эквивалентна решению проблемы остановки для T на ω. Так как проблема остановки алгоритмически неразрешима, это приводит к противоречию с предположением о существовании алгоритма A. □

TODO Неплохо. Исправить обозначения и вклеить в существующие слайды. Очень важно иметь во введении неразрещимость проблемы остановки. Также пока не было сказано, что такое симуляция работы машины Тьюринга.

1.1.5. Неукорачивающие грамматики (1/5)

Грамматика $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$ называется *неукорачивающей*, если длина левой части любого правила не превосходит длины правой части: $\forall\, r = \alpha \to \beta \in \Pi \ (|\alpha| \leqslant |\beta|)$. При любом выводе в такой грамматике длины цепочек могут разве что возрастать. Поэтому, в частности, пустое слово ε не выводимо в такой грамматике.

Пример. Контекстно-зависимые грамматики являются неукорачивающими.

Язык $L \subset \Sigma^*$ называется *разрешимым* (по аналогии с понятием *разрешимое множество*), если существует алгоритм, который для произвольной цепочки $\alpha \in \Sigma^*$ за конечное число шагов даёт ответ на вопрос, входит ли эта цепочка в язык, т. е. $\alpha \in L$ или $\alpha \notin L$. Если при этом число шагов алгоритма определяется по длине цепочки α и может быть указано заранее, то язык называется *легко разрешимым*. Если же алгоритм даёт ответ, когда $\alpha \in L$, и может не давать ответа (алгоритм не заканчивает работу), когда $\alpha \notin L$, то язык называется *полуразрешимым*. Алгоритм, разрешающий язык, называют *распознающим*.

Теорема. *Язык $\mathfrak{L}(G)$, порождаемый неукорачивающей грамматикой $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$, легко разрешимый.*

Доказательство. Если $\alpha \in \mathfrak{L}(G)$, то $S \Rightarrow^* \alpha$, существует бесповторный вывод $\sigma_0, \ldots, \sigma_k$, где $\sigma_0 = S$ и $\sigma_k = \alpha$, причём $\forall\, i \in 0..k \ (|\sigma_i| \leqslant |\alpha|)$. Обозначим $n := |\alpha|$.

1.1.5. Неукорачивающие грамматики (2/5)

Доказательство. (Окончание) Число $r(n)$ различных цепочек длиной не более n:

$$r(n) = \sum_{i=1}^{n} |\mathrm{M} \cup \Sigma|^i = \sum_{i=1}^{n} (|\mathrm{M}| + |\Sigma|)^i = (|\mathrm{M}| + |\Sigma|) \frac{(|\mathrm{M}| + |\Sigma|)^n - 1}{|\mathrm{M}| + |\Sigma| - 1}.$$

Далее, длина бесповторной последовательности, составленной из таких цепочек, также ограничена (например, величиной $r(n)$). Общее количество $R(n)$ всех бесповторных последовательностей цепочек длиной не более n также ограничено:

$$R(n) = \sum_{i=0}^{r(n)} A(r(n), i) = \sum_{i=0}^{r(n)} \frac{r(n)!}{(r(n) - i)!} = r(n)! \sum_{i=0}^{r(n)} \frac{1}{i!} \leqslant r(n)! \sum_{i=0}^{\infty} \frac{1}{i!} = r(n)! \cdot e.$$

Тем более ограничено количество таких бесповторных последовательностей, которые начинаются с S и заканчиваются α. Отсюда немедленно вытекает схема алгоритма распознавания. Нужно перебрать все бесповторные последовательности цепочек длиной не более n, которые начинаются с S и заканчиваются α, и для каждой последовательности проверить, не является ли она допустимым выводом α в G. Если такой вывод найдётся, то можно прекратить дальнейший перебор и сделать вывод, что $\alpha \in \mathfrak{L}(G)$. В противном случае $\alpha \notin \mathfrak{L}(G)$.

1.1.5. Неукорачивающие грамматики (3/5)

Замечание. Число шагов перебора, о котором идёт речь в доказательстве, очень велико. Это не случайно. Можно показать, что в общем случае не существует эффективного алгоритма распознавания языка, порождённого неукорачивающей грамматикой. Другими словами, хотя число шагов работы алгоритма распознавания можно предсказать заранее, но это число может быть столь велико, что алгоритм практически не применим.

Отступление. Грамматически можно описать любое перечислимое множество, в том числе и неразрешимое. Поскольку грамматики типа $\mathfrak{T}_1$ описывают распознаваемые, то есть разрешимые языки, ясно что описание нераспознаваемых языков является уделом грамматик типа $\mathfrak{T}_0$. Хотя грамматики типа $\mathfrak{T}_1$ удобны для описания языков (контекст очень полезен) и порождают распознаваемые языки, но алгоритмы распознавания имеют слишком большую трудоёмкость. Поэтому грамматики типа $\mathfrak{T}_0$ и $\mathfrak{T}_1$ *никогда* не применяются для описания искусственных языков программирования. В информационных технологиях применяются грамматики типа $\mathfrak{T}_2$ и $\mathfrak{T}_3$. Лингвисты иногда применяют грамматики типа $\mathfrak{T}_0$ и $\mathfrak{T}_1$ для описания естественных языков, но это только потому, что они не имеют права изменять язык в угоду удобству описания, а программисты могут и должны это делать. Современная тенденция в языкотворчестве ведет к исключению из формальных языков конструкций, которые невозможно описать грамматиками типа $\mathfrak{T}_2$ и $\mathfrak{T}_3$.

1.1.5. Неукорачивающие грамматики (4/5)

Замечание. Любую грамматику можно доопределить так, чтобы все символы, входящие в исходные правила, были нетерминалами. Для этого достаточно для каждого терминального символа a ввести новый нетерминальный символ A и правило $A \to a$. После этого нужно все вхождения терминальных символов в старые правила заменить новыми нетерминальными символами.

Теорема. *Для всякой неукорачивающей грамматики существует эквивалентная ей контекстно-зависимая грамматика.*

Доказательство. Построим правила для контекстно-зависимой грамматики G_2 на основе правил неукорачивающей грамматики G_1. Каждое правило

$$X_1 X_2 \dots X_n \to Y_1 Y_2 \dots Y_{n+m} \in \Pi, \quad n > 0, m \geqslant 0$$

заменим на ряд из $2n$ следующих правил: $X_1 X_2 \dots X_n \to Z_1 X_2 \dots X_n$,
$Z_1 X_2 \dots X_n \to Z_1 Z_2 \dots X_n$, $\dots$, $Z_1 Z_2 \dots X_n \to Z_1 Z_2 \dots Z_n$, и
$Z_1 Z_2 \dots Z_n \to Y_1 Z_2 \dots Z_n$, $Y_1 Z_2 \dots Z_n \to Y_1 Y_2 \dots Z_n$, $\dots$,
$Y_1 Y_2 \dots Z_n \to Y_1 Y_2 \dots Y_n \dots Y_{n+m}$, где $Z_1, Z_2, \dots, Z_n$ — новые нетерминалы, для каждого исходного правила они свои. □

1.1.5. Неукорачивающие грамматики (5/5)

Доказательство. (продолжение) Правила с новыми нетерминалами могут применяться только в одной указанной последовательности. Другими словами, если будет применено правило $X_1X_2 \dots X_n \to Z_1X_2 \dots X_n$, то все последующие правила тоже будут применены, потому что иначе нетерминалы $Z_1, Z_2, \dots Z_n$ будут присутствовать в выведенном слове, чего быть не должно. Построенная грамматика эквивалентна исходной, потому что в результате применения новых правил цепочка $X_1X_2 \dots X_n$ перейдет в $Y_1Y_2 \dots Y_{n+m}$. Новая грамматика является контекстно-зависимой грамматикой по построению. □

Из доказанной теоремы и предшествующих наблюдений следует важный вывод: классы языков, порождаемых неукорачивающими грамматиками и грамматиками непосредственных составляющих, совпадают.

Замечание. Неукорачивающие грамматики, очевидно, не могут порождать языки, содержашие пустое слово ε. В большинстве случаев это обстоятельство не является существенным. Если, однако, требуется, чтобы пустое слово входило в язык, то достаточно в неукорачивающей грамматике разрешить иметь одно исключительное правило: $S \to \varepsilon$.

1.1.6. Контекстно-свободные грамматики (1/5)

Замечание. Для краткости оборот «контекстно-свободная грамматика» далее заменяется аббревиатурой КСГ.

Пусть задана КСГ $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$ и вывод $S \Rightarrow_G^* \alpha$ в этой грамматике. Поскольку все правила имеют вид $A \to \alpha$, то в выводе $S = \sigma_0, \sigma_1, \ldots, \sigma_n = \alpha$ для каждой пары σ_i, σ_{i+1} соседних цепочек следующая получается из предыдущей заменой *одного* нетерминального символа. Вообще говоря, возможно заменить любой нетерминальный символ, поэтому последовательность цепочек в выводе отнюдь не является однозначно определённой. Вывод называется *левосторонним* (*правосторонним*), если на каждом шаге заменяется самое левое (соответственно, самое правое) вхождение нетерминального символа в цепочку.

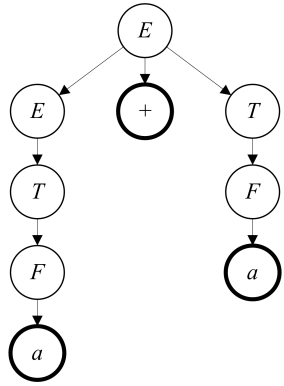
Вывод в КСГ удобно представить в виде *дерева разбора* (или *дерева вывода*) — упорядоченного ордерева, корнем которого является аксиома грамматики, нелистовыми узлами являются нетерминалы, а листьями — терминалы. Если в выводе было применено правило $A \to \alpha$, то в дереве разбора проводятся дуги из узла A ко всем узлам, соответствующим буквам цепочки α, причем эти узлы располагаются в том же порядке, что и буквы в цепочке α.

1.1.6. Контекстно-свободные грамматики (2/5)

Дерево вывода компактно представляет множество эквивалентных выводов, в которых применены одни и те же правила, но в разном порядке.

Пример. Рассмотрим КСГ G_1 с правилами $E \to E + T$, $E \to T$, $T \to T * F$, $T \to F$, $F \to a$ — упрощенную грамматику арифметических выражений. Дерево разбора справа в компактной форме представляет 10 (десять!) возможных выводов цепочки $a + a$.

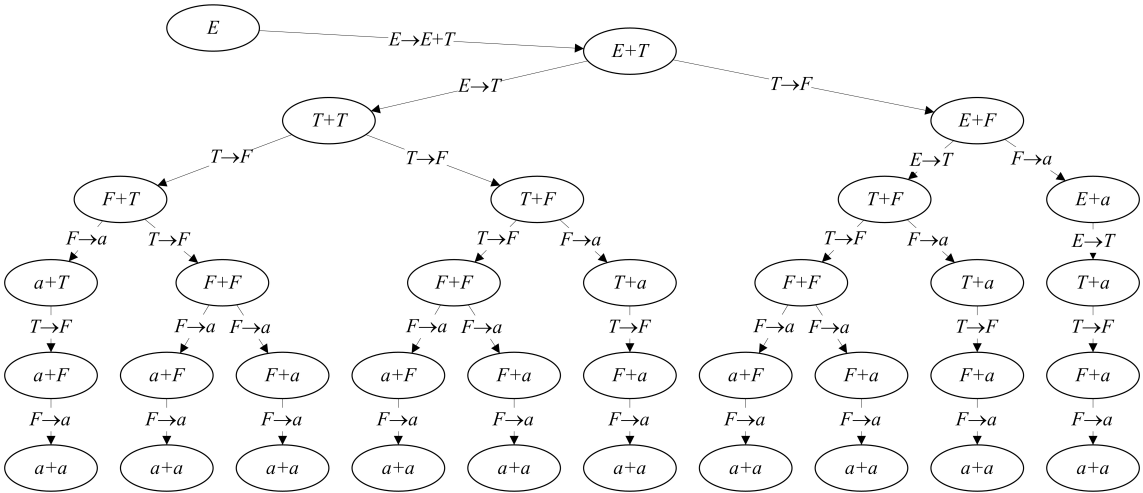
Ясно, что по дереву вывода можно однозначно восстановить левосторонний (равно как и правосторонний) вывод в данной грамматике.



Ещё одной наглядной формой представления выводов в КСГ является *дерево порождения*. Дерево порождения строится следующим образом. Корнем дерева является узел, помеченный аксиомой грамматики S. Если в грамматике есть правило $A \to \alpha$, и в дереве построен узел, помеченный цепочкой $\beta A \gamma$, то добавляется узел, помеченный цепочкой $\beta\alpha\gamma$, и проводится дуга $(\beta A\gamma, \beta\alpha\gamma)$, помеченная правилом $A \to \alpha$.

1.1.6.Контекстно-свободные грамматики (3/5)

Пример. На рисунке представлено дерево порождения, в котором все десять выводов предыдущего примера показаны в явной форме.



1.1.6.Контекстно-свободные грамматики (4/5)

Один и тот же язык может порождаться разными (но эквивалентными) контекстно-свободными грамматиками.

Пример. Рассмотрим ещё раз КСГ G_1 с правилами $E \to E{+}T, E \to T, T \to T{*}F, T \to F, F \to a$ и КСГ G_2 с правилами $E \to E + E$, $E \to E * E$, $E \to a$. Грамматика G_2 задает тот же упрощенный язык арифметических выражений, что и грамматика G_1. Чтобы убедиться в этом, достаточно заметить, что любой вывод в грамматике G_1 легко перестраивается в вывод в грамматике G_2 той же терминальной цепочки. Действительно, вместо правила $E \to E + T$ надо применить правило $E \to E + E$, вместо правила $T \to T * F$ надо применить правило $E \to E * E$, вместо правила $F \to a$ надо применить правило $E \to a$, а применения правил $E \to T$ и $T \to F$ надо просто исключить из вывода.

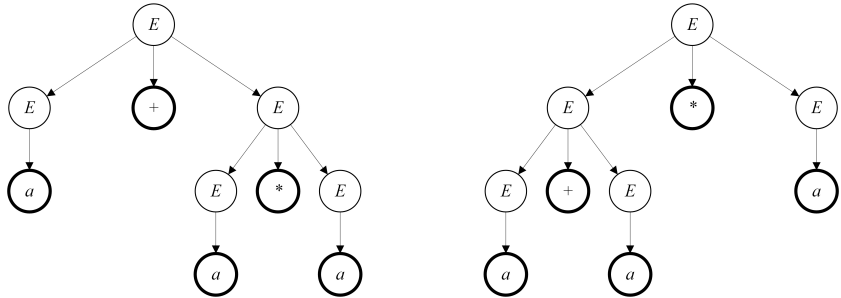
Представленные грамматики G_1 и G_2 задают один и тот же язык, но грамматика G_2 более компактна, то есть содержит меньше правил и меньше нетерминалов, чем грамматика G_1. Возникает вопрос: на основании каких свойств следует предпочесть одну КСГ другой? Ответ на поставленный вопрос зависит от целей построения грамматического описания языка.

Например, для *публикации* языка предпочтительна более компактная грамматика G_2

1.1.6.Контекстно-свободные грамматики (5/5)

Однако с точки зрения использования языка для *программирования* грамматика G_2 обладает двумя существенными недостатками.

Во-первых, КСГ G_2 *неоднозначна* — на рисунке представлены два различных дерева разбора для выражения $a + a * a$ в этой грамматике.



Во-вторых, грамматика G_2 не учитывает семантику языка (приоритет арифметических операций). Если использовать дерево разбора для вычислений (что и делается в большинстве реальных компиляторов), то дерево на рисунке справа даст неверный результат.

1.1.7. Линейные, регулярные и автоматные грамматики (1/5)

Пусть задана порождающая грамматика $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$. Правило грамматики называется *начальным*, если в левой части стоит аксиома грамматики S. Правило грамматики называется *заключительным*, если оно имеет вид $A \to \alpha$, где $A \in \mathrm{M}$, $\alpha \in \Sigma^*$. Правило называется *линейным* (соответственно, *леволинейным* или *праволинейным*), если оно имеет вид $A \to \alpha B \beta$ (соответственно, $A \to B\beta$ или $A \to \alpha B$), где $A, B \in \mathrm{M}$ и $\alpha, \beta \in \Sigma^*$.

Порождающая грамматика, все правила которой заключительные или линейные (соответственно, леволинейные или праволинейные) называется *линейной* (соответственно, *леволинейной* или *праволинейной*). Очевидно, что праволинейные и леволинейные грамматики являются линейными, а линейные грамматики являются контекстно-свободными.

Язык, порождаемый линейной (соответственно, леволинейной или праволинейной) грамматикой называется *линейным* (соответственно, *леволинейным* или *праволинейным*).

Пример. Язык $\{a^n b^n \mid n \in \mathbb{N}\}$ является линейным языком, потому что порождается линейной грамматикой $\langle \{S, A\}, \{a, b\}, \{S \to aAb, A \to \varepsilon, A \to aAb\}, S \rangle$.

Замечание. Любое дерево вывода в линейной грамматике имеет один нетерминальный «ствол», по бокам от которого висят терминальные «листья», причём в леволинейных деревьях листья только справа, а в праволинейных деревьях листья только слева.

1.1.7. Линейные, регулярные и автоматные грамматики (2/5)

Теорема. *Для любой праволинейной грамматики существует эквивалентная леволинейная грамматика и наоборот.*

Доказательство. Пусть задана праволинейная грамматика. Построим для неё эквивалентную леволинейную грамматику: для каждого начального правила $S \to \alpha A$ введём заключительное правило $A \to \alpha$, для каждого праволинейного правила $A \to \alpha B$ введём леволинейное правило $B \to A\alpha$, для каждого заключительного правила $A \to \alpha$ введём начальное правило $S \to A\alpha$. Теперь любой вывод в исходной грамматике преобразуется в вывод в построенной грамматике: достаточно применять новые правила начиная с конца исходного вывода. □

Таким образом праволинейные и леволинейные грамматики порождают один и тот же класс языков. Поэтому определение в п. 1.1.3, гласящее, что леволинейные и праволинейные грамматики называются *регулярными*, оказывается вполне корректным. Для определённости далее рассматриваются леволинейные грамматики. Регулярная грамматика называется *автоматной*, если все терминальные цепочки α в правилах состоят ровно из одного символа. Другими словами, леволинейная автоматная грамматика содержит правила вида $A \to a$ и $A \to Ba$, где $A, B \in \mathrm{M}$, $a \in \Sigma$.

1.1.7. Линейные, регулярные и автоматные грамматики (3/5)

Терминальные цепочки в правилах регулярной грамматики могут быть пустыми, и значит, язык, порождаемый регулярной грамматикой G, может содержать пустое слово ε. В то же время язык, порождаемый автоматной грамматикой, очевидно, не содержит пустого слова ε. Таким образом, формально эти классы языков различны, но на самом деле они почти эквивалентны.

Чтобы установить эту связь, необходимо обозначить некоторые специальные свойства грамматик, которые понадобятся также и в дальнейшем.

Нетерминальный символ называется *непорождающим*, если из него невозможно вывести терминальную цепочку. Нетерминальный или терминальный символ называется *недостижимым*, если его невозможно вывести из аксиомы. Если правило имеет вид $A \to B$, то оно называется *цепным*. Если в правиле $A \to \alpha$ цепочка α пустая, $\alpha = \varepsilon$, то правило принимает вид $A \to$ и называется *ε-правилом*. Если в одном из начальных правил $S \to \alpha$ аксиома входит в правую часть, то правило называется *S-правилом*.

Недостижимые и непопрождающие символы, цепные правила, ε-правила и S-правила грамматик создают различные помехи в рассуждениях и применении алгоритмов, но при этом все такие особенности грамматик являются *устранимыми* (см. п. 1.3.2).

1.1.7. Линейные, регулярные и автоматные грамматики (4/5)

Грамматики, в которых нет непорождающих и недостижимых символов, а также нет цепных правил, ε-правил и S-правил называются *приведёнными*.

На практике языки с пустыми словами, как правило, не используются, а грамматики составляются квалифицированными авторами и не содержат непорождающих и недостижимых символов, а также неподходящих правил. Поэтому мы докажем утверждение об эквивалентности регулярных и автоматных грамматик в существенно более слабой, но практически более полезной форме, а именно в форме алгоритма преобразования *приведённой* грамматики к нужному виду.

Теорема. *Для любой приведённой регулярной грамматики существует эквивалентная автоматная грамматика.*

Доказательство. Пусть задана леволинейная регулярная грамматика $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$, где каждое правило $r \in \Pi$ имеет вид $A \to \alpha$ или $A \to B\alpha$, причем $A, B \in \mathrm{M}$, $\alpha \in \Sigma^*$ и $|\alpha| \geqslant 1$. Построим автоматную грамматику $G' = \langle \mathrm{M}', \Sigma, \Pi', S \rangle$ в соответствии с алгоритмом на следующем слайде. Очевидно, что всякому выводу в грамматике G однозначно соответствует вывод в грамматике G'. Таким образом, грамматики эквивалентны.

1.1.7. Линейные, регулярные и автоматные грамматики (5/5)

Вход: Регулярная леволинейная грамматика $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$

Выход: Автоматная леволинейная грамматика $G' = \langle \mathrm{M}', \Sigma, \Pi', S \rangle$.

$\mathrm{M}' := \mathrm{M}; \Pi' := \varnothing$ // $r = A \to \alpha$ или $r = A \to B\alpha$

for $r \in \Pi$ **do**

  if $|\alpha| = 1$ **then** $\Pi' := \Pi' \cup r$; **next for** r **end if** // переносим без изменений

  $n := |\alpha| - 1$ // количество новых нетерминалов

  $\mathrm{M}' := \mathrm{M}' \cup$ **new** A_n // добавляем нетерминал A_n

  $\Pi' := \Pi' \cup (A \to A_n \alpha[n+1])$ // добавляем правило $A \to A_n a_{n+1}$

  for i **from** $n-1$ **to 1 do**

    $\mathrm{M}' := \mathrm{M}' \cup$ **new** A_i // добавляем нетерминал A_i

    $\Pi' := \Pi' \cup (A_{i+1} \to A_i \alpha[i+1])$ // добавляем правило $A_{i+1} \to A_i a_{i+1}$

  end for

  if $r = A \to \alpha$ **then** $\Pi' := \Pi' \cup (A_1 \to \alpha[1])$ // заключительное правило

  else $\Pi' := \Pi' \cup (A_1 \to B\alpha[1])$ **end if** // незаключительное правило

end for

1.1.8. Языки записи грамматик (1/9)

Запись правил грамматики в общей форме довольно многословна, не всегда легко читается и существенно использует рекурсию.

Пример. Набор правил $N \to D$, $N \to ND$, $D \to 0$, $D \to 1$, $D \to 2$, $D \to 3$, $D \to 4$, $D \to 5$, $D \to 6$, $D \to 7$, $D \to 8$, $D \to 9$ по существу означает, что целое без знака — это последовательность из одной или более цифр, что не всегда очевидно без дополнительных пояснений.

Были предложены многочисленные усовершенствования записи грамматик, в особенности контекстно-свободных и автоматных грамматик, которые сводятся к введению дополнительных *метасимволов*. Дополнительные метасимволы не меняют выразительной силы грамматик, но делают запись грамматики более удобной для восприятия человеком. Обычно рассматриваются следующие метасимволы.

1. Вертикальная черта $|$ — разделение альтернатив. Правило

$$A \to \alpha \,|\, \beta$$

эквивалентно паре правил

$$A \to \alpha \qquad A \to \beta$$

1.1.8. Языки записи грамматик (2/9)

2. Круглые скобки $(\ )$ — введение неявного нетерминала. Круглые скобки могут содержать несколько альтернатив, разделенных метасимволом $|$. Правило

$$A \to \alpha\,(\,\beta\,)\,\gamma$$

эквивалентно паре правил

$$A \to \alpha B \gamma \qquad B \to \beta$$

где B — это новый нетерминал.

3. Квадратные скобки $[\ ]$ — необязательная конструкция. Цепочка, заключенная в квадратные скобки, может присутствовать или отсутствовать. Правило

$$A \to \alpha\,[\,\beta\,]\,\gamma$$

эквивалентно паре правил

$$A \to \alpha\gamma \qquad A \to \alpha\beta\gamma.$$

1.1.8. Языки записи грамматик (3/9)

4. Фигурные скобки $\{\ \}$ — повторение конструкции. Цепочка, заключенная в фигурные скобки, может повторяться несколько раз. Правило

$$A \to \alpha \{ \beta \} \gamma$$

эквивалентно тройке правил

$$A \to \alpha B \gamma \qquad B \to \beta \qquad B \to B\beta$$

где B — это новый нетерминал.

5. Диез $\#$ в фигурных скобках — итерирование одной конструкции через другую конструкцию. Этот метасимвол является обобщением предыдущего: $\{ \alpha \} = \{ \alpha \# \varepsilon \}$. Правило

$$A \to \alpha \{ \beta \# \gamma \} \delta$$

эквивалентно тройке правил

$$A \to \alpha B \delta \qquad B \to \beta \qquad B \to B\gamma\beta$$

где B — это новый нетерминал.

1.1.8.Языки записи грамматик (4/9)

6. Точка с запятой ; — разделитель правил в множестве правил. Строчка

$$A \to \alpha \,;\, B \to \beta$$

эквивалентна паре правил

$$A \to \alpha \qquad B \to \beta$$

7. Точка . — признак конца множества правил.
8. Двоеточие : — замена метасимвола $\to$, которого нет на стандартной клавиатуре.

Замечание. Обычно считают, что метасимволы [ ], { } и # также вводят неявный нетерминал, а также что символ разделения альтернатив имеет приоритет перед всеми скобками, так что писать дополнительные круглые скобки не обязательно.

Пример. В приведённых обозначениях грамматику целых чисел с возможным знаком можно описать следующим образом:

$$N \to [\,+\,|\,-\,]\,(1\,|\,2\,|\,3\,|\,4\,|\,5\,|\,6\,|\,7\,|\,8\,|\,9)\,\{0\,|\,1\,|\,2\,|\,3\,|\,4\,|\,5\,|\,6\,|\,7\,|\,8\,|\,9\}\;\;|\;\;0\;.$$

Здесь не порождаются ведущие нули, а ноль знака не имеет. В такой записи грамматики отсутствует явная рекурсия и не используются вспомогательные нетерминалы.

1.1.8. Языки записи грамматик (5/7)

Иногда метасимволы { } определяют так, чтобы конструкция повторялась 0 или более раз, т.е. могла отсутствовать. В этом случае правило

$$A \to \alpha \{ \beta \} \gamma$$

считается эквивалентным трём правилам

$$A \to \alpha B \gamma \qquad B \to \varepsilon \qquad B \to B\beta$$

где B — это новый нетерминал, а ε — пустая цепочка.

Замечание. Символы : | ( ) [ ] { } # ; . присутствуют на клавиатуре и ими удобно пользоваться при описании грамматики. Однако эти же символы часто используются как терминальные символы языка, и тогда встаёт вопрос, как отличить терминальный символ от метасимвола? Здесь этот вопрос решён *визуально*, за счёт использования цвета. Если система программирования не допускает различения символов по цвету, то приходится придумывать дополнительные средства, например, заключать терминальные символы в кавычки ” ”.

Нетрудно убедиться, что все эти расширения (а также некоторые другие, применяемые на практике) не выводят за пределы контекстно-свободных грамматик типа $\mathfrak{T}_2$.

1.1.8. Языки записи грамматик (6/9)

Таким образом, сосуществуют и используются *несколько языков* для записи порождающих формальных грамматик. Такая множественность языков и диалектов является скорее правилом, нежели исключением. Какой вариант использовать — дело вкуса.

Очевидно, что использование метасимволов сокращает запись грамматики делает её более наглядной (при том условии, однако, что читатель грамматики точно знает значения всех метасимволов). С другой стороны, использование метасимволов существенно усложняет, например, доказательство эквивалентности грамматик, поскольку приходится рассматривать в доказательстве каждый метасимвол отдельно.

Пример. На следующем слайде для сравнения приведены четыре варианта записи грамматики двух языков: строгого языка записи грамматик с использованием $\rightarrow$ и расширенного языка с использованием метасимволов. Для наглядности в этих записях использованы одинаковые нетерминальные символы: G — собственно грамматика, N — алфавит нетерминальных символов целевого языка, T — алфавит терминальных символов целевого языка, R — правило грамматики, S — цепочка символов. Все терминальные символы заключены в кавычки ” ”. Правила строгого языка считаются разделенными некоторым непечатаемым символом (конец строки, пробел, ...), который здесь обозначен $\sqcup$.

1.1.8. Языки записи грамматик (7/9)

Грамматика строгого языка, записанная на строгом языке

$G \to R$
$G \to R \sqcup G$
$R \to N \text{ "}\to\text{" } S$
$S \to N$
$S \to T$
$S \to NS$
$S \to TS$

Грамматика расширенного языка, записанная на строгом языке

$G \to R \text{ "."}$
$G \to R \text{ ";" } G$
$R \to N \text{ ":" } S$
$S \to N$
$S \to T$
$S \to NS$
$S \to TS$
$S \to S \text{ "|" } S$
$S \to \text{"\{" } S \text{ "\#" } S \text{ "\}"}$
$S \to \text{"(" } S \text{ ")"}$
$S \to \text{"[" } S \text{ "]"}$
$S \to \text{"\{" } S \text{ "\}"}$

Грамматика строгого языка, записанная на расширенном языке

$G \ : \ \{ \ R \ \# \sqcup \ \} \ ;$
$R \ : \ N \text{ "}\to\text{" } S \ ;$
$S \ : \ \{ \ N \mid T \} \ .$

Грамматика расширенного языка, записанная на расширенном языке

$G \ : \ \{ \ R \ \# \text{ ";"} \} \text{ "."} \ ;$
$R \ : \ N \ \text{ ":" } S \ ;$
$S \ : \ \{ \ N \mid T \} \quad | \quad \text{"("} S \ \# \text{ " } | \text{ " } \text{")"} \quad |$
$\text{"[" } S \ \# \text{ " } | \text{ " } \text{"]"} \quad | \quad \text{"\{"} S \ [\text{ "\#" } S \ ] \text{ "\}"} \ .$

1.1.8. Языки записи грамматик (8/9)

Почти одновременно с появлением теории формальных грамматик для практических целей был предложен целый ряд других средств. Исторически самым заметным явилось описание языка программирования Алгол-60 с помощью *нормальных форм Бэкуса-Наура* (сокращенно БНФ). Описание языка с помощью нормальных форм Бэкуса[2]–Наура[3] состоит из набора *основных символов* языка, набора названий конструкций (или *металингвистических переменных*) и набора *металингвистических формул*. Каждая металингвистическая формула состоит из левой части, в которой стоит металингвистическая переменная, *металингвистической связки* $::=$ (читается «это есть») и правой части, в которой указывается один или несколько вариантов построения конструкции в левой части. Каждый вариант — это последовательность металингвистических переменных и основных символов. Варианты разделяются символом $|$ (читается «или»). Металингвистические переменные заключаются в угловые скобки, чтобы не путать их с основными символами. Считается, что метасимволы $<, >, |$ и $::=$ не входят в число основных символов.

[2] Джон Бэкус (1924–2007)

[3] Питер Наур (1928–2016)

1.1.8. Языки записи грамматик (9/9)

Пример. Язык целых десятичных чисел со знаком в обозначениях Бэкуса-Наура:
<Целое со знаком> ::= <Целое без знака> | $+$ <Целое без знака> | $-$ <Целое без знака>
<Целое без знака> ::= 0 | <Цифра не ноль> <Последовательность цифр>
<Последовательность цифр> ::= | <Последовательность цифр> <Цифра>
<Цифра> ::= 0 | <Цифра не ноль>
<Цифра не ноль> ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Совершенно ясно, что нормальные формы Бэкуса-Наура — это один из способов записать контекстно-свободную грамматику, то есть грамматику типа $\mathfrak{T}_2$. Действительно, основные символы — это алфавит терминальных символов Σ, металингвистические переменные — это алфавит нетерминальных символов M, металингвистические формулы — это множество правил вывода Π, в которых вместо символа $\to$ использована связка ::=, а аксиома грамматики обычно ясна из контекста.

Замечание. Форма Бэкуса–Наура была предназначена, по замыслу авторов, прежде всего для точного и в то же время *удобочитаемого* описания синтаксиса языков программирования, поэтому используются развёрнутые названия конструкций.

1.1.9. Альтернативные средства задания синтаксиса (1/9)

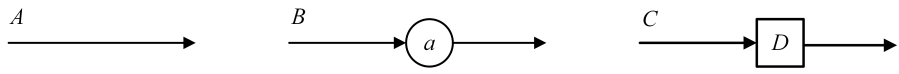
Отступление. Нормальные формы Бэкуса–Наура оказали большое влияние на развитие техники определения искусственных языков, в частности, языков программирования. В язык БНФ были добавлены различные усовершенствования, подобные рассмотренным выше метасимволам, и появились различные диалекты расширенной БНФ (РБНФ). До сих пор эта форма описания синтаксиса признаётся в качестве стандарта и используется как авторами языков, так и разработчиками трансляторов. Если бы дело ограничивалось сравнительно немногочисленными языками программирования общего назначения, то металингвистических формул было бы достаточно. Но для специализированных языков предметных областей формы Бэкуса–Наура оказались недостаточно *удобны*. Требуются более лаконичные, более наглядные, более современные средства, поскольку языков предметных областей в сотни раз больше, время их жизни в десятки раз меньше, а пользователями описания языка являются специалисты предметной области, не знающие и не нуждающиеся в знаниях тонкостей синтаксического анализа и компиляции.

В этом параграфе рассматриваются три разных подхода к заданию синтаксиса: использование особых графов (синтаксические диаграммы Вирта), использование языков разметки (XML) и использования языков задания иерархических структур (YAML).

1.1.9. Альтернативные средства задания синтаксиса (2/9)

Синтаксические диаграммы — это особый вид орграфов, предложеный Виртом[4] для записи синтасиса языка Паскаль. Каждая синтаксическая диаграмма — это сеть с одним источником и одним стоком, которые называются «вход» и «выход», соответственно. По традиции вход и выход не изображают на диаграмме как узлы, а рисуют приходящую «из ниоткуда» входную стрелку и уходящую «в никуда» выходную стрелку. Узлы на синтаксической диаграмме изображают двух типов: с прямыми углами (они соответствуют нетерминалам) и со скругленными углами (они соответствуют терминалам). Диаграммы имеют имя, оно обычно пишется в левом верхнем углу диаграммы.

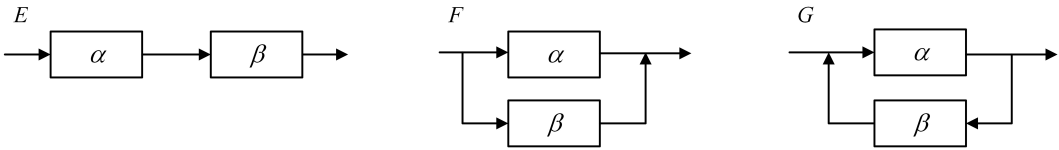
Пример. На рисунке приведены три диаграммы, A, B и C. Диаграмма A — это *тривиальная* диаграмма, она имеет только вход, выход и соединяющую их дугу, которую мы видим на диаграмме. Диаграммы B и C — это *элементарные* диаграммы, они имеют один узел, помеченный буквой (терминалом или нетерминалом) и входную и выходную дуги.



[4] Никлаус Вирт (1934–2024)

1.1.9. Альтернативные средства задания синтаксиса (3/9)

Над синтаксическими диаграммами определены операции *последовательного соединения* (рисунок ниже, диаграмма E), *параллельного соединения* (диаграмма F) и *итерации* (диаграмма G), которые позволяют строить более сложные диаграммы из более простых. На рисунке α и β — это *любые* диаграммы, сколь угодно сложные, но имеющие один вход и один выход.



Язык в целом описывается *совокупностью* синтаксических диаграмм. Каждая конструкция языка (то есть нетерминальный символ или металингвистическая переменная) определяется на отдельной диаграмме и записывается в качестве названия этой диаграммы. Один нетерминал должен быть указан как начальный (аксиома). При этом диаграмма определяет нетерминал в следующем смысле. Каждый путь на синтаксической диаграмме, ведущий от входа к выходу — это цепочка, выводимая из данного нетерминала.

1.1.9. Альтернативные средства задания синтаксиса (4/9)

Пример. На рисунке представлена пара синтаксических диаграмм, определяющих целые числа со знаком.

TODO диаграммы

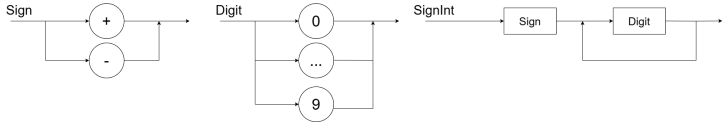
Пример. На рисунке представлены синтаксические диаграммы строгого языка описания грамматик из предыдущего параграфа.

TODO диаграммы

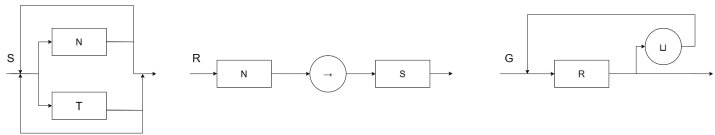
Говорят, что совокупность синтаксических диаграмм *правильно построена*, если для любого нетерминала есть ровно одна синтаксическая диаграмма и все синтаксические диаграммы построены из тривиальных и элементарных с помощью операций последовательного и параллельного соединения и итерации.

1.1.9. Альтернативные средства задания синтаксиса (4/9)

Пример. На рисунке представлена пара синтаксических диаграмм, определяющих целые числа со знаком.



Пример. На рисунке представлены синтаксические диаграммы строгого языка описания грамматик из предыдущего параграфа.



Говорят, что совокупность синтаксических диаграмм *правильно построена*, если для любого нетерминала есть ровно одна синтаксическая диаграмма и все синтаксические диаграммы построены из тривиальных и элементарных с помощью операций последовательного и параллельного соединения и итерации.

1.1.9. Альтернативные средства задания синтаксиса (5/9)

Теорема. *Для любой правильно построенной совокупности синтаксических диаграмм существует эквивалентная контекстно-свободная грамматика.*

Доказательство. Рассмотрим синтаксическую диаграмму для некоторого нетерминала A. Построим совокупность контекстно-свободных правил, так что каждому пути в диаграмме будет соответствовать вывод из A по данным правилам и обратно. Для этого введем необходимое количество новых нетерминалов, соответствующих структурным элементам диаграммы, построенной по правилам, приведённым на предыдущих слайдах, и добавим правила в соответствии со следующей таблицей.

Диаграмма	Рисунок	Правила
Тривиальная	A	$A \to \varepsilon$
Элементарная с терминалом a	B	$B \to a$
Элементарная с нетерминалом D	C	$C \to D$
Последовательное соединение α и β	E	$E \to \alpha\beta$
Параллельное соединение α и β	F	$F \to \alpha \mid \beta$
Итерация α через β	GE	$G \to \alpha \mid E\beta\alpha$

Очевидно, что правила в правом столбце контекстно-свободные.

1.1.9. Альтернативные средства задания синтаксиса (6/9)

Доказательство. (Окончание) Далее, по всякому пути на диаграмме можно построить вывод цепочки, прочитываемой вдоль пути. Вывод строится в обратном порядке: сначала мы имеем путь и заменяем в нем прохождение пустых ребер на нетерминалы по ε-правилам (см. первую строку таблицы), затем заменяем элементарные диаграммы на нетерминалы, затем пары нетерминалов в соответствии с операциями и так далее, пока не получим один нетерминал — название диаграммы.

Обратно, по всякому выводу в построенном множестве правил, который заканчивается цепочкой, состоящей из терминалов и нетерминалов, присутствующих на диаграмме в виде узлов, можно указать путь, соответствующий данному выводу.

Построение правил (введение дополнительных нетерминалов) производится для каждой диаграммы независимо (все новые нетерминалы различны). Поэтому выводу в грамматике в целом соответствует путь в иерархии диаграмм и обратно. □

Отступление. Синтаксические диаграммы полезно сопоставить с источниками (см. п. 1.2.3). И в том и в другом случае для описания языка используется граф специального вида, а язык определяется как множество некоторых путей в таком графе. Но в источниках «лингвистическая» нагрузка приходится на ребра, а в синтаксических диаграммах — на вершины.

1.1.9. Альтернативные средства задания синтаксиса (7/9)

Широко используемым на практике средством задания синтаксиса являются *языки разметки*.

TODO Кратко идея SGML, далее HTML, XML и XML Scheme с обязательным сопоставлением с КСГ. Ещё можно (но не очень нужно) замечание про Mark Down

1.1.9. Альтернативные средства задания синтаксиса (8/9)

Сосуществуют и активно используются возникшие в разное время и по различным причинам несложные, но полезные средства описания иерархических текстовых структур.

TODO Далее можно YAML или JSON или что угодно, но обязательно с переводом с птичьего языка на язык КСГ.

1.1.9. Альтернативные средства задания синтаксиса (9/9)

TODO Опционально: слова про метод раскрутки (bootstapping), поднять себя за шнурки ботинок, с демонстрацией выразительной силы любого из рассмотренных средств. Нечто похожее на слайд 1.1.8.(7/9)

1.1.8. Языки записи грамматик (4/9)

6. Точка с запятой ; — разделитель правил в множестве правил. Строчка

$$A \to \alpha \,;\, B \to \beta$$

эквивалентна паре правил

$$A \to \alpha \qquad B \to \beta$$

7. Точка . — признак конца множества правил.
8. Двоеточие : — замена метасимвола $\to$, которого нет на стандартной клавиатуре.

Замечание. Обычно считают, что метасимволы [ ], { } и # также вводят неявный нетерминал, а также что символ разделения альтернатив имеет приоритет перед всеми скобками, так что писать дополнительные круглые скобки не обязательно.

Пример. В приведённых обозначениях грамматику целых чисел с возможным знаком можно описать следующим образом:

$$N \to [\,+\,|\,-\,]\,(1\,|\,2\,|\,3\,|\,4\,|\,5\,|\,6\,|\,7\,|\,8\,|\,9)\,\{0\,|\,1\,|\,2\,|\,3\,|\,4\,|\,5\,|\,6\,|\,7\,|\,8\,|\,9\}\;\;|\;\;0\;.$$

Здесь не порождаются ведущие нули, а ноль знака не имеет. В такой записи грамматики отсутствует явная рекурсия и не используются вспомогательные нетерминалы.

1.1.8. Языки записи грамматик (5/9)

Иногда метасимволы { } определяют так, чтобы конструкция повторялась 0 или более раз, т.е. могла отсутствовать. В этом случае правило

$$A \to \alpha \{ \beta \} \gamma$$

считается эквивалентным трём правилам

$$A \to \alpha B \gamma \qquad B \to \varepsilon \qquad B \to B\beta$$

где B — это новый нетерминал, а ε — пустая цепочка.

Замечание. Символы : | ( ) [ ] { } # ; . присутствуют на клавиатуре и ими удобно пользоваться при описании грамматики. Однако эти же символы часто используются как терминальные символы языка, и тогда встаёт вопрос, как отличить терминальный символ от метасимвола? Здесь этот вопрос решён *визуально*, за счёт использования цвета. Если система программирования не допускает различения символов по цвету, то приходится придумывать дополнительные средства, например, заключать терминальные символы в кавычки ” ”.

Нетрудно убедиться, что все эти расширения (а также некоторые другие, применяемые на практике) не выводят за пределы контекстно-свободных грамматик типа $\mathfrak{T}_2$.

1.1.9. Альтернативные средства задания синтаксиса (7/9)

Широко используемым на практике средством задания синтаксиса являются *языки разметки* — это языки для описания структуры и представления текста.

- *SGML* — это стандарт, определяющий правила построения языков разметки. Он использует *DTD* (Document Type Definition), чтобы задать множество тегов (нетерминалов) и правила их вложенности. Так как SGML допускает множество сокращений (например, незакрытые теги), написание парсеров для него затруднительно.
- *HTML* — это язык разметки, изначально написанный на SGML, его множество нетерминалов и правила фиксированы стандартом.
- *XML* — это доработанный SGML со строгим синтаксисом, используемый для хранения и передачи структурированных данных. Благодаря строгому синтаксису парсеры для XML проще и работают быстрее. Для описания структуры XML-документов используется *XML Schema*, который позволяет задать грамматику.

Замечание. SGML и XML являются инструментами для определения грамматик, DTD и XML Scheme — грамматиками, а HTML — языком, порожденным грамматикой.

1.1.9. Альтернативные средства задания синтаксиса (8/9)

Пример. Сопоставление с КСГ $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$:

1. Схема документа (DTD, XML Schema) — Грамматика G.
2. Множество тегов (Elements) — Алфавит нетерминалов M.
3. Текстовое содержимое (PCDATA) — Алфавит терминалов Σ.
4. Правила вложенности тегов — Правила вывода Π.
5. Корневой элемент (Root) — Аксиома грамматики S.

Например, язык целых десятичных чисел со знаком XML-документ с внутренней схемой (Internal DTD):

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE signedInteger [
  <!ELEMENT signedInteger (sign?, unsignedInteger)>
  <!ELEMENT sign (#PCDATA)>
  <!ELEMENT unsignedInteger (zero | (nonzero, digit*))>
  <!ELEMENT zero (#PCDATA)> <!ELEMENT nonzero (#PCDATA)>
  <!ELEMENT digit (#PCDATA)> ]>
```

TODO Замечания ФАН 21 января. Вклеить в структуру, доработать примеры.

1.1.A. Онтология формальных языков, грамматик и автоматов

1.1.Б. Темы для самостоятельной работы студентов

1. СНИРС => Грамматические описания нетекстовых языков

   В некоторых случаях формализм порождающих грамматик удаётся применить, когда знаками языка являются не символы в тексте, а другие объекты. Например, фигуры и линии на диаграмме графа, жесты пользователей смартфонов, ноты в партитуре и тому подобное. Требуется подготовить небольшой, но представительный набор убедительных примеров.

2. СНИРС => История развития и перспективы теории формальных грамматик

   Требуется подготовить познавательный научно-популярный доклад с фотографиями ученых, датами, событиями, примерами применения и прогнозами на будущее.

3. СНИРС => Классификация формальных языков на основе ...

   Классификация языков на основе формальных грамматик уже развита Хомским и другими учёными. Предлагается попробовать построить классификацию на основе использования иных формализмов. Например, провести классификацию языков, которые могут быть заданы с помощью графов и орграфов различных видов.

1.2. Регулярные выражения (1/3)

Порождающие грамматики — не единственное средство определения языков. Языки являются множествами, поэтому над ними определены все операции над множествами (объединение, пересечение, разность и симметрическая разность). Кроме того, для элементов языков — слов — определена операция конкатенации, что позволяет определить для языков и другие операции. Таким образом, существует возможность определить язык *алгебраически*, как выражение, построенное из более простых элементов.
Основополагающий вклад в развитие теории регулярных выражений внес Клини.[5]

Удивительный факт: далёкие как от исчислений (формальные грамматики), так и от алгебраических структур (регулярные выражения) формализмы, в частности, ориентированные *графы*, при использовании для определения языков приводят к одним и тем же результатам. Обсуждение этого наблюдения начинается в этом разделе, а продолжается и заканчивается в разделе 2.2.

[5] Стивен Коул Клини (1909–1994)

1.2. Регулярные выражения (2/3)

№	Название параграфа	Ключевые термины и обозначения
1.2.1.	Регулярные операции	регулярные операции, объединение, альтернация, сложение, умножение, конкатенация, сцепление, итерация, замыкание Клини
1.2.2.	Алгебра регулярных выражений	элементарный язык, регулярный язык, регулярное выражение, алгебра регулярных множеств $\mathcal{R}$, алгебра регулярных выражений $\mathfrak{R}$, эквивалентность регулярных выражений
1.2.3.	Языковые графы	граф языка, источник, полный источник, эквивалентные источники, двухполюсник, начальные и заключительные узлы, значение пути, размер источника, замкнутое множество узлов
1.2.4.	Детерминированные и недетерминированные источники	теорема детерминизации

1.2. Регулярные выражения (3/3)

№	Название параграфа	Ключевые термины и обозначения
1.2.5.	Расширение языка регулярных выражений	операции повторения, итерация Цейтина, символы шаблона, символьные классы
1.2.6.	Программирование в регулярных выражениях	POSIX, PERL, SNOBOL4, REFAL
1.2.7.	Уравнения с регулярными коэффициентами	
1.2.8.	Характеристика класса регулярных языков	язык Дика

1.2.1. Регулярные операции (1/3)

Пусть $L_1, L_2 \subset A^*$. Рассмотрим следующие три операции над языками L_1, L_2.

Объединение (*альтернация*, *сложение*), обозначается знаками $\cup$, $\mid$, $+$:

$$L_1 \cup L_2 = L_1 \mid L_2 = L_1 + L_2 \stackrel{\text{Def}}{=} \{\alpha \in A^* \mid \alpha \in L_1 \vee \alpha \in L_2\}.$$

Замечание. В определении указано, что языки определены над одним алфавитом, $L_1, L_2 \subset A^*$, но это сделано только ради удобства обозначений и не ограничивает общности. Можно считать, что $L_1 \subset A_1^*$, $L_2 \subset A_2^*$, и тогда $L_1 \cup L_2 \subset (A_1 \cup A_2)^*$.

Умножение (*конкатенация*, *сцепление*), обозначается знаком $\cdot$ или опускается:

$$L_1 L_2 = L_1 \cdot L_2 \stackrel{\text{Def}}{=} \{\alpha \in A^* \mid \alpha = \alpha_1\alpha_2 \;\&\; \alpha_1 \in L_1 \;\&\; \alpha_2 \in L_2\}.$$

Замечание. Можно сцеплять язык сам с собой: $L, LL, LLL, \ldots$, что естественно обозначить с помощью показателя степени, то есть $L^1 := L, L^2 := LL, L^3 := LLL$ и так далее. При этом по определению считают, что $L^0 := \{\varepsilon\}$.

Итерация (*замыкание Клини*), обозначается знаком $*$:

$$L_1^* \stackrel{\text{Def}}{=} \varepsilon \cup L_1 \cup L_1 L_1 \cup \cdots = \bigcup_{i=0}^{\infty} L_1^i.$$

1.2.1. Регулярные операции (2/3)

Введённые операции называются *регулярными*. Эти операции имеют особое значение для теории языков и автоматов.

Поскольку результатом регулярной операции над языками является язык, из этих операций с помощью суперпозиции можно строить сколь угодно сложные выражения, которые называются *регулярными*.

Для регулярных операций обычно устанавливают следующий приоритет: $*$, $\cdot$, $|$, что позволяет опускать лишние скобки при записи выражений.

Замечание. Операция альтернации является привычной операцией объединения из наивной теории множеств, но операции конкатенации и итерации не являются и не выражаются операциями наивной теории множеств.

Примеры. Пусть язык $L_1 = \{a, b, c, \ldots, x, y, z\}$ — это латинский алфавит, то есть множество однобуквенных слов, а язык $L_2 = \{1, 2, 3, \ldots\}$ — это натуральные числа в десятичной записи. Тогда $L_1 L_2$ — это множество идентификаторов, которые начинаются с буквы и продолжаются цифрами, L_1^* — это множество всех слов, составленных из букв латинского алфавита, включая пустое слово ε, а $L_2^* = L_2$.

1.2.1. Регулярные операции (3/3)

Теорема. *Пусть $L_1, L_2, L_3 \subset A^*$. Тогда регулярные операции обладают следующими свойствами.*

1. *Альтернация коммутативна:*
   $L_1 \mid L_2 = L_2 \mid L_1$;
2. *Альтернация ассоциативна:*
   $L_1 \mid (L_2 \mid L_3) = (L_1 \mid L_2) \mid L_3$;
3. *Конкатенация дистрибутивна относительно альтернации слева:*
   $L_1(L_2 \mid L_3) = L_1L_2 \mid L_1L_3$;
4. *Конкатенация дистрибутивна относительно альтернации справа:*
   $(L_1 \mid L_2)L_3 = L_1L_3 \mid L_2L_3$;
5. *Конкатенация ассоциативна:*
   $(L_1L_2)L_3 = L_1(L_2L_3)$;
6. *Итерация идемпотентна:*
   $(L_1^*)^* = L_1^*$;
7. *Альтернация идемпотентна:*
   $L_1 \mid L_1 = L_1$.

Доказательство. Очевидно из определений.

1.2.2. Алгебра регулярных выражений (1/8)

Пусть задан конечный алфавит $A = \{a_1, \ldots, a_k\}$. Язык $L_{a_i} := \{a_i\}$, состоящий из одного однобуквенного слова, называется *элементарным* и обозначается a_i. Пустое слово ε тоже считается элементарным языком.

Отступление. Используемые обозначения многозначны. Поясним это утверждение примером. Пусть $A = \{a, b, c, \ldots, x, y, z\}$. Тогда символ a именует одновременно три различных объекта:

1. a — это буква латинского алфавита, $a \in A$;
2. a — это однобуквенное слово, $a \in A^*$;
3. a — это элементарный язык, $a \subset A^*$.

Аналогично символ ε именует одновременно три различных объекта:

1. ε — это символ *греческого* алфавита, $\varepsilon \notin A$;
2. ε — это обозначение пустого слова, $\varepsilon \in A^*$;
3. ε — это элементарный язык, $\varepsilon \subset A^*$.

Такие обозначения удобны, потому что лаконичны, и не создают путаницы, потому что тип объекта всегда восстанавливается из контекста. В частности, аргументы регулярных операций — это всегда языки, но не слова и не буквы.

1.2.2. Алгебра регулярных выражений (2/8)

Язык (то есть множество слов в алфавите A) называется *регулярным*, если он может быть построен из элементарных языков (букв алфавита A и пустого слова ε) с помощью суперпозиции регулярных операций. Семейство регулярных языков является подмножеством множества всех языков над алфавитом A. Множество регулярных языков обозначим $\mathcal{R}$: $\mathcal{R} \subset 2^{A^*}$. Алгебра $\langle \mathcal{R};\ |\ ,\ \cdot\ ,\ {}^*\rangle$ называется *алгеброй регулярных множеств*. Эта алгебра конечнопорождённая; ее образующими являются элементарные языки.

Каждый регулярный язык может быть описан формулой, которая называется *регулярным выражением*. Операндами регулярных выражений являются обозначения элементарных языков (то есть буквы алфавита A), а операциями — регулярные операции $|\ \cdot\ {}^*$. Множество регулярных выражений обозначим $\mathfrak{R}$. Алгебра $\langle \mathfrak{R};\ |\ ,\ \cdot\ ,\ *\rangle$ называется *алгеброй регулярных выражений*. Эта алгебра также конечнопорождённая; ее образующими являются буквы алфавита A и обозначение ε.

Каждое регулярное выражение порождает единственный регулярный язык. Один и тот же язык может быть порождён бесконечным числом различных регулярных выражений.

Пример. Различные регулярные выражения $\varepsilon,\ \varepsilon\varepsilon,\ \varepsilon\,|\,\varepsilon,\ \varepsilon^*,\ \ldots$ порождают один и тот же регулярный язык $\{\varepsilon\}$.

1.2.2. Алгебра регулярных выражений (3/8)

Соответствие регулярных выражений и регулярных языков является *гомоморфизмом*, который уважает все регулярные операции. Говорят, что регулярные выражения *эквивалентны*, если они задают один и тот же язык. Эквивалентность регулярных выражений обозначают, как обычно, знаком $=$. Таким образом, по теореме о гомоморфизме алгебра классов эквивалентности регулярных выражений *изоморфна* алгебре регулярных множеств. На этом обстоятельстве основана общепринятая в математике практика: манипулировать конечными дескрипторами (в данном случае регулярными выражениями) бесконечных объектов (в данном случае, регулярных языков).

Примеры.

1. Обозначение A^* — это правильное регулярное выражение: $A^* = (a_1 \mid \ldots \mid a_k)^*$.
2. Любое слово регулярно: оно построено из букв конкатенацией.
3. Любой конечный язык регулярен: $L = \{\alpha_1, \ldots, \alpha_k\}, \quad L = \alpha_1 \mid \ldots \mid \alpha_k$.
4. Язык $(aba)^*$ регулярен.

В предыдущем параграфе установлены семь важных эквивалентностей, с помощью которых можно установить многие другие полезные свойства операций алгебры регулярных выражений

1.2.2. Алгебра регулярных выражений (4/8)

Пустое множество $\varnothing \subset A^*$ является языком, этот язык считается регулярным по определению, $\varnothing \in R, \varnothing \neq \varepsilon$. Напомним, что $A^* = \varepsilon + A + AA + \cdots = \bigcup_{i=0}^{\infty} A^i$ и введём дополнительное обозначение: $A^+ \stackrel{\text{Def}}{=} A + AA + \cdots = \bigcup_{i=1}^{\infty} A^i$.

Пользуясь свойствами регулярных операций и определениями, нетрудно установить, что в алгебре регулярных выражений выполняются следующие соотношения:

8. $A + \varnothing = A$;
9. $A\varnothing = \varnothing A = \varnothing$;
10. $A^* + A = A^*$;
11. $\varnothing^* = \varepsilon$;
12. $\varepsilon^* = \varepsilon$;
13. $A^* + \varepsilon = A^*$;
14. $A\varepsilon = \varepsilon A = A$;
15. $A^+ = AA^*$;
16. $A^+ + A = A^+$;
17. $\varnothing^+ = \varnothing$;
18. $\varepsilon^+ = \varepsilon$;
19. $A^+ + \varepsilon = A^*$.

Замечание. Алгебра регулярных выражений является моноидом относительно конкатенации и языка ε, является коммутативным моноидом относительно альтернации и пустого языка $\varnothing$, и является некоммутативным полукольцом с единицей относительно указанных операций и элементов.

1.2.2. Алгебра регулярных выражений (5/8)

Пользуясь указанными простыми свойствами, в алгебре регулярных выражений можно доказывать различные равенства.

Пример. Доказать, что $(b + aa^*b) + (b + aa^*b)(a + ba^*b)^*(a + ba^*b) = a^*b(a + ba^*b)^*$.

Доказательство: $(b + aa^*b) + (b + aa^*b)(a + ba^*b)^*(a + ba^*b) =$

$= (b + aa^*b)(\varepsilon + (a + ba^*b)^*(a + ba^*b)) = (b + aa^*b)(\varepsilon + (a + ba^*b)^+) =$

$= (b + aa^*b)(a + ba^*b)^* = (\varepsilon + aa^*)b(a + ba^*b)^* = (\varepsilon + a^+)b(a + ba^*b)^* = a^*b(a + ba^*b)^*$.

Имеют место некоторые менее очевидные свойства.

Лемма. $\forall\, L \subset A^*\ (\forall\, \alpha \in L^*, \alpha \neq \varepsilon\ (\exists\, \alpha_1, \ldots, \alpha_k \in L\ (\alpha = \alpha_1 \ldots \alpha_k)))$.

Другими словами, если слово принадлежит итерации языка, то это слово всегда можно разложить в конкатенацию слов, принадлежащих языку и обратно.

Теорема. $(A + B)^* = (A^* + B^*)^* = (A^*B^*)^*$.

Доказательство. Обозначим $L_1 := (A + B)^*, L_2 := (A^* + B^*)^*, L_3 = (A^*B^*)^*$.

[$L_1 \subset L_2$] $A + B \subset A^* + B^* \Longrightarrow (A + B)^* \subset (A^* + B^*)^*$.

[$L_2 \subset L_1$] Если $\alpha \in (A^* + B^*)^*$, то по лемме $\exists\, \alpha_1, \ldots, \alpha_k \in A^* + B^*\ (\alpha = \alpha_1 \ldots \alpha_k)$. Но по той же лемме $\forall\, \alpha_i\ \left(\exists\, \alpha_{i_1}, \ldots, \alpha_{i_j} \in A + B\ \left(\alpha_i = \alpha_{i_1} \ldots \alpha_{i_j}\right)\right)$. Значит $\alpha \in (A + B)^*$.

1.2.2. Алгебра регулярных выражений (6/8)

Доказательство. (Окончание)

[$L_1 \subset L_3$] Если $\alpha \in (A+B)^*$, то по лемме $\exists\, \alpha_1, \ldots, \alpha_k \in A+B \ \ (\alpha = \alpha_1 \ldots \alpha_k)$. Но если $\alpha_i \in A+B$, то $\alpha_i \in A^*B^*$ и значит $\alpha \in (A^*B^*)^*$.

[$L_3 \subset L_1$] Если $\alpha \in (A^*B^*)^*$, то по лемме $\exists\, \alpha_1, \ldots, \alpha_k \in A^*B^* \ \ (\alpha = \alpha_1 \ldots \alpha_k)$. Далее, если $\alpha_i \in A^*B^*$, то $\exists\, \beta_i \in A^{k_1}, \gamma_i \in B^{k_2} \ \ (\alpha_i = \beta_i\gamma_i)$. Но слова β_i и γ_i также можно разложить по лемме: $\exists\, \beta_{i_1}, \ldots, \beta_{i_{k_1}} \in A \ \left(\beta_i = \beta_{i_1} \ldots \beta_{i_{k_1}}\right)$, $\exists\, \gamma_{i_1}, \ldots, \gamma_{i_{k_2}} \in B \ \left(\gamma_i = \gamma_{i_1} \ldots \gamma_{i_{k_2}}\right)$, и значит слово α_i разложено в конкатенацию слов, каждое из которых принадлежит $A+B$, откуда $\alpha \in (A+B)^*$. □

Следствие. [1] $(AB)^*A = A(BA)^*$.

Доказательство. Раскрасим буквы в доказываемом равенстве, чтобы было видно, какому языку принадлежит какое слово: $(AB)^*A = A(BA)^*$. Если $\alpha \in (AB)^*A$, то $\alpha = \alpha_1\alpha_2$, причём $\alpha_1 \in (AB)^* \ \&\ \alpha_2 \in A$. Далее, если $\alpha_1 \in (AB)^*$, то по лемме $\exists\, \alpha_{11}, \ldots, \alpha_{1k} \in AB \ \ (\alpha_1 = \alpha_{1_1} \ldots \alpha_{1_k})$, причём $\forall\, i \ \ (\alpha_{1_i} = \gamma_i\beta_i \ \&\ \gamma_i \in A \ \&\ \beta_i \in B)$. Таким образом, $\alpha = \gamma_1\beta_1 \ldots \gamma_k\beta_k\alpha_2 \ \&\ \forall\gamma, \alpha \in A, \forall\beta \in B$. По ассоциативности конкатенации: $\alpha = \gamma_1(\beta_1\gamma_2 \ldots \beta_{k-1}\gamma_k\beta_k\alpha_2)$. Значит $\alpha \in A(BA)^*$. Мы доказали, что $(AB)^*A \subset A(BA)^*$. Вложение $A(BA)^* \subset (AB)^*A$ доказывается аналогично.

1.2.2. Алгебра регулярных выражений (7/8)

Следствие. [2] $(A^*B)^* = (A+B)^*B + \varepsilon$.

Доказательство. Пусть $\alpha = \varepsilon$. Тогда $\alpha \in (A^*B)^* \ \& \ \alpha \in (A+B)^*B + \varepsilon$ и можно сразу предположить, что $\alpha \neq \varepsilon$.

[$(A^*B)^* \subset (A+B)^*B + \varepsilon$] Если $\alpha \in (A^*B)^*$, то по лемме $\exists\, \alpha_1, \ldots, \alpha_k \in A^*B \ \ (\alpha = \alpha_1 \ldots \alpha_k)$, причём $\alpha_i = \gamma_i\beta_i \ \& \ \gamma_i \in A^{j_i} \ \& \ \beta_i \in B$. По лемме $\exists\, \gamma_{i_1}, \ldots, \gamma_{i_{j_i}} \in A \ \left(\gamma_i = \gamma_{i_1} \ldots \gamma_{i_{j_i}}\right)$. В итоге $\alpha = \gamma_{1_1} \ldots \gamma_{1_{j_1}}\beta_1 \ldots \gamma_{k_1} \ldots \gamma_{k_{j_k}}\beta_k$, где $\gamma_s \in A, \beta_r \in B$. Расставим скобки и получим $\alpha = (\gamma_{1_1} \ldots \gamma_{1_{j_1}}\beta_1 \ldots \gamma_{k_1} \ldots \gamma_{k_{j_k}})\beta_k$, $\alpha \in (A+B)^*B$ или, расширяя множество, $\alpha \in (A+B)^*B + \varepsilon$.

[$(A+B)^*B + \varepsilon \subset (A^*B)^*$] . Пусть $\alpha \in (A+B)^*B + \varepsilon$ и $\alpha \neq \varepsilon$, тогда $\alpha \in (A+B)^*B$. По определению конкатенации $\alpha = \gamma\beta$, где $\gamma \in (A+B)^j, \beta \in B$. Слово γ раскладываем по лемме: $\exists\, \gamma_1, \ldots, \gamma_j \in (A+B) \ \ (\gamma = \gamma_1 \ldots \gamma_j)$, $\alpha = \gamma_1 \ldots \gamma_j\beta$. Получили $\alpha \in (A^*B)^*$. □

Следствие. [3] $(AB^*)^* = A(A+B)^* + \varepsilon$.

Доказательство. Упражнение. □

Доказанные эквивалентности позволяют упрощать регулярные выражения.

1.2.2. Алгебра регулярных выражений (8/8)

Пример. Доказать, что $(ABA(BA)^*B + AB)(A(BA)^*B)^* = A(BA)^*B$.

Для начала проведем несколько преобразований

$(ABA(BA)^*B + AB)(A(BA)^*B)^* \overset{14}{=} (ABA(BA)^*B + A\varepsilon B)(A(BA)^*B)^* \overset{3,4}{=}$
$\overset{3,4}{=} (A(BA(BA)^* + \varepsilon)B)(A(BA)^*B)^* \overset{15}{=} (A((BA)^+ + \varepsilon)B)(A(BA)^*B)^* \overset{19}{=}$
$\overset{19}{=} (A(BA)^*B)(A(BA)^*B)^* \overset{15}{=} (A(BA)^*B)^+$

Теперь осталось доказать, что $(A(BA)^*B)^+ = A(BA)^*B$

[$A(BA)^*B \subset (A(BA)^*B)^+$] По определению введенной операции L^+.

[$A(BA)^*B)^+ \subset A(BA)^*B$] Заметим, что если пустое слово ε не содержится ни в языке A, ни в языке B, то оно не содержится и в выражениях. Поэтому можно доказывать не $(A(BA)^*B)^+ \subset A(BA)^*B$, а $(A(BA)^*B)^* \subset A(BA)^*B + \varepsilon$. Пусть $\alpha \in (A(BA)^*B)^*$. По лемме $\exists\, \alpha_1, \ldots, \alpha_k \in A(BA)^*B \;\; (\alpha = \alpha_1 \ldots \alpha_k)$, причём $\alpha_i = \gamma_i\theta_i\beta_i$, $\gamma_i \in A$, $\theta_i \in (BA)^{l_i}$, $\beta_i \in B$. Далее $\exists\, \theta_{i_1}, \ldots, \theta_{i_{l_i}} \in (BA) \;\; \left(\theta_i = \theta_{i_1} \ldots \theta_{i_{l_i}}\right)$. Наконец, $\theta_{i_j} = \lambda_{ij}\mu_{ij}$, где $\lambda_{ij} \in B$, $\mu_{ij} \in A$. Имеем $\alpha = \gamma_1\lambda_{11}\mu_{11} \ldots \lambda_{1l_1}\mu_{1l_1}\beta_1\gamma_2 \ldots \beta_{k-1}\gamma_k\lambda_{k1}\mu_{k1} \ldots \lambda_{kl_k}\mu_{kl_k}\beta_k$, где $\gamma, \mu \in A$, $\lambda, \beta \in B$. Получили $\alpha \in A(BA)^*B$.

Особенность этого примера: мы построили такой язык L, что $L^+ = L$.

1.2.3. Языковые графы (1/5)

Для описания языка можно использовать граф. *Граф языка над алфавитом* A (или *источник*) — это ориентированный (мульти)граф, на дугах которого написаны буквы (или пустое слово ε) и в котором выделены *начальные* и *заключительные* узлы.

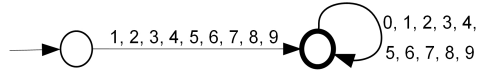
Далее используются следующие обозначения. Если s и t — узлы источника H, то $\langle s,t\rangle$ обозначает путь из s в t, а $\alpha = V(\langle s,t\rangle)$ означает слово, полученное конкатенацией букв, написанных на дугах пути $\langle s,t\rangle$. Такое слово $\alpha \in A^*$ будем называть *значением* пути. Очевидно, что $|V(\langle s,t\rangle)| \leqslant |\langle s,t\rangle|$ и $|V(\langle s,t\rangle)| = |\langle s,t\rangle|$ тогда и только тогда, когда $\langle s,t\rangle$ не содержит дуг, нагруженных ε.

Говорят, что источник *представляет* некоторый язык, а именно, язык, состоящий из слов, которые являются значениями всех путей, ведущих из начальных узлов в заключительные. Очевидно, что всякий источник представляет некоторый язык (но не всякий язык может быть представлен графом).

Замечание. Множества начальных и заключительных узлов могут пересекаться и даже совпадать. На диаграммах начальные узлы помечают входной стрелкой «из ниоткуда», а заключительные узлы выделяют жирной границей. Мультидугу обычно изображают одной линией и указывают сразу множество букв через запятую.

1.2.3. Языковые графы (2/5)

Пример. Справа представлена диаграмма источника десятичных записей натуральных чисел.



Наряду с путями, которые ведут из начальных узлов в заключительные узлы и представляют «правильные» слова языка, в источнике могут быть пути, которые не ведут из начальных узлов в заключительные, и тем самым представляют «неправильные» слова, то есть не принадлежащие языку. Источник, в котором существуют пути для *всех* слов в алфавите, естественно назвать *полным*. Если два источника представляют один язык, то их называют *эквивалентными*.

Теорема. *Для любого источника существует эквивалентный полный источник.*

Доказательство. Если в графе из любого узла исходят дуги, помеченные всеми буквами алфавита, то источник, очевидно, полный. Любой источник можно превратить в полный следующим образом. Нужно добавить ещё один узел «поглотитель» и назначить его не заключительным. Для каждого из старых узлов, в которых не все буквы алфавита использованы на исходящих дугах, нужно провести новые дуги в узел поглотитель и пометить их недостающими буквами. В самом этом узле поглотителе нужно добавить петли для всех букв. Источник станет полным, а представляемый язык не изменится.

1.2.3. Языковые графы (3/5)

Размером (*величиной*) источника H называется количество узлов в нем (обозначение $|H|$).

Подмножество $X \subset H$ узлов источника называется *замкнутым*, если

$$x \in X \;\&\; x \xrightarrow{\varepsilon} y \Longrightarrow y \in X$$

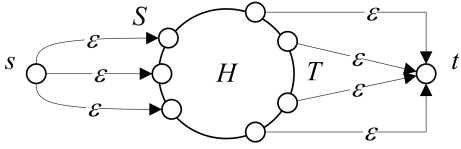
(то есть в замкнутое подмножество входят все узлы, в которые можно попасть «даром» — по слову ε).

Замечание. Если в источнике нет пустых дуг, то все подмножества узлов замкнутые.

Языковой граф (источник) называется *двухполюсным* или *двухполюсником*, если он имеет один начальный и один заключительный узел. Обозначим их s и t соответственно.

Теорема. *Для любого источника существует эквивалентный ему двухполюсник.*

Доказательство. Пусть H — заданный источник, S — множество начальных узлов, T — множество заключительных узлов. Построим эквивалентный двухполюсный источник так, как показано на рисунке. □



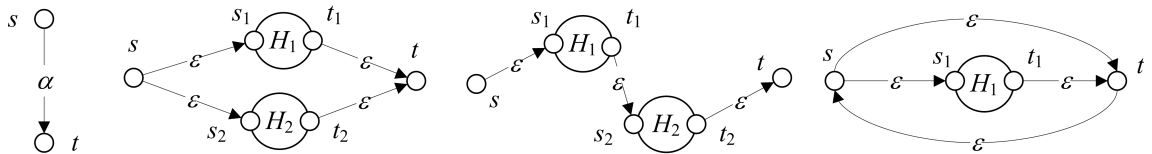
1.2.3. Языковые графы (4/5)

Введенное понятие двухполюсника исчерпывающим образом характеризует множество языков, описываемых регулярными выражениями.

Теорема. *Для любого регулярного выражения E существует двухполюсник H, представляющий E.*

Доказательство. Построим H индукцией по структуре E.

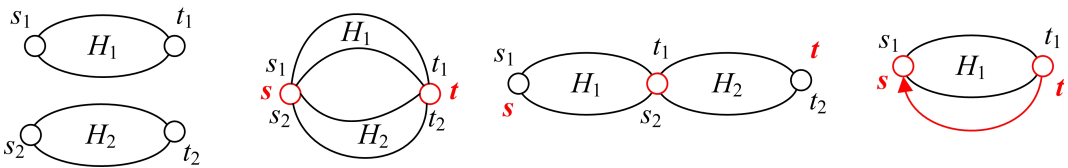
Буквы: для каждой буквы a, входящей в выражение E, построим источник, как показано на рисунке *а*. Альтернация: построим параллельное соединение источников, как показано на рисунке *б*. Конкатенация: построим последовательное соединение источников, как показано на рисунке *в*. Итерация: построим зацикливание источников, как показано на рисунке *г*. □



а *б* *в* *г*

1.2.3. Языковые графы (5/5)

Замечание. Пустые дуги нужны для наглядности построения и простоты доказательства. При проведении реального построения графа по регулярному выражению, как правило, можно обойтись склейкой узлов, как показано на рисунке. Склеенные узлы и добавленные дуги выделены красным.



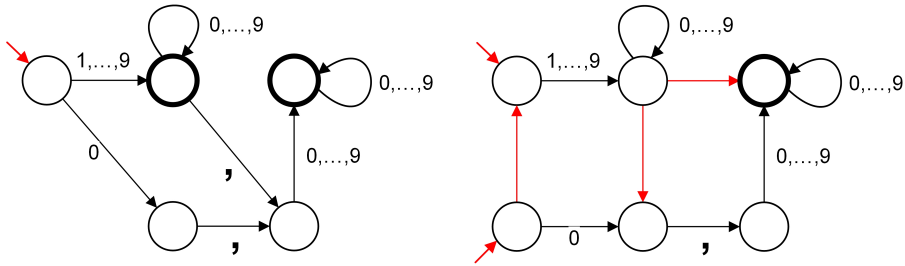
Пример. На рисунке показан двухполюсник, представляющий десятичные числа со знаком.

TODO рисунок

1.2.4. Детерминированные и недетерминированные источники (1/5)

Источник (граф языка) называется *детерминированным*, если он содержит одну начальную вершину, не содержит пустых дуг и все дуги, исходящие из одного узла, помечены разными буквами. В противном случае, то есть если нарушено хотя бы одно из трёх условий, источник называется *недетерминированным*.

Пример. На рисунке слева показан детерминированный источник, представляющий общепринятый язык записи вещественных чисел с фиксированной запятой, а справа показан эквивалентный недетереминированный источник, в котором нарушены все три условия детерминированности.



1.2.4. Детерминированные и недетерминированные источники (2/5)

Теорема. [Детерминизации] *Для любого источника H с n узлами существует эквивалентный ему детерминированный источник H', имеющий не более 2^n узлов.*

Доказательство. Пусть $\bar{H}$ — множество узлов исходного источника H, причем $S \subset \bar{H}$ — множество начальных узлов, а $T \subset \bar{H}$ — множество заключительных узлов. Построим источник H'. Рассмотрим все замкнутые подмножества $\bar{H}$ (включая пустое). Обозначим их $\bar{H}' := 2^{[\bar{H}]}$ и будем считать их узлами источника H'. Таких узлов не более 2^n, $|H'| \leqslant 2^n$. Положим $S' \in \bar{H}'$ — наименьшее замкнутое подмножество $\bar{H}$, содержащее все начальные узлы H, $S' := [S]$. Все элементы $\bar{H}'$, содержащие хотя бы один заключительный узел $\bar{H}$, объявим заключительными и обозначим $T' := \left\{ q' \in \bar{H}' \mid \exists\, q \in \bar{H} \;\; (q \in q' \;\&\; q \in T) \right\}$. Построим теперь дуги источника H'. Рассмотрим $q'_i \in \bar{H}'$ ($q'_i \subset \bar{H}$) и символ a. Положим $q'_j := \left\{ q_j \in \bar{H} \mid \exists\, q_i \in q'_i \;\; (\exists \langle q_i, q_j \rangle \;\; (V(\langle q_i, q_j \rangle) = a)) \right\}$ — множество узлов, в которые ведут пути из q'_i со значением a. Если q'_j не замкнуто, то положим $q'_j := [q'_j]$. Если нет путей со значением a, исходящих из узлов множества q'_i, то положим $q'_j := \varnothing$, $\varnothing \in \bar{H}'$. Множество q'_j является узлом источника H', $q'_j \in \bar{H}'$. Проведем дугу $q'_i \xrightarrow{a} q'_j$. Таким образом, каждому q'_i и каждому a соответствует ровно одна дуга, так что источник H' детерминированный по построению.

1.2.4. Детерминированные и недетерминированные источники (3/5)

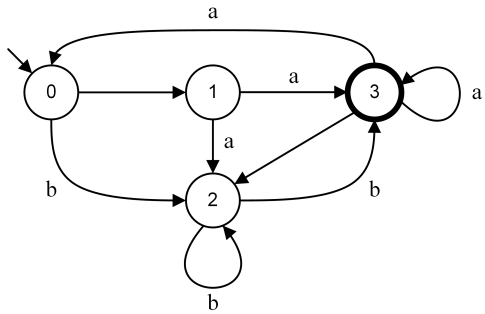
Доказательство. (Продолжение) Осталось показать, что $H \sim H'$. Для этого сначала покажем, что $\exists\, \langle S', q'_j \rangle \ \left(V(\langle S', q'_j \rangle) = \alpha\right) \iff \forall\, q_j \in q'_j \ (\exists\, q_i \in S \ (V(\langle q_i, q_j \rangle) = \alpha))$. Индукция по длине α. База: если $\alpha = \varepsilon$, то по построению $q'_j = S'$, так как в H' нет пустых дуг. Если $\alpha = a$, то, база индукции верна по построению H'. Индукционный переход: пусть доказываемое утверждение верно для $|\alpha| \leqslant k$. Рассмотрим αa.

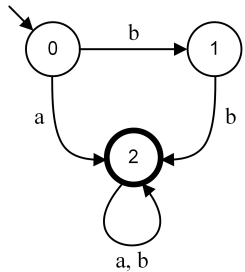
Пусть $\exists\, \langle S', q'_j \rangle \ \left(V(\langle S', q'_j \rangle) = \alpha a\right)$. Обозначим q'_ℓ — предпоследний узел на этом пути, так что $V(\langle S', q'_\ell \rangle) = \alpha$ и $q'_\ell \xrightarrow{a} q'_j$. По индукционному предположению в источнике H $\forall\, q_\ell \in q'_\ell \ (\exists\, q_i \in S \ (V(\langle q_i, q_\ell \rangle) = \alpha))$. По построению источника H', если $V(\langle q'_\ell, q'_j \rangle) = a$, то $\forall\, q_j \in q'_j \ (\exists\, q_\ell \in q'_\ell \ (V(\langle q_\ell, q_j \rangle) = a))$ и, таким образом, в источнике H имеем путь $\langle q_i, \ldots, q_\ell, q_j \rangle$, причём $V(\langle q_i, q_\ell \rangle) = \alpha$ и $V(\langle q_\ell, q_j \rangle) = a$. Пусть теперь $\forall\, q_j \in q'_j \ (\exists\, q_i \in S \ (V(\langle q_i, q_j \rangle) = \alpha a))$. Рассмотрим множество q'_ℓ узлов, в которые можно попасть из q_i по путям со значением α. Аналогичным образом имеем путь $\langle S', \ldots, q'_\ell, q'_j \rangle$, причём $V(\langle S', q'_\ell \rangle) = \alpha$ и $V(\langle q'_\ell, q'_j \rangle) = a$. Из доказанного свойства и определения заключительных узлов следует, что в источнике H' путь α из начального узла S' в какой-либо заключительный узел существует тогда и только тогда, когда в источнике H существует путь α из некоторого начального узла в какой-либо заключительный узел.

1.2.4. Детерминированные и недетерминированные источники (4/5)

Замечание. Если интерпретировать источник как алгоритм порождения слов языка, то теорема детерминизации показывает, что в некоторых случаях по недетерминированному алгоритму можно построить функционально эквивалентный детерминированный.

Пример. В доказательстве теоремы используется построение, при котором количество узлов экспоненциально возрастает. Однако это не означает, что детерминированный источник всегда имеет больше узлов, чем недетерминированный. На рисунке приведён соответствующий пример.





1.2.4. Детерминированные и недетерминированные источники (5/5)

Замечание. Детерминированный источник — это граф автомата без выходов (см. п. 2.2.1). Недетерминированный источник — это недетерминированный конечный автомат, то есть автомат, в котором нарушено условие автоматной детерминированности (см. п. 2.1.4). Если интерпретировать источник как алгоритм порождения слов языка, то недетерминированный источник на каждом шаге «выбирает» одно из ребер, по которому можно перейти. Теорема детерминизации показывает, что с точки зрения порождения языков, возможности этих двух классов автоматов совпадают, хотя эти классы автоматов различны с точки зрения программной реализации.

TODO Вопрос о программной реализации детерминированных и детерминированных источников обозначен, но не закончен, это открытый TODO. Нужно либо подготовить хороший атом с примером, либо перенести этот слайд в следующий семестр.

Замечание. Свойства источника, как графа, связаны со свойствами языка, как множества слов. Например, источник конечного языка всегда можно представить в виде леса корневых деревьев, то есть бесконтурного орграфа. Источник бесконечного языка, напротив, обязательно содержит хотя бы один контур.

1.2.5. Расширения языка регулярных выражений (1/4)

Трёх введённых регулярных операций (альтернация, конкатенация и замыкание Клини) *достаточно* для описания класса регулярных языков. Обнако набор регулярных операций может быть *расширен* многими способами, например, введением дополнительных операций, которые не меняют выразительной мощности регулярных выражений, но делают их более *удобными*.

Рассмотрим операцию дополнения:

$$\overline{L} \stackrel{\text{Def}}{=} \{\alpha \in A^* \mid \alpha \notin L\}.$$

Теорема. *Если L — регулярный язык, то его дополнение $\overline{L}$ также регулярно.*

Доказательство. Если в полном источнике языка L заключительные узлы объявить незаключительными и наоборот, то полученный граф будет источником языка $\overline{L}$. □

Имея регулярную операцию дополнения, легко вывести, что операция пересечения также регулярна:

$$L_1 \cap L_2 = \overline{\overline{L}_1 \mid \overline{L}_2}.$$

Замечание. Алгебра $\langle \mathcal{R}; \cup, \cap, \overline{\ } \rangle$ является булевой алгеброй.

1.2.5. Расширения языка регулярных выражений (2/4)

Помимо введения дополнительных операций над регулярными языками, можно вводить сокращённые обозначения для регулярных выражений.

Пример. *Символы шаблона*, которые используются в операционных системах для задания маски имени файла, имеют следующий смысл: знак «?», который обозначает один произвольный символ, это сокращение для регулярного выражения $a_1|\ldots|a_n$, а знак «*», соответственно, сокращение для $(a_1|\ldots|a_n)^*$, где $a_1, \ldots, a_n$ — все символы, которые разрешается использовать в именах файлов.

Замечание. Важно понимать, что конкретное значение символов шаблона зависит от контекста, потому что в разных операционных системах и даже в разных версиях одной системы наборы допустимых символов различны.

Пусть далее в этом параграфе $A = \{a_1, \ldots, a_n\}$ — некоторый фиксированный конечный алфавит, причём буквы алфавита считаются перенумерованными, а P и Q — произвольные регулярные выражения над этим алфавитом. Между алгеброй регулярных языков и алгеброй регулярных выражений установлен изоморфизм, и это означает, что к регулярным выражениям можно применять регулярные операции, получая новые регулярные выражения.

1.2.5. Расширения языка регулярных выражений (3/4)

В частности, в соответствии с п. 1.2.1 и п. 1.2.2

$PQ \stackrel{\text{Def}}{=} \{\alpha\beta \mid \alpha \in P \;\&\; \beta \in Q\}$, $P^1 \stackrel{\text{Def}}{=} P$, $P^k \stackrel{\text{Def}}{=} PP^{k-1}$, $P^* \stackrel{\text{Def}}{=} \bigcup_{i=0}^{\infty} P^i$, $P^+ \stackrel{\text{Def}}{=} PP^*$.

Введем дополнительное обозначение для опциональной, необязательной конструкции, которое выражается через базовые регулярные операции:

$$P? \stackrel{\text{Def}}{=} \varepsilon \mid P.$$

Пример. $(ab)?c = c|abc$.

Замечание. Операции P^*, P^+ и $P?$ можно рассматривать как *операции повторения*: P^* — нуль или более повторений, P^+ — одно или более повторений, $P?$ — нуль или одно повторение P.

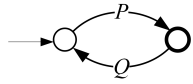
Удобно ввести *итерацию с разделителем* (обобщённую итерацию, итерацию Цейтина[6]). Пусть P и Q — регулярные выражения:

$$P \# Q \stackrel{\text{Def}}{=} P(QP)^*.$$

[6] Григорий Самуилович Цейтин (1936–2022)

1.2.5. Расширения языка регулярных выражений (4/4)

Отступление. Выразительную силу и пользу итерации Цейтина легко оценить, взглянув на источник выражения $P \# Q$.



Часто используют *символьные классы*, то есть подмножества символов заданного алфавита A. Подмножество можно задать перечислением элементов или же диапазоном, поскольку алфавит линейно упорядочен.

Пример. $[abcde] = [a - e] = (a|b|c|d|e)$.

Символьному классу можно дать имя и многократно использовать это имя в других регулярных выражениях

Пример. Символьный класс «цифра»: $\backslash d = [0-9] = [0123456789] = (0|1|2|3|4|5|6|7|8|9)$.

Используют в том числе и инвертированные символьные классы: $[\hat{}\, S] = [\, \overline{S}\,]$

Замечание. Операция дополнения определена только относительно фиксированного алфавита. Следует быть осторожными при использовании инвертированных символьных классов, поскольку результат зависит от ряда трудно учитываемых факторов: версии операционной системы, действующей в данный момент схемы кодирования и т.д.

Пример. Если $A = \{0, 1, 2, 3\}$, то $[\hat{}01] = 2|3$.

1.2.6. Программирование в регулярных выражениях (1/5)

Регулярные выражения являются элегантным и эффективным средством решения многих задач обработки текстов и встроены в большинство унаследованных и современных систем программирования.

TODO Посмотреть POSIX, PERL, SNOBOL4, REFAL. Понять основные идеи.
Эффективный алгоритм сопоставления с образцом = парсинг с присваиваниями.
Обусловленные (условные, с условиями) регулярные выражения.
Обязательно хороший пример. Я люблю такой 08.01.2026 => 2026, Jan, 8
но можно взять любой, только понятный до конца и без ошибок в коде.
Если это пример вдобавок будет выдавать :) в ответ на дату 31.06.2026, то честь и слава.

1.2.6. Программирование в регулярных выражениях (1/5)

TODO Замечания ФАН 20 января.
В целом хорошо = 4 * 4 = 16.
Слить слайды 1 и 2.
Расшифровки всех аббревиатур.
Извлечение значений для переменных = присваивание переменным значений, это важнейшее место сказано коряво.
Backtracing = поиск с возвратами. Есть великолепные графики, показывающие сравнительную производительность юниксоидных регулярных выражений и питона, из которых хорошо видно, что питон = отстой. Это неплохо бы дать в отступлении.
В примерах надо объяснять (кратко) все буквы и имена методов

Регулярные выражения являются элегантным и эффективным средством решения многих задач обработки текстов и встроены в большинство унаследованных и современных систем программирования.

1.2.6. Программирование в регулярных выражениях (2/5)

Исторически и практически регулярные выражения развивались в разных традициях.

- *POSIX* (BRE/ERE) — стандарт для многих UNIX-утилит (`grep`, `sed`, `awk`): акцент на переносимости и предсказуемости.
- *PERL/PCRE* — «расширенные» регулярные выражения: захваты, подстановки, дополнительные проверки контекста, обратные ссылки и др.
- *SNOBOL4* — сопоставление с образцом является центральной идеей языка: образцы можно комбинировать, успешное сопоставление естественно связывает переменные с найденными фрагментами.
- *REFAL* — сопоставление и переписывание выражений по правилам: акцент не на поиске в строке, а на систематических преобразованиях структур.

1.2.6. Программирование в регулярных выражениях (3/5)

В программировании важная задача при работе с регулярными выражениями — *сопоставление с образцом*: проверка соответствия строки или её фрагмента некоторому образцу, и извлечение значений для переменных.

Существует два подхода:

- Автоматный подход (POSIX): регулярное выражение преобразуется в автомат, сопоставление выполняется проходом по слову.
- Backtracking (Perl): перебираются альтернативы и разбиения, фиксируется первый успешный разбор.

Замечание. Если выражение допускает несколько разборов одной строки, то результат определяется правилами сопоставления в выбранной системе.

Пример. Выражение `(a|aa)`, строка `'aa'`.

PERL (backtracking): совпадение `'a'`. POSIX (leftmost-longest): совпадение `'aa'`.

1.2.6. Программирование в регулярных выражениях (4/5)

В некоторых системах регулярных выражений встречаются *условные регулярные выражения* — конструкции, позволяющие выбирать один из образцов в зависимости от условия:

```
(?(cond) yes | no).
```

Замечание. Условные конструкции не входят в POSIX-стандарт. Они подчёркивают, что на практике регулярные выражения нередко дополняются средствами управления ходом сопоставления.

Пример. Требуется распознавать время в одном из форматов: `HH` или `HH:MM`, при условии: минуты обязательны, если присутствует двоеточие:

```
(\d\d)(:)?(?(2)\d\d).
```

Группа 2 соответствует символу `':'`. Условная часть `(?(2)\d\d)` требует две цифры минут, если двоеточие было сопоставлено.

Строки `'09'` и `'09:30'` подходят, а `'09:'` и `'0930'` — нет.

1.2.6. Программирование в регулярных выражениях (5/5)

Пример. Строка задаёт дату в формате `'DD.MM.YYYY'`. Требуется преобразовать её к виду `'YYYY, Mon, D'`, при некорректной дате вывести `':)'`.

Input: s

$R_{\text{valid}} :=$ `^(?: 31\.(?:01|03|05|07|08|10|12)\.\d{4}`
`|30\.(?:01|03|04|05|06|07|08|09|10|11|12)\.\d{4}`
`|29\.(?:01|03|04|05|06|07|08|09|10|11|12)\.\d{4}`
`|(?:0[1-9]|1\d|2[0-8])\.(?:0[1-9]|1[0-2])\.\d{4})$`

$R_{\text{extract}} :=$ `^0?(\d+)\.0?(\d+)\.(\d{4})$`

$Mon[1..12] :=$ [`Jan`, `Feb`, `Mar`, `Apr`, `May`, `Jun`, `Jul`, `Aug`, `Sep`, `Oct`, `Nov`, `Dec`]

if $match(R_{\text{valid}}, s)$ **then**
  $(d, m_{\text{str}}, y) := capture(R_{\text{extract}}, s)$
  $m := toInt(m_{\text{str}})$
  print $y,\ Mon[m],\ d$
else
  print `:)`
end if

1.2.7. Уравнения с регулярными коэффициентами (1/3)

В алгебре регулярных выражений можно составлять и решать *уравнения*, пользуясь свойствами регулярных операций, установленными в п. 1.2.2. Коэффициентами и неизвестными таких уравнений являются регулярные множества слов, поэтому их называют *уравнениями с регулярными коэффициентами*.

Пример. Уравнение $X = aX + b$ имеет решение $X = a^*b$.

Действительно, $X = a^*b = (\varepsilon + aa^*)b = \varepsilon b + aa^*b = b + aX = aX + b$.

Отступление. Регулярные операции обладают многими удобными свойствами, но они не имеют обратных операций, а потому «школьные» приёмы решения алгебраических уравнений, наподобие правила «перенести неизвестные в левую часть» оказываются неприменимыми.

В уравнении коэффициентами могут быть не только отдельные символы или конкретные слова, как в предыдущем примере, но и любые регулярные выражения. В этом случае решение ищется в форме регулярного выражения над коэффициентами уравнения.

Пример. Уравнение $X = EX + B$ имеет решение $X = E^*B$ при любых регулярных коэффициентах E и B.

Однако даже для линейного уравнения решение может не быть единственным.

1.2.7. Уравнения с регулярными коэффициентами (2/3)

Пример. Если в уравнении $X = EX + B$ окажется, что $\varepsilon \in E$ (т. е. $E^+ = E^*$), то выражение $X = E^*(B + C)$ является решением для произвольного C. Действительно, $EX+B = EE^*(B+C)+B = E^+(B+C)+B = (E^+B+B)+E^+C = (E^+ +\varepsilon)B+E^+C$, и учитывая, что $E^+ = E^+ + \varepsilon = E^*$ в данном случае,
имеем $(E^+ + \varepsilon)B + E^+C = E^*B + E^*C = E^*(B + C) = X$.

TODO Связь с теоремой о наименьшей неподвижной точке функции (опционально)
См. Ахо – Ульман Т1 стр.126 и далее.
Системы уравнений с регулярными коэффициентами
Развернутый пример : система уравнений 1(0|1)*
то есть, двоичные записи натуральных чисел
Очень нужно доделать параграф.

1.2.7. Уравнения с регулярными коэффициентами (1/7)

В алгебре регулярных выражений можно составлять и решать *уравнения*, пользуясь свойствами регулярных операций, установленными в п. 1.2.2. Коэффициентами и неизвестными таких уравнений являются регулярные множества слов, поэтому их называют *уравнениями с регулярными коэффициентами*.

Пример. Уравнение $X = aX + b$ имеет решение $X = a^*b$.

Действительно, $X = a^*b = (\varepsilon + aa^*)b = \varepsilon b + aa^*b = b + aX = aX + b$.

Отступление. Регулярные операции обладают многими удобными свойствами, но они не имеют обратных операций, а потому «школьные» приёмы решения алгебраических уравнений, наподобие правила «перенести неизвестные в левую часть» оказываются неприменимыми.

В уравнении коэффициентами могут быть не только отдельные символы или конкретные слова, как в предыдущем примере, но и любые регулярные выражения. В этом случае решение ищется в форме регулярного выражения над коэффициентами уравнения.

Пример. Уравнение $X = EX + B$ имеет решение $X = E^*B$ при любых регулярных коэффициентах E и B.

Однако даже для линейного уравнения решение может не быть единственным.

1.2.7. Уравнения с регулярными коэффициентами (2/7)

Пример. Если в уравнении $X = EX + B$ окажется, что $\varepsilon \in E$ (т. е. $E^+ = E^*$), то выражение $X = E^*(B + C)$ является решением для произвольного C. Действительно, $EX+B = EE^*(B+C)+B = E^+(B+C)+B = (E^+B+B)+E^+C = (E^++\varepsilon)B+E^+C$, и учитывая, что $E^+ = E^+ + \varepsilon = E^*$ в данном случае,
имеем $(E^+ + \varepsilon)B + E^+C = E^*B + E^*C = E^*(B + C) = X$.

Замечание. Причина неоднозначности: при $\varepsilon \in E$ уравнение $X = EX + B$ имеет целое семейство неподвижных точек (решений), отличающихся добавкой E^*C. Ниже будет выделено *наименьшее* (по включению) решение.

Теорема. *[Лемма Ардена] Рассмотрим уравнение $X = EX + B$ над регулярными множествами.*

1. *Множество E^*B является решением.*
2. *E^*B — наименьшее решение по включению.*
3. *Если $\varepsilon \notin E$, то решение единственно и равно E^*B.*

1.2.7. Уравнения с регулярными коэффициентами (3/7)

Доказательство. (1) уже проверено в примере: $E(E^*B)+B = E^+B+B = (E^+ + \varepsilon)B = E^*B$.

(2) Пусть X — любое решение, т. е. $X = EX + B$. Тогда $X \supseteq B$. Подставляя рекуррентно, получаем $X = EX + B \supseteq EB + B$, далее $X \supseteq E(EB + B) + B = E^2B + EB + B$, и вообще $X \supseteq B + EB + \cdots + E^nB$ для всех $n \geqslant 0$. Переходя к объединению по n, имеем $X \supseteq \bigcup_{n\geqslant 0} E^nB = E^*B$.

(3) Пусть $\varepsilon \notin E$ и X — решение. Покажем $X \subseteq E^*B$. Возьмём слово $w \in X$ минимальной длины среди слов из $X \setminus E^*B$ (если такие есть). Из равенства $X = EX+B$ следует: $w \in B$ или $w = uv$, где $u \in E$, $v \in X$. Если $w \in B$, то $w \in E^0B \subseteq E^*B$ — противоречие. Если $w = uv$ с $u \in E$, то поскольку $\varepsilon \notin E$, имеем $|u| \geqslant 1$, значит $|v| < |w|$. По минимальности w получаем $v \in E^*B$, следовательно $w = uv \in EE^*B = E^+B \subseteq E^*B$ — противоречие. Итак, $X \subseteq E^*B$, вместе с (2) получаем $X = E^*B$. □

Отступление. Связь с теоремой о наименьшей неподвижной точке (ср. Ахо–Ульман, т. 1, стр. 126 и далее): уравнение $X = EX + B$ можно понимать как задачу найти неподвижную точку функции $F(X) = EX + B$ на решётке регулярных языков.

1.2.7. Уравнения с регулярными коэффициентами (4/7)

Теорема. *[Наименьшая неподвижная точка для $F(X) = EX + B$] Пусть $\mathcal{P}(\Sigma^*)$ упорядочено по включению. Функция $F(X) = EX + B$ монотонна, и её наименьшая неподвижная точка равна*

$$\mu F = \bigcup_{n \geqslant 0} F^n(\varnothing) = \bigcup_{n \geqslant 0} E^n B = E^* B.$$

Доказательство. Монотонность: если $X \subseteq Y$, то $EX \subseteq EY$, значит $F(X) \subseteq F(Y)$. Обозначим $X_0 = \varnothing$, $X_{n+1} = F(X_n)$. Тогда $X_1 = B$, $X_2 = EB + B$, $X_3 = E^2B + EB + B$ и по индукции $X_n = \bigcup_{i=0}^{n-1} E^i B$. Следовательно, $\bigcup_{n \geqslant 0} X_n = \bigcup_{i \geqslant 0} E^i B = E^* B$.
Проверим неподвижность: $F(E^*B) = E(E^*B) + B = E^+B + B = E^*B$. Пусть X — любая неподвижная точка F, т. е. $X = F(X)$. Тогда X является решением уравнения $X = EX + B$, а значит по лемме Ардена $E^*B \subseteq X$. Итак, E^*B — наименьшая неподвижная точка. □

1.2.7. Уравнения с регулярными коэффициентами (5/7)

Система линейных уравнений с рег. коэффициентами — это набор уравнений вида

$$\begin{cases} X_1 = E_{11}X_1 + E_{12}X_2 + \cdots + E_{1n}X_n + B_1, \\ \quad \cdots \\ X_n = E_{n1}X_1 + E_{n2}X_2 + \cdots + E_{nn}X_n + B_n, \end{cases}$$

где все E_{ij} и B_i — регулярные выражения (языки), а X_i — неизвестные языки.

Замечание. Удобно писать в векторной форме: $X = EX + B$, где $X = (X_1, \ldots, X_n)^T$, $B = (B_1, \ldots, B_n)^T$, а $E = (E_{ij})$ — матрица регулярных коэффициентов.

Теорема. *[Существование наименьшего решения системы] Система $X = EX + B$ имеет наименьшее (по покомпонентному включению) решение, равное наименьшей неподвижной точке монотонной функции $F(X) = EX + B$. Это решение получается итерациями Клини: $X^{(0)} = \varnothing$, $X^{(k+1)} = F(X^{(k)})$, и $X^{(\infty)} = \bigcup_{k \geqslant 0} X^{(k)}$.*

Доказательство. Функция F монотонна покомпонентно (как и в одномерном случае). На полной решётке $\mathcal{P}(\Sigma^*)^n$ существует наименьшая неподвижная точка, равная объединению итераций от $\varnothing$. Это объединение даёт вектор языков, который удовлетворяет $X = EX + B$ и минимален среди решений.

1.2.7. Уравнения с регулярными коэффициентами (6/7)

Пример.[Развёрнутый пример: двоичные записи натуральных чисел] Рассмотрим язык

$$L = 1(0 \mid 1)^*,$$

то есть все двоичные записи натуральных чисел без ведущих нулей (кроме 0). Построим систему уравнений по автомату:

начальное состояние q_0 — прочитали ещё ничего;
состояние q_1 — уже прочитали начальную 1 и затем читаем любые $0/1$;
оба состояния принимающие: только q_1 (пустая строка после стартовой 1 допустима).
Обозначим X_0 — язык слов, ведущих из q_0 в принимающее состояние, X_1 — язык слов, ведущих из q_1 в принимающее состояние. Тогда:

$$\begin{cases} X_0 = 1X_1, \\ X_1 = 0X_1 + 1X_1 + \varepsilon. \end{cases}$$

1.2.7. Уравнения с регулярными коэффициентами (7/7)

Во втором уравнении сгруппируем: $X_1 = (0+1)X_1 + \varepsilon$. Так как $\varepsilon \notin (0+1)$, по лемме Ардена имеем

$$X_1 = (0+1)^*\varepsilon = (0+1)^* = (0 \mid 1)^*.$$

Подставляя в первое уравнение, получаем

$$X_0 = 1X_1 = 1(0 \mid 1)^*.$$

Замечание. Если нужно включить число 0, то язык записей натуральных чисел будет $L_0 = \{0\} + 1(0 \mid 1)^*$.

TODO Неплохо.
Немного пример с автоматом недоделан.
Обознчения: греческие буквы, названия теорем, сноски ...
11 удовлетворительных атомов

1.2.8. Характеристика класса регулярных языков (1/3)

Следующее наблюдение помогает очертить класс регулярных языков.

Лемма. [о разрастании] *Пусть L — регулярный язык над алфавитом Σ. Тогда существует такое $n \in \mathbb{N}$, что для любого слова $\omega \in L$ длины $|\omega| \geqslant n$ найдутся слова $\alpha, \beta, \gamma \in \Sigma^*$, для которых верно:* $\alpha\beta\gamma = \omega, \quad \beta \neq \varepsilon, \quad |\alpha\beta| \leqslant n, \quad \forall\, i \geqslant 0 \ (\alpha\beta^i\gamma \in L)$.

Доказательство. Так как язык L регулярен, существует полный детерминированный источник H языка L (см. п. 1.2.3, п. 1.2.4). Обозначим через n число узлов источника H. Рассмотрим слово $\omega \in L$, $|\omega| \geqslant n$, и зафиксируем путь в H, порождающий слово ω. Пусть q_k — узел, в котором оказывается источник после порождения первых k символов слова ω (в частности, q_0 — начальный узел). Среди узлов $q_0, q_1, \ldots, q_n$ имеется совпадение: $q_k = q_m$ для некоторых $0 \leqslant k < m \leqslant n$ (принцип Дирихле). Положим α — префикс слова ω длины k, β — подслово слова ω, прочитанное на участке пути от q_k до q_m, и γ — оставшийся суффикс, так что $\omega = \alpha\beta\gamma$. Тогда $|\beta| = m - k > 0$ и $|\alpha\beta| = m \leqslant n$.

Поскольку участок пути $q_k \xrightarrow{\beta} q_m$ является циклом ($q_k = q_m$), его можно пройти i раз, после чего продолжить путь по γ. Следовательно, для любого $i \geqslant 0$ существует путь со значением $\alpha\beta^i\gamma$ из начального узла в заключительный, то есть $\alpha\beta^i\gamma \in L$. □

1.2.8. Характеристика класса регулярных языков (2/3)

Замечание. Лемма о разрастании применима только к *бесконечным* языкам, в которых длина слов не ограничена сверху. Все конечные языки регулярны.

Пример. Язык $D := \{a^n b^n \mid n \geqslant 1\}$ не является регулярным языком. Действительно, рассмотрим слово $\omega = a^n b^n \in D$ при некотором достаточно большом n и допустим, что оно разбито на три части $\omega = \alpha\beta\gamma$ так, что $|\beta| > 0$. Рассмотрим слово $\alpha\beta\beta\gamma$. Если $\beta \in a^+$ или $\beta \in b^+$, то $\alpha\beta\beta\gamma \notin D$, потому что нарушается баланс a и b. Если же $\beta \in a^+b^+$, то $\alpha\beta\beta\gamma \notin D$, потому что возникает чередование ba.

Отступление. Рассмотренный пример является частным случаем *языка Дика*[7], то есть языка правильно построенных скобочных выражений в математике. Правильно построенные последовательности открывающих и закрывающих скобок являются контекстно-свободным языком, поскольку порождаются грамматикой $S \to () \mid (S) \mid SS$, но не являются регулярным языком, то есть не могут быть описаны и/или распознаны никаким регулярным выражением. В этом проявляется ограниченность регулярных выражений: такие выражения «не умеют считать», точнее говоря, умеют считать только до числа n — количество узлов в графе источнике языка.

[7] Вальтер Франц Антон фон Дик (1856–1934)

1.2.8. Характеристика класса регулярных языков (3/3)

Теорема. *Язык регулярных выражений $\mathfrak{R}$ не является регулярным языком.*

Доказательство. От противного. Пусть Σ содержит некоторый исходный алфавит плюс используемые метасимволы регулярных выражений $(\ ) \mid *$. По лемме о разрастании существует число n, такое что если $|\omega| \geqslant n \ \& \ \omega \in \mathfrak{R}$, то существует такое слово β, $|\beta| > 0$, что $\omega = \alpha\beta\gamma$ и $\forall\, i \ \ (\alpha\beta^i\gamma \in \mathfrak{R})$, причём $|\alpha\beta| \leqslant n$. Рассмотрим слово

$$\omega = \underbrace{(\ldots(\ldots(}_{n \text{ символов } (} \ \cdots \ \underbrace{)\ldots)\ldots)}_{n \text{ символов })}.$$

Это корректное регулярное выражение, содержащее n пар сложенных скобок. Не имеет значения, что обозначено многоточием $\cdots$. Так как первые n символов слова ω — это открывающие скобки, то слово β состоит только из открывающих скобок, и не содержит закрывающих скобок. Но тогда при разрастании слова β нарушается баланс скобок и значит $\alpha\beta^i\gamma \notin \mathfrak{R}$ для $i > 1$. Противоречие, число n не существует. □

Замечание. Если вложенность скобок ограничена константой, то язык математических выражений со скобками (язык Дика) оказывается регулярным.

1.2.A. Онтология формальных языков, грамматик и автоматов

1.2.Б. Темы для самостоятельной работы студентов

1. СНИРС => Языки обработки символьной информации

   Существуют и используются языки программирования, в частности, REFAL, SNOBOL и многие другие, специально предназначенные для работы с формальными текстами и опирающиеся, в частности, на регулярные выражения. Кроме общих слов, необходим один красивый и убедительный пример элегантного решения нетривиальной задачи на обсуждаемом языке.

2. СНИРС => Программные инструменты работы с регулярными выражениями

   В различных системах программирования разные инструменты показывают большой разброс по производительности Требуется подготовить обзорный научно-популярный доклад с графиками и рекомендациями.

3. СНИРС => Эволюция регулярных выражений

   Предлагаются различные нововведения, которые как расширяют класс регулярных выражений с целью охвата новых задач, так и сужают класс регулярных выражений с целью повышения эффективности. Требуется выбрать те или иные расширения/сужения и провести сравнительный анализ и составление с тем, что в учебнике.

1.3. Контекстно-свободные грамматики

Для определения синтаксиса реальных языков программирования и языков предметной области чаще всего используются контекстно-свободные грамматики, п. 1.1.3. (напомним также введённое обозначение КСГ, п. 1.1.3.). Это обстоятельство объясняется тем, что выразительной силы регулярных языков иногда оказывается недостаточно для достижения практических целей, а для контекстно-зависимых грамматик неизвестно достаточно эффективных алгоритмов синтаксического анализа. Таким образом, КСГ образуют «золотую середину»: эти грамматики позволяют, с одной стороны, формально описать «почти весь» реальный язык программирования, и, с другой стороны, позволяют «достаточно эффективно» реализовать все необходимые алгоритмы работы с реальным языком программирования.

Поэтому изучение свойств КСГ и алгоритмов, связанных с этим классом грамматик, необходимо для всех последующих построений. В этом разделе изложена *теория* контекстно-свободных грамматик, которая используется в прикладных построениях других разделов.

1.3.1. Выводимость в контекстно-свободных грамматиках (1/3)

Выводимость в КСГ является частным случаем общего понятия выводимости, рассматриваемого в п. 1.1.2. Однако, поскольку форма правил вывода (продукций) в КСГ ограничена (левая часть правила — это всегда одиночный нетерминал), выводимость в КСГ обладает рядом дополнительных полезных свойств и допускает наглядное представление в виде *упорядоченного корневого дерева*, которое называется *дерево разбора* или *дерево вывода* (см. п. 1.1.5).

Замечание. Для грамматик более общего вида, контекстно-зависимых грамматик и грамматик с фразовой структурой (п. 1.1.3), также можно предложить графоподобные представления выводов в этих грамматиках. Однако такие представления оказываются принадлежами более сложным классам объектов (в частности, гиперорграфам), диаграммы которых не столь наглядны, как привычные диаграммы корневых деревьев. В то же время, хотя представление вывода деревом разбора безусловно возможно в случае линейных грамматик (поскольку в таких грамматиках левая часть каждого правила — это одиночный нетерминал, п. 1.1.6), но представление вывода в виде корневого дерева не добавляет наглядности, поскольку диаграмма такого дерева всегда имеет форму «ствола», состоящего из одной линии, к которой подвешены терминалы.

1.3.1. Выводимость в контекстно-свободных грамматиках (2/3)

Напомним, что в корневом ориентированном упорядоченном дереве *сечением* называется множество узлов, которое содержит ровно по одному излу из каждого пути из корня в лист. В частности, сам корень, а также крона, являются сечениями.

Дерево разбора, или *дерево вывода* в КСГ $G = \langle N, T, R, S \rangle$ — это ориентированное упорядоченное дерево, все узлы которого помечены символами из $N \cup T \cup \{\varepsilon\}$ таким образом, что если в дереве какой-то узел помечен символом A, а потомки этого узла помечены символами, образующими слово α, то в множестве правил грамматики R существует правило $A \to \alpha$.

Теорема. *Если $G = \langle N, T, R, S \rangle$ — контекстно-свободная грамматика, то $S \Rightarrow^* \alpha$ тогда и только тогда, когда существует дерево вывода с корнем s и кроной $\alpha \in T^*$.*

Доказательство. TODO Эссе 1.3. часть 1. Доказательство. По выводу можно построить дерево, а по дереву можно построить вывод. □

Замечание. Теорема утверждает существование дерева вывода, но не утверждает, что такое дерево единственно (см. п. 1.3.6).

1.3.1. Выводимость в контекстно-свободных грамматиках (3/3)

TODO Эссе 1.3. часть 2. Отступление. Аналогия с логическими исчислениями. Секвенциальная форма = сечение = узел в дереве порождения, см. 1.1.5.

TODO Эссе 1.3. часть 3. Отступление. Аналогия с орграфами. Правило $A \to \alpha B \beta$ порождает дугу $A \to B$. Отношение достижимости соответствует выводимости.

Таким образом, вопрос о выводимости в КСГ сводится к вопросу о построении дерева разбора.

Понятие выводимости в КСГ полезно обобщить, введя выводимость из одного символа и из цепочки символов, как терминальных, так и нетерминальных. Действительно, пусть задана грамматика $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$ и произвольная цепочка $\alpha \in (\mathrm{M} \cup \Sigma)^*$. Положим $G' := \langle \mathrm{M} + S', \Sigma, \Pi + (S' \to \alpha), S' \rangle$, то есть добавим в грамматику новую аксиому S' и новое правило $S' \to \alpha$. Легко видеть, что $\forall \omega \in \Sigma^* \ (S \Rightarrow_G^* \omega \iff S' \Rightarrow_{G'}^* \omega)$. Это наблюдение позволяет считать, что $\alpha \Rightarrow_G^* \omega \stackrel{\mathsf{Def}}{=} S' \Rightarrow_{G'}^* \omega$. Другими словами, любой символ или цепочка символов α в грамматике G также порождают некоторый язык, а именно множество выводимых слов, поэтому правомерно использовать обозначение $\mathfrak{L}(\alpha)$:

$$\mathfrak{L}(\alpha) \stackrel{\mathsf{Def}}{=} \{\omega \in \Sigma^* \mid \alpha \Rightarrow_G^* \omega\} .$$

1.3.1. Выводимость в КСГ: доказательство (4/7)

Доказательство.

[Необходимость: по выводу строим дерево.] Пусть $S = \alpha_0 \Rightarrow \alpha_1 \Rightarrow \ldots \Rightarrow \alpha_n = \alpha$ — вывод длины n. Построим последовательность деревьев $D_0, D_1, \ldots, D_n$, где крона D_i есть α_i.
Положим D_0 — дерево из одной вершины, помеченной S.
Пусть $\alpha_i = \beta A \gamma$ и применяется правило $A \to X_1 \ldots X_k$. Тогда D_{i+1} получается из D_i добавлением к листу, помеченному A, потомков $X_1, \ldots, X_k$. Крона D_{i+1} будет $\alpha_{i+1} = \beta X_1 \ldots X_k \gamma$.
Дерево D_n — искомое: его корень — S, его крона — α.
[Достаточность: по дереву строим вывод.] Рассмотрим последовательность сечений $C_0, C_1, \ldots$ где $C_0 = \{S\}$ (корень), а C_m — крона дерева. Каждое C_{i+1} получается заменой нетерминала $A \in C_i$ его потомками в дереве. Последовательность крон сечений образует вывод $S \Rightarrow^* \alpha$. □

1.3.1. Пример: дерево разбора выражения (5/7)

Пример. Рассмотрим КСГ арифметических выражений $G = \langle N, T, R, E \rangle$, где $N = \{E, T, F$
$T = \{a, +, *, (,)\}$, с правилами:

$$(1)\ E \to E + T, \qquad (2)\ E \to T,$$
$$(3)\ T \to T * F, \qquad (4)\ T \to F,$$
$$(5)\ F \to (E), \qquad (6)\ F \to a.$$

Левосторонний вывод цепочки $a + a * a$:

$$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T \Rightarrow a+T*F \Rightarrow a+F*F \Rightarrow a+a*F \Rightarrow a+a*a.$$

Левый разбор: $\pi = (1, 2, 4, 6, 3, 4, 6, 6)$.
Дерево разбора отражает приоритет: $a + (a * a)$, т.е. умножение выполняется раньше.

1.3.1. Аналогии: логика и орграфы (6/7)

Отступление.[Аналогия с логическими исчислениями] В секвенциальном исчислении доказательство представляется деревом, где каждый узел — секвенция (утверждение), а рёбра соответствуют правилам вывода.

Аналогия: *секвенциальная форма соответствует сечению дерева разбора* (см. п. 1.1.5).

В обоих случаях корень — исходное утверждение (аксиома S), а листья — элементарные факты (терминалы).

Отступление.[Аналогия с ориентированными графами] Построим граф достижимости нетерминалов: для каждого правила $A \to \alpha B \beta$, где $B \in \mathrm{M}$, добавим дугу $A \to B$.

Тогда *отношение достижимости* в этом графе соответствует выводимости: если B достижим из A в графе, то существует ω, такое что $A \Rightarrow^* \omega$ и ω содержит B.

TODO Хорошая работа.

Семь удовлетворительных атомов.

Ошибки форматирования, нет картинок.

1.3.1. Пример: граф достижимости нетерминалов (7/7)

Пример. Для грамматики арифметических выражений с правилами

$$E \to E + T \mid T, \quad T \to T * F \mid F, \quad F \to (E) \mid a$$

построим граф достижимости нетерминалов.

Дуги графа:

- Из $E \to E + T$: дуги $E \to E$, $E \to T$
- Из $E \to T$: дуга $E \to T$
- Из $T \to T * F$: дуги $T \to T$, $T \to F$
- Из $T \to F$: дуга $T \to F$
- Из $F \to (E)$: дуга $F \to E$

Получаем: $E \leftrightarrow T \leftrightarrow F$, причём $F \to E$. Все нетерминалы взаимно достижимы (граф сильно связен).

Замечание. Взаимная достижимость означает: из любого нетерминала можно вывести цепочку, содержащую любой другой нетерминал.

1.3.2. Преобразования контекстно-свободных грамматик (1/9)

Если $\mathfrak{L}(G_1) = \mathfrak{L}(G_2)$, то грамматики G_1 и G_2 называются *эквивалентными* и записывают это так: $G_1 \sim G_2$.

Замечание. Ясно, эквивалентность грамматик рефлексивна, симметрична и транзитивна, а потому введённая терминология и обозначения вполне оправданы.

TODO Эссе 1.3. Часть 4. Пример двух существенно различных, но эквивалентных грамматик. Пример с арифметическими выражениями.

Эквивалентные грамматики порождают один язык, и с этой точки зрения неразличимы, но эквивалентные грамматики могут иметь различные свойства, которые при применении, например, алгоритмов синтаксического анализа, могут давать различные практические результаты.

TODO Эссе 1.3. Часть 4. Ещё пример с арифметическими выражениями. Разные результаты вычислений, приоритет операций.

Известно несколько частных приёмов, которые позволяют придать контекстно-свободной грамматике желательные свойства, не меняя языка. Эти приёмы называются *эквивалентными преобразованиями* грамматик.

1.3.2. Преобразования контекстно-свободных грамматик (2/9)

Первым приёмом является устранение бесполезных символов из грамматики.

Символ X называется *полезным* в грамматике $G = \langle N, T, R, S \rangle$, если существует некоторый *терминальный* вывод (по правилам грамматики G) вида $S \Rightarrow^* \alpha X \beta \Rightarrow^* \omega$, где $\omega \in T^*$.

Замечание. Символ X может быть как терминальным, так и нетерминальным. Выводимая цепочка $\alpha X \beta$ может быть первой или последней в порождении.

Символ X называется *бесполезным*, если он не является полезным. Бесполезные символы не нужны в грамматике, и от них можно и нужно избавиться. Для выявления бесполезных символов определим свойства, характерные для полезных символов:

1. Символ X называется *порождающим*, если $\exists\, \omega \in T^* \ (X \Rightarrow^* \omega)$.
2. Символ X называется *достижимым*, если $\exists\, \alpha, \beta \in (N \cup T)^* \ (S \Rightarrow^* \alpha X \beta)$.

Полезный символ является одновременно порождающим и достижимым. Таким образом, если удалить из грамматики все непорождающие и все недостижимые символы, то останутся полезные символы и только они.

1.3.2. Преобразования контекстно-свободных грамматик (3/9)

Пример. Рассмотрим грамматику $\langle \{S, A\}, \{b\}, \{S \to A|b, B \to b\}, S\rangle$. Все символы, кроме A, являются порождающими: b порождает себя, S порождает b, B порождает b. Все символы, кроме B, являются достижимыми.

Зададим алгоритм индуктивного построения множества порождающих символов.

1. Каждый терминальный символ порождающий.
2. Если все символы в правой части α правила $A \to \alpha$ порождающие, то символ A — порождающий.

Зададим аналогичное индуктивное построение множества достижимых символов.

1. Начальный символ S достижимый.
2. Если символ A достижимый, то все символы в правой части α правила $A \to \alpha$ достижимые.

Таким образом, построив множество порождающих символов и множество достижимых символов, и взяв пересечение этих множеств, получаем множество полезных символов данной грамматики.

Пример. $\langle \{S, A\}, \{b\}, \{S \to A|b, B \to b\}, S\rangle \sim \langle \{S\}, \{b\}, \{S \to b\}, S\rangle$.

1.3.2. Преобразования контекстно-свободных грамматик (4/9)

Из определений следует, что любой *терминальный* вывод $S \Rightarrow^* \omega$, где $\omega \in T^*$, содержит только *полезные* символы, а значит бесполезные символы, хотя и могут присутствовать в грамматике, но не влияют на язык, определяемый грамматикой.

Теорема. *Если G_1 — произвольная контекстно-свободная грамматика, а грамматика G_2 получена из грамматики G_1 удалением всех бесполезных символов и всех содержащих их правил, то $\mathfrak{L}(G_1) = \mathfrak{L}(G_2)$.*

Замечание. Возможен вырожденный случай, когда начальный символ S является бесполезным в грамматике G_1, то есть непорождающим, и язык, порождаемый грамматикой G_1 — пустое множество. В таком случае грамматика G_2 оказывается пустой, $\langle \varnothing, \varnothing, \varnothing, \quad \rangle$, но заключение теоремы остаётся справедливым, поскольку язык, определяемый пустой грамматикой, естественно считать пустым. Поэтому, не ограничивая общности, можно считать, что начальные символы в обеих грамматиках совпадают.

Доказательство. Обозначим $L_1 := \mathfrak{L}(G_1), L_2 := \mathfrak{L}(G_2)$. Пусть $\alpha \in L_2$, т.е. $S \Rightarrow^*_{G_2} \alpha$. Тогда $S \Rightarrow^*_{G_1} \alpha$, поскольку всякое правило G_2 является правилом G_1 по построению G_2. Отсюда $\alpha \in L_1$ и $L_2 \subset L_1$. Пусть $\alpha \in L_1$, т.е. $S \Rightarrow^*_{G_1} \alpha$. Тогда $S \Rightarrow^*_{G_2} \alpha$, поскольку терминальный вывод не может содержать бесполезных символов. Отсюда $\alpha \in L_2$ и $L_1 \subset L_2$.

1.3.2. Преобразования контекстно-свободных грамматик (5/9)

Вторым приёмом построения эквивалентных грамматик является *устранение* некоторых правил за счёт *подстановки* одних правил в другие правила.

Теорема. *Пусть КСГ* $G_1 = \langle N, T, R, S\rangle$, $A \in N$, $(A \to \alpha_1 B \alpha_2) \in R$, $B \to \beta_1 | \ldots | \beta_k$. *Построим КСГ* $G_2 = \langle N, T, R_2, S\rangle$, *где* $R_2 = R \backslash \{A \to \alpha_1 B \alpha_2\} \cup \{A \to \alpha_1\beta_1\alpha_2 | \ldots | \alpha_1\beta_k\alpha_2\}$ *Тогда* $\mathfrak{L}(G_1) = \mathfrak{L}(G_2)$.

Доказательство. Обозначим $L_1 := \mathfrak{L}(G_1), L_2 := \mathfrak{L}(G_2)$. Пусть $\omega \in L_2$, т.е. $S \Rightarrow^*_{G_2} \omega$. Тогда заменим в этом выводе каждое применение нового правила вида $A \to \alpha_1\beta_i\alpha_2$ последовательным применением двух старых правил $A \to \alpha_1 B \alpha_2$, $B \to \beta_i$ и получим вывод $S \Rightarrow^*_{G_2} \omega$. Отсюда $\omega \in L_1$ и $L_2 \subset L_1$. Пусть теперь $\omega \in L_1$, т.е. $S \Rightarrow^*_{G_1} \omega$. Тогда заменим в этом выводе каждое применение удаленного правила $A \to \alpha_1 B \alpha_2$ и соответствующего (не обязательного соседнего!) применения правила $B \to \beta_i$ применением нового правила $A \to \alpha_1\beta_i\alpha_2$, и получим вывод $S \Rightarrow^*_{G_2} \omega$. Отсюда $\alpha \in L_2$ и $L_1 \subset L_2$. □

Следствие. *Пусть в КСГ* $G_1 = \langle N, T, R, S\rangle$ *есть некоторый вывод* $A \Rightarrow^*_{G_1} \alpha$, *где* $A \in N, \alpha \in (N \cup T)^*$. *Построим КСГ* $G_2 = \langle N, T, R_2, S\rangle$, *где* $R_2 = R \cup \{A \to \alpha\}$. *Тогда* $\mathfrak{L}(G_1) = \mathfrak{L}(G_2)$.

Замечание. В исчислениях любой вывод можно использовать как *производное* правило.

1.3.2. Преобразования контекстно-свободных грамматик (6/9)

Во многих случаях желательно избавляться от *цепных* правил $A \to B$ (см. п. 1.1.6.). Цепные правила можно удалять по одному на основании предыдущей теоремы, а можно удалить все разом, пользуясь следующим индуктивным построением.

1. Для каждого нетерминала A построим множество $\mathcal{C}(A) := \{B \in N \mid A \Rightarrow^* B\}$ нетерминалов, выводимых из нетерминала A цепными правилами из заданного множества правил R_1 (при этом $A \in \mathcal{C}(A)$).
2. Для каждого нетерминала B построим множество $\mathcal{B}(B) := \{A \in N \mid B \in \mathcal{C}(A)\}$ нетерминалов, из которых выводим нетерминал B цепными правилами из заданного множества правил R_1 (при этом $B \in \mathcal{B}(B)$)
3. Для каждого *нецепного* правила $A \to \alpha$ из заданного множества правил R_1 добавим в новое множество правил R_2 множество правил вида $B \to \alpha$ для каждого нетерминала $B \in \mathcal{B}(A)$.

Пример. Устранение цепных правил в грамматике арифметических выражений.

$E \to E + T \mid T$	$\mathcal{C}(E) := \{E, T, F\}, \mathcal{B}(E) := \{E\}$	$E \to E + T \mid T * F \mid (E) \mid a$
$T \to T * F \mid F$	$\mathcal{C}(T) := \{T, F\}, \mathcal{B}(E) := \{E, T\}$	$T \to T * F \mid (E) \mid a$
$F \to (E) \mid a$	$\mathcal{C}(E) := \{F\}, \mathcal{B}(E) := \{E, T, F\}$	$F \to (E) \mid a$

1.3.2. Преобразования контекстно-свободных грамматик (7/9)

Теорема. *Пусть* $G_1 = \langle N, T, R_1, S\rangle$ *содержит цепные правила, а из* $G_2 = \langle N, T, R_2, S\rangle$ *цепные правила удалены указанным построением. Тогда* $\mathfrak{L}(G_1) = \mathfrak{L}(G_2)$.

Доказательство. Обозначим $L_1 := \mathfrak{L}(G_1), L_2 := \mathfrak{L}(G_2)$. Пусть $\omega \in L_2$, т.е. $S \Rightarrow^*_{G_2} \omega$. Тогда если в этом выводе применяется правило $(B \to \alpha) \in R_2$, то это значит, что $\exists A \in G_1 \ \left(B \Rightarrow^*_{G_1} A \,\&\, (A \to \alpha) \in R\right)$ (возможно, что $B = A$). Заменим правило из R_2 последовательностью правил из R_1 и получим вывод $S \Rightarrow^*_{G_1} \omega$. Отсюда $\omega \in L_1$ и $L_2 \subset L_1$. Пусть теперь $\omega \in L_1$, т.е. $S \Rightarrow^*_{G_1} \omega$. Тогда заменим в этом выводе каждое применение последовательности цепных правил, которая заканчивается нецепным правилом $A \to \cdots \to B \to \alpha$ применением правила $A \to \alpha$ и и получим вывод $S \Rightarrow^*_{G_2} \omega$. Отсюда $\omega \in L_2$ и $L_1 \subset L_2$. □

Если в начальном правиле $S \to \alpha$ аксиома S входит в правую часть, $S \in \alpha$ то правило называется *S-правилом*. Покажем, что случай S-правил является частным случаем цепных правил, и к нему можно применить алгоритмы избавления от цепных правил.

Действительно, для устранения S-правил достаточно ввести новое *цепное* правило $S_1 \to S$ и объявить S_1 новой аксиомой грамматики. Множество выводимых терминальных цепочек при этом, очевидно, не изменится.

1.3.2. Преобразования контекстно-свободных грамматик (8/9)

К нежелательным часто относят ε-правила, или *укорачивающие* правила вида $A \to \varepsilon$.

Пример. Алгоритм устранения цепных правил может оказаться неприменимым в случае наличия ε-правил.

Устранить укорачивающие правила возможно следующим построением, во многом следующим идеям предыдущих построений.

1. Построим множество $\mathcal{E} := \{A \in N \mid A \Rightarrow^+ \varepsilon\}$ нетерминалов, из которых выводимо пустое слово.
2. Каждое *неукорачивающее* правило из множества R_1 представим в виде $A \to \alpha_0 B_1 \alpha_1 \dots \alpha_{k-1} B_k \alpha_k$, где $\forall\, i \in 1..k \ \ (B_i \in \mathcal{E})$, все символы в α_i, напротив, не входят в $\mathcal{E}$ и $k \geqslant 0$.
3. Для каждого неукорачивающего правила, содержащего k символов из $\mathcal{E}$, в множество R_2 включаются 2^k правил вида $A \to \alpha_0 X_1 \alpha_1 \dots \alpha_{k-1} X_k \alpha_k$, где X_i означает либо ничего, пустое слово, либо символ B_i. Такие правила соответствуют всем возможным подмножествам множества $\{B_1, \dots, B_k\}$.
4. Если $S \in \mathcal{E}$, то добавлятеся правило $S \to \varepsilon$ для возможности порождения пустого слова.

1.3.2. Преобразования контекстно-свободных грамматик (9/9)

Третий приём, добавление нового нетерминала, очень прост: часть цепочки, составляющей правую часть правила исходной грамматики можно обозначить новым нетерминалом, введя дополнительное правило, определяющее новый нетерминал.

Пример. Пусть в исходной грамматике G одно из правил имеет вид $A \to abBc$. Обозначив bB через новый нетерминал C, заменим это правило двумя: $A \to aCc, C \to bB$.

Грамматика, в которой нет бесполезных символов, а также ε-правил (за исключением разве что одного правила $S \to \varepsilon$), S-правил и цепных правил называется *приведённой*.

Теорема. *Для любой контекстно-свободной грамматики G_1 существует такая приведённая контекстно-свободная грамматика G_2, что $\mathfrak{L}(G_2) = \mathfrak{L}(G_1)$.*

Доказательство. Существование требуемой грамматики обеспечивается ранее перечисленными алгоритмами устранения нежелательных элментов. Важно соблюдать порядок преобразований.

1. Устранить ε-правила.
2. Устранить S-правила.
3. Устранить цепные правила.
4. Устранить бесполезные символы.

1.3.3. Нормальная форма Хомского

Говорят, что КСГ находится в *нормальной форме Хомского*, если все правила этой грамматики имеют одну из двух следующих форм: либо $A \to BC$, либо $A \to a$,
где $A, B, C \in N, a \in T$.

Теорема. *Для любой контекстно-свободной грамматики G_1 существует такая контекстно-свободная грамматика G_2 в нормальной форме Хомского, что $\mathfrak{L}(G_2) = \mathfrak{L}(G_1) - \varepsilon$.*

Доказательство. TODO Эссе 1.3. Часть 5. □

Пример. TODO Эссе 1.3. Часть 5. Уупрощенная грамматика выражений в привычной форме
и в форме Хомского

Замечание. Если грамматика имеет нормальную форму Хомского, то дерево разбора в такой грамматике является двоичным деревом. Для двоичных деревьев известно множество полезных и эффективных представлений и алгоритмов.

1.3.4. Лемма о разрастании

TODO Эссе 1.3. Часть 6. Лемма о разрастании. (теорема Огдена).
Очень важно неформально объяснить, что лемма о разрастании относится к языкам, хотя формулируется и доказывается с помощью грамматик.
Можно сначала лемму о разрастании для регулярных языков.
Доказательство лучше не прямое а через нормальную форму Хомского.
Обязательно пример: как с помощью леммы проверяется, что язык контекстно-свободный или нет.

1.3.3. Нормальная форма Хомского (2/4)

Доказательство. Пусть G_1 — приведённая грамматика (см. п. 1.3.2), не содержащая ε-правил. Построим G_2 в два этапа.

1. *Устранение смешанных правил*. Для каждого терминала $a \in \Sigma$, входящего в правила длины более 1, введём новый нетерминал X_a и правило $X_a \to a$. В исходных правилах заменим все вхождения a на X_a. Теперь правила имеют вид $A \to a$ или $A \to B_1 \dots B_k$, где $B_i \in \mathrm{N}$.
2. *Бинаризация длинных правил*. Для каждого правила вида $A \to B_1 B_2 \dots B_k$, где $k \geqslant 3$, введём новые нетерминалы $D_1, \dots, D_{k-2}$ и заменим исходное правило на цепочку:

$$A \to B_1 D_1, \quad D_1 \to B_2 D_2, \quad \dots, \quad D_{k-2} \to B_{k-1} B_k.$$

Полученная грамматика G_2 содержит только правила вида $A \to BC$ и $A \to a$. Язык не изменился, так как каждое преобразование является эквивалентным. □

1.3.3. Нормальная форма Хомского (3/4)

Пример. Рассмотрим приведённую грамматику арифметических выражений (без скобок): $E \to E + T \mid T * F \mid a$.

1. Устраняем терминалы в длинных правилах. Вводим $P \to +$ и $M \to *$. Правила принимают вид: $E \to EPT \mid TMF \mid a$.
2. Разбиваем длинные правила. Правило $E \to EPT$ заменяем на $E \to EZ_1, Z_1 \to PT$. Правило $E \to TMF$ заменяем на $E \to TZ_2, Z_2 \to MF$.

Результирующая грамматика в НФХ:

$$E \to EZ_1 \mid TZ_2 \mid a,$$
$$Z_1 \to PT, \quad P \to +,$$
$$Z_2 \to MF, \quad M \to *.$$

1.3.3. Нормальная форма Хомского (4/4)

Замечание. Важнейшим свойством грамматик в нормальной форме Хомского является структура дерева разбора. Поскольку каждое правило (кроме правил вида $A \to a$) содержит ровно два нетерминала в правой части, дерево разбора любой цепочки длины $n \geqslant 2$ является двоичным.

Следствие. *Если G — грамматика в нормальной форме Хомского, и $S \Rightarrow^* \alpha$, где $\alpha \in \Sigma^*$ и $|\alpha| = n$, то глубина дерева разбора не менее $\log_2 n$.*

Это свойство ограничивает «ширину» дерева относительно его высоты, что используется при доказательстве свойств контекстно-свободных языков, в частности, леммы о разрастании.

TODO Необходимо слить два текста в один без повторов и в разумном порядке, распределить по слайдам, ваправить форматы и обозначения.

1.3.4. Лемма о разрастании (1/4)

Для контекстно-свободных языков существует аналог леммы о накачке для регулярных языков. Более сильный вариант известен как *лемма Огдена*.

Теорема. *Для любого контекстно-свободного языка L существует такая константа $n \in \mathbb{N}$, что любую цепочку $z \in L$, в которой отмечено не менее n позиций, можно представить в виде $z = uvwxy$, так что:*

1. *цепочка vwx содержит **не более** n отмеченных позиций;*
2. *цепочка vx содержит **хотя бы одну** отмеченную позицию;*
3. $\forall\, i \geqslant 0 \ \ (uv^iwx^iy \in L)$.

Замечание. Классическая лемма о разрастании для КС-языков (*Bar-Hillel lemma*) является частным случаем леммы Огдена, где в цепочке отмечены все позиции.

1.3.4. Лемма о разрастании (2/4)

Доказательство. Пусть $L = \mathfrak{L}(G)$, где $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$ — грамматика в нормальной форме Хомского. Пусть $k = |\mathrm{M}|$. Положим $n = 2^{k+1}$. Рассмотрим дерево разбора цепочки z. Назовём узел дерева *ветвящимся*, если оба его поддерева содержат отмеченные позиции. Построим путь от корня к листу, каждый раз выбирая потомка, содержащего больше отмеченных позиций. Так как отмеченных позиций $\geqslant 2^{k+1}$, на этом пути встретится не менее $k + 1$ ветвящихся узлов. □

1.3.4. Лемма о разрастании (3/4)

Доказательство. (Окончание) На выбранном пути находится более k нетерминалов. Согласно принципу Дирихле, найдётся нетерминал A, который встречается на пути дважды (как A_1 и A_2, где A_2 — потомок A_1). Это позволяет разбить вывод на части:

$$S \Rightarrow^* uA_1y, \quad A_1 \Rightarrow^* vA_2x, \quad A_2 \Rightarrow^* w.$$

В силу того, что A_1 — ветвящийся узел, в поддеревьях, соответствующих v и x, содержится как минимум одна отмеченная позиция, следовательно $vx \neq \varepsilon$. Повторяя вывод $A \Rightarrow^* vAx$ произвольное число раз (или пропуская его), получаем цепочки uv^iwx^iy, принадлежащие языку L. □

1.3.4. Лемма о разрастании (4/4)

Пример. Докажем, что язык $L = \{a^m b^m c^m \mid m \geqslant 1\}$ не является контекстно-свободным. Предположим противное. Пусть n — константа леммы Огдена. Рассмотрим цепочку $z = a^n b^n c^n$. Отметим в ней все позиции с символом a. Разложим $z = uvwxy$. Согласно лемме, vx обязан содержать отмеченные символы (то есть a). В то же время, vwx содержит не более n отмеченных символов. Если мы «накачаем» цепочку ($i = 2$), количество символов a увеличится. Однако, так как v и x не могут содержать символы c (из-за структуры цепочки и ограничения на длину vwx), количество c останется неизменным. Получим цепочку с разным количеством a и c, которая не принадлежит L. Противоречие.

TODO Хороший материал, сделано в точности то, что требовалось.
Требуются следующие улучшения:
Перемещение материала по слайдам с обычной плотностью.
Примечание про Огдена и наведение порядка с именованием.
Греческие буквы для цепочек и переносы формул.
Семь хороших атомов.

1.3.5. Нормальная форма Грейбах (1/2)

Говорят, что КСГ находится в *нормальной форме Грейбах*, если все правила этой грамматики имеют следующую форму: $A \to a\alpha$, где $A \in N, a \in T, \alpha \in N^*$.

Построение нормальной формы Грейбах основано на устранении левой рекурсии.

Нетерминал $A \in N$ называется *рекурсивным*, если $A \Rightarrow^+ \alpha A \beta$. Если при этом $\alpha = \varepsilon$, то нетерминал называется *леворекурсивным*, а если $\beta = \varepsilon$, то нетерминал называется *праворекурсивным*.

Грамматика, в которой существуют леворекурсивные (праворекурсивные) нетерминалы, называется *леворекурсивной* (соответственно, *праворекурсивной*).

Теорема. *Для любой приведённой контекстно-свободной грамматики G_1 существует не леворекурсивная контекстно-свободная грамматика G_2, такая что $\mathfrak{L}(G_2) = \mathfrak{L}(G_1)$.*

Доказательство. TODO Эссе 1.3. Часть 7.

Алгоритм устранения левой рекурсии

желательно связать с уравнениями с регулярными коэффициентами □

Пример. TODO Эссе 1.3. Часть 7.

Упрощенная грамматика выражений в привычной форме

и с устранением левой рекурсии

1.3.5. Нормальная форма Грейбах (2/2)

Теорема. *Для любой не леворекурсивной контекстно-свободной грамматики G_1 существует контекстно-свободная грамматика G_2 в нормальной форме Грейбах, такая что $\mathfrak{L}(G_2) = \mathfrak{L}(G_1)$.*

Доказательство. TODO Эссе 1.3. Часть 8.

Алгоритм приведения к нормальной форме Грейбах □

Пример. TODO Эссе 1.3. Часть 8. Упрощенная грамматика выражений в привычной форме и в нормальной форме Грейбах

TODO Замечания ФАН 20 января

Удовлетворительно, большая часть материала пригодна, но необходимо связать с другими частями и исправить огромное количество ошибок.

Существенная дырка: а зачем это вообще нужно?

Зеленов, долг = 15 баллов

1.3.5. Нормальная форма Грейбах (1/6)

Говорят, что КСГ находится в *нормальной форме Грейбах*, если все правила этой грамматики имеют следующую форму: $A \to a\alpha$, где $A \in N, a \in T, \alpha \in N^*$.

Построение нормальной формы Грейбах основано на устранении левой рекурсии.

Нетерминал $A \in N$ называется *рекурсивным*, если $A \Rightarrow^+ \alpha A \beta$. Если при этом $\alpha = \varepsilon$, то нетерминал называется *леворекурсивным*, а если $\beta = \varepsilon$, то нетерминал называется *праворекурсивным*.

Грамматика, в которой существуют леворекурсивные (праворекурсивные) нетерминалы, называется *леворекурсивной* (соответственно, *праворекурсивной*).

Теорема. *Для любой приведённой контекстно-свободной грамматики G_1 существует не леворекурсивная контекстно-свободная грамматика G_2, такая что $\mathfrak{L}(G_2) = \mathfrak{L}(G_1)$.*

Доказательство. Доказательство основано на двух стандартных преобразованиях и на алгоритме последовательного устранения косвенной и непосредственной левой рекурсии. □

Лемма. *[Устранение правила] Пусть в грамматике есть правило $A \to B\alpha$, а все B-правила имеют вид $B \to \beta_1 \mid \cdots \mid \beta_k$. Тогда замена правила $A \to B\alpha$ на правила $A \to \beta_1\alpha \mid \cdots \mid \beta_k\alpha$ не меняет язык грамматики.*

1.3.5. Нормальная форма Грейбах (2/6)

Доказательство. Правило $A \to B\alpha$ допускает ровно те выводы, где сначала выводится некоторая цепочка из B, а затем приписывается α. Замена на $A \to \beta_i\alpha$ непосредственно воспроизводит все эти случаи. □

Лемма. *[Устранение непосредственной левой рекурсии] Пусть все A-правила имеют вид $A \to A\alpha_1 \mid \cdots \mid A\alpha_k \mid \beta_1 \mid \cdots \mid \beta_m$, где ни одна β_i не начинается с A. Тогда при введении нового нетерминала A' и замене этих правил на*

$$A \to \beta_1 A' \mid \cdots \mid \beta_m A', \qquad A' \to \alpha_1 A' \mid \cdots \mid \alpha_k A' \mid \varepsilon$$

язык грамматики не меняется.

Доказательство. В исходной грамматике из A выводятся в точности цепочки $\beta_i\alpha_{j_1}\alpha_{j_2}\cdots\alpha_{j_\ell}$ где $\ell \geqslant 0$. В новой грамматике: $A \Rightarrow \beta_i A' \Rightarrow \beta_i\alpha_{j_1}A' \Rightarrow \cdots \Rightarrow \beta_i\alpha_{j_1}\cdots\alpha_{j_\ell}A' \Rightarrow \beta_i\alpha_{j_1}\cdots\alpha_{j_\ell}$. Обратно, любое применение A' порождает ровно такую же последовательность α-блоков (или ε), значит множества выводимых цепочек совпадают. □

1.3.5. Нормальная форма Грейбах (3/6)

Замечание. Для устранения *косвенной* левой рекурсии упорядочим нетерминалы $N = \{A_1, \ldots, A_n\}$ и последовательно для $i = 1, \ldots, n$ заменяем каждое правило $A_i \to A_j\alpha$ при $j < i$ по лемме об устранении правила, после чего устраняем непосредственную левую рекурсию для A_i по предыдущей лемме. Тогда в результате не остаётся правил, из которых может начаться левый вывод $A_i \Rightarrow^+ A_i\gamma$.

Отступление. Связь с уравнениями: непосредственная левая рекурсия $A \to A\alpha_1 \mid \cdots \mid A\alpha_k \mid \beta_1 \mid \cdots \mid \beta_m$ соответствует “линейному” уравнению языков $X = XE + B$, где $E = \alpha_1 + \cdots + \alpha_k$, $B = \beta_1 + \cdots + \beta_m$. Преобразование с A' реализует фактор E^*: из A' выводятся любые повторения α-блоков.

Пример.[Упрощённая грамматика выражений и устранение левой рекурсии] Пусть $T = \{x, +, *, (,)\}$. Обычная (леворекурсивная) грамматика арифметических выражений:

$$E \to E + T \mid T,$$
$$T \to T * F \mid F,$$
$$F \to (E) \mid x.$$

1.3.5. Нормальная форма Грейбах (4/6)

Устраняя левую рекурсию, получаем эквивалентную грамматику:

$$
\begin{aligned}
&E \to TE',\\
&E' \to +TE' \mid \varepsilon,\\
&T \to FT',\\
&T' \to *FT' \mid \varepsilon,\\
&F \to (E) \mid x.
\end{aligned}
$$

Замечание. Полученная грамматика удобна для нисходящего разбора: ни один нетерминал не является леворекурсивным.

Теорема. *Для любой не леворекурсивной контекстно-свободной грамматики G_1 существует контекстно-свободная грамматика G_2 в нормальной форме Грейбах, такая что $\mathfrak{L}(G_2) = \mathfrak{L}(G_1)$.*

Отступление. Обычно перед построением НФГ грамматику упрощают (удаляют бесполезные символы, ε-правила и цепные правила), а затем приводят к форме, где каждое правило имеет вид $A \to a$ или $A \to BC$ (при $\varepsilon \notin L$). Эти преобразования сохраняют язык.

1.3.5. Нормальная форма Грейбах (5/6)

Доказательство. Опишем конструкцию (для случая $\varepsilon \notin L$), которая после предварительного упрощения строит НФГ из грамматики, у которой правила имеют вид $A \to a$ и $B \to CD$.

Введём новые нетерминалы $(A\backslash B)$ для всех $A, B \in N$ и положим $N' = N \cup \{(A\backslash B) \mid A, B \in N\}$. Смысл: $(C\backslash F)$ порождает такие $\gamma \in N^*$, что $F \Rightarrow^* C\gamma$.

Зададим правила:

$$(A\backslash A) \to \varepsilon \quad (A \in N),$$

$$(C\backslash E) \to A(A\backslash D)(B\backslash E) \quad \text{для всех } (B \to CD) \in R,\ A \in N,\ E \in N,$$

$$A \to a \quad \text{для всех } (A \to a) \in R, \qquad S \to a(A\backslash S) \text{ для всех } (A \to a) \in R.$$

Теперь заменим в правилах вида $(C\backslash E) \to A(A\backslash D)(B\backslash E)$ первый нетерминал A на терминал a, используя все варианты правил $A \to a$. Получим правила

$$(C\backslash E) \to a(A\backslash D)(B\backslash E),$$

то есть каждое правило начинается с терминала, а далее идут только нетерминалы: получена нормальная форма Грейбах.

1.3.5. Нормальная форма Грейбах (6/6)

Корректность (вкратце) доказывается индукцией по длине $\gamma \in N^*$:

1. Если $F \Rightarrow^* C\gamma$ в исходной грамматике, то $(C\backslash E) \Rightarrow^* \gamma(F\backslash E)$ в новой.
2. Для всех C, F верно: $F \Rightarrow^* C\gamma$ тогда и только тогда, когда $(C\backslash F) \Rightarrow^* \gamma$.

Из (2) следует равенство языков, в частности для стартового символа.

Пример.[Упрощённая грамматика выражений в НФГ] Ниже приведён один из возможных вариантов НФГ для языка выражений из примера (3/6) (используем вспомогательный нетерминал $R \to)$ для закрывающей скобки). Все правила имеют вид $A \to a\alpha$.

$$
\begin{aligned}
&R \to),\\
&F \to x \mid (ER,\\
&M \to *FM \mid *F,\\
&P \to +TP \mid +T,\\
&T \to x \mid xM \mid (ER \mid (ERM,\\
&E \to x \mid xM \mid xP \mid xMP \mid (ER \mid (ERM \mid (ERP \mid (ERMP.
\end{aligned}
$$

Замечание. Грамматика получена из нелеворекурсивной формы подстановкой правил для T в правила для E так, чтобы в каждой продукции первым символом был терминал.

1.3.6. Алгоритмические свойства грамматик (1/X)

TODO Повторное Определение термина неоднозначные грамматики и двусмысленные грамматки. Два развёрнутых примера. Первый пример Зубковой 1.1.5. Второй пример листочки Чепулис. Войнова Алёна

1.3.6. Алгоритмические свойства грамматик (2/X)

TODO Замкнутость, детерминированность, неоднозначность.

Теоретический параграф про разрешимое и неразрешимое.

Является ли проблема однозначности грамматики алгоритмически разрешимой?

Теорема. *Проблема распознавания однозначности контекстно-свободной грамматики алгоритмически неразрешима.*

TODO Параграф повис в воздухе. Яков Караев, сделан доклад. Нужны хорошие атомы.

К сожалению, в общем случае проблема эквивалентности двух контекстно-свободных грамматик, равно как и проблема определения неоднозначности контекстно-свободной грамматики оказываются алгоритмически неразрешимыми.

TODO разобраться и доложить!

TODO Проблемы: Неоднозначность. Пустота языка.

TODO Замечание ФАН 21 января. Параграф трудный, но очень важный и полезный для самообразования. Помогу с источниками.

1.3.7. Специальные классы грамматик (1/9)

Как показано в п. 1.4.4, наилучшие построенные алгоритмы синтаксического анализа контекстно-свободных языков без ограничений на грамматику имеют асимптотическую оценку трудоёмкости разбора $\mathcal{O}(n^3)$, где n — длина входной цепочки.

В этом параграфе рассматриваются такие подклассы КСГ, для которых возможно построить эффективные анализаторы, расходующие на обработку входной цепочки длины n время и память порядка $\mathcal{O}(n)$.

За эту эффективность приходится расплачиваться тем, что ни один из классов грамматик, для которых можно построить такие эффективные анализаторы, *не* порождает *все* контекстно-свободные языки. Однако эти ограниченные классы грамматик оказываются *достаточными* для описания синтаксиса современных языков программирования.

Отступление. По традиции рассматриваемые здесь подклассы КСГ принято обозначать $LL(k)$ и $LR(k)$. В этих обозначениях первая буква означает направление просмотра входной цепочки (L — слева направо), вторая буква означает ориентацию вывода (L — левосторонний вывод, R — правосторонний вывод, п. 1.1.5), а k — число символов, на которые анализатор должен «заглянуть вперёд» для того, чтобы разбор оставался безвозвратным.

1.3.7. Специальные классы грамматик (2/9)

Пусть задана однозначная КСГ $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$ и её язык $L = \mathfrak{L}(G)$. Рассмотрим функцию, по традиции обозначаемую FIRST_k^G, доставляющую мнжество всех возможных префиксов терминального порождения слова $\alpha \in (\mathrm{M} \cup \Sigma)^*$:

$$\mathrm{FIRST}_k^G(\alpha) \stackrel{\mathrm{Def}}{=} \{\omega \in \Sigma^* \mid (|\omega| < k \ \& \ \omega \in \mathfrak{L}(\alpha)) \ \vee \ (|\omega| = k \ \& \ \exists\, \beta \in \Sigma^* \ (\omega\beta \in \mathfrak{L}(\alpha)))\}\,.$$

Замечание. Для доказательства теорем необходимо рассматривать функцию FIRST_k^G как определённую на множестве всех цепочек, но для реализации простых алгоритмов разбора достаточно определить эту функцию только для нетерминальных символов: $\mathrm{FIRST}_k^G : \mathrm{M} \to 2^{\Sigma^*}$. Если индексы G и k ясны из контекста, их опускают.

КСГ $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$ называется *LL(k)-грамматикой*, если из существования двух левых выводов $S \Rightarrow_l^* \omega A\alpha \Rightarrow_l \omega\beta\alpha \Rightarrow^* \omega\delta$ и $S \Rightarrow_l^* \omega A\alpha \Rightarrow_l \omega\gamma\alpha \Rightarrow^* \omega\lambda$ для которых $\mathrm{FIRST}_k(\delta) = \mathrm{FIRST}_k(\lambda)$, вытекает, что $\beta = \gamma$.

Замечание. $LL(k)$ условие означает, что при левом разборе цепочки $\omega A\alpha$, где ω уже разобрана и A первый слева нетерминал, существует *не более одного правила* для A, позволяющего продолжить вывод с данными k терминалами.

1.3.7. Специальные классы грамматик (3/9)

Пример. Рассмотрим грамматику G_1 с правилами $S \to aAS \mid b, A \to a \mid bSA$ и покажем, что G_1 является $LL(1)$-грамматикой. Чтобы убедиться в этом, надо рассмотреть две цепочки вывода:

$$S \Rightarrow_l^* \omega S\alpha \Rightarrow_l \omega\beta\alpha \Rightarrow^* \omega\delta, \quad S \Rightarrow_l^* \omega S\alpha \Rightarrow_l \omega\gamma\alpha \Rightarrow^* \omega\lambda,$$

где $\mathrm{FIRST}_1(\delta) = \mathrm{FIRST}_1(\lambda)$. Если δ и λ начинаются символом a, то в выводе участвует $S \to aAS$ и $\beta = \gamma = aAS$. Если же δ и λ начинаются b, то применяется $S \to b$ и $\beta = \gamma = b$.

Теорема. *Приведенная контекстно-свободная грамматика $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$ является $LL(k)$-грамматикой тогда и только тогда, когда для двух различных правил $A \to \beta$ и $A \to \gamma$ пересечение $\mathrm{FIRST}_k(\beta\alpha) \cap \mathrm{FIRST}_k(\gamma\alpha) = \varnothing$ при всех таких $\omega A\alpha$, что $S \to_l^* \omega A\alpha$.*

Доказательство.

[Необходимость.] Допустим, что ω, A, α, β и γ удовлетворяют условиям теоремы, а $\mathrm{FIRST}_k(\beta\alpha) \cap \mathrm{FIRST}_k(\gamma\alpha)$ содержит δ. Тогда по определению FIRST для некоторых λ и μ найдутся выводы $S \Rightarrow^* \omega A\alpha \Rightarrow^* \omega\beta\alpha \Rightarrow^* \omega\delta\lambda$ и $S \Rightarrow^* \omega A\alpha \Rightarrow^* \omega\gamma\alpha \Rightarrow^* \omega\delta\mu$. Если $|\delta| < k$, то $\lambda = \mu = \varepsilon$. Так как $\beta \neq \gamma$, то G не $LL(k)$-грамматика. □

Замечание. Здесь использован тот факт, что M не содержит бесполезных нетерминалов.

1.3.7. Специальные классы грамматик (???)

TODO Это вставной слайд с заданием. В трёх слайдах ранее обозначения выправил ФАН. Основное: вместо w, x, y, z нужны $\omega, \delta, \lambda, \mu$, всегда $LL(k)$-грамматика, никогда $\rightarrow$ вместо $\Rightarrow$, $\mathrm{M} \neq M$!!!. Хотя слайды списаны с правильного источника и в целом имеют нужную структуру, но ошибки в обозначениях и в форматировании требуют полной переделки. ФАН 15.01.26

TODO На следующем вставном слайде важный текст, для которого надобно найти место и переделать всё последующее с оглядкой на этот факт.

1.3.7. Специальные классы грамматик (???)

КСГ называется *простой* (или *разделённой*), если для любого нетерминала $A \in \mathrm{M}$ правые части всех правил для этого нетерминала начинаются с различных терминальных символов. Другими словами, для любой пары $a \in \Sigma$, $A \in \mathrm{M}$ существует не более одного правила вида $A \to a\alpha$.

Замечание. Очевидно, что простая грамматика не содержит ε-правил и является $LL(1)$-грамматикой.

Отступление. Современное языкотворчество имеет тенденцию к использованию преимущественно разделённых грамматик, ввиду наличия простых и эффективных алгоритмов реализации. В частности, например, условие разделённости грамматики проверяется элементарно по записи правил, в то время как проверка выполнения свойства $LL(k)$ требует применения специального алгоритма. Более того, проблема определения того, существует ли такое число k, что грамматика является $LL(k)$-грамматикой, является алгоритмически неразрешимой.

TODO по возможности пересказать Дополнение Агафонова и подготовить почву для рассказа про управляющую таблицу и k-предсказывающий алгоритм синтаксического анализа. Всё дальнейшее изложение следует сосредоточить на разделённых грамматиках.

1.3.7. Специальные классы грамматик (4/9)

[Достаточность.] Допустим, что G не $\mathrm{LL}(k)$-грамматика. Тогда найдутся такие два вывода $S \to_i^* \omega A\alpha \to_l \omega\beta\alpha \to_i^* \omega x$ и $S \to_i^* \omega A\alpha \to_l \omega\gamma\alpha \to_i^* \omega y$, что цепочки x и y совпадают в первых k позициях, но $\beta \neq \gamma$. Поэтому $A \to \beta$ и $A \to \gamma$ — различные правила из Π и каждое из множеств $\mathrm{FIRST}_k(\beta\alpha)$ и $\mathrm{FIRST}_k(\gamma\alpha)$ содержит цепочку $\mathrm{FIRST}_k(x)$, совпадающую с цепочкой $\mathrm{FIRST}_k(y)$.

Пример. Грамматика G, состоящая из двух правил $S \to aS \mid a$, не будет $\mathrm{LL}(1)$-грамматикой так как $\mathrm{FIRST}_1(aS) = \mathrm{FIRST}_1(a) = a$. Интуитивно это можно объяснить так: видя при разборе цепочки, начинающейся символом a, только этот первый символ, мы не знаем, какое из правил $S \to aS$ или $S \to a$ надо применить к S.

С другой стороны, G — это $\mathrm{LL}(2)$-грамматика. В самом деле, если $S \to_l^* wA\alpha$, то $A = S$ и $\alpha = \varepsilon$. Так как для S даны только два указанных правила, то $\beta = aS$ и $\gamma = a$. Поскольку $\mathrm{FIRST}_2(aS) = aa$ и $\mathrm{FIRST}_2(a) = a$, то по теореме G будет $\mathrm{LL}(2)$-грамматикой.

1.3.7. Специальные классы грамматик (5/9)

Пусть $\langle \mathrm{M}, \Sigma, \Pi, S \rangle$ – контекстно-свободная грамматика. *Пополненной грамматикой*, полученной из G, называется грамматика

$$G' = \langle \mathrm{M} \cup \{S'\}, \Sigma, \Pi \cup \{S' \to S\}, S' \rangle,$$

где S' — новый начальный символ, не принадлежащий N, $S' \to S$ — нулевое правило грамматики G', а остальные правила занумерованы числами $1, 2, \ldots, p$.

Замечание. Это начальное правило добавляется для того, чтобы свёртку, в которой используется нулевое правило, можно было интерпретировать как сигнал о том, что цепочка допускается.

Пусть $\langle \mathrm{M}, \Sigma, \Pi, S \rangle$ — КСГ, а $G' = \langle \mathrm{M}', \Sigma, \Pi', S' \rangle$ — полученная из неё пополненная грамматика. G называется *LR(k)-грамматикой* для $k \geqslant 0$, если из условий

$$(1)\ S' \to_r^* \alpha A w \to_r \alpha \beta w,$$
$$(2)\ S' \to_r^* \gamma B x \to_r \gamma \beta y,$$
$$(3)\ FIRST_k(w) = FIRST_k(y)$$

следует, что $\alpha A y = \gamma B x$, то есть $\alpha = \gamma$, $A = B$ и $x = y$.

1.3.7. Специальные классы грамматик (6/9)

Пусть $\langle \mathrm{M}, \Sigma, \Pi, S \rangle$ — КС-грамматика. $[A \to \beta_1 \cdot \beta_2,\ u]$ называется *LR(k)-ситуацией*, если $A \to \beta_1\beta_2$ — правило из Π, а $u \in \Sigma^{*k}$.

Будем говорить, что LR(k)-ситуация $[A \to \beta_1 \cdot \beta_2,\ u]$ *допустима для активного префикса* $\alpha\beta_1$, если существует вывод $S' \to_r^* \alpha A w \to_r \alpha\beta_1\beta_2 w$, такой, что $u = FIRST_k(w)$.

Определим функцию $EFF_k^G(\alpha)$ так:

1. если α начинается терминалом, то $EFF_k(\alpha) = FIRST_k(\alpha)$;
2. если α начинается нетерминалом, то $EFF_k(\alpha) = \{\, w \mid w \in FIRST_k(\alpha)$ и существует $\beta \to_r wx,$ где $\beta \neq Awx$ для $A \in N \,\}$.

Пример. Рассмотрим грамматику G с правилами $S \to C|D, C \to aC|b, D \to aD|c$. Ситуация $[C \to a \cdot C, e]$ допустима для aaa, так как существует вывод $S \to_r^* aaC \to_r aaaC$, т. е. в этом примере $\alpha = aa$ и $w = e$

1.3.7. Специальные классы грамматик (7/9)

Теорема. *Пусть $\langle \mathrm{M}, \Sigma, \Pi, S\rangle$ — КС-грамматика. Тогда G является LR(k)-грамматикой тогда и только тогда, когда для каждой цепочки $u \in \Sigma^*$ выполняется условие:*
Пусть $\alpha\beta$ — активный префикс правовыводимой цепочки в пополненной грамматике G'. Если LR(k)-ситуация $[A \to \beta\cdot, u]$ допустима для $\alpha\beta$, то не существует другой LR(k)-ситуации $[A_1 \to \beta_1 \cdot \beta_2, v]$, допустимой для той же $\alpha\beta$, где $u \in EFF_k(\beta_2 v)$.

Доказательство.

[Необходимость.] Пусть $[A \to \beta\cdot, u]$ и $[A_1 \to \beta_1\cdot\beta_2, v]$ — различные ситуации, допустимые для $\alpha\beta$. Тогда существуют выводы:

$$S' \to_r^* \alpha A w \to_r \alpha\beta w,$$
$$S' \to_r^* \alpha_1 A_1 x \to_r \alpha_1\beta_1\beta_2 x,$$

где $\mathrm{FIRST}_k(w) = u$, $\mathrm{FIRST}_k(x) = v$ и $\alpha\beta = \alpha_1\beta_1$. Кроме того, $\beta_2 x \to_r^* uy$ для некоторой цепочки y и в этом (возможно, пустом) выводе терминал левом конце никогда не заменяется пустой цепочкой.

□

Специальные классы грамматик (8/9)

Рассмотрим 3 случая:

1. $\beta_2 = \varepsilon$. Тогда $u = v$, и выводы имеют вид $S' \to_r^* \alpha A w \to_r \alpha\beta w$, $S' \to_r^* \alpha A_1 x \to_r \alpha\beta_1 x$. Так как ситуации различны, то либо $A \neq A_1$, либо $\beta \neq \beta_1$, и нарушается LR(k)-условие.
2. $\beta_2 = z, z \in \Sigma^+$. Тогда $S' \to_r^* \alpha A w \to_r \alpha\beta w$, $S' \to_r^* \alpha A_1 x \to_r \alpha\beta_1 z x$. По условию $u \in EFF_k(\beta_2 v) = EFF_k(zv)$ Так как $\beta_2 = z$ состоит только из терминалов: $EFF_k(zv) = FIRST_k(zv)$ Таким образом $u \in FIRST_k(zv)$ Противоречие аналогично
3. β_2 содержит нетерминал. Пусть $\beta_2 \to_r^* u_1 B u_3 \to_r u_1 u_2 u_3$, где $u_1 u_2 \neq \varepsilon$. Тогда:

$$S' \to_r^* \alpha A w \to_r \alpha\beta w,$$
$$S' \to_r^* \alpha_1 A_1 x \to_r \alpha_1\beta_1\beta_2 x \to_r \alpha_1\beta_1 u_1 B u_3 x \to_r \alpha_1\beta_1 u_1 u_2 u_3 x.$$

В LR(k)-грамматике требуется, чтобы $\alpha A u_1 u_2 u_3 x = \alpha_1\beta_1 u_1 B u_3 x$, т.е. $\alpha A u_1 u_2 = \alpha_1\beta_1 u_1 B$, $A u_1 u_2 = \beta u_1 B$, но это невозможно, так как $u_1 u_2 \neq \varepsilon$.

Замечание. именно здесь используется условие, что $u \in EFF_k(\beta_2, v)$. Если бы в формулировке использовать только множество FIRST$_k$, цепочка $u_1 u_2$ могла бы быть пустой, и доказательство бы не прошло.

1.3.7. Специальные классы грамматик (9/9)

[Достаточность.] Пусть G — не LR(k)-грамматика. Тогда существуют два вывода:

$$S' \to_r^* \alpha A w \to_r \alpha\beta w,$$
$$S' \to_r^* \gamma B x \to_r \gamma\delta x = \alpha\beta y,$$

где $\text{FIRST}_k(w) = \text{FIRST}_k(y) = u$, но $\alpha A y \neq \gamma B x$. Выберем $\alpha\beta$ минимальной длины. Пусть $\alpha_1 A_1 y_1$ — последняя открытая цепочка в выводе: $S' \to_r^* \gamma B x \to_r^* \alpha_1 A_1 y_1 \to_r \alpha_1\beta_1\beta_2 y_1 \to_r^* \alpha_1\beta_1 y$. Тогда $\alpha_1\beta_1 = \alpha\beta$, и $u = \text{FIRST}_k(y) \in EFF_k(\beta_2, y)$. Ситуация $[A_1 \to \beta_1 \cdot \beta_2, v]$ допустима для $\alpha\beta$, где $v = FIRST_k(y_1)$. В то же время ситуация $[A \to \beta\cdot, u]$ допустима для $\alpha\beta$, поэтому осталось доказать, что $A_1 \to \beta_1 \cdot \beta_2$ отличается от $A \to \beta$. Пусть $A_1 \to \beta_1 \cdot \beta_2$ совпадает с $A \to \beta$. Тогда $S' \to_r^* \alpha_1 A y \to_r \alpha_1\beta y$, где $\alpha_1\beta = \alpha\beta$, поэтому $\alpha_1 = \alpha$ и $\alpha A y = \alpha B x$, что противоречит предположению о том, что G не $LR(k)$-грамматика

1.3.7. Специальные классы грамматик (1/9)

Как показано в п. 1.4.4, наилучшие построенные алгоритмы синтаксического анализа контекстно-свободных языков без ограничений на грамматику имеют асимптотическую оценку трудоёмкости разбора $\mathcal{O}(n^3)$, где n — длина входной цепочки.

В этом параграфе рассматриваются такие подклассы КСГ, для которых возможно построить эффективные анализаторы, расходующие на обработку входной цепочки длины n время и память порядка $\mathcal{O}(n)$.

За эту эффективность приходится расплачиваться тем, что ни один из классов грамматик, для которых можно построить такие эффективные анализаторы, *не* порождает *все* контекстно-свободные языки. Однако эти ограниченные классы грамматик оказываются *достаточными* для описания синтаксиса современных языков программирования.

Отступление. По традиции рассматриваемые здесь подклассы КСГ принято обозначать $LL(k)$ и $LR(k)$. В этих обозначениях первая буква означает направление просмотра входной цепочки (L — слева направо), вторая буква означает ориентацию вывода (L — левосторонний вывод, R — правосторонний вывод, п. 1.1.5), а k — число символов, на которые анализатор должен «заглянуть вперёд» для того, чтобы разбор оставался безвозвратным.

1.3.7. Специальные классы грамматик (2/9)

Пусть задана однозначная КСГ $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$ и её язык $L = \mathfrak{L}(G)$. Рассмотрим функцию, по традиции обозначаемую FIRST_k^G, доставляющую множество всех возможных префиксов терминального порождения слова $\alpha \in (\mathrm{M} \cup \Sigma)^*$:

$$\mathrm{FIRST}_k^G(\alpha) \stackrel{\mathrm{Def}}{=} \{\omega \in \Sigma^* \mid (|\omega| < k \ \& \ \omega \in \mathfrak{L}(\alpha)) \ \vee \ (|\omega| = k \ \& \ \exists\, \beta \in \Sigma^* \ (\omega\beta \in \mathfrak{L}(\alpha)))\}\,.$$

Замечание. Для доказательства теорем необходимо рассматривать функцию FIRST_k^G как определённую на множестве всех цепочек, но для реализации простых алгоритмов разбора достаточно определить эту функцию только для нетерминальных символов: $\mathrm{FIRST}_k^G \colon \mathrm{M} \to 2^{\Sigma^*}$. Если индексы G и k ясны из контекста, их опускают.

КСГ $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$ называется *$LL(k)$-грамматикой*, если из существования двух левых выводов $S \Rightarrow_l^* \omega A \alpha \Rightarrow_l \omega\beta\alpha \Rightarrow^* \omega\delta$ и $S \Rightarrow_l^* \omega A \alpha \Rightarrow_l \omega\gamma\alpha \Rightarrow^* \omega\lambda$ для которых $\mathrm{FIRST}_k(\delta) = \mathrm{FIRST}_k(\lambda)$, вытекает, что $\beta = \gamma$.

Замечание. $LL(k)$ условие означает, что при левом разборе цепочки $\omega A \alpha$, где ω уже разобрана и A первый слева нетерминал, существует *не более одного правила* для A, позволяющего продолжить вывод с данными k терминалами.

1.3.7. Специальные классы грамматик (3/9)

Пример. Рассмотрим грамматику G_1 с правилами $S \to aAS \mid b, A \to a \mid bSA$ и покажем, что G_1 является $LL(1)$-грамматикой. Чтобы убедиться в этом, надо рассмотреть две цепочки вывода:

$$S \Rightarrow_l^* \omega S\alpha \Rightarrow_l \omega\beta\alpha \Rightarrow^* \omega\delta \quad S \Rightarrow_l^* \omega S\alpha \Rightarrow_l \omega\gamma\alpha \Rightarrow^* \omega\lambda,$$

где $\mathrm{FIRST}_1(\delta) = \mathrm{FIRST}_1(\lambda)$. Если δ и λ начинаются символом a, то в выводе участвует $S \to aAS$ и $\beta = \gamma = aAS$. Если же δ и λ начинаются b, то применяется $S \to b$ и $\beta = \gamma = b$.

Теорема. *Приведенная КСГ $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$ является $LL(k)$-грамматикой тогда и только тогда, когда для двух различных правил $A \to \beta$ и $A \to \gamma$ пересечение $\mathrm{FIRST}_k(\beta\alpha) \cap \mathrm{FIRST}_k(\gamma\alpha) = \varnothing$ при всех таких $wA\alpha$, что $S \Rightarrow_l^* wA\alpha$.*

Доказательство.

[Необходимость.] Допустим, что ω, A, α, β и γ удовлетворяют условиям теоремы, а $\mathrm{FIRST}_k(\beta\alpha) \cap \mathrm{FIRST}_k(\gamma\alpha)$ содержит δ. Тогда по определению FIRST для некоторых λ и μ найдутся выводы $S \Rightarrow_i^* \omega A\alpha \Rightarrow_i^* \omega\beta\alpha \Rightarrow_i^* \omega\delta\lambda$ и $S \Rightarrow_i^* \omega A\alpha \Rightarrow_i^* \omega\gamma\alpha \Rightarrow_i^* \omega\delta\mu$. Если $|\delta| < k$, то $\lambda = \mu = \varepsilon$. Так как $\beta \neq \gamma$, то G не $LL(k)$-грамматика. □

Замечание. Здесь использован тот факт, что M не содержит бесполезных нетерминалов

1.3.7. Специальные классы грамматик (???)

TODO Это вставной слайд с заданием. В трёх слайдах ранее обозначения выправил ФАН. Основное: вместо w, x, y, z нужны $\omega, \delta, \lambda, \mu$, всегда $LL(k)$-грамматика, никогда $\rightarrow$ вместо $\Rightarrow$, $\mathrm{M} \neq M$!!!. Хотя слайды списаны с правильного источника и в целом имеют нужную структуру, но ошибки в обозначениях и в форматировании требуют полной переделки. ФАН 15.01.26

TODO На следующем вставном слайде важный текст, для которого надобно найти место и переделать всё последующее с оглядкой на этот факт.

1.3.7. Специальные классы грамматик (4/9)

[Достаточность.] Допустим, что G не $LL(k)$-грамматика. Тогда найдутся такие два вывода $S \Rightarrow_i^* \omega A\alpha \Rightarrow_l \omega\beta\alpha \Rightarrow_i^* \omega\delta$ и $S \Rightarrow_i^* \omega A\alpha \Rightarrow_l \omega\gamma\alpha \Rightarrow_i^* \omega\lambda$, что цепочки δ и λ совпадают в первых k позициях, но $\beta \neq \gamma$. Поэтому $A \to \beta$ и $A \to \gamma$ — различные правила из Π и каждое из множеств $\mathrm{FIRST}_k(\beta\alpha)$ и $\mathrm{FIRST}_k(\gamma\alpha)$ содержит цепочку $\mathrm{FIRST}_k(\delta) = \mathrm{FIRST}_k(\lambda)$.

Пример. Грамматика G, состоящая из двух правил $S \to aS \mid a$, не будет $LL(1)$ - грамматикой, так как $\mathrm{FIRST}_1(aS) = \mathrm{FIRST}_1(a) = a$. Интуитивно это можно объяснить так: видя при разборе цепочки, начинающейся символом a, только этот первый символ, мы не знаем, какое из правил $S \to aS$ или $S \to a$ надо применить к S.

С другой стороны, G — это $LL(2)$-грамматика. В самом деле, если $S \Rightarrow_l^* \omega A\alpha$, то $A = S$ и $\alpha = \varepsilon$. Так как для S даны только два указанных правила, то $\beta = aS$ и $\gamma = a$. Поскольку $\mathrm{FIRST}_2(aS) = aa$ и $\mathrm{FIRST}_2(a) = a$, то по теореме G будет $LL(2)$-грамматикой.

1.3.7. Специальные классы грамматик (5/9)

КСГ называется *простой* (или *разделённой*), если для любого нетерминала $A \in \mathrm{M}$ правые части всех правил для этого нетерминала начинаются с различных терминальных символов. Другими словами, для любой пары $a \in \Sigma$, $A \in \mathrm{M}$ существует не более одного правила вида $A \to a\alpha$.

Замечание. Очевидно, что простая грамматика не содержит ε-правил и является $LL(1)$-грамматикой.

Отступление. Современное языкотворчество имеет тенденцию к использованию преимущественно разделённых грамматик, ввиду наличия простых и эффективных алгоритмов реализации. В частности, например, условие разделённости грамматики проверяется элементарно по записи правил, в то время как проверка выполнения свойства $LL(k)$ требует применения специального алгоритма. Более того, проблема определения того, существует ли такое число k, что грамматика является $LL(k)$-грамматикой, является алгоритмически неразрешимой.

TODO по возможности пересказать Дополнение Агафонова и подготовить почву для рассказа про управляющую таблицу и k-предсказывающий алгоритм синтаксического анализа. Всё дальнейшее изложение следует сосредоточить на разделённых грамматиках.

1.3.7. Специальные классы грамматик (6/9)

Введение понятия $LL(1)$-грамматики — это один из возможных способов наложить на грамматику такие условия, чтобы шаг предсказания очередного правила вывода можно было сделать детерминированным, используя только текущий входной символ. Существует обобщающий подход разбора, соответствующий разделенной грамматике. Для разделенной грамматики шаг предсказания максимально прост: если a — текущий входной символ, A — верхний символ магазина и в грамматике есть правило $A \to aa$, то в магазине надо заменить A на a и сдвинуть входную головку.

Идея обобщения: к A-правилам разделенной грамматики разрешается добавить e-правило $A \to e$, а на шаге предсказания для текущих символов a и A разрешается применить правило $A \to e$ (т. е. удалить A из магазина), если в грамматике нет правила вида $A \to aa$.

1.3.7. Специальные классы грамматик (7/9)

КСГ $G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle$ называется грамматикой в *слабой нормальной форме Грейбах* если каждое правило из P имеет вид $A \to a\alpha$, где $\alpha \in \Sigma$, $a \in \mathrm{M}^*$, или $A \to e$. Если к тому же у нее правые части правил вида $A \to a\alpha$ и $A \to b\beta$ начинаются разными символами, т. е. $a \neq b$, то она называется *псевдоразделенной*.

Для формализации выбора правила по текущему символу для псевдоразделенной грамматики строится простой МП-распознаватель $M_G = \langle \Sigma, \mathrm{M}, \delta, S \rangle$, где функция переходов δ определяется следующим образом:

1. Если в стеке нетерминал A, а на входе символ a, и существует правило $A \to a\alpha$, то A заменяется на $a\alpha$ в стеке без сдвига входной головки. $\delta(a, A) = (a, 1)$
2. Если правила для символа a нет, но существует e-правило $A \to e$, то нетерминал A удаляется из стека (также без сдвига головки). $\delta(b, S) = (e, 0)$

Автомат служит для детерминизации шага предсказания в левом выводе. Он позволяет сопоставить нетерминал на вершине стека с текущим входным символом без преждевременного сдвига входной головки

1.3.7. Специальные классы грамматик (8/9)

Введение МП-распознавателя позволяет выделить иерархию грамматик, пригодных для эффективного анализа по одному символу:

1. **Псевдоразделенная грамматика**: КСГ G, для которой порождаемый язык $\mathfrak{L}(G)$, в точности совпадает с языком, допускаемым её МП - распознавателем $\mathfrak{L}(M_G)$.
2. **Разделенная грамматика**: частный случай, где для каждого нетерминала выбор правила по первому терминалу тривиален (отсутствуют e-правила и пересечения начальных символов).
3. **Слаборазделенная грамматика**: расширение, накладывающее ограничения на контекст применения e-правил, чтобы избежать конфликтов с другими альтернативами нетерминала.

Замечание. Данный МП-распознаватель является упрощенной моделью предсказывающего алгоритма. Управляющая таблица $M(A, a)$ фактически реализует функцию δ этого распознавателя, превращая теоретическую модель в практический инструмент синтаксического анализа.

1.3.7. Специальные классы грамматик (9/9)

Пример. Пусть G_1 — псевдоразделенная грамматика с правилами $S \to aSA \mid e, A \to a \mid b$.

Тогда $M_{G_1} = (\{a, b\}, \{S, A\}, \delta, S)$, где

$$\delta(a, S) = (SA, 1)$$
$$\delta(b, S) = (e, 0)$$
$$\delta(\$, S) = (e, 0)$$
$$\delta(a, A) = (e, 1)$$
$$\delta(b, A) = (e, 1)$$

МП-распознаватель ограничивается анализом одного текущего символа. Обобщением этого подхода является k-предсказывающий алгоритм, где решение принимается на основе аванцепочки длиной k. Конфигурация алгоритма представляет собой необработанный вход, содержимое магазина и выходную ленту (номера правил).

Управляющая таблица для таких алгоритмов строится на основе функции $\mathrm{FIRST_k}$, расширяя идеи «разделенности» на цепочки символов.

1.3.8. Классификация контекстно-свободных грамматик

TODO Итоговый обзор на тему: в каких случаях какие грамматики целесообразно применять. Здесь же про грамматики в регулярной форме.
TODO Общая диаграмма онтология всех классов грамматик. Здесь же про класс КС-языков, что язык $\{a^n b^n c^n\}$ НС-язык, но не КС-язык, и при этом $\{a^n b^n\}$ КС-язык, но не регулярный язык.

TODO Параграф повис в воздухе. Кто???

1.3.8. Классификация контекстно-свободных грамматик (2/5)

В курсе рассмотрены различные классы грамматик. Выбор подходящего класса зависит от задачи:

Класс грамматики	**Когда применять**
Регулярные грамматики	Лексический анализ, токенизация, поиск по образцу. Разбор за $\mathcal{O}(n)$, минимальные ресурсы.
$LL(k)$*-грамматики*	Ручное написание парсера (рекурсивный спуск). Простая реализация, понятная структура.
$LR(k)$*-грамматики*	Автоматическая генерация парсера (yacc, bison). Мощнее $LL(k)$, но сложнее для понимания.
Общие КСГ	Когда важна естественность описания, а не эффективность разбора. Алгоритмы CYK, Эрли: $\mathcal{O}(n^3)$.

1.3.8. Классификация контекстно-свободных грамматик (3/5)

Иерархия классов грамматик по Хомскому и их свойства:

Тип	Название	Распознаватель	Сложность
0	С фразовой структурой	Машина Тьюринга	Полуразрешимо
1	Контекстно-зависимые (НС)	ЛО-автомат	$\mathcal{O}(c^n)$
2	Контекстно-свободные	МП-автомат	$\mathcal{O}(n^3)$
3	Регулярные	Конечный автомат	$\mathcal{O}(n)$

Замечание. Внутри класса КСГ (тип 2) существует своя иерархия подклассов:

$$\text{Регулярные} \subset LL(1) \subset LL(k) \subset \text{Однозначные КСГ} \subset \text{КСГ}.$$

При этом для каждого фиксированного k: $LL(k) \subset LR(k)$.

1.3.8. Классификация контекстно-свободных грамматик (4/5)

Примеры языков, разделяющие классы:

- **Регулярный язык:** $L_1 = \{a^n \mid n \geqslant 0\} = a^*$ — распознаётся конечным автоматом.
- **КС-язык, но не регулярный:** $L_2 = \{a^n b^n \mid n \geqslant 0\}$ — требует МП-автомат (нужен стек для подсчёта a). Доказательство нерегулярности — лемма о разрастании.
- **НС-язык, но не КС-язык:** $L_3 = \{a^n b^n c^n \mid n \geqslant 0\}$ — требует линейно-ограниченный автомат. Доказательство — лемма Огдена (см. п. 1.3.4).
- **Рекурсивно перечислимый, но не КЗ:** язык $\{w \mid$ машина Тьюринга M_w останавливается проблема остановки неразрешима линейно ограниченным автоматом.

Отступление. Каждый следующий класс строго содержит предыдущий. Это обосновывает необходимость изучения всей иерархии Хомского.

1.3.8. Классификация контекстно-свободных грамматик (5/5)

Грамматика в регулярной форме — это КСГ, в которой правые части правил записаны с использованием регулярных операций (объединение, конкатенация, итерация).

Пример. Вместо правил $A \to aA \mid a$ можно записать $A \to a^+$ или $A \to aa^*$.

Преимущества регулярной формы:

- Более компактная запись грамматики
- Естественное представление повторяющихся конструкций
- Используется в РБНФ (п. 1.1.8) и современных парсер-генераторах

Замечание. Грамматика в регулярной форме эквивалентна обычной КСГ — регулярные операции можно раскрыть, введя дополнительные нетерминалы. Например, $A \to \alpha^*$ заменяется на $A \to \varepsilon \mid \alpha A$.

TODO Отличный ПЛАН!
Теперь каждый пункт раскрыть, обосновать, насытить ссылками и примерами.
Интересно, кто такие современные парсер-генераторы?

1.4. Методы синтаксического анализа

В самом общем случае синтаксический анализ (или разбор) — это сопоставление заданному выражению некоторого языка специальной структуры, характерной для языков этого класса.

Пример. Синтаксический анализ предложения на русском языке по типу, членам предложения, частям речи и т. д.

Применительно к языкам предметной области можно ограничиться языками, которые порождаются контекстно-свободными грамматиками. В таком случае результатом синтаксического анализа является дерево разбора.

Замечание. Если грамматика неоднозначна, то требуется найти *все* деревья разбора.

Отступление. Задача синтаксического анализа имеет больше практической значимости по сравнению с задачей распознавания принадлежности слова языку. Дело в том, что в результате синтаксического анализа получается больше информации: кроме ответа «да» мы получаем дерево разбора, которое можно использовать для дальнейшей работы, а кроме ответа «нет» мы получаем сообщения о синтаксических ошибках, которые также можно использовать для дальнейшей работы.

1.4.1. Постановка задачи синтаксического анализа

TODO Синтез и анализ на основе грамматики. Что требуется от парсера. Разбор с возвратами и без возвратов.
Что является результатом разбора для грамматик разных типов: регулярное выражение, дерево разбора, семантические падежи Филлмора и т.д. в отступлениях.

TODO Синтаксический разбор как логический вывод. Разрешимость и полуразрешимость. Разбор перебором. Требования к эффективности. Безвозвратный разбор и разбор с возвратами. Ограничения на алгоритмы разбора для грамматик разных классов.

1.4.1. Постановка задачи синтаксического анализа (1/5)

Синтаксический анализ — по заданным G и $w \in \Sigma^*$ решить задачу распознавания $w \in \mathfrak{L}(S)$ и, при необходимости, построить структурное свидетельство.

Пусть грамматика

$$G = \langle \mathrm{M}, \Sigma, \Pi, S \rangle, \qquad \mathfrak{L}(S) = \{\, w \in \Sigma^* \mid S \Rightarrow^* w \,\}.$$

Синтез:

построить $w \in \mathfrak{L}(S)$ посредством вывода $S \Rightarrow^* w$.

Анализ:

по w установить $(w \in \mathfrak{L}(S)?)$ и/или построить свидетельство $S \Rightarrow^* w$.

Задача синтаксического анализа включает два варианта:

1. *Распознавание (membership)*:

   вход: (G, w), выход: $(w \in \mathfrak{L}(S)?)$.

2. *Построение структуры (parsing)*:

   вход: (G, w), выход: структура, кодирующая вывод $S \Rightarrow^* w$.

1.4.1. Постановка задачи синтаксического анализа (2/5)

Результат (как свидетельство) зависит от класса грамматики (иерархия Хомского) и постановки задачи анализа.

- Тип 3 (регулярные) Распознавание: принимающий прогон конечного автомата на w (цепочка состояний/переходов). Парсинг по регулярному выражению: дерево разбора регулярного выражения.
- Тип 2 (КС-грамматики) Парсинг: дерево разбора T_w, удовлетворяющее $S \Rightarrow^* w$; при неоднозначности — множество деревьев $\{T_w\}$.
- Тип 1 (КЗ-грамматики) Результат парсинга задаётся как вывод $S \Rightarrow^* w$ с указанием применённых правил и позиций применения, удовлетворяющих контекстным условиям правил.
- Тип 0 (неограниченные) Результат парсинга (при успехе) — некоторый вывод $S \Rightarrow^* w$; универсальной гарантии завершения для всех w нет.

Отступление. В задачах обработки естественного языка построенная синтаксическая структура может служить входом для семантического этапа, например для назначения *семантических ролей*; это не является частью синтаксического распознавания.

1.4.1. Постановка задачи синтаксического анализа (3/5)

Требования к парсеру формулируются для фиксированных G и входа w относительно языка $\mathfrak{L}(S)$:

- *Корректность*: если парсер принимает w, то $w \in \mathfrak{L}(S)$; при parsing выдаваемая структура корректно кодирует вывод $S \Rightarrow^* w$.
- *Полнота*: для всех $w \in \mathfrak{L}(S)$ парсер принимает w (и строит структуру, если это требуется постановкой).
- *Детерминизм*: выбор действия однозначен и не использует перебор альтернатив.
- *Эффективность*: ресурсы ограничены функциями от $n = \mathrm{Length}(w)$, приемлемыми для практики.
- *Диагностика*: при $w \notin \mathfrak{L}(S)$ формируется сообщение об ошибке и её локализации.

Неоднозначность: $\exists w \in \mathfrak{L}(S)$, для которого существует более одного дерева разбора. В задачах компиляции неоднозначность устраняется переписыванием грамматики или фиксированными правилами разрешения конфликтов, поскольку разным деревьям могут соответствовать разные интерпретации.

1.4.1. Постановка задачи синтаксического анализа (4/5)

Разбор как логический вывод. Правила Π трактуются как правила вывода; утверждение $w \in \mathfrak{L}(S)$ эквивалентно существованию доказательства:

$$w \in \mathfrak{L}(S) \iff \exists \text{ вывод } S \Rightarrow^* w.$$

Парсер реализует алгоритм поиска такого доказательства; дерево разбора является внешним представлением найденного доказательства.

Разрешимость и полуразрешимость задачи распознавания:

- Тип 3: разрешима (конечный автомат завершает вычисление на любом входе).
- Тип 2: разрешима (существуют общие алгоритмы разбора, завершающиеся на любом (G, w)).
- Тип 0: полуразрешима: при $w \in \mathfrak{L}(S)$ перебор выводов находит свидетельство; при $w \notin \mathfrak{L}(S)$ алгоритм может не закончить работу.

1.4.1. Постановка задачи синтаксического анализа (5/5)

Разбор с возвратами (backtracking) — перебор альтернатив применения правил с возвратом и попыткой применить следующее правило при неудаче; в худшем случае число частичных разборов экспоненциально по $n = |w|$, что нарушает требования эффективности.

Безвозвратный разбор (deterministic parsing) — выбор действия однозначен по текущей конфигурации и ограниченному предпросмотру входа; перебор отсутствует, типично достигается линейная сложность по n.

Замечание. Ограничения на алгоритмы разбора задаются классом грамматик: общий разбор применим к более широким классам при более высокой асимптотике; линейные безвозвратные методы применимы лишь к подклассам, обеспечивающим детерминизм выбора действий.

TODO Январские тезисы.
Работать, работать и работать.

1.4.2. Нисходящий разбор

При нисходящем разборе результат синтаксического анализа — дерево разбора — строится, начиная с корня, то есть с аксиомы КСГ.

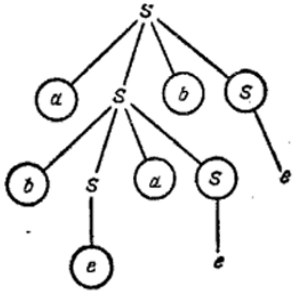
TODO Возможно, несколько параграфов.
Связь нисходящего разбора с левым выводом.
Рекурсивный спуск. Пример с арифметическими выражениями.
$LL(k)$-грамматики. Отступление про ANTL-R.
$LL(*)$???
TODO Игнатьев–Караев–Кириллова, очень неровный материал, доклады сделаны.

1.4.2 Нисходящий разбор (1/7)

Пусть D — дерево вывода в грамматике G. TODO Это всё не в тему.

Сечение дерева — множество вершин, пересекающее *все* пути от корня к листьям так, чтобы на каждом пути была ровно *одна* вершина из этого множества.

Крона сечения — последовательность меток вершин из этого сечения, записанных слева направо.



Пример. Дерево вывода с кроной $abaSbS$.

Лемма. *Пусть $S = \alpha_0, \alpha_1, \ldots, \alpha_n$ - вывод цепочки α_n из S в КС-грамматике $G = (N, \Sigma, P, S)$. Тогда в G можно построить дерево вывода D, для которого α_n — крона, а $\{\alpha_0, \alpha_1, \ldots, \alpha_{n-1}\}$ — некоторые из крон сечений.*

1.4.2 Нисходящий разбор (2/7)

Доказательство. Построим последовательность деревьев вывода D_i, где $0 \leqslant i \leqslant n$, таких что α_i — крона дерева D_i. Пусть D_0 — дерево, состоящее из единственной вершины, помеченной S.

Допустим, что

$$\alpha_i = \beta_i A \gamma_i$$

и после применения правила

$$A \to X_1 X_2 \dots X_k$$

к выделенному вхождению A получается

$$\alpha_{i+1} = \beta_i X_1 X_2 \dots X_k \gamma_i.$$

Тогда дерево D_{i+1} получается из D_i добавлением к листу, помеченному выделенным вхождением A (оно является $(|\beta_i| + 1)$-м символом кроны D_i), новых потомков, помеченных $X_1, X_2, \dots, X_k$ соответственно.

Очевидно, что крона дерева D_{i+1} будет α_{i+1}. Дерево D_n будет искомым деревом вывода D. TODO Эти леммы с доказательствамим нужны, но не здесь, а в другом месте. □

1.4.2 Нисходящий разбор (3/7)

Лемма. *Пусть D — дерево вывода в КС-грамматике $G = (N, \Sigma, P, S)$ с кроной α. Тогда $S \Rightarrow^* \alpha$.*

Доказательство. Рассмотрим последовательность сечений $(C_0, \ldots, C_n)$, где

1. C_0 содержит только корень дерева D,
2. C_{i+1} для $0 \leqslant i < n$ получается из C_i заменой одной нетерминальной вершины её прямыми потомками,
3. C_n — крона дерева D.

Если α_i — крона сечения C_i, то $(\alpha_0, \alpha_1, \ldots, \alpha_n)$ — вывод цепочки α_n из α_0 в G. □

Если в доказательстве леммы сечение C_{i+1} получается из C_i заменой самой левой нетерминальной вершины её прямыми потомками, то вывод $(\alpha_0, \alpha_1, \ldots, \alpha_n)$ называется *левым выводом* цепочки α_n из α_0 в грамматике G. TODO Это в другом месте. Ссылка.

Если $S = \alpha_0, \alpha_1, \ldots, \alpha_n = w$ — левый вывод терминальной цепочки w, то каждая цепочка α_i $(0 \leqslant i < n)$ имеет вид:

$$\alpha_i = x_i A_i \beta_i, \text{ где } x_i \in \Sigma^*, A_i \in N, \text{ и } \beta_i \in (N \cup \Sigma)^*.$$

и α_{i+1} получается из α_i заменой самого левого нетерминала A_i правой частью правила.

1.4.2 Нисходящий разбор (4/7)

Для некоторой КС-грамматики G цепочка $w \in L(G)$ считается *разобранной* (или *проанализированной*), если известно одно (а возможно, и все) из её деревьев вывода.

Definition

TODO Ужас! Пожалуйста, не надо! Пусть $G = (N, \Sigma, P, S)$ — КС-грамматика, правила которой занумерованы числами $1, 2, \ldots, p$.

Пусть $\alpha \in (N \cup \Sigma)^*$. *Левым разбором* цепочки α называется последовательность правил, применённых при левом выводе цепочки α из S.

Пусть $\pi = \{i_1, \ldots, i_n\}$ — левый разбор цепочки $w \in L(G)$, где $G = (N, \Sigma, P, S)$ — КС-грамматика. Зная π, можно построить дерево разбора цепочки w следующим нисходящим образом.

Начнём с корня, помеченного S. Тогда i_1 даёт правило, которое надо применить к S. Допустим, что i_1 — номер правила

$$S \Rightarrow X_1 X_2 \ldots X_k.$$

Присоединим к вершине, помеченной S, потомков $X_1, X_2, \ldots, X_k$ и пометим их этими символами.

1.4.2 Нисходящий разбор (5/7)

Если X_i — первый слева нетерминал в цепочке $X_1X_2\dots X_k$, то первыми $i-1$ символами цепочки w должны быть $X_1X_2\dots X_{i-1}$. Правило с номером i_2, например, должно тогда иметь вид

$$X_i \Rightarrow Y_1Y_2\dots Y_\ell,$$

и можно продолжить построение дерева разбора цепочки w, применяя ту же процедуру к вершине, помеченной X_i.

Продолжая в том же духе, можно построить всё дерево разбора цепочки w, соответствующее *левому разбору* этой цепочки.

Связь с левым выводом:

Если на каждом шаге выбирается *самый левый* нетерминал текущей строки α_i, то последовательность подстановок

$$S = \alpha_0 \Rightarrow \alpha_1 \Rightarrow \dots \Rightarrow \alpha_k = w$$

является *левым выводом* в грамматике G. Следовательно, нисходящий разбор моделирует левый вывод.

TODO Это первая фраза в тему. Но она непонятна. Что значит моделирует?

1.4.2 Нисходящий разбор (6/7)

Рекурсивный вывод – метод доказательства принадлежности цепочки w языку $L(A)$: если для правила $A \to X_1 X_2 \dots X_k$ выполнено $w_i \in L(X_i)$ для всех i, то

$$w = w_1 w_2 \dots w_k \in L(A).$$

Пусть $G = (N, \Sigma, P, E)$, где

$$N = \{E, E', T, T', F\},$$
$$\Sigma = \{+, *, (,), \text{num}\},$$
$$E - \text{стартовый символ.}$$

Правила P:

$$E \to TE'$$
$$E' \to +TE' \mid \varepsilon$$
$$T \to FT'$$
$$T' \to *FT' \mid \varepsilon$$
$$F \to (E) \mid \text{num}$$

1.4.2 Нисходящий разбор (7/7)

Вход:
КС-грамматика $G = \langle N, \Sigma, P, E \rangle$
поток токенов $a_1 \dots a_m \in \Sigma,\ \$$
Выход: accept / reject

parse(tokens):
продвинуться к первому токену
parseE(tokens)
if next_token = $ **then**
    ACCEPT
else
    REJECT
end if

parseE(tokens): //$E \to T\ E'$
parseT(tokens)
while next_token = $+$ **do**
    consume($+$)
    parseT(tokens)
end while

parseT(tokens): //$T \to F\ T'$
parseF(tokens)
while next_token = $*$ **do**
    consume($*$)
    parseF(tokens)
end while

parseF(tokens): //$F \to (E) \mid$ num
if next_token = num **then**
    consume(num)
elseif next_token = (**then**
    consume(()
    parseE(tokens)
    consume())
end if

TODO Ровно два слайда в тему, но оформлено неверно. 1) Определение рекурсивного спуска. 2) Пример с арифметическими выражениями. 3) Анализатор арифметических выражений.
Посмотрите другие слайды!

1.4.2. Нисходящий разбор (???)

TODO Вторая попытка.
Стало намного лучше, но всё подлежит переделыванию. Неправильные обозначения, нет ссылок на уже введенный материал, взятые в интернете безапелляционные и при этом НЕВЕРНЫЕ оценочные суждения.
Очень плохо аргументированы примеры: можно перейти к прямой польской записи, а можно в инфиксной записи избавиться от левой рекурсии.
Напрасно Вы убрали рекурсивный спуск — с него надо начинать!
Совершенно не используется идея простой или разделённой грамматики и её последствия для современного языкотворчества и реализации DSL.
Атомов много, а хороших нет.

1.4.2. Нисходящий разбор (1/7)

Нисходящий разбор — метод синтаксического анализа, при котором дерево вывода строится от корня (аксиомы грамматики) к листьям (входной цепочке). Алгоритмически это соответствует поиску *левого вывода* $S \Rightarrow_{lm}^{*} \omega$ для входной строки ω.

Пусть задана КС-грамматика $G = \langle N, T, R, S \rangle$. На каждом шаге анализатор рассматривает текущую сентенциальную форму $\alpha A \gamma$, где $\alpha \in T^*$ — обработанный префикс, $A \in N$ — левый нетерминал, подлежащий замене. Требуется выбрать продукцию $A \to \beta$ так, чтобы существовало продолжение вывода $\alpha\beta\gamma \Rightarrow_{lm}^{*} \omega$.

Основная проблема — недетерминизм выбора продукции. В общем случае требуется перебор (backtracking), что неэффективно. Практический интерес представляют грамматики, допускающие детерминированный выбор на основе ограниченного предпросмотра (lookahead).

Отступление. Левая рекурсия делает грамматику непригодной для нисходящего разбора — процедура анализа зацикливается. Методы устранения левой рекурсии рассмотрены в п. 1.3.4.

1.4.2. Нисходящий разбор (2/7)

Ключевая идея — возможность выбора продукции $A \to \beta$ на основе следующего символа входной цепочки. Если для любого нетерминала A и любого входного символа a существует не более одной применимой продукции, разбор может выполняться без возвратов.

Для формализации вводятся два множества:

- $\mathrm{FIRST}(\alpha)$ — терминалы, которые могут начинать вывод из цепочки α
- $\mathrm{FOLLOW}(A)$ — терминалы, которые могут следовать непосредственно за A в выводе из S

Грамматика G называется *LL(1)-грамматикой*, если для каждого нетерминала A и любых двух различных продукций $A \to \alpha$ и $A \to \beta$ выполняется:

$$\mathrm{FIRST}(\alpha) \cap \mathrm{FIRST}(\beta) = \varnothing,$$

а если $\varepsilon \in \mathrm{FIRST}(\alpha)$, то дополнительно:

$$\mathrm{FIRST}(\beta) \cap \mathrm{FOLLOW}(A) = \varnothing.$$

1.4.2. Нисходящий разбор (3/7)

Пример. Рассмотрим грамматику арифметических выражений:

$$\begin{aligned} E &\to T\,E' \\ E' &\to +\,T\,E' \mid \varepsilon \\ T &\to F\,T' \\ T' &\to *\,F\,T' \mid \varepsilon \\ F &\to (\,E\,) \mid \mathbf{id} \end{aligned}$$

Вычислим необходимые множества (опуская детали):

$$\begin{aligned} &\mathrm{FIRST}(E') = \{+, \varepsilon\}, \quad \mathrm{FOLLOW}(E') = \{), \$\} \\ &\mathrm{FIRST}(T') = \{*, \varepsilon\}, \quad \mathrm{FOLLOW}(T') = \{+,), \$\} \end{aligned}$$

Проверим условие LL(1) для E': $\mathrm{FIRST}(+\,T\,E') = \{+\}$, $\mathrm{FIRST}(\varepsilon) = \varnothing$, и $\{+\} \cap \{), \$\} = \varnothing$. Условие выполняется.

1.4.2. Нисходящий разбор (4/7)

Для LL(1)-грамматики строится таблица $M[A, a]$, где $A \in N$, $a \in T \cup \{\$\}$.

Алгоритм построения: Для каждой продукции $A \to \alpha$:

1. Для каждого $a \in \mathrm{FIRST}(\alpha)$ положить $M[A, a] = A \to \alpha$.
2. Если $\varepsilon \in \mathrm{FIRST}(\alpha)$, то для каждого $b \in \mathrm{FOLLOW}(A)$ положить $M[A, b] = A \to \alpha$.

	id	**+**	*****	**(**	**)**	**$**
E	$E \to T E'$			$E \to T E'$		
E'		$E' \to + T\, E'$			$E' \to \varepsilon$	$E' \to \varepsilon$
T	$T \to F\, T'$			$T \to F\, T'$		
T'		$T' \to \varepsilon$	$T' \to * F\, T'$		$T' \to \varepsilon$	$T' \to \varepsilon$
F	$F \to \mathbf{id}$			$F \to (\, E\,)$		

Замечание. Все ячейки либо содержат ровно одну продукцию, либо пусты — грамматика действительно LL(1).

1.4.2. Нисходящий разбор (5/7)

Алгоритм разбора по таблице: Используется стек. Начальное состояние: стек содержит $[S, \$]$.
На каждом шаге пусть X — верхний символ стека, a — текущий входной символ.

1. Если $X \in T$ и $X = a$, то снять X со стека и продвинуть вход.
2. Если $X \in T$ и $X \neq a$, то ошибка.
3. Если $X \in N$, то найти в таблице $M[X, a] = X \to Y_1 \dots Y_k$.
   - Если продукция найдена, снять X и положить на стек символы $Y_k, \dots, Y_1$ (чтобы Y_1 оказался сверху). Для ε-продукции просто снять X.
   - Если не найдено — ошибка.
4. Если $X = \$$ и $a = \$$, разбор успешно завершён.

Замечание. Этот алгоритм работает за линейное время $O(|\omega|)$ и использует память $O(|\omega|)$ для стека.

1.4.2. Нисходящий разбор (6/7)

Пример разбора цепочки $\mathbf{id} + \mathbf{id}$

Стек	Вход	Действие
$E\$$	$\mathbf{id} + \mathbf{id}\$$	$M[E, \mathbf{id}] = E \to T\,E'$
$T\,E'\$$	$\mathbf{id} + \mathbf{id}\$$	$M[T, \mathbf{id}] = T \to F\,T'$
$F\,T'\,E'\$$	$\mathbf{id} + \mathbf{id}\$$	$M[F, \mathbf{id}] = F \to \mathbf{id}$
$\mathbf{id}\,T'\,E'\$$	$\mathbf{id} + \mathbf{id}\$$	совпадение $\mathbf{id}$
$T'\,E'\$$	$+\mathbf{id}\$$	$M[T', +] = T' \to \varepsilon$
$E'\$$	$+\mathbf{id}\$$	$M[E', +] = E' \to +\,T\,E'$
$+\,T\,E'\$$	$+\mathbf{id}\$$	совпадение $+$
$T\,E'\$$	$\mathbf{id}\$$	$M[T, \mathbf{id}] = T \to F\,T'$
$F\,T'\,E'\$$	$\mathbf{id}\$$	$M[F, \mathbf{id}] = F \to \mathbf{id}$
$\mathbf{id}\,T'\,E'\$$	$\mathbf{id}\$$	совпадение $\mathbf{id}$
$T'\,E'\$$	$\$$	$M[T', \$] = T' \to \varepsilon$
$E'\$$	$\$$	$M[E', \$] = E' \to \varepsilon$
$\$$	$\$$	успех

Замечание. Применённые продукции образуют левый вывод.

1.4.2. Нисходящий разбор (7/7)

- Каждая LL(1)-грамматика однозначна.
- Существуют однозначные грамматики, не являющиеся LL(k) ни для какого k.
- Проверка принадлежности грамматики классу LL(1) разрешима.
- Класс языков, порождаемых LL(1)-грамматиками, строго включает регулярные языки и строго содержится в классе детерминированных КС-языков.
- **LL(k)**: использование k символов предпросмотра расширяет класс грамматик, но таблицы имеют размер $\mathcal{O}(|N| \cdot |T|^k)$.
- **LL($*$)**: современные инструменты (ANTLR) используют динамический анализ с помощью конечных автоматов.
- **Рекурсивный спуск**: альтернативная реализация, где каждый нетерминал соответствует процедуре. Удобен для ручного кодирования и встраивания семантических действий.

Отступление. Нисходящий разбор широко применяется благодаря своей наглядности и простоте интеграции семантических действий. Однако он менее мощен, чем восходящие методы (LR), и требует устранения левой рекурсии.

1.4.3. Восходящий разбор

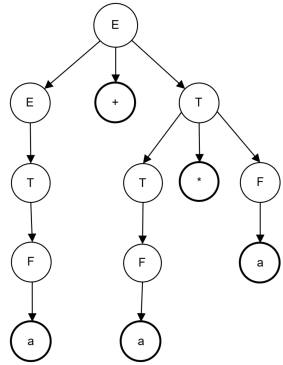
При восходящем разборе результат синтаксического анализа — дерево разбора — строится, начиная с кроны, то есть с входной цепочки.

TODO Возможно, несколько параграфов.
Связь восходящего разбора с обращенным правым выводом.
Перенос–свёртка. Пример с арифметическими выражениями.
$LR(k)$-грамматики. Связь с МП-автоматами.

1.4.3. Восходящий разбор(1/6)

Если при построении вывода α, всегда заменяется самый правый нетерминальный символ промежуточной цепочки, то вывод называется *правым* или *правосторонним выводом* α.

Пример. Рассмотрим КСГ G_1 с правилами $E \to E+T \mid T$, $T \to T*F \mid F$, $F \to (E) \mid a$. Дерево разбора справа соответствует правостороннему выводу цепочки $a+a*a$, описанному следующим образом: $E \Rightarrow E+T \Rightarrow E+T*F, \Rightarrow E+T*a \Rightarrow E+F*a \Rightarrow E+a*a \Rightarrow T+a*a \Rightarrow F+a*a \Rightarrow a+a*a$. Этот вывод соответствует стандартной интерпретации с учётом приоритета операций (умножение выполняется раньше сложения), то есть $a+(a*a)$. Данная грамматика является неоднозначной, поэтому для цепочки $a+a*a$ существует также альтернативный левосторонний вывод, приводящий к неверной группировке $(a+a)*a$.



1.4.3. Восходящий разбор(2/6)

Напротив левого разбора, *правый разбор* цепочки α — это обращение последовательности правил, примененных при правом выводе цепочки α из стартового символа S.
Вообще *правый разбор* цепочки α в грамматике G = $\langle \mathrm{M}, \Sigma, \Pi, S \rangle$ - это последовательность правил, с помощью которых можно свернуть цепочку α начальному символу S. В терминах деревьев выводов *правый анализ* цепочки α представляет собой последовательность отсечений основ, сводящую дерево вывода с кроной α к одной вершине, помеченной S. В сущности это равносильно процессу *заполнения* дерева вывода, начинающемуся с одной только кроны и идущему от листьев от корню, то есть снизу вверх. Поэтому с процессом порождения правого разбора часто ассоциируется термин *восходящий анализ* или *восходящий разбор*.

1.4.3. Восходящий разбор (3/6)

Метод *перенос-свёртка* представляет собой фундаментальную технику восходящего синтаксического анализа в теории компиляторов. Большинство восходящих анализаторов используют стек и два основных действия:

- *Перенос (Shift)* - перемещение следующего входного символа в стек.
- *Свёртка (Reduce)* - если на вершине стека находится последовательность символов, совпадающая с правой частью правила $A \to \beta$, она заменяется на нетерминал A.

Процесс продолжается до тех пор, пока стек не будет содержать только стартовый символ грамматики, а входной поток не исчерпан (accept).

1.4.3. Восходящий разбор(4/6)

Пример. Вернемся к приведенному выше примеру, но уже с использованием метода перенос-свёртка. Ниже приведена последовательность шагов перенос-свёртка парсера (восходящий разбор).

Шаг	Стек	Остаток входа	Действие	Правило
0	ϵ	a + a * a $	Shift	–
1	a	+ a * a $	Reduce	F $\rightarrow$ a
2	F	+ a * a $	Reduce	T $\rightarrow$ F
3	T	+ a * a $	Reduce	E $\rightarrow$ T
4	E	+ a * a $	Shift	–
5	E +	a * a $	Shift	–
6	E + a	* a $	Reduce	F $\rightarrow$ a
7	E + F	* a $	Reduce	T $\rightarrow$ F
8	E + T	* a $	Shift	–
9	E + T *	a $	Shift	–
10	E + T * a	$	Reduce	F $\rightarrow$ a
11	E + T * F	$	Reduce	T $\rightarrow$ T * F
12	E + T	$	Reduce	E $\rightarrow$ E + T
13	E	$	Accept	–

1.4.3. Восходящий разбор(5/6)

Замечание. Однако «чистый» перенос-свёртка (без дополнительных механизмов) часто сталкивается с конфликтами:

- *Перенос-свёртка*: в текущей ситуации возможно как выполнение переноса, так и свёртки.
- *Свёртка-свёртка*:на вершине стека одновременно подходят правые части нескольких разных правил.

Поэтому перенос-свёртка работает эффективно и детерминированно только при наличии механизма однозначного разрешения конфликтов.
$LR(k)$ как раз и является усовершенствованной и стандартизированной версией алгоритма перенос-свёртка, которая гарантирует детерминизм за счёт использования предпросмотра (lookahead) на k символов. Он сохраняет базовую механику shift и reduce, но добавляет предварительно построенную таблицу разбора, полностью устраняющую конфликты для грамматик класса $LR(k)$. На практике подавляющее большинство компиляторов использует подклассы с $k = 1$: $SLR(1)$, $LALR(1)$ (yacc, bison) или полные $LR(1)$.

1.4.3. Восходящий разбор(6/6)

Восходящий разбор по $LR(k)$ (в частности, $LR(1)$) напрямую реализуется на ДМП-автомате с помощью алгоритма перенос-свёртка : магазин (стек) хранит последовательность состояний и символов, соответствующих префиксу правого разбора; таблица ACTION определяет действия перенос или свёртка на основе текущего состояния и просмотра k символов вперёд; таблица GOTO управляет переходами после свёртки. Отсутствие конфликтов в таблицах гарантирует детерминизм. Таким образом, существование $LR(k)$-грамматики эквивалентно возможности эффективного восходящего синтаксического анализа языка детерминированным магазинным автоматом.

TODO Структурно материал раздела построен верно.
Источник взят какой-то очень поверхностный.
Все абзацы, начиная с первого, содержат неточности и неверные утверждения, текст надобно весь переработать.
Восемь атомов с оценкой удовлетворительно.

1.4.4. Специальные методы синтаксического анализа (1/9)

Методы синтаксического анализа, рассмотренные в предыдущих параграфах (п. 1.4.2 и 1.4.3), ориентированы на использование контекстно-свободных грамматик, обладающих дополнительными структурными свойствами, такими как детерминированность, отсутствие левой рекурсии или принадлежность к классам $LL(k)$ и $LR(k)$. Эти ограничения позволяют строить эффективные алгоритмы разбора, однако существенно сужают класс допустимых грамматик.

Для анализа языков, заданных контекстно-свободными грамматиками с другими ограничениями или без ограничений разработаны специальные методы, основанные на иных математических идеях. В разделе разделе рассматриваются три знаменитых алгоритма:

1. *Алгоритм Дейкстры* (*алгоритм сортировочной станции*) для анализа арифметических выражений;
2. *Алгоритм Эрли* для произвольных КС-грамматик;
3. *Алгоритм Кока–Янгера–Касами* (*алгоритм CYK*) для грамматик в нормальной форме Хомского.

Эти алгоритмы демонстрируют поучительные компромиссы между общностью, эффективностью и сложностью реализации. TODO именные сноски

1.4.4. Специальные методы синтаксического анализа (2/9)

Алгоритм Дейкстры (shunting-yard algorithm) решает задачу синтаксического анализа арифметических выражений за один проход ($\mathcal{O}(n)$). Его основная идея — использование двух стеков: *стека операндов* V и *стека операторов* O. TODO оператор операция, буква O

Правила алгоритма: Для каждого символа x во входной строке: TODO псевдокод

1. Если x - операнд ($a, b, c, \ldots$), то поместить x в стек V.
2. Если x - оператор ($+, *, \ldots$), то пока $O \neq \varepsilon$ и $\mathrm{Pr}(\mathrm{Top}(O)) \geqslant \mathrm{Pr}(x)$, выполнять *свёртку*. Затем поместить x в O.
3. Если $x =$ (, то поместить его в O.
4. Если $x =$), то выполнять свёртки, пока $\mathrm{Top}(O) \neq$ (, после чего удалить (из O.
5. В конце строки выполнить все оставшиеся свёртки. Результат - верхний элемент стека V.

Операция свёртки:

1. Извлечь оператор $\oplus$ из O.
2. Извлечь операнды y и x из V.
3. Поместить в V результат $x \oplus y$ (или поддерево разбора $(\oplus, x, y)$).

1.4.4. Специальные методы синтаксического анализа (3/9)

Пример. Рассмотрим выражение $a + b * c$ и грамматику:

$$E \to E + T \mid T, \qquad T \to T * F \mid F, \qquad F \to a \mid b \mid c.$$

Алгоритм Дейкстры неявно строит правосторонний вывод. Процесс разбора: TODO таблица

- **Шаг 1:** Вход a. $V = [a]$, $O = [\,]$.
- **Шаг 2:** Вход $+$. $V = [a]$, $O = [+]$.
- **Шаг 3:** Вход b. $V = [a, b]$, $O = [+]$.
- **Шаг 4:** Вход $*$. Так как $\Pr(*) > \Pr(+)$, оператор $*$ помещается в стек. $V = [a, b]$, $O = [+, *]$.
- **Шаг 5:** Вход c. $V = [a, b, c]$, $O = [+, *]$.
- **Шаг 6:** Конец строки. Выполняются свёртки:
  1. Для $*$: $V = [a, (b * c)]$, $O = [+]$.
  2. Для $+$: $V = [(a + (b * c))]$, $O = [\,]$.

Итоговое дерево разбора соответствует выводу: TODO картинка

$$E \Rightarrow E + T \Rightarrow E + T * F \Rightarrow E + T * c \Rightarrow E + b * c \Rightarrow a + b * c.$$

1.4.4. Специальные методы синтаксического анализа (4/9)

Алгоритм Эрли (1970) представляет собой универсальный метод синтаксического анализа для произвольных КС-грамматик. Его основная идея — динамическое построение множества *ситуаций* S_j для каждой позиции j во входной строке $w = a_1 \dots a_n$.

Ситуацией называется тройка $[A \to \alpha \cdot \beta, i]$, где: TODO Ситуация ? Конфигурация

- $A \to \alpha\beta \in R$ — правило грамматики;
- Точка $\cdot$ указывает текущую позицию в правиле;
- i — позиция во входной строке, где началось применение этого правила.

Алгоритм инициализирует S_0 и последовательно строит множества $S_1, \dots, S_n$ с помощью трёх операций:

1. **Предсказание:** Если $[A \to \alpha \cdot B\beta, i] \in S_j$, добавить $[B \to \cdot\gamma, j]$ для всех правил $B \to \gamma \in R$.
2. **Сканирование:** Если $[A \to \alpha \cdot a\beta, i] \in S_j$ и $a = w_{j+1}$, добавить $[A \to \alpha a \cdot \beta, i]$ в S_{j+1}.
3. **Завершение:** Если $[A \to \gamma\cdot, i] \in S_j$, то для всех $[B \to \alpha \cdot A\beta, k] \in S_i$ добавить $[B \to \alpha A \cdot \beta, k]$ в S_j.

Строка допускается, если в S_n присутствует ситуация $[S' \to S\cdot, 0]$, где S' — новый стартовый нетерминал, добавленный к грамматике. TODO Слова = Агафонов/Курочкин

1.4.4. Специальные методы синтаксического анализа (5/9)

Теорема. *Алгоритм Эрли корректен и полон: $w \in \mathfrak{L}(G)$ тогда и только тогда, когда $[S' \to S\cdot, 0] \in S_n$.*

Доказательство. Индукция по длине вывода. Корректность: каждая добавляемая ситуация соответствует возможному выводу в грамматике. Полнота: для любого вывода $S \Rightarrow^* w$ в грамматике соответствующая ситуация будет построена алгоритмом. □

Теорема. *[О сложности] В худшем случае алгоритм Эрли работает за время $\mathcal{O}(n^3)$ и использует память $\mathcal{O}(n^2)$ для грамматики фиксированного размера.*

Доказательство. Каждая ситуация $[A \to \alpha \cdot \beta, i]$ определяется нетерминалом A, позицией в правиле, начальной позицией i и текущей позицией j. Число возможных ситуаций есть $\mathcal{O}(n^2 \cdot |G|)$. Обработка одной ситуации (операции предсказания и завершения) может, в худшем случае, породить $\mathcal{O}(n)$ новых ситуаций (из-за цикла по k в завершении). Итоговая верхняя оценка времени работы: $\mathcal{O}(n^2 \cdot |G|) \cdot \mathcal{O}(n) = \mathcal{O}(n^3 \cdot |G|)$. Для фиксированной грамматики $|G|$ — константа, что даёт $\mathcal{O}(n^3)$. □

Пример. Для грамматики $E \to E{+}T \mid T,\ T \to a$ и строки $a{+}a$ алгоритм последовательно строит множества ситуаций S_0, S_1, S_2, S_3. В S_3 появляется ситуация $[E' \to E\cdot, 0]$, что подтверждает принадлежность строки языку. TODO Переписать слайд. Развернуть при-

1.4.4. Специальные методы синтаксического анализа (6/9)

Алгоритм Кока–Янгера–Касами (CYK) предназначен для синтаксического анализа КС-грамм приведённых к *нормальной форме Хомского* (НФХ, см. п. 1.3.3). В НФХ каждое правило имеет вид $A \to BC$ или $A \to a$.

Пусть $w = a_1 \dots a_n$ — входная строка. Построим треугольную таблицу P размером $n \times n$, где $P[i, j]$ — множество нетерминалов, порождающих подстроку $a_i \dots a_j$. TODO псевдо-код

1. **Инициализация** для подстрок длины 1:
$$P[i, i] = \{A \mid A \to a_i \in R\}.$$
2. **Рекуррентное правило** для длины $l > 1$:
$$P[i, j] = \bigcup_{k=i}^{j-1} \{A \mid A \to BC \in R,\ B \in P[i, k],\ C \in P[k+1, j]\}.$$

Строка принадлежит языку, если $S \in P[1, n]$.

Замечание. Для корректной работы алгоритма грамматика должна быть приведена к НФХ и не содержать ε-правил (кроме, возможно, $S \to \varepsilon$).

1.4.4. Специальные методы синтаксического анализа (7/9)

Теорема. *Алгоритм CYK корректен и полон для грамматик в НФХ:* $w \in \mathfrak{L}(G)$ *тогда и только тогда, когда* $S \in P[1, n]$.

Доказательство. Индукция по длине подстроки $l = j - i + 1$.

База: $l = 1$. По определению НФХ, $A \Rightarrow^* a_i$ тогда и только тогда, когда $A \to a_i \in R$, что соответствует инициализации.

Шаг: Если $A \Rightarrow^* a_i \dots a_j$ для $l > 1$, то существует правило $A \to BC$ и разбиение подстроки такое, что $B \Rightarrow^* a_i \dots a_k$ и $C \Rightarrow^* a_{k+1} \dots a_j$. По предположению индукции, $B \in P[i, k]$ и $C \in P[k + 1, j]$, следовательно, алгоритм добавит A в $P[i, j]$. □

Пример. Рассмотрим грамматику в НФХ и строку $w = baaba$:

$$\begin{aligned} S &\to AB \mid BC, \\ A &\to BA \mid a, \\ B &\to CC \mid b, \\ C &\to AB \mid a. \end{aligned}$$

Инициализация ($l = 1$): $P[1, 1] = \{B\}$, $P[2, 2] = \{A, C\}$, $P[3, 3] = \{A, C\}$, $P[4, 4] = \{B\}$, $P[5, 5] = \{A, C\}$.

1.4.4. Специальные методы синтаксического анализа (8/9)

Пример. (*Продолжение*) Для $l > 1$:

- $l = 2$**:** Например, $P[1,2] = \{A, S\}$, т.к. $A \to BA$ ($B \in P[1,1]$, $A \in P[2,2]$) и $S \to BC$ ($B \in P[1,1]$, $C \in P[2,2]$).
- $l = 3$**:** $P[2,4] = \{B\}$, т.к. $B \to CC$ ($C \in P[2,3]$, $C \in P[4,4]$? Нет, требуется аккуратная проверка. Фактически, $P[2,4]$ получается из разбиений $k = 2$ и $k = 3$).
- $l = 5$**:** $P[1,5] = \{A, S, C\}$.

Итоговая треугольная таблица P:

	1	2	3	4	5
1	$\{B\}$	$\{A, S\}$	$\varnothing$	$\varnothing$	$\{A, S, C\}$
2		$\{A, C\}$	$\{B\}$	$\{B\}$	$\{A, S, C\}$
3			$\{A, C\}$	$\{S, C\}$	$\{B\}$
4				$\{B\}$	$\{A, S\}$
5					$\{A, C\}$

Так как $S \in P[1,5]$, строка $baaba$ принадлежит языку.

1.4.4. Специальные методы синтаксического анализа (9/9)

Отступление. Алгоритм CYK допускает расширения для решения смежных задач:

1. Построение деревьев разбора:

   Если в $P[i, j]$ хранить не только нетерминалы, но и соответствующие им поддеревья, то можно восстановить все возможные деревья разбора.

2. Вероятностные КС-грамматики (ВКСГ):

   Для грамматик с вероятностями правил $p(A \to \alpha)$ алгоритм вычисляет наиболее вероятный разбор:

   $$V[i, j, A] = \max_{\substack{A \to BC \in R \\ i \leqslant k < j}} p(A \to BC) \cdot V[i, k, B] \cdot V[k + 1, j, C]$$

   Вероятность строки: $V[1, n, S]$.

Замечание. Эти расширения демонстрируют гибкость алгоритма CYK, хотя основное его назначение — проверка принадлежности строки языку, порождаемому грамматикой в НФХ.

Пример. В задачах обработки естественного языка ВКСГ и модифицированный алгоритм CYK используются для синтаксического анализа предложений.

1.5. Методы определения семантики

Определить семантику языка — это означает задать правила приписывания смысла всем или некоторым словам или предложениям языка (п. 0.1.1).

Для конечных языков задать семантику нетрудно, например, таблицей.

Пример. Англо-русский разговорник задаёт семантику конечного (и очень небольшого) подмножества выражений английского языка.

Для бесконечных языков требуются алгоритмические средства задания потенциально бесконечных соответствий, т. е. правила *вычисления* смысла предложения. Такие правила существенно зависят от того, как именно определён язык и как именно задан смысл. Универсальных правил задания семантики, применимых во всех случаях, в настоящее время не известно.

Пример. Методы, рассматриваемые в этом разделе, неприменимы для извлечения смысла (музыки) из нотной записи.

1.5.1. Постановка задачи определения семантики

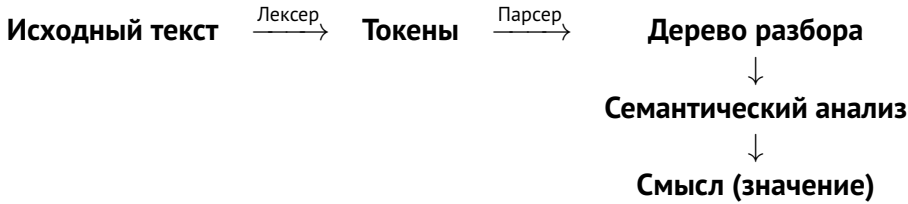
TODO Семантика языков, определённых грамматически. Схема. ФАН

TODO Общая философия. Абстракция выполнения. Доказательство правильности.
Императивные и декларативные языки.
Что требуется от семантики? ФАН

TODO ≈ 5 атомов, Петрунин

1.5.1. Постановка задачи определения семантики (2/6)

Схема определения семантики языка, заданного грамматикой:



Замечание. Синтаксис языка определяет *форму* предложений (что корректно записано).
Семантика языка определяет *смысл* предложений (что это означает).

Пример. Для арифметического выражения $2 + 3 * 4$:

- синтаксис определяет дерево разбора с учётом приоритета операций,
- семантика определяет результат вычисления: 14.

TODO Здесь и далее неплохой ПЛАН.
Уточнить и реализовать.

1.5.1. Постановка задачи определения семантики (3/6)

Абстракция выполнения — формальное описание того, как программа изменяет состояние вычислительной системы, без привязки к конкретной реализации.

Отступление. Зачем нужна формальная семантика?

- **Точная спецификация языка** — устранение двусмысленностей
- **Доказательство свойств программ** — верификация, корректность
- **Эквивалентность реализаций** — разные компиляторы дают одинаковый результат
- **Оптимизация** — преобразования, сохраняющие смысл

Замечание. Доказательство правильности программы — это математическое обоснование того, что программа соответствует своей спецификации для всех допустимых входов.

1.5.1. Постановка задачи определения семантики (4/6)

Два подхода к определению смысла программ:

Императивные языки	**Декларативные языки**
Описывают *как* вычислять	Описывают *что* вычислять
Последовательность команд	Система уравнений или правил
Изменяемое состояние (переменные)	Неизменяемые значения
Семантика: переходы состояний	Семантика: подстановки, редукции
Примеры: C, Java, Python	Примеры: Haskell, Prolog, SQL

Замечание. Для императивных языков естественна *операционная* семантика (описание через абстрактную машину).

Для декларативных языков естественна *денотационная* семантика (описание через математические функции).

1.5.1. Постановка задачи определения семантики (5/6)

Основные методы определения семантики:

1. *Атрибутные грамматики* (п. 1.5.2) — семантика вычисляется как атрибуты узлов дерева разбора
2. *Синтаксически управляемый перевод* (п. 1.5.3) — семантика как перевод в другой язык (например, в байт-код)
3. *Операционная семантика* (п. 1.5.5) — описание через выполнение на абстрактной машине
4. *Денотационная семантика* (п. 1.5.7) — сопоставление программе математической функции
5. *Исчисления программ* (п. 1.5.8) — описание через пред- и постусловия (аксиомы Хоара)

1.5.1. Постановка задачи определения семантики (6/6)

Пример. Рассмотрим семантику оператора присваивания $x := e$ в императивном языке.

Операционная семантика:

- Вычислить значение выражения e в текущем состоянии σ
- Обновить состояние: $\sigma' = \sigma[x \mapsto \text{значение}(e, \sigma)]$

Денотационная семантика:

$$[\![x := e]\!](\sigma) = \sigma[x \mapsto [\![e]\!](\sigma)]$$

Деривационная семантика (исчисление программ):

$$\{P\}\, x := e\, \{P \;\&\; x = e\}$$

Замечание. Все три подхода описывают одну и ту же семантику разными способами. Выбор метода зависит от целей: реализация, верификация или спецификация.

1.5.2. Атрибутные грамматики

TODO Идея Кнута.
Синтезированные и унаследованные атрибуты.
Пример = школьный калькулятор.
Пример = символьное дифференцирование.

TODO Замечания ФАН 20 января:
По теме и структуре почти хорошо, по качеству выполнения совершенно никуда не годится. Это безобразная халтура, позорище. Делал Олег Жохов.
Неправильные обозначения, неверное форматирование, невнятные слова, недоделанные примеры.
Далее: не выполнено задание (см. выше), нет позиционирования, нет ссылки на Кнута, нет финальных выводов.
Что делать: полностью переделать и расширить до 9 слайдов.

1.5.2. Атрибутные грамматики (1/7)

Рассмотрим формальную КСГ $G = \langle N, T, R, S \rangle$. Она хорошо описывает синтаксис языка, но её недостаточно для описания *семантики* языка, то есть значений в нём. Для описания семантики введем *атрибуты* - дополнительные данные, ассоциированные с грамматическими символами.

Атрибутной грамматикой называется четверка $AG = \langle G, A, F \rangle$, где – конечное множество всех атрибутов, $A = A_0 + A_1$, где A_0 - конечное множество синтезируемых атрибутов, A_1 - конечное множество наследуемых атрибутов, F - конечное множество семантических правил.

Замечание. $A_0 \cap A_1 = \oslash$

Рассмотрим атрибутную грамматику AG. С каждым символом $X \in V, V = N \cup T$, связывается конечное множество атрибутов A, которое делится на A_0 и A_1. Будем обозначать атрибут α символа X как $\alpha(X)$. Каждое $\alpha(X) \in A(X)$ имеет множество значений V_α. Принадлежность атрибута α символу X обозначим как $X.\alpha$.

Замечание. $A_1(S) = \oslash, A_0(t) = \oslash$, где $t \in T$

Замечание. Символ α не имеет отношения к цепочкам.

1.5.2. Атрибутные грамматики (2/7)

Пусть F состоит из набора m правил, где p-е правило имеет вид: $X_{p0} \to X_{p1}X_{p2}...X_{pn_p}$, где $n_p \geqslant 0, X_{p0} \in N, X_{pj} \in N$ при $1 \leqslant j \leqslant n_p$

Пример. p-е правило имеет вид $X_{p0} \to X_{p1}X_{p2}...X_n$

Семантическими правилами называются функции f_{pja}, определенные для всех $1 \leqslant p \leqslant m, 0 \leqslant j \leqslant n_p$ и $\alpha \in A_0(X_{pj})$, если $j = 0$; или $\alpha \in A_1(X_{pj})$, если $j > 0$.

$f_{pj\alpha} : V_{\alpha 1} \times V_{\alpha 2} \times ... \times V_{\alpha t} \to V_\alpha$ для $t = t(p, j, \alpha) \geqslant 0$, где все $\alpha_i = \alpha_i(p, j, \alpha)$ являются атрибутами X_{pk_i} при $0 \leqslant k_i = k_i(p, j, \alpha) \leqslant n_p, 1 \leqslant i \leqslant t$.

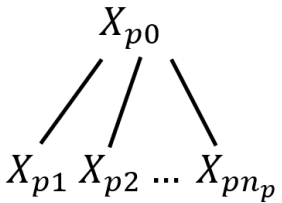
Пример.

$X_{p0}.\alpha_i = f_{p,p0\ ,\alpha_i}\,(X_{p1}.\alpha_{j_1}, X_{p1}.\alpha_{j_2}, ..., X_{p1}.\alpha_{j_{t_1}}, X_{p2}.\alpha_{j_1}, X_{p1}.\alpha_{j_2}, ..., X_{p1}.\alpha_{j_{j_2}}, ..., X_{pk}.\alpha_{j_{t_k}})$

Здесь $X_{p0}.\alpha_i$ — i-й атрибут символа X_{p0}, где $1 \leqslant p \leqslant m, 0 \leqslant p_0 \leqslant n_p, \alpha_i \in A(X_{p0})$; $f_{p,p0\ ,\alpha_i}$ — функция вычисления атрибута α_i символа X_{p_0} правила вывода p; p_β — индексы символов правила p, где $1 \leqslant \beta \leqslant k \leqslant n_p$; $\alpha_{j_{t_1}}, ..., \alpha_{j_{t_k}}$ — атрибуты символов X_{p_β}, где $1 \leqslant \beta \leqslant k \leqslant n_p, t_\beta \leqslant t\ \forall \beta \in [1, k]$

Замечание. Последняя запись означает, что необязательно каждый символ $X_{p\beta}$ будет иметь одинаковое число атрибутов. t_β, варьируется от символа к символу и не превышает общего количества атрибутов своего символа.

1.5.2. Атрибутные грамматики (3/7)



Семантические правила используются для сопоставления цепочкам КС языка «значения» при помощи дерева вывода. Для любого вывода терминальной цепочки $t \in S$ при помощи синтаксических правил построим обычное дерево вывода. Корнем дерева будет S, каждый узел помечается либо терминальным символом, либо нетерминалом X_{p0},

соответствующим применению k-го правила для некоторого p. В последнем случае у этого узла будет n_p прямых потомков.

Пусть теперь X – метка некоторого узла дерева, $\alpha \in A(X)$ – атрибут . Если $\alpha \in A_0(X)$, то $X = X_{po}$ для некоторого p, если же $\alpha \in A_1(X)$, то $X = X_{pj}$ для некоторых j и p. В обоих случаях дерево на участке этого узла имеет вид, представленный в схеме. По определению атрибут α имеет в этом узле значение $\upsilon = f_{pj\alpha}(\upsilon_1, ..., \upsilon_t)$, если в соответствующем семантическом правиле $f_{pj\alpha} : V_{\alpha 1} \times V_{\alpha 2} \times ... \times V_{\alpha t} \to V_\alpha$ все атрибуты $\alpha_1, ..., \alpha_t$ уже определены и имеют в узлах с метками $X_{pk_1}, ..., X_{pk_t}$ значения $\upsilon_1, ..., \upsilon_t$ соответственно.

1.5.2. Атрибутные грамматики (4/7)

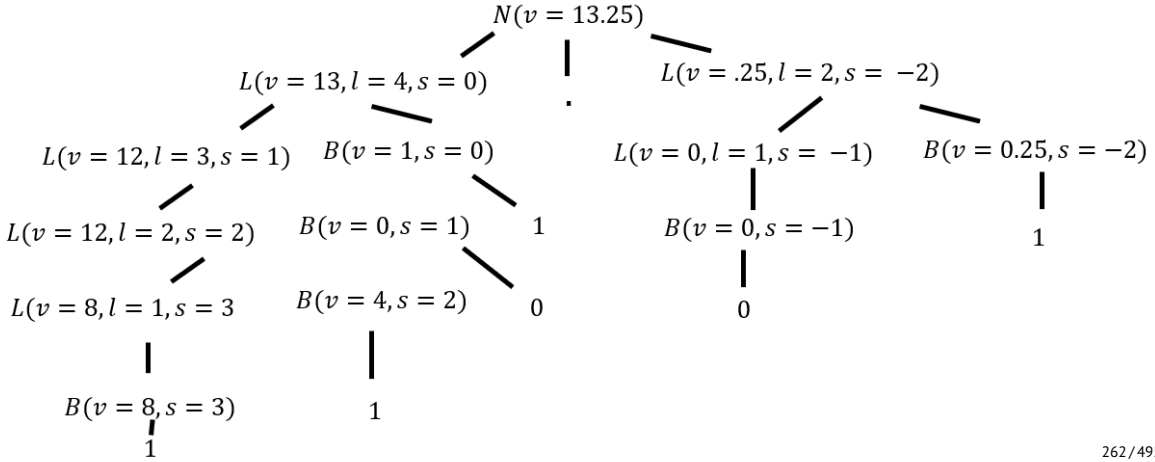
Пример.Нужно дать точное определение двоичной системы записи чисел. Для этого используем КСГ $G = \langle\langle B, L, N\rangle \langle "0\,"1\,"."\rangle, R, N\rangle$, где: $R = \langle B \to 0, B \to 1, L \to B, L \to LB, N \to L, N \to L \cdot L\rangle$

Зададим следующие атрибуты. B: $v(B) \in Q$ - значение, $s(B) \in Q$ - масштаб; L: $v(L) \in Q$ - значение, $s(L) \in Q$ - масштаб, $l \in Z$ - длина; N: $v(B) \in Q$ - значение.

Синтаксические правила	Семантические правила
$B \to 0$	$v(B) := 0$
$B \to 1$	$v(B) := 2^{s(B)}$
$L \to B$	$v(L) := v(B), s(B) := s(L), l(L) := 1$
$L_1 \to L_2 B$	$v(L_1) := v(L_2) + v(B), s(B) := s(L_1),$ $s(L_2) := s(L_1) + 1, l(L_1) := l(L_2) + 1$
$N \to L$	$v(N) := v(L), s(L) := 0$
$N \to L_1 \cdot L_2$	$v(N) := v(L_1) + v(L_2), s(L_1) := 0, s(L_2) := -l(L_2)$

1.5.2. Атрибутные грамматики (5/7)

Замечание. В данной семантике индексы используются для обозначения одного и того же нетерминала в рамках одного правила. 1 - индекс родительского узла, 2 - дочернего. Построим дерево разбора с сопоставленными каждой вершине атриубами - *атрибутированное* дерево разбора для представления числа 13.25 в двоичной системе счисления.



1.5.2. Атрибутные грамматики (6/7)

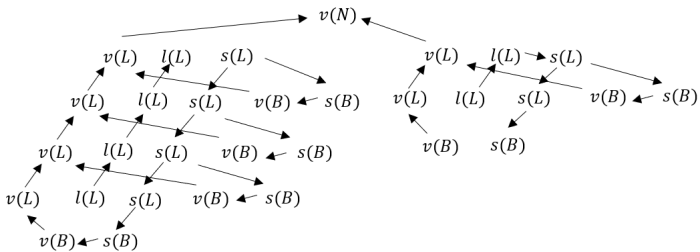
Получили $13.25_{10} = 1101.01_2$. Атрибуты $\upsilon(B), \upsilon(L), \upsilon(N), l(L)$ узлов вычисляются через знаяения атрибутов дочерних узлов. Такие атрибуты называют *синтезированными*. Атрибуты $s(B), s(L)$ узлов вычисляются через значения атрибутов узлов-предков. Это *унаследованные* атрибуты. Грамматики, в которых все атрибуты являются синтезированными, называют *S-атрибутными грамматиками*. Грамматики, в которых все наследуемые атрибуты зависят только от наследуемых атрибутов предка и/или от любых атрибутов левых братьев называют *L-атрибутными грамматиками*

Замечание. Всякая S-атрибутная грамматика является L атрибутной, поскольку в S-атрибутн грамматике нет наследуемых атрибутов. *Графом зависимостей* D_p $\forall p \in R$ является ориентированный граф, вершинами которого являются атрибуты символов, соответствующих p и проведена дуга из $X_m.\alpha$ в $X_k.\alpha_j$ тогда и только тогда, когда $\forall p \in R\, \exists f \mid X_m.\alpha_i = f(..., X_k.\alpha_j, ...)$ *Графом зависимостей* $D(\alpha)$ выводимой цепочки $S \Rightarrow^*_{AG} \alpha$ является ориентированный граф, полученный объединением графов зависимостей правил вывода p, применённых в α. Узлы, соответствующие одним и тем же атрибутам одного и того же символа в разных частях дерева, связанные правилами, отождествляются

1.5.2. Атрибутные грамматики (7/7)

Граф зависимостей $D'(T)$ - это ориентированный граф с множеством узлов $A(X)$, в котором дуга из α в α' существует тогда и только тогда, когда в полном графе зависимостей $D(T)$ существует ориентированный путь из узла (X, α) в узел (X, α') для дерева вывода T с корнем X.

Пример.Граф зависимостей для дерева из слайда 5/7.



Атрибутная грамматика называется *незацикленной* или *корректной*, если граф зависемостей $D'(T)$ не содержит циклов.

Замечание.В примере нет циклов. Грамматика корректна.

1.5.3. Синтаксически управляемый перевод

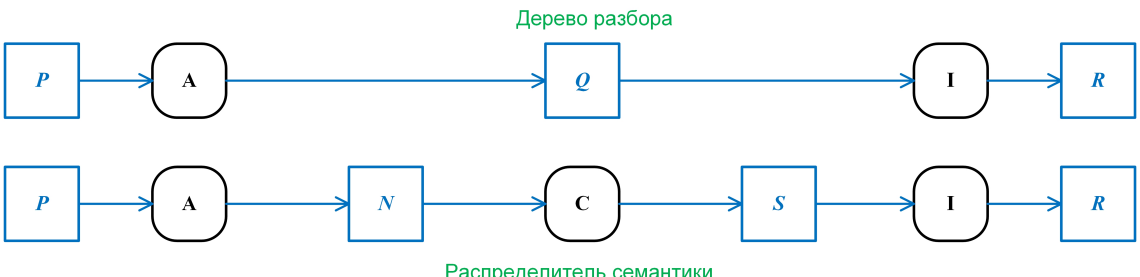
TODO Семантика перевода (трансляции).
Грамматика анализа и грамматика синтеза.
Пример = перевод в обратную польскую запись.
См. Ахо Ульман
Бурков Александр

1.5.2. Контекстные условия

TODO Что невыразимо в КСГ. Перенос части синтаксисиса в семантику. Проверка типов. Грамматики Ван-Вейнгаардена

1.5.5. Операционная семантика

TODO Абстрактная машина. Перевод в команды интерпретатора.
Семантические процедуры, распределитель семантик.



1.5.6. Венский метод определения языков программирования

TODO Колосков Александр. Доклад сделан, но материал для вклейки в слайды пока не готов

1.5.7. Денотационная семантика

TODO λ-исчисление. Денотационная семантика.

TODO ≈ 25 атомов. Петрунин

1.5.8. Исчисления программ

TODO Аксионы Хоара

Величко Арсений

TODO ≈ 25 атомов. Кто это сделал?

1.5.8. Исчисления программ (2/9)

Исчисление программ — формальная система для доказательства свойств программ, предложенная Ч. Хоаром (1969).

Замечание. Исчисление программ — это частный случай исчисления предикатов, в котором формулы описывают связь между состоянием программы до и после выполнения оператора.

Основная формула исчисления программ:

$$\{P\}\, S\, \{Q\}$$

где:

- P — *предусловие* (условие на состояние до выполнения S)
- S — оператор (фрагмент программы)
- Q — *постусловие* (условие на состояние после выполнения S)

1.5.8. Исчисления программ (3/9)

Содержательный смысл формулы $\{P\}\,S\,\{Q\}$:
Если до выполнения оператора S программа находилась в состоянии w, для которого выполнено предусловие $P(w)$, то после выполнения S программа перейдёт в состояние $w' = S(w)$, в котором выполнено постусловие $Q(w')$.

$$\{P\}\,S\,\{Q\} \stackrel{\text{Def}}{=} \forall\, w\ \ (P(w) \Longrightarrow \exists\, w'\ \ (w' = S(w)\ \&\ Q(w')))$$

Пример. $\{x = 5\}\,x := x + 1\,\{x = 6\}$ — корректная формула.
$\{x > 0\}\,x := x * 2\,\{x > 0\}$ — также корректная формула.
Замечание. Различают *частичную* корректность (если S завершается, то Q выполнено) и *полную* корректность (дополнительно доказывается завершаемость S).

1.5.8. Исчисления программ (4/9)

Основные аксиомы и правила вывода:

1. **Аксиома присваивания:** $\{P\}\, v := e\, \{P\ \&\ v = e\}$
2. **Правило последовательности:** $\dfrac{\{P\}\, S_1\, \{Q\} \quad \{Q\}\, S_2\, \{R\}}{\{P\}\, S_1; S_2\, \{R\}}\ 1$
3. **Правило условного оператора:** $\dfrac{\{P\ \&\ B\}\, S_1\, \{Q\} \quad \{P\ \&\ \neg B\}\, S_2\, \{Q\}}{\{P\}\ \textbf{if}\ B\ \textbf{then}\ S_1\ \textbf{else}\ S_2\, \{Q\}}\ 2$
4. **Правило цикла:** $\dfrac{\{I\ \&\ B\}\, S\, \{I\}}{\{I\}\ \textbf{while}\ B\ \textbf{do}\ S\, \{I\ \&\ \neg B\}}\ 3$
   где I — *инвариант цикла*

1.5.8. Исчисления программ (5/9)

Применение исчисления программ:

- Формальная верификация программ
- Доказательство корректности алгоритмов
- Автоматическая генерация тестов
- *Аннотирование программ* — добавление утверждений в виде комментариев

Отступление. На практике полностью формальное доказательство корректности трудоёмко и применяется редко. Однако неформальное обоснование с использованием идей Хоара (инварианты, пред- и постусловия) чрезвычайно полезно при разработке и отладке программ.

1.5.8. Исчисления программ (6/9)

Дополнительные правила вывода:

Правила эквивалентных преобразований условий:

$$\frac{P \Longleftrightarrow P_1 \quad \{P_1\}\, S\, \{Q_1\} \quad Q_1 \Longleftrightarrow Q}{\{P\}\, S\, \{Q\}}\, 4$$

Правила усиления предусловия и ослабления постусловия:

$$\frac{P \Longrightarrow P_1 \quad \{P_1\}\, S\, \{Q_1\} \quad Q_1 \Longrightarrow Q}{\{P\}\, S\, \{Q\}}\, 5$$

Замечание. Правило (5) позволяет использовать более сильное предусловие и более слабое постусловие, что упрощает доказательства.

1.5.8. Исчисления программ (7/9)

Пример. Рассмотрим *оператор обмена значениями* $x :=: y$.
Семантика этого оператора:

$$\{P(x,y)\}\, x :=: y\, \{P(y,x)\}$$

Рассмотрим фрагмент программы сортировки трёх значений:

if $x > y$ **then** $x :=: y$ **endif**
if $y > z$ **then** $y :=: z$ **endif**
if $x > y$ **then** $x :=: y$ **endif**

Цель: Доказать, что постусловием является $x \leqslant y \leqslant z$.

1.5.8. Исчисления программ (8/9)

Введём процедуру $S2(a,b) := \textbf{if } a > b \textbf{ then } a :=: b \textbf{ endif}$.

Свойство процедуры $S2$:

$$\{P(a,b)\}\, S2(a,b)\, \{(P(a,b) \vee P(b,a)) \,\&\, a \leqslant b\}$$

Доказательство.

1. $\{P(a,b) \,\&\, a > b\}\, a :=: b\, \{P(b,a) \,\&\, b > a\}$ аксиома
2. $P(b,a) \,\&\, b > a \Longrightarrow (P(a,b) \vee P(b,a)) \,\&\, a \leqslant b$ общезначимо
3. $\{P(a,b) \,\&\, a > b\}\, a :=: b\, \{(P(a,b) \vee P(b,a)) \,\&\, a \leqslant b\}$ правило 1
4. $P(a,b) \,\&\, \neg(a > b) \Longrightarrow (P(a,b) \vee P(b,a)) \,\&\, a \leqslant b$ общезначимо
5. $\{P(a,b)\}\, S2(a,b)\, \{(P(a,b) \vee P(b,a)) \,\&\, a \leqslant b\}$ правило 2

□

1.5.8. Исчисления программ (9/9)

Отступление. Формальное доказательство даже простой программы (сортировка трёх элементов — частный случай пузырьковой сортировки) оказывается громоздким. Доказательство всегда длиннее и сложнее самой программы.

Однако как программисты используют языки высокого уровня вместо машинного кода, так и математики используют менее формальные, но более выразительные методы.

Практическая ценность исчисления программ:

- Неформальное обоснование с использованием инвариантов полезно при разработке
- *Аннотирование программ* — вставка утверждений о состоянии в виде комментариев
- Современные инструменты (Dafny, Frama-C, SPARK) автоматизируют верификацию

Замечание. Подробное изложение с полным доказательством — см. раздел 3.3.2 «Исчислен программ».

TODO хороший план
Как перераспределить материал между частями курса?
Что можно сказать про «современные инструменты» ?

1.5.9. Преобразователи предикатов

TODO Преобразователи предикатов Дейкстры.
Слабейшие предусловия.
Завершаемость.
Величко Арсений

1.6. Трансляция языков программирования

TODO Лексический и синтаксичесикй анализ. Оптимизация синтаксического анализа.
Пошаговая компиляция и интерпретация. Обработка синтаксических ошибок.
Генерация кода. Оптимизация.
Построение моделей программ из кода. Потоковый анализ. Глобальная экономия памяти.

TODO Раздел повис в воздухе. Кто??? $\rightarrow$ ФАН

TODO Замечание ФАН 22 января
Расширить, обновить, согласовать с Соломатиным и перенести в главу 3.

2. Состояния и переходы

Алгоритм в неформальном смысле можно мыслить как *последовательность действий* (приводящих к изменению состояния) или как *переходы состояний* (происходящих в определённой последовательности). Эти две точки зрения на алгоритмы в известном смысле двойственны и взаимодополнительны. Последовательности действий хорошо всем известны в форме машин с неограниченными регистрами, блок-схем или императивных языков программирования. В этой главе рассматриваются переходы состояний.

Конечный автомат является, видимо, наипростейшей возможной моделью устройств обработки и преобразования информации, относящейся к концепции переходов состояний. Несмотря на свою простоту, понятие конечного автомата оказывается достаточно содержательным, чтобы послужить основой большого числа важнейших приложений.

Замечание. Наряду с термином «конечный автомат» (finite automaton) в англоязычной литературе употребляются термины *машина состояний* (state machine) и *машина переходов состояний* (state transition machine). Эти термины обозначают один и тот же класс объектов, но различаются по контексту употребления. В формальном математическом контексте обычно говорят «конечный автомат», а в неформальном программистском контексте говорят «машина состояний».

2.1. Конечные автоматы Мили и Мура

Так же, как и в случае с графами, для конечных автоматов имеется множество определений, различающихся некоторыми деталями. Мы начнем с классических определений автоматов, принадлежащих Мили[8] и Муру[9].

[8] George H. Mealy (1927–2010)
[9] Edward F. Moore (1925–2003)

2.1.1. Автомат Мили (1/2)

Конечным автоматом (Мили) называется система $S = \langle A, Q, B, \delta, \lambda \rangle$, где

1. $A = \{a_1, \ldots, a_m\}$ — конечный алфавит *входных символов (входной алфавит)*;
2. $Q = \{q_1, \ldots, q_n\}$ — конечный алфавит *состояний*;
3. $B = \{b_1, \ldots, b_k\}$ — конечный алфавит *выходных символов (выходной алфавит)*;
4. $\delta : Q \times A \to Q$ — *функция переходов*, определяющая по состоянию и входному символу новое состояние;
5. $\lambda : Q \times A \to B$ — *функция выходов*, определяющая по состоянию и входному символу выходной символ.

Данная математическая модель может быть использована для абстрактного описания поведения в следующем смысле. Если автомат находится в состоянии $q_i \in Q$ и на вход поступает символ $a_j \in A$, то автомат переходит в состояние $q_x = \delta(q_i, a_j)$ и выдает на выход символ $b_y = \lambda(q_i, a_j)$.

Замечание. В базовое определение автомата не включено понятие начального состояния (см. п. 2.1.5). Это позволяет задавать одним описанием сразу некоторый класс родственных объектов, но требует дополнительного внимания при разборе примеров.

2.1.1. Автомат Мили (2/2)

Пример. Автомат $\langle \{1\}, \{0,1\}, \{0,1\}, \delta(0,1) := 1, \delta(1,1) := 0, \lambda(0,1) := 1, \lambda(1,1) := 0 \rangle$ с двумя состояниями по входному символу инвертирует своё состояние и выдаёт его в качестве выходного символа. Такой автомат называется *триггер*.

Отступление. Идею конечного автомата можно интерпретировать разными способами. Можно считать, что автомат последовательно читает из входного потока данных входные символы, записывая выходные символы в выходной поток данных. С другой стороны, можно считать, что входной символ — это событие, которое произошло в системе, а выходной символ — это процедура реакции, которую нужно выполнить в ответ на возникновение данного события в данном состоянии. С третьей стороны, можно считать, что автомат — это устройство, которое работает «по тактам», опрашивая на каждом такте состояния своих входных регистров (входной символ) и меняя состояния выходных регистров (выходной символ).

Ценность понятия «конечный автомат» заключается в том, что оно отлично подходит для описания всех здесь перечисленных, а также многих других конкретных интерпретаций.

Пример. Автомат $\langle \{0,1\}, \{1\}, \{0,1\}, \delta(1,0) := 1, \delta(1,1) := 1, \lambda(1,0) := 1, \lambda(1,1) := 0 \rangle$ с одним состоянием инвертирует входной символ и выдаёт его в качестве выходного символа. Такой автомат называется *инвертор*.

2.1.2. Задание автомата таблицей (1/2)

Поскольку функции δ и λ определены на конечных множествах, их можно задать *таблицей конечного автомата*:

	a_1	$\dots$	a_j	$\dots$	a_m
q_1					
$\dots$					
q_i	$\dots$	$\dots$	q_x, b_y	$\dots$	$\dots$
$\dots$					
q_n					

где $q_x = \delta(q_i, a_j)$, а $b_y = \lambda(q_i, a_j)$.

Пример. Пусть N — натуральное число, а строка $\#a_1 \dots a_s$ — запись числа в двоичной системе счисления, $a_i \in \{0, 1\}$. Рассмотрим автомат $S = \langle A, Q, B, \delta, \lambda \rangle$, где $A = \{0, 1, \#\}$, $Q = \{1, 2, 3\}$, $B = \{0, 1, \#\}$, а функции δ и λ заданы следующей таблицей:

	0	1	#
1	3, 1	2, 0	1, #
2	3, 1	2, 0	1, 1
3	3, 0	3, 1	1, #

2.1.2. Задание автомата таблицей (2/2)

Пример. (Продолжение) Если автомат находится в состоянии 1 и на его вход подана последовательность символов $a_s, a_{s-1}, \ldots, a_1, \#$ (запись числа N, прочитанного справа налево; символ # является маркером, ограничивающим запись), то автомат выдаст последовательность цифр в двоичной записи числа $N+1$ (также в порядке справа налево). Например, в следующей таблице приведен протокол работы автомата при обработке записи чисел 5 ($101\#$) слева и 7 ($111\#$) справа.

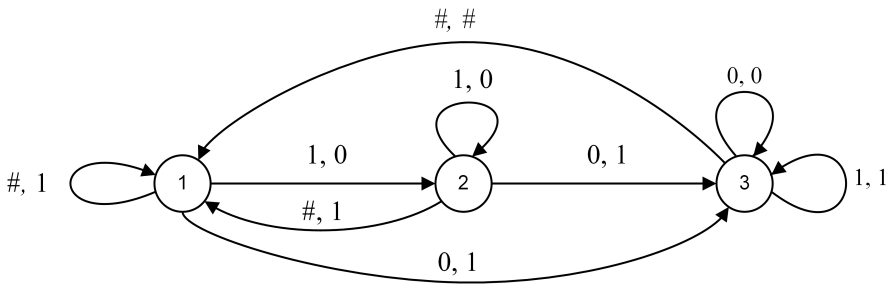
a	q	δ	λ	a	q	δ	λ
1	1	2	0	1	1	2	0
0	2	3	1	1	2	2	0
1	3	3	1	1	2	2	0
$\#$	3	1	$\#$	$\#$	2	1	1

Отступление. Конечное число состояний позволяет «запомнить и учесть» только *конечное* число поступивших символов или событий. Поэтому невозможно составить конечный автомат, который вычисляет $N+1$ для *любого* числа N, если цифры числа поступают на вход автомата в привычном порядке — слева направо. В этом проявляется ограниченность конструкции конечного автомата.

2.1.3. Задание автомата графом переходов состояний (1/2)

Конечный автомат можно задать *графом переходов* (*графом переходов состояний*, *диаграммой состояний*) — ориентированным нагруженным псевдомультиграфом, узлы которого помечены состояниями автомата и, если $\delta(q_i, a_j) = q_x$ и $\lambda(q_i, a_j) = b_y$, то в графе имеется дуга (q_i, q_x), помеченная парой (a_j, b_y).

Пример. Диаграмма графа переходов автомата из предыдущего примера ($N := N + 1$) представлена на рисунке.

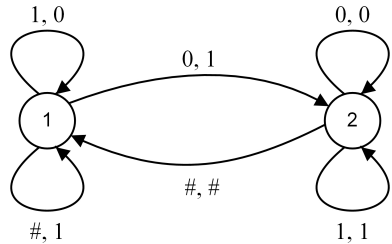


2.1.3. Задание автомата графом переходов состояний (2/2)

Одно и то же (или очень близкое) поведение может быть задано существенно различными автоматами, подобно тому, как одну и ту же функцию можно запрограммировать разными способами.

Пример. На рисунке приведены диаграмма и таблица автомата, выполняющего то же преобразование ($N := N + 1$), что и автомат из предыдущих примеров.

	0	1	#
1	2, 1	1, 0	1, #
2	2, 0	2, 1	1, #



Отступление. Поскольку автомат может служить средством описания алгоритмов, составление автоматов является программированием. В этом контексте рисование диаграмм является одним из немногих примеров визуального программирования, где слово «визуальный» употреблено по праву.

2.1.4. Полные и частичные автоматы (1/2)

Автомат называется *полным*, если функции δ и λ тотальны, то есть определены для всех пар значений аргументов. В таблице полного автомата все клетки заполнены, а граф переходов является регулярным графом степени $m = |A|$ (то есть в любом состоянии исходящие дуги определены для всех входных символов).

Замечание. Для каждого состояния все исходящие дуги переходов должны быть помечены *разными* входными символами — это *условие детерминированности* графа переходов конечного автомата. В некоторых случаях оказывается полезным рассмотрение графов, подобных графам переходов, но с невыполненным условием детерминированности. В этом случае из одного состояния могут выходить несколько дуг, помеченных одним и тем же входным символом. Такая конструкция называется *недетерминированным автоматом*. Если не оговорено противное, далее в этой главе подразумевается, что условие детерминированности выполнено и рассматриваются детерминированные автоматы.

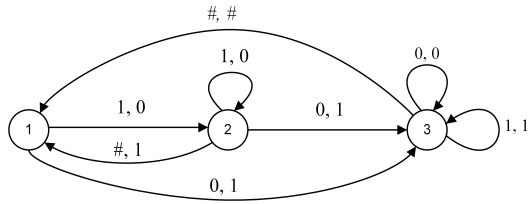
Автомат называется *частичным*, если δ и/или λ — частичные функции. В таблице частичного автомата некоторые клетки могут быть не заполнены, а в графе переходов различные состояния могут иметь разное количество исходящих дуг.

2.1.4. Полные и частичные автоматы (2/2)

Отступление. Частичный автомат может быть неработоспособен для некоторых последовательностей входных символов. Однако это обстоятельство не мешает практическому применению частичных автоматов. Дело в том, что не все возможные последовательности входных символов фактически реализуются. Могут быть такие состояния, в которых определённые события в принципе невозможны, а потому нет нужды предусматривать реакцию на них.

Пример. Автоматы из предыдущих примеров допускают *любые* последовательности символов 0, 1, $\#$. Но если на вход автомата подается *правильная* двоичная запись числа (то есть непустая последовательность 0 и 1, которая заканчивается $\#$), то достаточно следующего частичного автомата (неопределённые значения функций δ и λ обозначены прочерками):

	0	1	#
1	3, 1	2, 0	—, —
2	3, 1	2, 0	1, 1
3	3, 0	3, 1	1, #



Для упрощения изложения далее, если не оговорено противное, автомат считается полным.

2.1.5. Начальное и заключительные состояния (1/2)

Автомат называется *инициальным*, если в нем выделено *одно* состояние, которое называется *начальным*. Обычно начальному состоянию дают наименьший номер: 1 (или 0). Инициальный автомат S с начальным состоянием q_1 обозначим $S_{q_1} = \langle A, Q, B, \delta, \lambda, q_1 \rangle$. Если индекс q_1 подразумевается, то его можно опустить. Начальное состояние — это то состояние, в котором находится автомат, когда на вход поступает первый символ. На диаграммах графов переходов мы будем отмечать начальное состояние входной стрелкой, приходящей извне.

Для данного автомата S существуют n инициальных автоматов, где $n = |Q|$. Поведение автомата может существенно зависеть от того, какое состояние является начальным.

Пример. Если в автомате примера п. 2.1.2 на вход автомата подать последовательность цифр в записи числа N в двоичной системе счисления (справа налево) и при этом считать, что начальным состоянием является состояние 1 или 2, то автомат выдает запись числа $N+1$, а если начальным состоянием является 3, то автомат выдает запись числа N, то есть просто копию входной последовательности.

В автомате может быть выделено некоторое подмножество состояний, которые называются *заключительными*.

2.1.5. Начальное и заключительные состояния (2/2)

Иногда удобно считать, что если автомат переходит в заключительное состояние, то он прекращает свою работу (то есть более не реагирует на входные символы). На диаграммах графов переходов мы будем отмечать заключительные состояния жирной границей.

Пример. В автомат примера п. 2.1.2 добавлено заключительное состояние 4, так что автомат завершает свою работу сразу по получении символа $\#$.

	0	1	$\#$
1	3,1	2,0	$4,-$
2	3,1	2,0	4,1
3	3,0	3,1	$4,-$
4	$-,-$	$-,-$	$-,-$

Замечание. Для заключительных состояний функции δ и λ в таблице автомата задавать не имеет смысла, поэтому строки для заключительных состояний пусты. Их можно не включать в таблицу, считая заключительными все состояния с номерами большими, чем фактическое количество строк в таблице. В графе переходов узлы заключительных состояний не имеют исходящих дуг.

2.1.6. Автоматное преобразование (1/2)

Функции переходов δ и выходов λ можно расширить, чтобы они воспринимали в качестве второго аргумента *последовательности* входных символов. Определим *расширенную функцию переходов* $\delta^*\colon Q \times A^* \to Q$ и *расширенную функцию выходов* $\lambda^*\colon Q \times A^* \to B$ следующим образом:

$$\forall\, a \in A\ (\forall\, q \in Q\ (\forall\, \alpha \in A^*\ (\delta^*(q,a) = \delta(q,a)\ \&\ \delta^*(q,\alpha a) = \delta(\delta^*(q,\alpha),a))))\,,$$

$$\forall\, a \in A\ (\forall\, q \in Q\ (\forall\, \alpha \in A^*\ (\lambda^*(q,a) = \lambda(q,a)\ \&\ \lambda^*(q,\alpha a) = \lambda(\delta^*(q,\alpha),a))))\,.$$

Инициальный автомат $S = \langle A, Q, B, \delta, \lambda, q\rangle$ определяет *автоматное преобразование* $\mathcal{S}_q\colon A^* \to B^*$ следующим образом:

$$\forall\, a \in A\ (\mathcal{S}_q(a) := \lambda(q,a)\ \&\ \forall\, \alpha \in A^*\ (\mathcal{S}_q(\alpha a) := \mathcal{S}_q(\alpha) + \lambda(\delta^*(q,\alpha),a)))\,,$$

где $+$ обозначает операцию *конкатенации (сцепления) строк*.

Замечание. Если начальное состояние известно из контекста, то индекс q можно опустить.

2.1.6. Автоматное преобразование (2/2)

Автоматное преобразование обладает следующими свойствами:

1. $\forall\, \alpha \in A^*\ \ (|\alpha| = |\mathcal{S}(\alpha)|)$ — сохранение длины;
2. $\forall \alpha \in A^*\ \ (\alpha = \alpha_1\alpha_2\ \&\ \beta = \mathcal{S}(\alpha)\ \&\ \beta = \beta_1\beta_2\ \&\ |\alpha_1| = |\beta_1| \Longrightarrow \mathcal{S}(\alpha_1) = \beta_1)$ — образ префикса есть префикс образа.

Замечание. В общем случае свойство 2 не выполняется для постфикса.

Автоматное преобразование нетрудно реализовать программно, например, как показано в следующем алгоритме. В этом алгоритме массивы δ и λ, задающие соответственно функции переходов и выходов, считаются глобальными.

Вход: входное слово α : **array** $[1..s]$ **of** A, начальное состояние $q \in Q$;
Выход: выходное слово β : **array** $[1..s]$ **of** B.

for i **from** 1 **to** s **do**
  $\beta[i] := \lambda\,[q, \alpha[i]]$ // выдать очередной выходной символ
  $q := \delta\,[q, \alpha[i]]$ // перейти в следующее состояние
end for

Замечание. Чтобы упростить обозначения, далее мы обозначаем автоматное преобразование той же буквой, что и сам автомат, если при этом не возникает недоразумений.

2.1.7. Изоморфизм автоматов (1/2)

Говорят, что состояние q_i *достижимо* из состояния q_j, если $\exists\, \alpha \in A^* \ (q_i = \delta^*(q_j, \alpha))$. Обозначение: $\langle q_j, q_i \rangle$. Автомат называется *сильно связным*, если $\forall\, q_i, q_j \ (\langle q_i, q_j \rangle)$. Сильно связный автомат имеет сильно связный граф переходов.

В инициальном автомате состояние называется *недостижимым*, если оно недостижимо из начального состояния. Недостижимые состояния являются заведомо лишними — их присутствие (или отсутствие) не влияет на работу автомата.

Пусть заданы два автомата $S = \langle A_S, Q_S, B_S, \delta_S, \lambda_S \rangle$ и $T = \langle A_T, Q_T, B_T, \delta_T, \lambda_T \rangle$. Говорят, что существует *гомоморфизм* из S в T (обозначение $S \rightsquigarrow T$), если существуют функции

$$f\colon A_S \to A_T, \quad g\colon Q_S \to Q_T, \quad h\colon B_S \to B_T$$

такие, что

$$\forall\, a \in A_S, q \in Q_S, b \in B_S \ (\delta_T(g(q), f(a)) = g(\delta_S(q, a)) \ \&\ \lambda_T(g(q), f(a)) = h(\lambda_S(q, a)))\,.$$

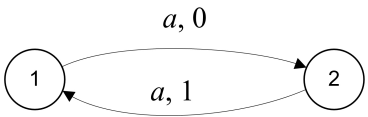
Если функции f, g, h взаимно-однозначны, то автоматы S и T *изоморфны* (обозначение $S \sim T$).

2.1.7. Изоморфизм автоматов (2/2)

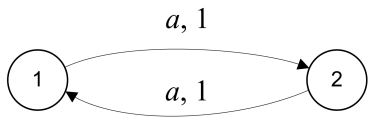
Изоморфизм автоматов означает, что можно так переименовать буквы в алфавитах A, Q и B, что таблицы автоматов совпадут.

Изоморфизм графов переходов (без учета нагрузки на дугах) не означает изоморфизма автоматов.

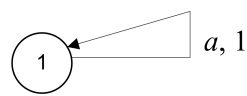
Пример. На рисунке автоматы S и T не изоморфны, хотя их графы переходов изоморфны. Существует очевидный гомоморфизм из T в U.



S



T



U

Действительно, $S \not\sim T$, поскольку легко подобрать функции f и g, которые являются биекциями (например, тождественные функции), но невозможно подобрать взаимно-однозначную функцию h, так как $|\{0,1\}| \neq |\{1\}|$. С другой стороны, $T \rightsquigarrow U$, поскольку достаточно взять функции f и h тождественными, и положить $g(1) := 1, g(2) := 1$.

2.1.8. Автомат Мура (1/2)

Конечный автомат называется *автоматом Мура*, если функция выходов не зависит от входов, то есть $\lambda\colon Q \to B$. Обычно в этом случае функция обозначается μ и называется *функцией пометок*, так как каждому состоянию сопоставляется пометка — выходной символ. В таблице выходные символы пишут рядом с состояниями, а на диаграмме графа переходов — в узлах, оставляя на дугах только входные символы.

Автомат Мура работает следующим образом:

Вход: α : **array** $[1..s]$ **of** A, $q \in Q$;
Выход: β : **array** $[1..s]$ **of** B.

```
for i from 1 to s do
    q := δ(q, α[i])  // переходим в состояние
    β[i] := μ(q)  // выдаем его пометку
end for
```

Очевидно, что автомат Мура является автоматом Мили. В следующем разделе показано, что возможности этих двух видов автоматов совпадают.

2.1.8. Автомат Мура (2/2)

Пример. Ниже приведены диаграмма и таблица для автомата Мура, выполняющего то же преобразование $(N \to N+1)$, что и автомат Мили из примера п. 2.1.2.

Переименование пар (состояние, выходной символ) из примера п. 2.1.2:
$(1, \# \to 1/\#)$
$(1, 1 \to 2/1)$
$(2, 0 \to 3/0)$
$(3, 0 \to 4/0)$
$(3, 1 \to 5/1)$.

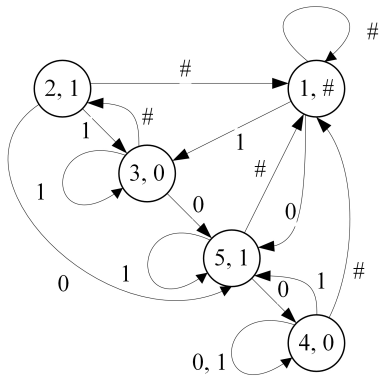


Таблица автомата Мура

	0	1	#
1/#	5	3	1
2/1	5	3	1
3/0	5	3	2
4/0	4	5	1
5/0	4	5	1

Понятия начальных и заключительных состояний, расширенных функций переходов и пометок, а также автоматного преобразования очевидным образом распространяются на случай автомата Мура.

2.1.A. Онтология формальных языков, грамматик и автоматов

2.1.Б. Темы для самостоятельной работы студентов

1. СНИРС => Современное состояние автоматного программирования
   Автоматное программирование — термин, предложенный А.А. Шалыто.
   На сайте is.ifmo.ru можно найти множество примеров. Требуется подготовить *качественный* пересказ одного из современных достижений.
2. СНИРС => Блок-схемы vs. графы переходов состояний
   В магистерской диссертации Александры Лукъяновой вопрос поставлен и решён. Требуется актуализировать исследование и, возможно, продвинуть вперёд.
3. СНИРС => … автоматы …
   Существует множество моделей, в которых используется идеи автоматов (переходов состояний). Например, абстрактные машины состояний Гуревича, клеточные автоматы, игра Конвея «жизнь», … . Требуется рассказать что-то в обозначениях и терминах этого курса.

2.2. Распознавание языков конечными автоматами

Конечные автоматы — это один из способов описания алгоритмов. Данный способ не универсален, но удивительно широко применим. В этом разделе применимость конечных автоматов иллюстрируется на примере задачи распознавания языков автоматами — одной из самых известных задач программирования.

2.2.1. Автоматы-распознаватели (1/2)

Рассмотрим автомат Мура с выходным алфавитом $B = \{0, 1\}$. Состояния, помеченные 1, будем называть *заключительными*. Функцию пометок μ можно задать, указав подмножество заключительных состояний $F \subset Q$. Такой автомат $S = \langle A, Q, \delta, F \rangle$ называется автоматом *без выходов*. Инициальный автомат без выходов $S = \langle A, Q, \delta, q_0, F \rangle$ называется *автоматом-распознавателем* (здесь $q_0 \in Q$ и $F \subset Q$).

Замечание. В данном случае попадание в заключительное состояние *не* подразумевает прекращения работы автомата.

Рассмотрим конечный алфавит A и язык $L \subset A^*$ над алфавитом A. Говорят, что язык L *распознается* автоматом $S = \langle A, Q, \delta, q_0, F \rangle$, если

$$\forall\, \alpha \in A^* \ \left(\alpha \in L \iff \delta^*(q_0, \alpha) \in F\right).$$

Другими словами, распознавание языка L автоматом S означает следующее. Если на вход автомата S подать «правильное» слово $\alpha \in L$, то автомат перейдет в заключительное состояние $f \in F$ (в этом случае говорят, что слово α *допускается* автоматом S). Если же на вход автомата S подать «неправильное» слово $\alpha \notin L$, то автомат перейдет в состояние g, которое не является заключительным, $g \notin F$ (говорят, что слово α *не допускается* автоматом S).

2.2.1. Автоматы-распознаватели (2/2)

Всякому автомату-распознавателю S соответствует язык L, распознаваемый этим автоматом. А именно, язык L — это множество значений всех путей (см. п. 1.2.3) из начального состояния в заключительное.

Замечание. Начальное состояние одно, а заключительных состояний может быть несколько, но это обстоятельство никак не ограничивает общность модели.

Таким образом, мы видим, что языки, распознаваемые конечными автоматами, существуют. Из общей теории множеств следует, что языки, не распознаваемые конечными автоматами, также существуют. Действительно, с одной стороны, множество символов A конечно, множество слов A^* счётно, а множество языков 2^{A^*} — не счётно. С другой стороны, множество автоматов с k состояниями конечно, поскольку это число различных орграфов с k узлами, обладающих свойством автоматной детерминированности. Но счётное число конечных множеств счётно, поэтому множество автоматов счётно. Между несчётным и счётным множествами взаимнооднозначного соответствия быть не может.

Возникает вопрос: каково же множество языков, распознаваемых конечными автоматами?

2.2.2. Распознаватели конечных языков (1/4)

Начнём описание счётного подмножества распознаваемых автоматами языков с тривиальных наблюдений.

Теорема. *Любой конечный язык распознается конечным автоматом.*

Доказательство. Пусть дан язык $L = \{\alpha_1, \ldots, \alpha_n\}$. Сгруппируем слова $\alpha_1, \ldots, \alpha_n$ по первой букве, так что в одну группу G_{a_i} попадут все слова языка L, начинающиеся с одной и той же буквы a_i. Множество всех первых букв обозначим A_1.

$$A_1 \subset A,\ L = \bigcup_{a_i \in A_1} G_{a_i},\ G_{a_i} := \{\alpha \mid \alpha \in L\ \&\ \alpha = a_i\beta\ \&\ a_i \in A_1\}.$$

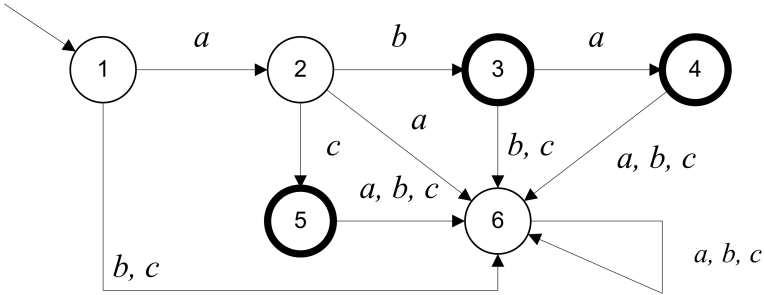
Множество непустых постфиксов (без первой буквы) слов из группы G_{a_i} обозначим L_i ($L_i := \{\beta \mid \alpha = a_i\beta\ \&\ \alpha \in G_{a_i}\ \&\ \beta \neq \varepsilon\}$).

Построим граф переходов автомата S следующим образом. Введем начальное состояние q_0 и состояние недопустимости $\bar{q}$. Введем состояния q_i для всех G_{a_i}. Проведем дуги из q_0 в q_i для всех $a_i \in A_1$ и дуги из q_0 в $\bar{q}$ для всех остальных символов. Если в группе G_{a_i} есть однобуквенное слово a_i, то считаем q_i заключительным состоянием. □

2.2.2. Распознаватели конечных языков (2/4)

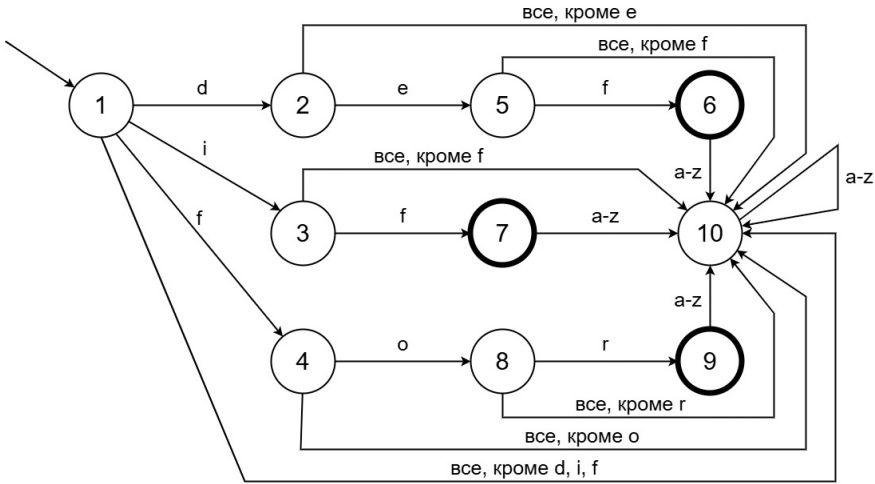
Доказательство. (Окончание) Далее рекурсивно применим эту процедуру к каждому языку L_i, используя q_i в качестве начального узла (состояние недопустимости $\bar{q}$ можно использовать одно для всех языков). Процесс, очевидно, закончится, так как язык L конечен. Построенный граф является графом переходов автомата, распознающего язык L. □

Пример. Для конечного языка $L = \{ab, ac, aba\}$ применение алгоритма, изложенного в доказательстве предыдущей теоремы, дает автомат-распознаватель, представленный на рисунке. Здесь состояние 1 — начальное, а состояние 6 соответствует недопустимости слова.



2.2.2. Распознаватели конечных языков (3/4)

Пример. Рассмотрим конечный язык ключевых слов Python: $L = \{\texttt{def}, \texttt{if}, \texttt{for}\}$. Здесь состояние 1 — начальное, а состояние 10 соответствует недопустимости слова. Алфавит A предполагается состоящим из строчных букв английского алфавита $\{a, b, \ldots, z\}$.



2.2.2. Распознаватели конечных языков (4/4)

Замечание. Напомним, что на рисунках здесь и далее использованы следующие обозначения. Начальное состояние отмечено входящей извне дугой. Заключительное состояние обозначено жирным кружком. Множество символов, написанных на дуге, означают множество соответствующих мультидуг.

Отступление. Если отбросить состояние недопустимости, то граф переходов автомата-распознавателя конечного языка окажется ордеревом. Такое ордерево является одним из самых эффективных способов представления конечных словарей (множеств слов) в компьютере. Время поиска слова в таком словаре пропорционально *длине* слова и *не* зависит от общего количества слов в словаре. Если словарь обладает *свойством префикса*, то есть ни одно слово в словаре не является собственным началом никакого другого слова, то листья дерева и только они оказываются заключительными состояниями. Однако свойство префикса не является обязательным: словарь одинаково хорошо работает для *любого* конечного множества слов. Более того, слова можно выровнять по длине, дополнив каким-либо нейтральным символом (пробелом) до максимальной длины, и словарь примет форму выровненного дерева, в котором свойство префикса выполнено автоматически.

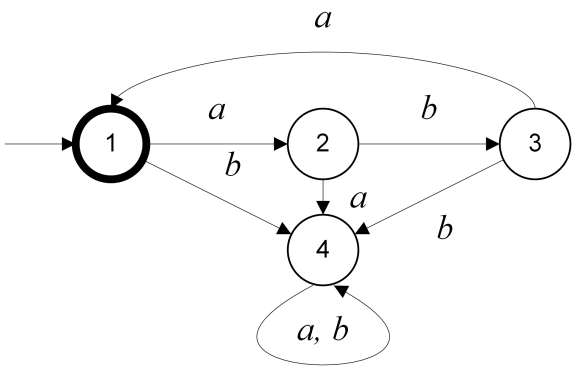
2.2.3. Распознаватели бесконечных языков (1/6)

Множество языков, распознаваемых автоматами, не исчерпывается конечными языками — автоматы могут распознавать и бесконечные языки.

Пример. Автомат на рисунке распознает бесконечный язык

$$L = \{\varepsilon, aba, abaaba, \ldots, (aba)^n, \ldots\} = \{(aba)^n \mid n \geqslant 1\}.$$

Состояние 1 является начальным и единственным заключительным, состояние 4 соответствует недопустимости.



2.2.3. Распознаватели бесконечных языков (2/6)

Язык данного примера эффективно порождается процедурой, описанной в следующем алгоритме.

$\alpha := \varepsilon$
while true do
  yield α
  $\alpha := \alpha +$ “aba”
end while

Однако существуют языки, в том числе эффективно порождаемые, которые не распознаются никаким конечным автоматом.

Говорят, что автомат *распознает* (бесконечную) последовательность букв, если он распознает язык, состоящий из всех префиксов данной последовательности.

Теорема. *Никакая бесконечная непериодическая последовательность не распознаваема конечным автоматом.*

Доказательство. От противного. Пусть бесконечная непериодическая последовательность $\alpha = a_{i_1} a_{i_2} \dots$ распознается автоматом $S = \langle A, Q, \delta, q_0, F \rangle$ с n состояниями. □

2.2.3. Распознаватели бесконечных языков (3/6)

Доказательство. (Продолжение) Заметим, что для любого начального отрезка $a_{i_1} \dots a_{i_j}$ состояние $q_{i_j} = \delta^*(q_0, a_{i_1} \dots a_{i_j})$ — заключительное, то есть автомат все время проходит через заключительные состояния. Возьмем начальный отрезок последовательности α длины больше n и рассмотрим соответствующую последовательность состояний автомата. Поскольку $|Q| = n$, в этой последовательности есть одинаковые состояния q_{i_j} и $q_{i_{j+s}}$, то есть $\delta^*(q_{i_j}, a_{i_{j+1}} \dots a_{i_{j+s}}) = q_{i_j}$. Тогда построим периодическую последовательность $\alpha_1 := a_{i_1} \dots a_{i_j}(a_{i_{j+1}} \dots a_{i_{j+s}})$. Автомат S для всех начальных отрезков последовательности α_1 будет находиться в заключительных состояниях, то есть автомат не отличает α от α_1 и, таким образом, не распознает α. □

Пример. Язык

$$L = \{a, ab, aba, abab, ababb, ababba, ababbab, ababbabb, ababbabbb, ababbabbbabbbb, \dots\},$$

который описывается бесконечной непериодической последовательностью

$$ababbabbbabbbb \dots,$$

не распознается никаким конечным автоматом, как видно из предыдущей теоремы.

2.2.3. Распознаватели бесконечных языков (4/6)

Однако этот язык эффективно порождается следующим простым алгоритмом.

$\alpha := \varepsilon$; $n := 1$
while true do
  $\alpha := \alpha +$ “a”
  yield α
  for i **from** 1 **to** n **do**
    $\alpha := \alpha +$ “b”
    yield α
  end for
  $n := n + 1$
end while

В двух последних параграфах приведены примеры языков, распознаваемых и нераспознаваемых конечными автоматами. К сожалению, не существует общего метода определения того, можно ли распознать произвольный заданный язык конечным автоматом.

2.2.3. Распознаватели бесконечных языков (5/6)

Теорема. *Свойство языка «быть распознаваемым конечным автоматом» алгоритмически неразрешимо.*

Доказательство. От противного. Предположим, существует алгоритм-анализатор $\mathbf{R}$, который для любого генератора определяет, является ли порождаемый им язык регулярным. Пусть $\mathbf{M}$ — произвольная машина Тьюринга (см. 2.5.1), ω — входное слово. Построим процедуру-генератор $\mathbf{P}$ над алфавитом $A = \{a, b\}$:

$k := 1$
while true do
  Симулировать работу $\mathbf{M}$ на ω ровно k шагов (определение шагов МТ см. 2.5.1).
  if $\mathbf{M}$ остановилась за эти k шагов **then**
    stop
  end if
  yield $a^k b^k$
  $k := k + 1$
end while

2.2.3. Распознаватели бесконечных языков (6/6)

Доказательство. (Продолжение) Рассмотрим язык $L(\mathbf{P})$, порождаемый этой процедурой. Возможны два случая:

1. $\mathbf{M}$ зацикливается на ω: в этом случае симуляция никогда не обнаруживает остановку, процедура порождает все слова $a^1b^1, a^2b^2, \ldots$.

$$L = \{a^k b^k \mid k \geqslant 1\}$$

   Это бесконечный язык, который нерегулярен (по лемме о разрастании).

2. $\mathbf{M}$ останавливается через k шагов: в этом случае порождается конечное множество

$$L = \{a^1b^1, \ldots, a^{k-1}b^{k-1}\}$$

   Любой конечный язык регулярен (см. 2.2.2).

Таким образом: $L(\mathbf{P})$ регулярен тогда и только тогда, когда $\mathbf{M}$ останавливается на ω. Другими словами, имея $\mathbf{R}$, мы могли бы решить проблему остановки машины Тьюринга, которая известна как алгоритмически неразрешимая. Получили противоречие. □

2.2.4. Построение автомата по грамматике (1/3)

Мы видим, что в общем случае невозможно узнать, можно распознать язык автоматом или нельзя.

Однако, если для задания языка используется какой-нибудь конкретный способ, иногда по свойствам *описания* языка можно сделать вывод о свойствах самого языка.

В предыдущей главе рассмотрены два средства задания языков: формальные порождающие грамматики и регулярные выражения. В этом разделе устанавливается эквивалентная связь между автоматами-распознавателями, регулярными выражениями и автоматными грамматиками.

Первый шаг: покажем, что по автоматной грамматике можно построить автомат, распознающий язык, порождённый грамматикой. Входной алфавит автомата — это терминальный алфавит грамматики T, а множество состояний состоит из нетерминалов N, начального состояния q_0 и состояния недопустимости $\bar{q}$.

2.2.4. Построение автомата по грамматике (2/3)

Вход: Леволинейная автоматная грамматика $G = \langle N, T, R, I \rangle$.

Выход: Автомат-распознаватель $S = \langle T, Q, \delta, q_0, F \rangle$, распознающий язык $L(G)$.

$Q := N \cup \{q_0\} \cup \{\bar{q}\}$ //по определению
for $r \in R$ **do**
  if $r = A \to a$ **then**
    $\delta(q_0, a) := \delta(q_0, a) \cup A$ // добавляем переход $q_0 \xrightarrow{a} A$
  else
    $\delta(B, a) := \delta(B, a) \cup A$ // добавляем переход $B \xrightarrow{a} A$
  end if
end for
for $q \in Q$ **do**
  for $a \in T$ **do**
    if $\neg\exists\delta(q, a)$ **then**
      $\delta(q, a) := \bar{q}$ // недопустимые символы
    end if
  end for
end for
$F := \{I\}$

2.2.4. Построение автомата по грамматике (3/3)

Обоснование. Леволинейная автоматная грамматика имеет правила вида $A \to a$ и $A \to Ba$. Для правила $A \to a$ в автомат добавляется переход $q_0 \xrightarrow{a} A$, а для правила $A \to Ba$ в автомат добавляется переход $B \xrightarrow{a} A$. После этого, для того, чтобы автомат был полным, добавляются недостающие переходы в состояние недопустимости $\bar{q}$. Построенный автомат S действительно распознаёт язык, порождаемый грамматикой G
Пусть $I \Rightarrow_G^* \alpha$ . . . .TODO доделать

□

В общем случае такой автомат может быть недетерминированным. Тогда его нужно преобразовать в детерминированный вид (аналогично тому, как это делается для недетерминированных источников, см. п. 1.2.5, исключить недостижимые состояния и минимизировать.

Пример.

TODO исправить обозначения
Пример с автоматом для чисел . . . .
причём тот же автомат подходит к 1.2.4

2.2.5. Синтез автоматов (1/3)

Задача синтеза автомата — это задача построения автомата для заданного языка.

Прежде всего, заметим, что детерминированный источник и автомат-распознаватель — это одно и то же — нагруженный символами орграф с выделенным начальным узлом и множеством заключительных узлов.

Теорема. [Клини о синтезе автоматов] *Для любого регулярного языка существует конечный автомат H, распознающий этот язык.*

Доказательство. Возьмем регулярный язык L. По определению он описывается регулярным выражением E. Согласно теореме 1.2.4, существует двухполюсник H, представляющий E (может быть, с большим количеством лишних пустых дуг). По теореме детерминизации существует детерминированный источник H', представляющий E.

Детерминированный источник является автоматом-распознавателем. □

Замечание. На практике в H' может быть меньше 2^n узлов, так как можно не включать те подмножества, которые не достижимы из начальных состояний. Однако в худшем случае число узлов равно всё-таки 2^n.

TODO В подходящем месте ввести сокращения НКА и ДКА.

2.2.5. Синтез автоматов (2/3)

Теорема. [Оценка детерминизации] *Пусть S — недетерминированный конечный автомат (НКА) с n состояниями над алфавитом A. Тогда эквивалентный ему детерминированный автомат (ДКА) S' имеет:*

- *в лучшем случае: n состояний и $n \cdot |A|$ переходов (если S уже детерминирован);*
- *в худшем случае: 2^n состояний и $2^n \cdot |A|$ переходов.*

Верхний предел 2^n достижим (см. п. 1.2.4 для доказательства теоремы детерминизации и п. 2.2.6 для анализа).

Пример. Рассмотрим язык $L = \{\omega \in A^* \mid$ символ на k-й позиции с конца равен $a\}$, где $A = \{a, b\}$.

- НКА для этого языка имеет $\mathcal{O}(k)$ состояний и $\mathcal{O}(k)$ переходов.
- Минимальный ДКА требует 2^k состояний и $2^k \cdot |A|$ переходов.

Замечание. При синтезе из регулярного выражения длины m, промежуточный НКА имеет $\mathcal{O}(m)$ состояний и переходов, но итоговый ДКА может иметь $2^{\mathcal{O}(m)}$ состояний и $2^{\mathcal{O}(m)} \cdot |A|$ переходов.

2.2.5. Синтез автоматов (3/3)

Пусть даны два детерминированных конечных автомата с n и m состояниями соответственно. При построении автомата для операций над языками число состояний и переходов может возрастать мультипликативно в худших случаях.

Операция	**Состояния**		**Переходы**	
	лучшее	**худшее**	**лучшее**	**худшее**
Объединение ($L_1 \cup L_2$)	$\mathcal{O}(n+m)$	$n \cdot m$	$\mathcal{O}((n+m) \cdot \lvert A \rvert)$	$nm \cdot \lvert A \rvert$
Пересечение ($L_1 \cap L_2$)	$\mathcal{O}(n+m)$	$n \cdot m$	$\mathcal{O}((n+m) \cdot \lvert A \rvert)$	$nm \cdot \lvert A \rvert$
Дополнение ($A^* \setminus L_1$)	n	n	$n \cdot \lvert A \rvert$	$n \cdot \lvert A \rvert$
Конкатенация ($L_1 L_2$)	$\mathcal{O}(n+m)$	$(n+m) \cdot 2^{n+m}$	$\mathcal{O}((n+m) \cdot \lvert A \rvert)$	$(n+m)2^{n+m} \cdot \lvert A \rvert$

Замечание. При анализе детерминированного конечного автомата с n состояниями по методу удаления состояний (см. п. 2.2.6) размер полученного регулярного выражения в лучшем случае составляет $\mathcal{O}(n^2)$, а в худшем — $\mathcal{O}(\frac{2^n}{\sqrt{n}})$. На практике применяют минимизацию (алгоритм Хопкрофта) с временной сложностью $\mathcal{O}(n \log n)$.

TODO ФАН 20 января. Хопкрофт?

2.2.6. Анализ автоматов (1/3)

Анализ автоматов — это построение языка для заданного автомата. Пусть задан автомат $S = \langle A, Q, \delta, q, F \rangle$. Определим индуктивно язык E_{ij}^k.

1. $E_{ij}^0 \stackrel{\text{Def}}{=} \{a \in A \mid \delta(q_i, a) = q_j\}$.
2. $E_{ij}^k \stackrel{\text{Def}}{=} E_{ij}^{k-1} \cup E_{ik}^{k-1}(E_{kk}^{k-1})^* E_{kj}^{k-1}$.

Обозначим через M_{ij}^k множество всех входных слов $\alpha \in A^*$, ведущих из q_i в q_j ($q_j = \delta^*(q_i, \alpha)$) и проходящих только через вершины $1..k$.

Лемма. $E_{ij}^k = M_{ij}^k$.

Доказательство. Индукция по k. База: $k = 0$, $E_{ij}^0 = \{a \in A \mid \delta(q_i, a) = q_j\} = M_{ij}^0$ — множество дуг, не проходящих через промежуточные вершины. Индукционный переход: пусть $E_{ij}^{k-1} = M_{ij}^{k-1}$. Рассмотрим M_{ij}^k. Это множество можно разбить на две части: $M_{ij}^k = \bar{M}_{ij}^k \cup \tilde{M}_{ij}^k$, где $\bar{M}_{ij}^k := \left\{\langle q_i, q_j \rangle \in M_{ij}^k \mid q_k \notin \langle q_i, q_j \rangle\right\}$, а $\tilde{M}_{ij}^k := \left\{\langle q_i, q_j \rangle \in M_{ij}^k \mid q_k \in \langle q_i, q_j \rangle\right\}$. Имеем $\bar{M}_{ij}^k = M_{ij}^{k-1} = E_{ij}^{k-1}$. Пусть $\alpha \in \tilde{M}_{ij}^k$ и проходит через q_k s раз (то есть образует $s-1$ петлю через узлы с номерами $1..k-1$). Тогда $\alpha = \alpha_0\alpha_1 \ldots \alpha_{s-1}\alpha_s$, где $\alpha_0 \in M_{ik}^{k-1}$, $\alpha_s \in M_{kj}^{k-1}$, $\alpha_1, \ldots, \alpha_{s-1} \in M_{kk}^{k-1}$. □

2.2.6. Анализ автоматов (2/3)

Доказательство. (Окончание) Таким образом,
$\alpha \in M_{ik}^{k-1}(M_{kk}^{k-1})^* M_{kj}^{k-1} = E_{ik}^{k-1}(E_{kk}^{k-1})^* E_{kj}^{k-1} = E_{ij}^k$.
Пусть теперь $\alpha \in E_{ik}^{k-1}(E_{kk}^{k-1})^* E_{kj}^{k-1}$. Тогда $\alpha \in M_{ik}^{k-1}(M_{kk}^{k-1})^* M_{kj}^{k-1}$, и значит, $\alpha \in \tilde{M}_{ij}^k$.
Наконец, $\tilde{M}_{ij}^k = E_{ik}^{k-1}(E_{kk}^{k-1})^* E_{kj}^{k-1}$ и, следовательно,
$M_{ij}^k = \bar{M}_{ij}^k \cup \tilde{M}_{ij}^k = E_{ij}^{k-1} \cup E_{ij}^{k-1}(E_{kk}^{k-1})^* E_{kj}^{k-1}$. □

Теорема. [Клини об анализе автоматов] *Всякий язык, представимый конечным автоматом, регулярен.*

Доказательство. Пусть задан автомат-распознаватель $S = \langle A, Q, \delta, q, F\rangle$, $|Q| = n$. Язык L, определяемый автоматом S — это множество всех путей из начального состояния в заключительное. В обозначениях предыдущей леммы

$$L = \bigcup_{z \in F} M_{qz}^n,$$

но $M_{qz}^n = E_{qz}^n$ и потому регулярно, а значит, L регулярен. □

2.2.6. Анализ автоматов (3/3)

Пример. Пусть задан автомат-распознаватель $S = \langle A, Q, \delta, q, F \rangle$, где $A = \{a, b\}$, $Q = \{q_1, q_2\}$, $q = q_1$, $F = \{q_2\}$, а функция переходов δ задана таблицей

δ	a	b
q_1	q_1	q_2
q_2	q_2	q_2

По доказательству предыдущего слайда имеем TODO картинка справа от таблицы просится

$$L = \bigcup_{z \in F} E_{qz}^{|Q|} = E_{12}^2.$$

$$k = 0: \quad E_{11}^0 = \{a\},\ E_{12}^0 = \{b\},\ E_{21}^0 = \varnothing,\ E_{22}^0 = \{a, b\}.$$

$$k = 1: \quad E_{12}^1 = E_{12}^0 \cup E_{11}^0 (E_{11}^0)^* E_{12}^0 = \{b\} \cup \{a\}\{a\}^*\{b\} = \{a\}^*\{b\}.$$

При этом $E_{22}^1 = \{a, b\}$.

$$k = 2: \quad E_{12}^2 = E_{12}^1 \cup E_{12}^1 (E_{22}^1)^* E_{22}^1 = \{a\}^*\{b\}(\{a, b\})^* = \{a\}^*\{b\}A^*.$$

Следовательно, $L = \{a\}^*\{b\}A^*$ — язык всех слов над $A = \{a, b\}$, содержащих хотя бы один символ b.

2.2.7. Множество языков, распознаваемых автоматами (1/2)

Конечные автоматы-распознаватели, регулярные выражения, регулярные порождающие грамматики и источники (графы языков) оказываются равнообъёмными понятиями — все они позволяют описать одно и то же множество языков (*регулярных* или *автоматных* языков).

Наличие нескольких альтернативных способов описания не является избыточным: на практике для описания регулярных языков применяют разные способы в зависимости от целей описания.

- Конечные автоматы удобнее всего применять, если нужно запрограммировать распознаватель языка.
- Регулярные выражения являются самым компактным описанием.
- Порождающая грамматика часто применяется для формального описания в стандартизующих документах.
- Граф языка в виде диаграммы является, видимо, наиболее наглядным.

2.2.7. Множество языков, распознаваемых автоматами (2/2)

На следующем рисунке просуммированы связи между этими понятиями.

2.2.A. Онтология формальных языков, грамматик и автоматов

2.2.Б. Темы для самостоятельной работы студентов

1. СНИРС => Вычисления, совмещённые с разбором

   При лексическом анализе можно и нужно вычислять значения некоторых лексем прямо во время анализа. Например, это нетрудно сделать для чисел, несколько сложнее для дат. При этом применяются различные трюки. Требуется найти и рассказать несколько примеров.

2. СНИРС => Экспериментальное сравнение сканеров

   Требуется найти несколько сканеров (лексических анализаторов) или подготовить свои и провести вычислительный эксперимент на представительном наборе нагрузочных тестов.

3. СНИРС => Регулярные подмножества нерегулярных языков

   Нерегулярный язык зачастую содержит регулярные полезные подмножества. Например, регулярный язык простейшего калькулятора является подмножеством контекстно свободного языка арифметических выражений. Требуется найти и привести другие примеры, желательно, подкрепленные реализацией.

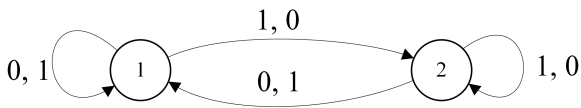
2.3. Эквивалентность автоматов

Конечный автомат реализует определённое поведение, реагируя на цепочки входных символов. Можно поставить следующий важный вопрос: в каких случаях разные автоматы реализуют одно и то же поведение? Другими словами, в каких случаях автоматы эквивалентны? Конечный автомат является, в сущности, одним из способов описания алгоритма. В общем случае (для произвольных алгоритмов) задача определения эквивалентности алгоритмически неразрешима. Однако для конечных автоматов это не так. В данном разделе устанавливаются необходимое и достаточное условия эквивалентности конечных автоматов и рассматриваются некоторые важные следствия.

2.3.1. Неотличимость состояний и автоматов (1/2)

Пусть даны два инициальных автомата над общими алфавитами входных и выходных символов, $S = \langle A, Q_S, B, \delta_S, \lambda_S, q\rangle$ и $T = \langle A, Q_T, B, \delta_T, \lambda_T, r\rangle$. Состояния $q \in Q_S$ и $r \in Q_T$ называются *неотличимыми* (обозначение $q \equiv r$), если $\forall\, \alpha \in A^*\ (S(\alpha) = T(\alpha))$, то есть если совпадают автоматные преобразования, определяемые инициальными автоматами.

Пример. Состояния автомата, диаграмма которого представлена на рисунке, неотличимы. $A = \{0, 1\}$, $B = \{0, 1\}$, $Q = \{1, 2\}$.



Неотличимость состояний, очевидно, является эквивалентностью.

Замечание. Неотличимые состояния обычно называют *эквивалентными*. Термин «неотличимость состояний» введен, чтобы избежать таких словосочетаний, как «отношение эквивалентности состояний является отношением эквивалентности».

2.3.1. Неотличимость состояний и автоматов (2/2)

Инициальные конечные автоматы называются *эквивалентными*, если их входные алфавиты совпадают, а начальные состояния неотличимы. Обозначение: $S \equiv T$.

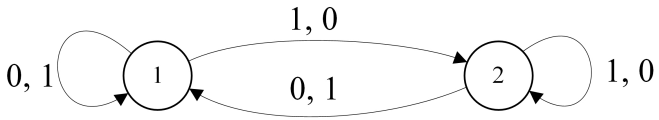
Конечные автоматы $S = \langle A, Q_S, B, \delta_S, \lambda_S \rangle$ и $T = \langle A, Q_T, B, \delta_T, \lambda_T \rangle$ называются эквивалентными, если

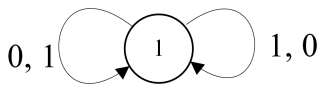
$$\forall\, q \in Q_S \ \left(\exists\, r \in Q_T \ \left(\langle A, Q_S, B, \delta_S, \lambda_S, q\rangle \equiv \langle A, Q_T, B, \delta_T, \lambda_T, r\rangle\right)\right),$$

$$\forall\, r \in Q_T \ \left(\exists\, q \in Q_S \ \left(\langle A, Q_T, B, \delta_T, \lambda_T, r\rangle \equiv \langle A, Q_S, B, \delta_S, \lambda_S, q\rangle\right)\right).$$

Изоморфные автоматы с общими алфавитами входных и выходных символов, очевидно, эквивалентны. В то же время эквивалентные автоматы не обязательно изоморфны.

Пример. На рисунке представлены диаграммы эквивалентных, но неизоморфных автоматов.





2.3.2. Прямое произведение автоматов (1/3)

Необходимое и достаточное условие эквивалентности инициальных конечных автоматов устанавливается путём введения следующей простой операции.

Прямым произведением инициальных конечных автоматов $S = \langle A, Q_S, B_S, \delta_S, \lambda_S, q\rangle$ и $T = \langle A, Q_T, B_T, \delta_T, \lambda_T, r\rangle$ (обозначается $S \times T$) называется инициальный автомат $S \times T = \langle A, Q_{S\times T} = Q_S \times Q_T, B_{S\times T} = B_S \times B_T, \delta_{S\times T}, \lambda_{S\times T}, (q, r)\rangle$, где

$$\forall\, q' \in Q_S\ (\forall\, r' \in Q_T\ (\forall\, a \in A\ (\delta_{S\times T}((q', r'), a) = (\delta_S(q', a), \delta_T(r', a))))),$$

$$\forall\, q' \in Q_S\ (\forall\, r' \in Q_T\ (\forall\, a \in A\ (\lambda_{S\times T}((q', r'), a) = (\lambda_S(q', a), \lambda_T(r', a))))).$$

Другими словами, прямое произведение автоматов — это просто два невзаимодействующих автомата, синхронно работающих с общим входом. Состояниями прямого произведения являются пары состояний сомножителей, а выходными символами — пары выходных символов сомножителей.

Теорема. [Мура] *Два инициальных конечных автомата эквивалентны тогда и только тогда, когда для любого достижимого состояния в их прямом произведении функции выходов совпадают.*

2.3.2. Прямое произведение автоматов (2/3)

Замечание. Если $S = \langle A, Q_S, B_S, \delta_S, \lambda_S, q\rangle$ и $T = \langle A, Q_T, B_T, \delta_T, \lambda_T, r\rangle$, то $S \equiv T \iff$
$\iff \forall\,(q', r') \in Q_{S\times T}\ \left(\exists\,\alpha \in A^*\ \left(\delta^*_{S\times T}((q, r), \alpha) = (q', r') \Longrightarrow \forall\, a \in A\ \left(\lambda_S(q', a) = \lambda_T(r', a)\right)\right)\right)$

Доказательство.

[Необходимость.] От противного. Пусть

$$\neg\forall\,(q', r') \in Q_{S\times T}\ \left(\exists\,\alpha \in A^*\ \left(\delta^*_{S\times T}((q, r), \alpha) = (q', r') \Longrightarrow \forall\, a \in A\ \left(\lambda_S(q', a) = \lambda_T(r', a)\right)\right)\right).$$

Тогда

$$\exists\,(q', r') \in Q_{S\times T}\ \left(\neg\exists\,\alpha \in A^*\ \left(\delta^*_{S\times T}((q, r), \alpha) = (q', r') \Longrightarrow \forall\, a \in A\ \left(\lambda_S(q', a) = \lambda_T(r', a)\right)\right)\right),$$

что можно преобразовать в

$$\exists\,(q', r') \in Q_{S\times T}\ \left(\forall\,\alpha \in A^*\ \left(\delta^*_{S\times T}((q, r), \alpha) = (q', r')\ \&\ \exists\, a \in A\ \left(\lambda_S(q', a) \neq \lambda_T(r', a)\right)\right)\right).$$

Положим $\beta := \alpha a$. Имеем

$$S(\beta) = S(\alpha a) = S(\alpha) + \lambda^*_S(\delta^*_S(q, \alpha), a) = T(\alpha) + \lambda^*_S(\delta^*_S(q, \alpha), a) = T(\alpha) + \lambda^*_S(q', a) \neq$$
$$\neq T(\alpha) + \lambda^*_T(r', a) = T(\alpha) + \lambda^*_T(\delta^*_T(r, \alpha), a) = T(\alpha a) = T(\beta).$$

Получили противоречие: $\exists\,\beta \in A^*\ (S(\beta) \neq T(\beta))$. □

2.3.2. Прямое произведение автоматов (3/3)

Доказательство. (Продолжение)

[Достаточность.] Индукция по $|\alpha|$, $\alpha \in A^*$. База: $|\alpha| = 1$, $\alpha = a \in A$. Имеем (q, r) достижимо по определению, следовательно, $\lambda_S(q, a) = \lambda_T(r, a)$ по условию, и значит, $S(a) = T(a)$. Индукционный переход: пусть $\forall\, \alpha \in A^k \ (\lambda_S^*(q, \alpha) = \lambda_T^*(r, \alpha))$, рассмотрим $\alpha \in A^{k+1}$, $|\alpha| = k + 1$. Для любой пары $(q', r') \in Q_{S\times T}$ имеем

$$\exists\, \alpha \in A^* \ \left(\delta^*_{S\times T}\left((q, r), \alpha\right) = (q', r') \Longrightarrow \forall\, a \in A \ \left(\lambda_S(q', a) = \lambda_T(r', a)\right)\right),$$

следовательно,

$$\forall\, a \in A \ \left(\lambda_S\left(\delta_S^*(q, \alpha), a\right) = \lambda_T\left(\delta_T^*(r, \alpha), a\right)\right),$$

откуда

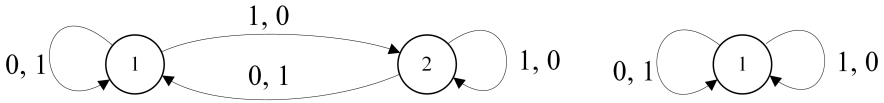
$$\forall\, a \in A \ \left(\forall\, \alpha \in A^k \ \left(\lambda_S^*(q, \alpha a) = \lambda_T^*(r, \alpha a)\right)\right),$$

и значит, $\forall\, \alpha \in A^{k+1} \ \left(\lambda_S^*(q, \alpha) = \lambda_T^*(r, \alpha)\right)$. □

2.3.3. Минимальный конечный автомат (1/2)

Автоматы могут функционировать одинаково, даже если у них разное число состояний.

Пример. Автоматы на рисунке эквивалентны независимо от того, какое состояние взять в качестве начального для левого автомата. Оба автомата инвертируют входную двоичную последовательность.



Таким образом, возникает задача *минимизации* конечных автоматов: для данного автомата найти эквивалентный автомат, имеющий минимальное число состояний. Такой автомат называется *минимальным*.

Теорема. *Для любого автомата S существует минимальный эквивалентный автомат T, причем количество состояний в минимальном автомате равно мощности фактормножества состояний по отношению неотличимости,*

$$\forall S \ \left(\exists T \ \left(S \equiv T \ \& \ |Q_T| = \min_{U \equiv S} |Q_U|\right)\right) \ \& \ |Q_T| = |Q_S/\equiv|.$$

2.3.3. Минимальный конечный автомат (2/2)

Доказательство. Пусть $\mathcal{C} = \{C_1, \ldots, C_t\}$ — разбиение множества состояний Q_S на классы эквивалентности по отношению неотличимости состояний $\equiv$:

$$\bigcup_{i=1}^{t} C_i = Q_S, \quad \forall\, i, j \in 1..t \ \left(i \neq j \Longrightarrow C_i \cap C_j = \varnothing\right), \quad \mathcal{C} = Q_S/\equiv .$$

Заметим, что $q_1 \in C_i \ \&\ q_2 \in C_i \Longrightarrow \forall\, a \in A \ (\delta(q_1, a) \in C_j \ \&\ \delta(q_2, a) \in C_j)$. Действительно, от противного, если $\delta(q_1, a) \not\equiv \delta(q_2, a)$ (обозначим $q_1' := \delta(q_1, a)$, $q_2' := \delta(q_2, a)$), то $\exists\, \alpha \in A^* \ (S_{q_1}(\alpha) \neq S_{q_2}(\alpha))$, но тогда $S_{q_1'}(a\alpha) \neq S_{q_2'}(a\alpha)$, то есть $q_1 \not\equiv q_2$, что противоречит тому, что $q_1 \in C_i \ \&\ q_2 \in C_i$.

Определим автомат $T = \langle A, Q_T, B, \delta_T, \lambda_T \rangle$ следующим образом: $Q_T := \{C_1, \ldots, C_t\}$, $\delta_T(C_i, a) := C_j$, такое что $\delta_S(q, a) \in C_j$, а $\lambda_T(C_i, a) := \lambda_S(q, a)$, где q — любое состояние из класса $C_i, q \in C_i$. Заметим, что $T \equiv S$ по построению и T не имеет неотличимых состояний. Покажем от противного, что T — минимальный.

Пусть $\exists\, T' \ (T' \equiv S \ \&\ |Q_{T'}| < |Q_T|)$. По транзитивности $T' \equiv T$. Но $|Q_{T'}| < |Q_T|$, следовательно, $\exists\, t' \in Q_{T'} \ (\exists\, t_1, t_2 \in Q_T \ (t_1 \equiv t' \ \&\ t_2 \equiv t'))$, откуда $t_1 \equiv t_2$, что противоречит построению.

2.3.4. Алгоритм минимизации Мили (1/5)

Приведённый ниже алгоритм Мили строит минимальный эквивалентный автомат путем последовательного разбиения множества состояний исходного автомата на блоки, пока эти блоки не станут классами эквивалентности по неотличимости.

Вход: Автомат $S =< A, Q, B, \delta, \lambda >$;

Выход: Разбиение $\mathcal{G}$ множества Q.

$\mathcal{C}_0 := Q/\equiv_\lambda$ // начальное разбиение
for i **from** 1 **to** $|Q|$ **do**
  $\mathcal{C}_i := \varnothing$ // инициализация очередного разбиения
  for $K \in \mathcal{C}_{i-1}$ **do**
    $\mathcal{C}_i := \mathcal{C}_i \cup K/\equiv_\delta^i$ // измельчение блока разбиения
  end for
  if $\mathcal{C}_i = \mathcal{C}_{i-1}$ **then**
    exit // прекращение процесса
  end if
end for

2.3.4. Алгоритм минимизации Мили (2/5)

В этом алгоритме операция вычисления фактормножества считается заданной, а отношения $\equiv_\lambda$ и $\equiv^i_\lambda$ определяются следующим образом:

$$q' \equiv_\lambda q'' \stackrel{\text{Def}}{=} \forall\, a \in A\ \left(\lambda(q', a) = \lambda(q'', a)\right),$$

$$q' \equiv^i_\delta q'' \stackrel{\text{Def}}{=} \forall\, a \in A\ \left(\exists\, K \in C_{i-1}\ \left(\delta(q', a) \in K\ \&\ \delta(q'', a) \in K\right)\right), \quad q', q'' \in C_i.$$

Замечание. При реализации алгоритма Мили хранить все разбиения не обязательно — достаточно двух последних. Явная индексация разбиений введена в алгоритме только для удобства обоснования.

Обоснование. Пусть алгоритм остановился на шаге с номером $k \leqslant |Q|$. Покажем, что полученное разбиение C_k множества Q является фактормножеством по неотличимости, то есть

$$q' \in K\ \&\ q'' \in K \in C_k \Longrightarrow q' \equiv q''$$

и

$$q' \in K' \in C_k\ \&\ q'' \in K'' \in C_k\ \&\ K' \neq K'' \Longrightarrow q' \not\equiv q''.$$

□

2.3.4. Алгоритм минимизации Мили (3/5)

Обоснование. (Продолжение) Действительно, пусть $q', q'' \in K \in C_k$. По условию остановки алгоритма $q' \equiv_\delta^k q''$, то есть $\forall\, a \in A\ (\exists\, K' \in C_{k-1}\ (\delta(q', a) \in K'\ \&\ \delta(q'', a) \in K'))$ (иначе блок K можно было бы измельчить на меньшие блоки). Таким образом, $\delta(q', a) \equiv_\delta^{k-1} \delta(q'', a)$, аналогично, $\forall\, b \in A\ \left(\delta(\delta(q', a), b) \equiv_\delta^{k-2} \delta(\delta(q'', a), b)\right)$ и т. д. Следовательно, $\exists\, K''\ (\forall\, \alpha \in A^*\ (\delta^*(q', \alpha) \in K''\ \&\ \delta^*(q'', \alpha) \in K''))$. Но если $q' \equiv_\delta^i q''$, то $q' \equiv_\lambda q''$ (потому что последующие разбиения получаются измельчением блоков исходного разбиения), и значит, $\lambda(q', a) = \lambda(q'', a)$, откуда $S_{q'}(\alpha) = S_{q''}(\alpha)$, то есть $q' \equiv q''$. Пусть теперь $q' \in K' \in C_k$, $q'' \in K'' \in C_k$ и $K' \neq K''$. Тогда

$$\exists\, a' \in A\ \left(\delta(q', a') \in K'_{k-1}\ \&\ \delta(q'', a') \in K''_{k-1}\right),$$

где $K'_{k-1} \neq K''_{k-1}$, K'_{k-1}, $K''_{k-1} \in C_{k-1}$. Другими словами, можно найти такое a', что $\delta(q', a')$ и $\delta(q'', a')$ обязательно принадлежат разным классам разбиения на предыдущем уровне (иначе q' и q'' попали бы в один класс на данном уровне). Но тогда $\exists\, a'' \in A\ \left(\delta\left(\delta(q', a'), a''\right) \in K'_{k-2}\ \&\ \delta\left(\delta(q'', a'), a''\right) \in K''_{k-2}\right)$. □

2.3.4. Алгоритм минимизации Мили (4/5)

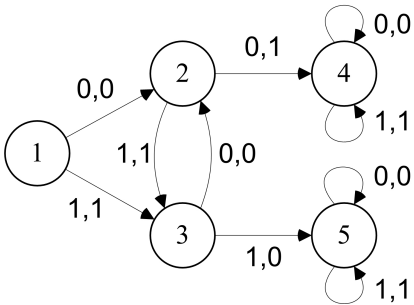
Обоснование. (Окончание) Заметим, что $K'_{k-2} \neq K''_{k-2}$, $K'_{k-2}, K''_{k-2} \in C_{k-2}$ и так далее, то есть можно построить слово α, $|\alpha| < k$, такое что $\delta^*(q', \alpha) \not\equiv_\lambda \delta^*(q'', \alpha)$. Это означает, что $\delta^*(q', \alpha)$ и $\delta^*(q'', \alpha)$ принадлежат разным классам начального разбиения $Q/\equiv_\lambda$, откуда $\lambda^*(q', \alpha) \neq \lambda^*(q'', \alpha)$, и значит, $q' \not\equiv q''$. □

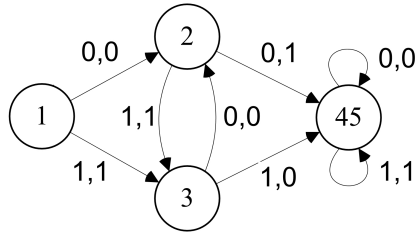
Пример. Применение алгоритма Мили к автомату, диаграмма которого приведена на следующем рисунке слева, дает автомат, диаграмма которого приведена на том же рисунке справа. При этом последовательность разбиений следующая:

1. $Q = \{1, 2, 3, 4, 5\}$ — множество состояний;
2. $C_0 = \{1, 4, 5\}, \{2\}, \{3\}$ — начальное разбиение по λ;
3. $C_1 = \{1\}, \{4, 5\}, \{2\}, \{3\}$ — измельчение блока $\{1, 4, 5\}$;
4. $C_2 = C_1$ — остановка алгоритма.

2.3.4. Алгоритм минимизации Мили (5/5)

Пример. (Продолжение – рисунок)





Автомат на рисунке выполняет следующее преобразование: если в последовательности битов встречаются два одинаковых подряд, то второй из них инвертируется. Остальные биты не изменяются.

Отступление. Минимизация автоматов, построенных вручную, мало что даёт, поскольку эти автоматы небольшие и, как правило, близки к оптимальным. Другое дело, когда речь идёт об автоматах, сгенерированных автоматически по формальным правилам. В таких автоматах бывают тысячи и десятки тысяч состояний, и минимизация даёт значительный эффект.

2.3.5. Эквивалентность автоматов Мура и Мили (1/2)

Очевидно, что автомат Мура является частным случаем автомата Мили. Действительно, достаточно перенести отметку вершины на входящие ребра, и из автомата Мура получится автомат Мили (с тем же числом состояний).

Следующая теорема устанавливает тот факт, что выразительные возможности двух классов автоматов совпадают.

Теорема. *Для любого автомата Мили существует эквивалентный автомат Мура.*

Доказательство. Пусть $S = \langle A, Q, B, \delta, \lambda \rangle$ — автомат Мили. Определим автомат Мура

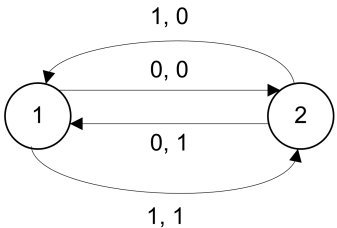
$$M = \langle A_M, Q_M, B_M, \delta_M, \mu \rangle$$

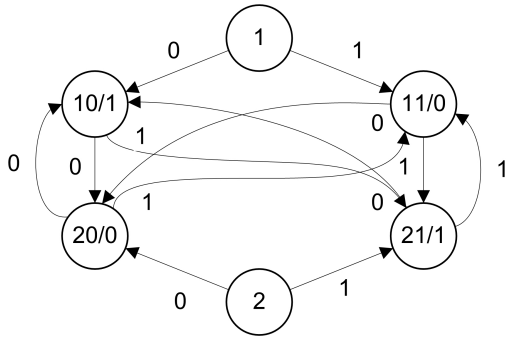
следующим образом: $A_M := A$; $B_M := B$; $Q_M := Q \times A \cup Q = \left\{ q_{ij}^M \right\}_{i=1,j=1}^{k,m} \cup \left\{ q_i^M \right\}_{i=1}^{k}$, где $\delta_M(q_i^M, a_j) := q_{ij}^M$, а $\delta_M(q_{ij}^M, a_s) := q_{ts}$ такое, что $q_t = \delta(q_i, a_s)$; $\mu(q_i^M) := \varepsilon$, $\mu(q_{ij}^M) := \lambda(q_i, a_j)$

Покажем индукцией по $|\alpha|$, что $S(q_i, \alpha) = M(q_i^M, \alpha)$. База: $|\alpha| = 1 \Longrightarrow \alpha = a$. По построению $\forall\, a \ \left(\forall\, i \ \left(S(q_i, a) = M(q_i^M, a) \right) \right)$. Индукционный переход: пусть $S(q_i, \alpha) = M(q_i^M, \alpha)$. Рассмотрим αa. Имеем $\delta^*(q_i, \alpha) = q_j \Longrightarrow \delta_M^*(q_i^M, \alpha) = q_{ij}^M$, $\lambda(\delta^*(a_i, \alpha), a) = \mu(\delta_M^*(q_i^M, \alpha))$, и значит, $S(q_i, \alpha a) = M(q_i^M, \alpha a)$. □

2.3.5. Эквивалентность автоматов Мура и Мили (2/2)

Пример. На рисунке слева приведен автомат Мили с двумя состояниями, а справа — эквивалентный автомат Мура с шестью состояниями. Эти автоматы инвертируют биты через один.





Замечание. После построения автомата Мура его можно минимизировать. Но в большинстве случаев минимальный автомат Мили содержит меньше состояний, чем эквивалентный ему минимальный автомат Мура.

2.3.A. Онтология формальных языков, грамматик и автоматов

2.3.Б. Темы для самостоятельной работы студентов

1. СНИРС => Современные алгоритмы минимизации автоматов

   Многие новые информационные технологии используют автоматическую генерацию конечных автоматов или родственных структур. В таких случаях требуется минимизировать автоматы с сотнями тысяч состояний, для чего используются другие алгоритмы. Требуется найти и рассказать несколько примеров.

2. СНИРС => Алгоритмическая неразрешимость эквивалентности

   Требуется разобраться и доказать алгоритмическую неразрешимость проблемы проверки эквивалентности различных моделей, более сложных, чем конечные автоматы.

2.4. Сети автоматов

Конечный автомат — это один из способов описания алгоритма. Для практики важно иметь возможность комбинировать эти описания таким образом, чтобы строить из простых алгоритмов более сложные. В этом разделе рассматриваются *некоторые* из известных способов организации взаимодействия автоматов.

2.4.1. Комбинационные и логические автоматы

Автомат называется *комбинационным*, если $\forall a \ (\forall q_i, q_j \ (\lambda(q_i, a) = \lambda(q_j, a)))$, то есть выходной символ зависит только от входного символа. Очевидно, что в комбинационном автомате все состояния эквивалентны и в минимальном эквивалентном автомате имеется одно состояние. Функция δ тривиальна и поведение комбинационного автомата описывается функцией выходов $\lambda : A \to B$.

Автомат называется *логическим*, если $A = \{0, 1\}^n$ и $B = \{0, 1\}^m$, то есть входными символами являются логические шкалы длины n, а выходными — логические шкалы длины m. Говорят, что логический автомат имеет n входов и m выходов: $a_1, \ldots, a_n$ и $b_1, \ldots, b_m$, соответственно.

Комбинационный логический автомат с n входами и m выходами — это просто система m булевых функций от n переменных:

$$\begin{cases} b_1 := f_1(a_1, \ldots, a_n), \\ \ldots \\ b_m := f_m(a_1, \ldots, a_n). \end{cases}$$

2.4.2. Автоматные блок-схемы (1/4)

Рассмотрим различные способы комбинирования автоматов. Начнем с простого.
Автоматной блок-схемой называется орграф, в котором есть узлы двух сортов: автоматы-преобразователи (в соответствии с традицией, они обозначаются прямоугольниками) и автоматы-распознаватели (они обозначаются ромбами). Эти узлы соединены дугами, которые называются *переходами* и показывают, в каком порядке блоки работают (так называемый «*поток управления*»).
Каждый блок должен «останавливаться» сам. Примем, что в преобразователях есть не менее одного заключительного состояния, попав в которое, автомат более не воспринимает входные символы, а в распознавателях имеется два типа заключительных состояний для ответов 0 и 1 и нет выходных символов.

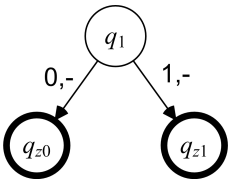
Блок-схема автоматов — это автомат. Его граф получается из графов блоков следующим образом: нужно «склеить» заключительные состояния q_z начал переходов с начальными состояниями q_1 концов переходов.

2.4.2. Автоматные блок-схемы (2/4)

Пример. Допустим, есть следующие исходные автоматы.

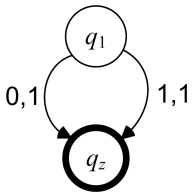
S_1 — распознаватель

	0	1
q_1	$q_{z0}, -$	$q_{z1}, -$
q_{z0}	—	—
q_{z1}	—	—



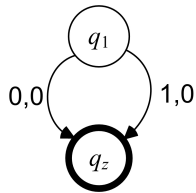
S_2 — константа 1

	0	1
q_1	$q_z, 1$	$q_z, 1$
q_z	—	—



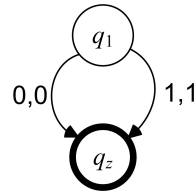
S_3 — константа 0

	0	1
q_1	$q_z, 0$	$q_z, 0$
q_z	—	—



S_4 — тождественный преобразователь

	0	1
q_1	$q_z, 0$	$q_z, 1$
q_z	—	—

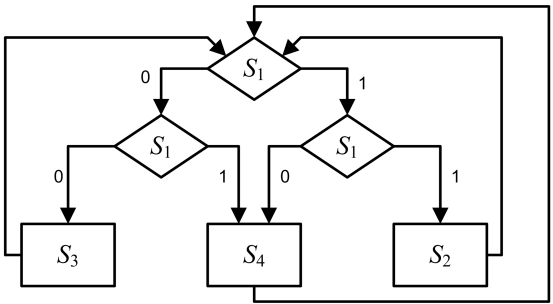


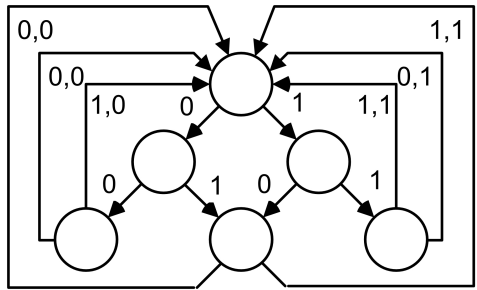
2.4.2. Автоматные блок-схемы (3/4)

В этих обозначениях на рисунке слева представлена блок-схема автомата «голосования» по трем разрядам. Данный преобразователь, например, выполняет следующее преобразование:

$$001110101 \mapsto 011.$$

Его граф переходов представлен на рисунке справа.





2.4.2. Автоматные блок-схемы (4/4)

Отступление. На графе переходов состояний справа не указаны имена состояний. Вообще говоря, имена состояний не нужны — поведение автомата Мили, задаваемого графом переходов состояний, полностью определяется тем, чем нагружены дуги, нагрузка на узлы играет роль комментария.

Замечание. В автоматной блок-схеме нет явной передачи информации между блоками. Поток управления определяет только порядок работы блоков. Каждый блок читает исходные данные и записывает результаты независимо.

Подведём итог. Блок-схема из конечных автоматов $S_1, \ldots, S_k$, работающих последовательно (то есть не одновременно) — это конечный автомат S, причем

$$|A| \leqslant \sum_{i=1}^{k} |A_i|, \quad |B| \leqslant \sum_{i=1}^{k} |B_i|, \quad |Q| \leqslant \sum_{i=1}^{k} |Q_i|.$$

Поэтому блок-схему иногда называют *суммой* автоматов.

2.4.3. Синхронные сети автоматов (1/8)

Рассмотрим теперь случай, когда соединяются входы и выходы автоматов, то есть явно описываются потоки данных. Такие конструкции называют *сетями автоматов*.

При описании сетей автоматов (в отличие от абстрактных конечных автоматов) явно нужно понятие времени. Есть два основных подхода к моделированию времени — синхронный и асинхронный. Здесь рассматривается более простой — синхронный. При этом подходе время считается дискретным, оно делится на такты, которые имеют номера $t = 0, 1, \ldots$. Входное слово (то есть последовательность сигналов) — это функция $a(t)$. Автоматные функции δ и λ реализуются с *задержкой*:

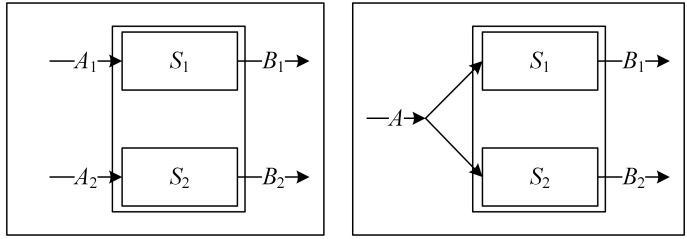
- $q(t+1) := \delta(q(t), a(t))$, $q(0)$ определено заранее.
- $b(t) := \lambda(q(t), a(t))$, но иногда полагают $b(t+1) := \lambda(q(t), a(t))$, тогда $b(0)$ должно быть определено заранее.

Все автоматы, соединенные в сеть, работают синхронно: в момент t они получают свои входные символы и переходят в свои новые состояния. Таким образом, в сети S, составленной из автоматов $S_1, \ldots, S_k$, состояние (на каждом такте t) характеризуется вектором $(q^1, \ldots, q^k)$.

2.4.3. Синхронные сети автоматов (2/8)

Сложные сети строятся из более простых, и в конечном счёте из элементарных автоматов с помощью четырёх основных операций (способов соединения): параллельного соединения, последовательного соединения, задержки и обратной связи.

На рисунке представлены два случая *параллельного соединения* автоматов. Слева автоматы имеют разные входные алфавиты и вообще никак не взаимодействуют, а справа автоматы имеют общий алфавит.

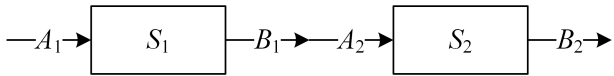


Здесь исходные автоматы $S_1 = \langle A_1, Q_1, B_1, \delta_1, \lambda_1 \rangle$ и $S_2 = \langle A_2, Q_2, B_2, \delta_2, \lambda_2 \rangle$, а результирующий автомат $S = S_1 \times S_2 = \langle A, Q, B, \delta, \lambda \rangle$, причём $A = A_1 \times A_2$, $Q = Q_1 \times Q_2$, $B = B_1 \times B_2$, $\delta(q, a) = \delta((q^1, q^2), (a^1, a^2)) = (\delta_1(q^1, a^1), \delta_2(q^2, a^2))$, $\lambda(q, a) = (\lambda_1(q^1, a^1), \lambda_2(q^2, a^2))$, где $q^1 \in Q_1$, $q^2 \in Q_2$, $a^1 \in A_1$, $a^2 \in A_2$. В случае общего входного алфавита $\delta(q, a) = (\delta_1(q^1, a), \delta_2(q^2, a))$.

Замечание. Случай справа рассмотрен в п. 2.3.2 (прямое произведение автоматов).

2.4.3. Синхронные сети автоматов (3/8)

На рисунке представлена операция *последовательного соединения* автоматов. Эта операция определена, если выходной алфавит первого автомата является входным алфавитом второго.



Здесь исходные автоматы $S_1 = \langle A_1, Q_1, B_1, \delta_1, \lambda_1 \rangle$ и $S_2 = \langle A_2, Q_2, B_2, \delta_2, \lambda_2 \rangle$, а результирующий автомат $S = \langle A, Q, B, \delta, \lambda \rangle$, причем $A := A_1; B := B_2; B_1 = A_2; Q = Q_1 \times Q_2$.
Для определения δ и λ нужно знать, как определена λ_1 — с задержкой или без.
Если задержка λ_1 равна нулю, то есть $\lambda_1(q^1(t), a(t)) = b^1(t)$, то

$$q(t+1) = (q^1(t+1), q^2(t+1)) = (\delta_1(q^1(t), a(t)), \delta_2(q^2(t), \lambda_1(q^1(t), a(t)))).$$

В этом случае $q(t+1)$ зависит только от $q(t) = (q^1(t), q^2(t))$ и от $a(t)$, то есть от предыдущего такта. Если же задержка λ_1 равна 1, то есть $\lambda_1(q^1(t), a(t)) = b^1(t+1)$, то

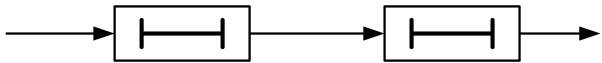
$$q(t+1) = (\delta_1(q^1(t), a(t)), \delta_2(q^2(t), \lambda_1(q^1(t-1), a(t-1))))$$

и зависимости только от предыдущего такта не получается.

2.4.3. Синхронные сети автоматов (4/8)

Пусть автомат имеет одно состояние, двоичные вход и выход, а на выход переписывает свой вход с задержкой на один такт: $\lambda(q(t), a(t)) := a(t-1)$, то есть $b(t+1) = a(t)$. Такой автомат называется *элементом задержки*. Для завершения описания элемента задержки необходимо определить $b(0)$. Пусть $b(0) := 0$. Однако использование функций выхода с задержкой приводит к трудностям.

Пример. На рисунке представлен автомат, являющийся последовательным соединением двух элементов задержки.



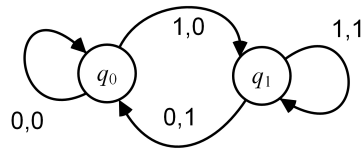
Казалось бы, такая сеть тривиальна: по определению предыдущего параграфа $Q = Q_1 \times Q_2$, $|Q| = 1$. Однако $S(\alpha) = 00\alpha[1..(|\alpha|-2)]$, а это преобразование невозможно реализовать автоматом с одним состоянием.

Отступление. Конечный автомат не может хранить больше информации, чем количество состояний. Образно говоря, чтобы уметь считать до двух, автомат должен иметь по меньшей мере два состояния.

2.4.3. Синхронные сети автоматов (5/8)

Выход состоит в том, чтобы рассматривать элемент задержки как автомат с двумя состояниями и без задержки выхода, как показано на рисунке.

	0	1
q_0	$q_0, 0$	$q_1, 0$
q_1	$q_0, 1$	$q_1, 1$



При этом

$$\lambda(q(t), a(t)) = b(t) \quad (= a(t-1)).$$

Таким образом, автомат с одним состоянием и единичной задержкой выхода эквивалентен автомату с двумя состояниями и без задержки выхода.

2.4.3. Синхронные сети автоматов (6/8)

В этом случае цепь из двух задержек описывается автоматом без задержек, в котором необходимо иметь уже четыре состояния:

q	0	1
00	00, 0	10, 0
01	00, 1	10, 1
10	01, 0	11, 0
11	01, 1	11, 1

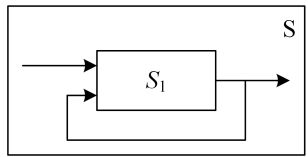
Таким образом, описана задержка на 2 такта. Аналогично описывается задержка на любое число тактов.

Определение последовательного соединения остаётся корректным при использовании элементов задержки.

Отступление. В теории информации мерой информации, присущей некоторой дискретной системе, принято считать энтропию этой системы, то есть логарифм числа состояний. Если для удобства взять основанием логарифма число 2, то единицей информации оказывается информация о том, в котором из двух возможных состояний находится некоторая система. Эта единица называется *бит*.

2.4.3. Синхронные сети автоматов (7/8)

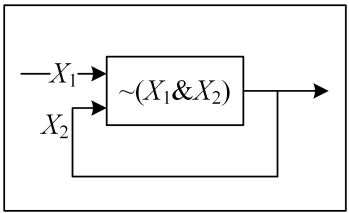
Операция введения *обратной связи* состоит в том, что выход автомата присоединяется к одному из входов, как показано на рисунке.

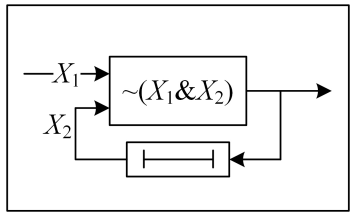


Замечание. Обратная связь может быть не прямой, а косвенной, то есть выход автомата присоединяется ко входу того же автомата не прямо, а через промежуточные автоматы. Другими словами, всякий контур в сети автоматов, рассматриваемой как орграф, является обратной связью.

2.4.3. Синхронные сети автоматов (8/8)

Пример. Пусть S_1 — логический комбинационный автомат, реализующий формулу $\overline{x_1} \vee \overline{x_2} = \overline{x_1 x_2}$. Применим операцию обратной связи, как показано на рисунке слева.





Рассмотрим работу данной сети. Пусть $b(t) = 0$, тогда $x_2(t) = 0$ и $b(t) = \overline{x_1} \vee \overline{x_2} = \overline{x_1} \vee 1 = 1$. Пусть $b(t) = 1$. Тогда $x_2(t) = 1$, и если $x_1(t) = 1$, то $b(t) = \overline{x_1} \vee \overline{x_2} = 0 \vee 0 = 0$. Получили противоречие (типа парадокса Рассела в теории множеств). Но если ввести в обратную связь задержку, то никаких противоречий не возникнет (рисунок справа).

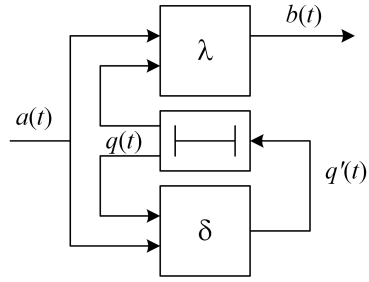
Таким образом, сеть с обратными связями без задержек может быть противоречивой. Поэтому рассмотрение обратных связей без задержек обычно сразу исключают.

2.4.4. Представление автомата сетью

Обратная связь с задержкой является мощным средством построения автоматов.

Теорема. *Любой конечный автомат при любом двоичном кодировании его алфавитов A, Q, B может быть реализован синхронной сетью логических комбинационных автоматов и двоичных задержек, причём необходимое число задержек не меньше чем $\log_2 |Q|$, где $A \subset \{0,1\}^m$, $Q \subset \{0,1\}^n$, $B \subset \{0,1\}^k$.*

Доказательство. Пусть задан автомат $S = \langle A, Q, B, \delta, \lambda \rangle$. Построим сеть, представленную на рисунке. Здесь λ — логический комбинационный автомат $Q \times A \to B$, $b(t) = \lambda(q(t), a(t))$; δ — логический комбинационный автомат, $Q \times A \to Q$, $q(t+1) = \delta(q(t), a(t))$; H — блок задержек (параллельное соединение задержек для всех разрядов q) на один такт, $Q \to Q$, причём $q'(t) = \delta(q(t), a(t)) = q(t+1)$. □



Очевидно, что данная сеть автоматов фактически реализует алгоритм работы конечного автомата S. Для кодирования $|Q|$ символов нужно не менее $\log_2 |Q|$ разрядов.

2.4.5. Представление сети автоматом (1/3)

Рассмотрим сеть, состоящую из логических комбинационных автоматов и двоичных элементов задержки. Такая сеть называется *правильно построенной*, если:

1. к каждому входу присоединен не более чем один выход,
2. в каждом контуре обратной связи имеется не менее чем один элемент задержки.

Замечание. В правильно построенной сети на один вход не должно подаваться несколько сигналов, но один выход может разветвляться и присоединяться к нескольким входам.

Входами (*выходами*) правильно построенной сети являются те входы (выходы) её элементов, к которым не присоединены никакие выходы (входы), соответственно.

Теорема. *Всякая правильно построенная сеть G со входами $x_1, \ldots, x_m$, выходами $z_1, \ldots, z_n$ и n элементами задержки, является конечным автоматом $S = \langle A, Q, B, \delta, \lambda \rangle$, причем $A = \{0,1\}^m$, $Q = \{0,1\}^n$, $B = \{0,1\}^k$.*

Доказательство. Рассмотрим случай, когда сеть G не имеет задержек. По определению в этом случае G не имеет контуров и является ациклическим орграфом. Рассмотрим произвольный выход z сети G. Логический комбинационный автомат, которому принадлежит этот выход, реализует некоторую булеву функцию $z = f(p_1, \ldots, p_s)$ от своих входов $p_1, \ldots, p_s$.

2.4.5. Представление сети автоматом (2/3)

Доказательство. (продолжение) Но каждый вход $p \in \{p_1, \ldots, p_s\}$ либо является входом сети, то есть $p \in \{x_1, \ldots, x_m\}$, либо выходом другого логического комбинационного автомата. В последнем случае p также реализован как булева функция $p = g(t_1, \ldots, t_r)$. Продолжая этот процесс и учитывая, что орграф бесконтурный, в конце концов получаем, что z является суперпозицией некоторых булевых функций от некоторого подмножества входов $x_1, \ldots, x_m$. Это верно для любого выхода z сети G, а значит правильно построенная сеть без задержек является логическим комбинационным автоматом.

Пусть теперь сеть G имеет n элементов задержки, входы $x_1, \ldots, x_m$ и выходы $z_1, \ldots, z_k$. Рассмотрим сеть G_0, которая получается из G удалением всех элементов задержки. По доказанному G_0 — логический комбинационный автомат со входами $x_1, \ldots, x_m, y_1, \ldots, y_n$ и выходами $z_1, \ldots, z_k, y'_1, \ldots, y'_n$, где $y'_1, \ldots, y'_n$ — входы, а $y_1, \ldots, y_n$ — выходы элементов задержки. □

2.4.5. Представление сети автоматом (3/3)

Доказательство. (продолжение) Таким образом, G_0 вычисляет две системы $z_1, \ldots, z_k$ и ${y'}_1, \ldots, {y'}_n$ логических функций от переменных $x_1, \ldots, x_m, y_1, \ldots, y_n$ (эти системы называются *каноническими уравнениями сети*). Положим $a(t) := (x_1(t), \ldots, x_m(t))$ — входной символ, $b(t) := (z_1(t), \ldots, z_k(t))$ — выходной символ, $q(t) := (y_1(t), \ldots, y_n(t))$ — состояние. Заметим, что

$$({y'}_1(t), \ldots, {y'}_n(t)) = (y_1(t+1), \ldots, y_n(t+1)) = q(t+1).$$

Таким образом, G_0 одновременно вычисляет функцию выходов $b(t) := \lambda\left(q(t), a(t)\right)$ и функцию переходов $q(t+1) := \delta\left(q(t), a(t)\right)$ конечного автомата S. □

Пример. Сеть на рисунке слева имеет канонические уравнения $y' = x \vee y$, $z = x \vee y$ и реализуется автоматом, диаграмма переходов которого изображена на рисунке справа.

TODO Два рисунка.

2.4.6. Структурный синтез автоматов (1/3)

Структурный синтез автоматов — это решение задачи декомпозиции, когда необходимо представить заданный автомат в виде сети из более простых автоматов. Наиболее важный и изученный случай — реализация автомата в заданном базисе (одна из основных задач схемотехники). Здесь можно поставить различные задачи, например:

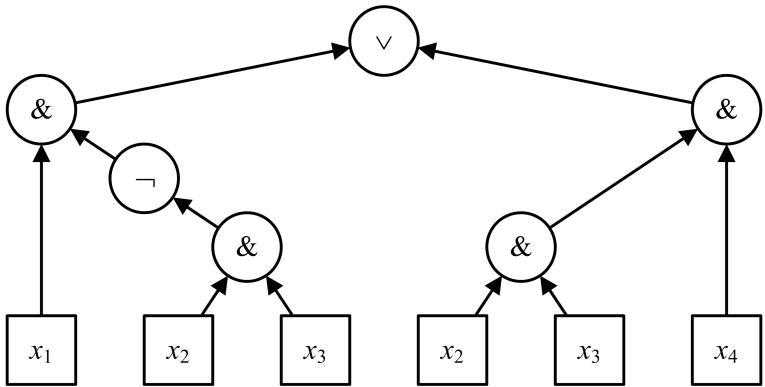
1. Выяснить, можно ли реализовать любой автомат в заданном базисе (такой базис называется *автоматно полным*).
2. Среди всех схем реализации найти обладающую определёнными свойствами, например, минимальную.

Мы начнем рассмотрение структурного синтеза автоматов с простейшего случая, а именно с задачи синтеза логических комбинационных автоматов, то есть со схемы реализации блока, вычисляющего заданную булеву функцию при условии, что заданы блоки, вычисляющие функции базиса (они называются *элементарными* или *элементами*).

Метод очень прост. Если функция задана формулой в данном базисе, то строим дерево разбора формулы и получаем древовидную блок-схему, реализующую функцию.

2.4.6. Структурный синтез автоматов (2/3)

Пример. Для формулы $x_1\overline{x_2x_3} \vee x_2x_3x_4$ схема реализации, соответствующая дереву разбора $(x_1 \,\&\, \neg(x_2 \,\&\, x_3)) \vee ((x_2 \,\&\, x_3) \,\&\, x_4)$, представлена на рисунке.



2.4.6. Структурный синтез автоматов (3/3)

Ответ на вопрос о существовании формулы, реализующей любую функцию в данном базисе, исчерпывающим образом дает теорема Поста о полноте системы булевых функций. Таким образом, ответ на первый вопрос, поставленный в начале параграфа, для простейшего случая логических комбинационных автоматов, сформулирован в следующей теореме.

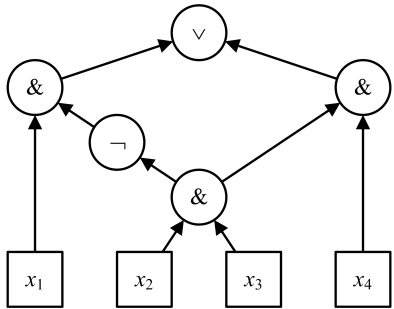
Теорема. *Над полным базисом можно реализовать любой логический комбинационный автомат.*

Для общего случая всё обстоит значительно сложнее. Из теорем двух предыдущих параграфов ясно, что набор, состоящий из функционально полного набора комбинационных автоматов плюс элемент задержки является автоматно полным. Однако это только один частный случай. В общем случае задача определения автоматной полноты произвольного набора элементов оказывается алгоритмически неразрешимой.

2.4.7. Минимизация схем (1/2)

Обратимся теперь ко второму вопросу, поставленному в предыдущем параграфе, — вопросу о минимизации схем. Как известно из курса дискретной математки минимизация формул — уже довольно трудная задача, но минимизация схем ещё труднее.

Пример. Рассмотрим схему, представленную на рисунке. Очевидно, что эта схема делает то же самое, что и схема на предыдущем рисунке, то есть вычисляет значение функции, представленной формулой $x_1\overline{x_2x_3} \vee x_2x_3x_4$. Однако представленная схема не древовидна и в ней меньше элементов, чем в *любой* формуле, реализующей функцию.



2.4.7. Минимизация схем (2/2)

До настоящего времени известно очень мало классов функций, для которых найдены минимальные схемные реализации. Получены только общие количественные характеристики, некоторые из которых мы приводим для справки без доказательства.

Пусть $L_\Sigma(f)$ — число элементов минимальной схемы, реализующей функцию f в базисе Σ. Функция $L_\Sigma(n) = \max_{f \in P_n} L_\Sigma(f)$ называется *функцией Шеннона* для базиса Σ. Она определяет минимальное число элементов, достаточное для реализации любой функции n переменных.

Теорема. [Шеннон–Лупанов]

1. *Функция Шеннона асимптотически равна $c_\Sigma \frac{2^n}{n}$, где коэффициент c_Σ зависит от базиса Σ:*
$$L_\Sigma(n) = O\left(\frac{2^n}{n}\right).$$
2. *С ростом n почти все функции f реализуются со сложностью, близкой к верхней границе.*
$$|\{g \mid L_\Sigma(g) < (1-\varepsilon)L_\Sigma(n)\}| = o\left(|P_n|\right).$$

2.4.8. Вложенные автоматы (1/X)

TODO В книге А.А.Шалыто и на сайте is.ifmo.ru, а также во многих других источниках, в частности, Поликарпова Шалыто = Автоматное программирование описывается ещё один способ взаимодействия автоматов: головной автомат вызывает подчинённые.
Необходимо разобраться и изложить основные идеи плюс один пример.

2.4.A. Онтология формальных языков, грамматик и автоматов

2.4.Б. Темы для самостоятельной работы студентов

1. СНИРС => Структурная теория автоматов

   Начатая в середине прошлого века развитая теория, которая имеет приложения в современных информационных тенологиях. Требуется найти и рассказать что-то интересное, отталкиваясь от параграфов 2.4.6 и 2.4.7.

2. СНИРС => Автоматное программирование

   Требуется сделать *качественный* обзор современных достижений автоматного программирования. Материала очень много, требуется выбрать полезную информацию в потоке рекламного информационного шума.

2.5. Автоматы с памятью (1/3)

В самом названии «конечные автоматы» слово «конечные» очень существенно, и во многом характеризует свойства этого способа описания алгоритмов.

Во всех формальных определениях алгоритмов, наряду со способностью абстрактных машин совершать некоторые действия, в той или иной форме присутствует *память*, то есть способность хранить и использовать результаты ранее совершенных действий. Заметим, что в универсальных моделях вычислимости память потенциально бесконечна: машина Тьюринга имеет бесконечную ленту (или ленты), регистры в МНР неограничены, слова в нормальных алгорифмах Маркова могут иметь любую длину.

В рассматриваемых моделях конечных автоматов в явном виде памяти нет. В неявном виде памятью автомата является множество состояний, но эта память ограничена: автомат «знает», в каком он пребывает состоянии, но не более того. Из этого следует, что если в конечном автомате n состояний, то он может «помнить» не более чем n предшествующих событий (входных символов).

2.5. Автоматы с памятью (2/3)

В частном случае комбинационных автоматов (см. п. 2.4.1), автомат вовсе не имеет памяти и является просто функцией над конечным алфавитом. Применительно к задаче распознавания языков, автомат без памяти может распознавать элементарные языки (то есть одиночные символы, см. п. 1.2.2), и не более того.

Конечный язык, то есть множество слов в заданном алфавите, проще всего распознаётся с помощью *дерева решений*, которое получится, если действовать, как указано в п. 2.2.2, но не добавлять состояние недопустимости.

Замечание. Дерево решений весьма эффективно, если алфавит не слишком велик, слова не очень длинные, и язык обладает *свойством префикса*, то есть никакое слово языка не является собственным началом никакого другого слова этого языка. Именно так устроены множества ключевых слов в современных языках программирования.

Конечные автоматы, то есть автоматы с конечной памятью состояний, могут распознавать регулярные языки (см. п. 1.2.5, п. 2.2.5 и п. 2.2.7), класс которых совпадает с классом языков, порождаемых грамматиками типа $\mathfrak{T}_3$.

2.5. Автоматы с памятью (3/3)

Как показано в п. 2.5.3, если допустить использование простейшей, но потенциально неограниченной памяти (стека), то конечные автоматы способны эффективно распознавать контекстно-свободные языки, и это обстоятельство является основой современных языков программирования.

Для того, чтобы распознавать контекстно-зависимые языки достаточно допустить наличие адресуемой памяти, ограниченной длиной входного слова.

Наконец, если снабдить автомат неограниченной памятью, то получается машина Тьюринга, способная распознавать любой перечислимы язык.

В этом разделе рассматриваются различные варианты добавления к конечному автомату памяти в том или ином виде, и рассматривается вопрос о том, как расширения модели конечных автоматов сказываются на их свойствах и возможностях в части распознавания языков в порядке от общего к частному.

2.5.1. Машины Тьюринга

Машина Тьюринга — это конечный автомат, к которому добавлена адресуемая память, в виде потенциально бесконечной ленты. Автомат наделяется способностью читать с ленты и писать на ленту один символ, находящийся по текущему адресу, а также уменьшать и увеличивать адрес на единицу, то есть сдвигаться по ленте влево и вправо.

Замечание. Можно рассматривать *многоленточные* машины Тьюринга, в которых используется *несколько* (конечное число) лент. Для любой многоленточной машины Тьюринга можно построить эквивалентную одноленточную машину ценой некоторого усложнения кодирования алгоритмов. Зато использование многоленточной модели иногда упрощает доказательства свойств алгоритмов.

Говорят, что машина Тьюринга *принимает* некоторое входное слово, записанное на ленте, если она останавливается в заключительном состоянии. Говорят, что машина Тьюринга *распознаёт* язык, если она принимает все слова этого языка и только их.

Теорема. *Если язык L порождается некоторой грамматикой G типа 0, то существует машина Тьюринга M, которая распознаёт язык L.*

TODO Эссе + недетерминированные машины Тьюринга.

2.5.1. Машины Тьюринга (2/5)

Недетерминированная машина Тьюринга (НМТ) — это обобщение машины Тьюринга, в котором функция переходов возвращает *множество* возможных конфигураций:

$$\delta : Q \times \Gamma \to 2^{Q \times \Gamma \times \{L,R\}},$$

где Q — множество состояний, Γ — алфавит ленты, L, R — сдвиг влево/вправо.

Замечание. НМТ *принимает* слово ω, если *существует* хотя бы одна последовательность переходов, приводящая к принимающему состоянию. Вычисление НМТ образует *дерево вычислений*: каждая ветвь соответствует одному возможному пути выполнения.

Теорема. *Для любой недетерминированной машины Тьюринга можно построить эквивалентную детерминированную машину Тьюринга.*

Доказательство. ДМТ моделирует НМТ *обходом дерева вычислений в ширину*: перебирает все возможные последовательности переходов длины $1, 2, 3, \ldots$, пока не найдёт принимающую ветвь (если она существует). □

2.5.1. Пример: МТ для языка $\{a^n b^n\}$ (3/5)

Пример. Построим ДМТ M, распознающую язык $L = \{a^n b^n \mid n \geqslant 0\}$.

Идея: На каждом проходе помечаем по одному a и одному b, проверяя, что они исчерпываются одновременно.

Алгоритм:

1. Если лента пуста (только $\sqcup$) — **принять**.
2. Заменить первый непомеченный a на X, сдвинуться вправо.
3. Пропустить все a и Y, найти первый непомеченный b.
4. Заменить этот b на Y, вернуться влево до первого X.
5. Сдвинуться вправо и повторить с шага 2.
6. Когда все a помечены: проверить, что все b тоже помечены.
7. Если остались непомеченные a или b — **отвергнуть**.

Сложность: $O(n^2)$ шагов для слова длины $2n$.

2.5.1. Пример: НМТ для проверки составности числа (4/5)

Пример. Построим НМТ для проверки, является ли число n (в двоичной записи) *составным*.

Идея: Недетерминированно «угадываем» делитель d, затем детерминированно проверяем делимость.

Алгоритм:

1. **Недетерминированный выбор:** записать на вспомогательную ленту двоичное число d, где $1 < d < n$ (выбор каждого бита — недетерминированный).
2. **Детерминированная проверка:** вычислить $n \mod d$.
3. Если $n \mod d = 0$ — **принять**, иначе — отвергнуть эту ветвь.

Если n составное, существует ветвь вычисления с делителем $d \mid n$, и НМТ принимает.
Если n простое, все ветви отвергают, и НМТ не принимает.

Замечание. Эквивалентная ДМТ перебирает все d от 2 до $n-1$ последовательно. Поскольку число n записано в двоичной системе (размер входа $\approx \log_2 n$), количество делителей $n-2$ экспоненциально по размеру входа.

2.5.1. Моделирование НМТ детерминированной МТ (5/5)

Пример. Пусть НМТ N имеет не более r вариантов перехода из каждой конфигурации. Построим эквивалентную ДМТ M (используя 3 ленты).

Структура лент:

- **Лента 1:** хранит входное слово ω (только для чтения).
- **Лента 2:** рабочая лента для симуляции одной ветви N.
- **Лента 3:** «адрес» текущей ветви — последовательность $(c_1, \ldots, c_\ell)$, $c_i \in \{1, \ldots, r\}$.

Алгоритм (обход в ширину):

1. Для $\ell = 0, 1, 2, \ldots$:
2. Для каждой последовательности $(c_1, \ldots, c_\ell)$ длины ℓ:
3. Скопировать ω на ленту 2, симулировать N, выбирая c_i-й переход на шаге i.
4. Если N приняла — **принять**.

TODO К сожалению, не сделано главного = не доказана основная теорема, которая необходима для связывания грамматик и автоматов и используется во многих других местах.

2.5.2. Линейно-ограниченные автоматы

Линейно-ограниченный автомат — это машина Тьюринга, в которой разрешается использовать только ту часть ленты, на которой записано входное слово. Автомат наделяется способностью читать с ленты и писать на ленту один символ, находящийся по текущему адресу, а также уменьшать и увеличивать адрес на единицу, то есть сдвигаться по ленте влево и вправо, но только в границах входного слова.

Замечание. Часто рассматривают модели, в которых размер доступной памяти ограничен *линейной* функцией от размера входа или же многоленточные модели. Также как и в случае мнолентоточных машин Тьюринга, алгоритмических свойств модели это принципиально не меняет, но иногда упрощает доказательства свойств алгоритмов.

Говорят, что линейно-ограниченный автомат *принимает* некоторое входное слово, записанное на ленте, если он останавливается в заключительном состоянии. Говорят, что линейно-ограниченный автомат *распознаёт* язык, если он принимает все слова этого языка и только их.

Теорема. *Если язык L порождается некоторой грамматикой G типа 1, то существует линейно-ограниченный автомат M, который распознаёт язык L.*

TODO Эссе + недетерминированные ДО-автоматы

2.5.2. Недетерминированные линейно-ограниченные автоматы (1/3)

Недетерминированный линейно-ограниченный автомат (НЛОА) задаётся как

$$M = \langle A, Q, \Gamma, \delta, q_0, F \rangle,$$

где $A \subseteq \Gamma$, Q конечно, $q_0 \in Q$, $F \subseteq Q$, а

$$\delta : Q \times \Gamma \to 2^{Q \times \Gamma \times \{-1,0,1\}}.$$

Зафиксируем $\# \notin \Gamma$. Для входа $w \in A^*$, $|w| = n$, положим

$$c_0(w) = \big(q_0, 1, \#w\#\big),$$

где $\#w\# \in (\Gamma \cup \{\#\})^{n+2}$.

Ограничение. Во всех достижимых конфигурациях $c = (q, i, t)$ выполняется

$$i \in \{0, \dots, n+1\}, \qquad t[0] = t[n+1] = \#,$$

а запись в клетки с $\#$ запрещена.

2.5.2. Одношаговый переход и недетерминизм (2/3)

Пусть вход $w \in A^*$ фиксирован, $|w| = n$. Для конфигурации $c = (q, i, t)$ обозначим $t[i]$ символ под головкой. Определим $\vdash_M$ так:

$$(q, i, t) \vdash_M (q', i{+}d, t[i \leftarrow b]) \iff t[i] \in \Gamma \wedge (q', b, d) \in \delta(q, t[i]) \wedge i{+}d \in \{0, \ldots, n{+}1\}.$$

Недетерминизм. Если $|\delta(q, a)| > 1$, то из одной конфигурации возможны несколько следующих.

Детерминированный случай. ДЛОА: $\forall q \in Q\ \forall a \in \Gamma :\ |\delta(q, a)| \leqslant 1$.

2.5.2. Принятие и язык $L(M)$ (3/3)

Конфигурация $c = (q, i, t)$ *конечная*, если $t[i] = \#$ или $\delta(q, t[i]) = \varnothing$. Конфигурация *принимающая*, если $q \in F$.

Принятие.

$$M \text{ принимает } w \iff \exists c : \; c_0(w) \vdash_M^* c \;\wedge\; c \text{ конечная} \;\wedge\; c \text{ принимающая}.$$

Распознаваемый язык.

$$L(M) = \{\, w \in A^* \mid M \text{ принимает } w \,\}.$$

TODO Это только начальная заготовка параграфа.
Все нуждается в переделке и расширении в три раза.

2.5.3. Автоматы с магазинной памятью

Автомат с магазинной памятью (коротко говорят МП-автомат) — это конечный автомат, в котором разрешается писать и читать символы из магазина (стека). Другими словами, это безадресная машина Тьюринга.

Говорят, что МП-автомат *принимает* некоторое входное слово, если он останавливается в заключительном состоянии.
Говорят, что МП-автомат *распознаёт* язык, если он принимает все слова этого языка и только их.

Замечание. Иногда рассматривают модели, в которых условиях остановки является *опустошение стека*. Символы в стек разрешают писать/читать не по одному, а по несколько за раз, символы разрешают брать из специального алфавита и/или из входного и выходного алфавитов и т.д. Алгоритмических свойств модели все эти вариации принципиально не меняют.

Теорема. *Если язык L порождается некоторой грамматикой G типа 2, то существует МП-автомат M, который распознаёт язык L.*

TODO Эссе + $LR(k)$ и Дональд Кнут.

2.5.3. Автоматы с магазинной памятью (1/5)

- *Автомат с магазинной памятью* (МП-автомат) — конечный автомат, дополненный магазином (стеком, LIFO).
- Входное слово $\alpha \in A^*$ читается слева направо; магазин хранит «контекст» вычисления (например, число не закрытых скобок).
- Формально

$$M = \langle A, Q, \Gamma, \delta, q_0, Z_0, F \rangle,$$

где A — входной алфавит, Q — множество состояний, Γ — алфавит магазина, $Z_0 \in \Gamma$ — нижний символ магазина, $F \subseteq Q$ — множество заключительных состояний.

- Переход имеет вид

$$\delta(q, a, X) \ni (q', \gamma), \quad a \in A \cup \{\varepsilon\},\ X \in \Gamma,\ \gamma \in \Gamma^*,$$

то есть читаем a (или ε), снимаем X с вершины и кладём на вершину слово γ.

- Приём обычно задают *по заключительному состоянию* (остановка в F); эквивалентный по выразимости вариант — *по пустому магазину*.
- *Детерминированный* МП-автомат: в каждой конфигурации нет конкурирующих действий (либо единственный переход по входному символу, либо единственный ε-перехо

2.5.3. МП-автоматы и КС-языки (2/5)

Теорема. *Язык $L \subseteq A^*$ контекстно-свободен $\Longleftrightarrow$ L распознаётся некоторым МП-автоматом*

- **Идея построения $G \Rightarrow M$.** Пусть $G = \langle N, \Sigma, P, S \rangle$ — КС-грамматика. Магазин симулирует *левосторонний вывод*:
  - если на вершине терминал a и следующий входной символ равен a, то делаем «сдвиг»: потребляем a и снимаем его с вершины;
  - если на вершине нетерминал A, то недетерминированно выбираем правило $(A \to \beta) \in P$ и заменяем A на β, записывая β в магазин в обратном порядке.
- **Идея построения $M \Rightarrow G$.** Нетерминалы кодируют «подпрограммы магазина»: например, символ вида $\langle p\, X\, q \rangle$ означает «из состояния p обработать некоторый префикс входа так, чтобы снять X с вершины и оказаться в q». Правила грамматики собираются по допустимым фрагментам вычислений автомата.
- Следствие: класс языков, распознаваемых МП-автоматами, совпадает с классом контек языков.

2.5.3. Мини-примеры и сложность (3/5)

- $L = \{a^n b^n \mid n \geqslant 0\}$: пока читаем a — кладём символ A в магазин; при первом b переходим в режим снятия; каждый b требует A на вершине и снимает его; при конце входа и пустом магазине — приём. Это детерминированный МП-автомат, время $O(|\alpha|)$.
- Сбалансированные скобки: «(» кладём в магазин, «)» требует «(» на вершине и снимает; при конце входа магазин пуст. Детерминированный МП-автомат, $O(|\alpha|)$.
- $L = \{ww^R \mid w \in \{a, b\}^*\}$ контекстно-свободен, но не детерминированно контекстно- нужно «угадать середину», поэтому требуется недетерминизм.
- Общая трудоёмкость проверки принадлежности: для произвольной КС-грамматики — полиномиально (CYK/Earley: $O(n^3)$), для детерминированного МП-автомата — линейно.

2.5.3. Замкнутость и решаемость (4/5)

- **Контекстно-свободные языки (CFL) замкнуты относительно:**

  $\cup$, $\cdot$, *, гомоморфизмов и обратных образов, $\cap$ с регулярным языком.

- **CFL не замкнуты относительно:** пересечения и дополнения.
- **Детерминированно КС-языки (DCFL) замкнуты относительно:** дополнения и пересечения с регулярными языками; **не замкнуты относительно:** объединения и звезды Клини.
- **Решаемость для CFL:** пустота и конечность — решаемы; принадлежность слова — решаема; эквивалентность, включение, универсальность — неразрешимы.

TODO Отказ. Всё не соответствует требованиям.

2.5.3. LR(k) и Кнут: парсер как МП-автомат (5/5)

- *LR(k)*: чтение слева направо; восстановление правостороннего вывода «с конца»; k-символьный просмотр. Класс введён Д. Кнутом (1965).
- Табличный LR-парсер: автомат по ситуациям (items) + две таблицы `ACTION` и `GOTO`. Магазин хранит пары «символ/состояние».
  - `shift`: потребить входной символ, записать его и новое состояние;
  - `reduce` по правилу $A \to \beta$: снять $2|\beta|$ ячеек и перейти по `GOTO` по A;
  - `accept/error`: приём или ошибка.
- Включения: для любого $k \geqslant 1$ имеем $LR(k) \subseteq DCFL$; объединение по всем k даёт $DCFL$.
- Практика: канонический $LR(1)$ наиболее выразителен, но тяжёл; $LALR(1)$ — индустриальный компромисс; $SLR(1)$ проще, но слабее.
- Вывод: табличный LR-парсер — частный случай детерминированного МП-автомата с гарантированным временем $O(|\alpha|)$.

Сеть Петри: P, T, F (1/7)

TODO В таком виде в работу нельзя взять.
Это прискорбно, потому что концепция сетей Петри важная, полезная и может быть использована весной.

Сеть Петри — ориентированный двудольный граф:

- вершины разбиты на P (позиции) и T (переходы), $P \cap T = \varnothing$;
- дуги идут только $P \to T$ или $T \to P$;
- множество дуг $F \subseteq (P \times T) \cup (T \times P)$; допускаются кратные дуги.

Сквозной пример. Позиции p_1, p_2 и переходы t_1, t_2 с дугами

$$p_1 \to t_1 \to p_2, \qquad p_2 \to t_2 \to p_1.$$

Маркировка μ: маркеры и вектор (2/7)

Маркировка — отображение $\mu : P \to \mathbb{N}$, где $\mu(p)$ равно числу *маркеров* (фишек) в позиции p.

Если $P = \{p_1, \ldots, p_m\}$, то удобно отождествлять μ с вектором

$$\mu = \big(\mu(p_1), \ldots, \mu(p_m)\big)^{\mathsf{T}} \in \mathbb{N}^m.$$

Пример. Для сквозного примера начальная маркировка:

$$\mu_0 = (1, 0)^{\mathsf{T}},$$

то есть в p_1 находится один маркер.

Запуск перехода: разрешённость и изменение μ (3/7)

Кратные дуги. Удобно рассматривать веса дуг (кратности)

$$W : F \to \mathbb{N} \setminus \{0\}.$$

Для перехода t определим вход и выход:

$$^\bullet t = \{p \in P \mid (p,t) \in F\}, \qquad t^\bullet = \{p \in P \mid (t,p) \in F\}.$$

Разрешённость. Переход t *разрешён* в маркировке μ, если

$$\forall p \in {}^\bullet t : \ \mu(p) \geqslant W(p,t).$$

Правило срабатывания. Если t разрешён, то при запуске получаем μ':

$$\mu'(p) = \mu(p) - W(p,t) + W(t,p) \qquad (W(x,y) = 0 \text{ если } (x,y) \notin F).$$

Пример. В $\mu_0 = (1,0)^\mathsf{T}$ разрешён t_1, и после его запуска:

$$\mu_1 = (0,1)^\mathsf{T}.$$

Линейная модель: матрица инцидентности C (4/7)

Матрицы C^-, C^+, C. Пусть $|P| = m, |T| = n$. Определим матрицы $C^-, C^+ \in \mathbb{N}^{m \times n}$:

$$C^-_{ij} = W(p_i, t_j), \qquad C^+_{ij} = W(t_j, p_i), \qquad C = C^+ - C^-.$$

Тогда запуск t_j соответствует прибавлению j-го столбца C.

Последовательность запусков. Если $\sigma \in \mathbb{N}^n$ — *вектор-счётчик* запусков (сколько раз сработал каждый переход), то формально:

$$\mu' = \mu + C \cdot \sigma.$$

Пример. (*Сквозной пример*) Для дуг $p_1 \to t_1 \to p_2$ и $p_2 \to t_2 \to p_1$:

$$C^- = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad C^+ = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad C = \begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix}.$$

Граф достижимости маркировок (5/7)

Определение. *Граф достижимости* строится так:

- вершины — маркировки μ;
- ребро $\mu \xrightarrow{t} \mu'$ есть, если переход t разрешён в μ и его запуск переводит μ в μ'.

Фрагмент графа для сквозного примера.

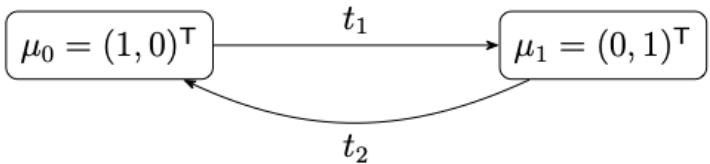


Рис.: Фрагмент графа достижимости для сквозного примера

По графу достижимости анализируют возможные поведения системы (достижимость, зацикливание, свойства).

Свойства сети: ограниченность, безопасность, живучесть (6/7)

Ограниченность и безопасность. Пусть $R(N, \mu_0)$ — множество достижимых маркировок.

- Сеть k-*ограничена*, если

$$\forall \mu \in R(N, \mu_0)\ \forall p \in P:\ \mu(p) \leqslant k.$$

- *Безопасность* — частный случай 1-ограниченности.

Живучесть. Переход t *жив*, если из любой достижимой μ можно попасть в такую μ', где t разрешён:

$$\forall \mu \in R(N, \mu_0)\ \exists \mu' \in R(N, \mu):\ t \text{ разрешён в } \mu'.$$

Пример. В сквозной сети с $\mu_0 = (1, 0)^\mathsf{T}$ все маркировки безопасны, сеть 1-ограничена, а переходы t_1, t_2 живы.

Сети Петри: параллелизм, проверка и (не)разрешимость (7/7)

Моделирование. Сети Петри естественно описывают параллельные процессы, синхронизацию и взаимодействие объектов: позиции — ресурсы/условия, переходы — события/действия, маркеры — текущее состояние.

Верификация по графу маркировок. Типовые вопросы (через граф достижимости/его аппроксимации):

- достижимость: можно ли попасть в нежелательную маркировку;
- ограниченность/безопасность;
- живучесть: может ли переход потерять возможность срабатывания навсегда.

Алгоритмическая сторона. Граф маркировок может расти очень быстро. Для ряда задач и обобщений сетей возникают трудные или даже алгоритмически неразрешимые вопросы; поэтому применяют ограничения моделей и специальные методы анализа.

2.6. Программирование автоматов

Конечные автоматы являются одним из самых распространенных приёмов описания алгоритмов и, следовательно, способ реализации конечного автомата в программе практически очень важен. В данном курсе рассматриваются только некоторые основные модели, связанные с понятием конечного автомата. На практике используется гораздо больше специфических конструкций, в основе которых лежит идея конечного графа переходов состояний. Буквально для каждой автоматной модели предложены свои (иногда многочисленные) способы представления автоматов в программе. Обсудить их все не представляется возможным — мы приводим только несколько характерных примеров.

TODO Различные реализации автоматов. Доделать.

2.6.1. Интерпретаторы таблиц (1/3)

Для начала рассмотрим способы представления, основанные на интерпретации хранимых данных.

Пусть задан инициальный конечный автомат-преобразователь $S = \langle A, Q, B, \delta, \lambda, q_1 \rangle$, см. п. 2.1.1, п. 2.1.2, п. 2.1.5.

Алфавиты, участвующие в описании конечного автомата, можно представить массивами:

A : **array** $[1..m]$ **of** ... — входной алфавит,

B : **array** $[1..k]$ **of** ... — выходной алфавит,

Q : **array** $[1..n]$ **of** ... — алфавит состояний.

Замечание. Здесь намеренно не указан тип элементов массивов — он может быть любым. В дальнейших построениях используются только индексы в этих массивах, то есть целые числа.

Функции переходов и выходов также можно представить массивами:

δ : **array** $[1..n, 1..m]$ **of** $1..n$,

λ : **array** $[1..n, 1..m]$ **of** $1..k$.

2.6.1. Интерпретаторы таблиц (2/3)

В таком случае автоматное отображение, определяемое инициальным автоматом-преобразователем $S = \langle A, Q, B, \delta, \lambda, q_1 \rangle$, реализуется следующим алгоритмом.

Вход: входное слово α : **array** $[1..s]$ **of** $1..m$
Выход: выходное слово β : **array** $[1..s]$ **of** $1..k$

$q := q_1$ // начальное состояние
for i **from** 1 **to** s **do**
  $a := \alpha[i]$ // очередной входной символ
  $\beta[i] := \lambda[q, a]$ // очередной выходной символ
  $q := \delta[q, a]$ // новое состояние
end for

Другие типы детерминированных автоматов также допускают подобное представление.

Важным обстоятельством является то, что алгоритм работы зависит не от конкретных таблиц функций переходов и выходов, а только от типа автомата. Таким образом, можно реализовать интерпретатор автомата нужного типа один раз, передавая ему необходимую информацию в виде аргументов.

2.6.1. Интерпретаторы таблиц (3/3)

Рассмотрим, например, универсальный автомат-распознаватель, представленный в следующем алгоритме.

Вход: Язык L, заданный таблицей переходов δ : **array** $[1..k, 1..m]$ **of** $1..n$;
начальное состояние $q_1 \in 1..k$;
подмножество заключительных состояний F: **set of** $1..n$;
входное слово α : **array** $[1..s]$ **of** $1..m$

Выход: **true** если $\alpha \in L$, **false** в противном случае.

$q := q_1$ // начальное состояние
for i **from** 1 **to** s **do**
  $q := \delta[q, \alpha[i]]$ // новое состояние
end for
return $q \in F$ // ответ распознавателя

Табличные интерпретаторы очень просты. Если массивы δ и λ не слишком велики, то такой способ представления автомата весьма эффективен.

TODO Верифицировать, вставить ссылки, насытить примерами.

2.6.2. Представление автомата в программном коде (1/3)

Альтернативой интерпретаторам, рассмотренным в предыдущем параграфе, является «компиляция» переходов автомата в программный код. Если используемый язык программирования допускает ветвление по значению переменной (оператор **case**), то код выполнения одного шага автомата $S = \langle A, Q, B, \delta, \lambda \rangle$ может иметь структуру, представленную в следующей схеме.

case q // ветвление по текущему состоянию
  q_1 : **case** a // ветвление по входному символу
    a_1 : $b := b_{i_1}$ // выходной символ
      $q := q_{j_1}$ // новое состояние
    a_2 : $b := b_{i_2}$
      $q := b_{j_2}$
      ...
    end case
  q_2 : **case** a
...
end case

2.6.2. Представление автомата в программном коде (2/3)

Замечание.

1. В правых частях операторов присваивания $q := \ldots$ и $b := \ldots$, реализующих функции переходов δ и функцию выходов λ, соответственно,
   стоят *константы* $b_{i_1}, b_{i_2}, \ldots, q_{i_1}, q_{j_2}, \ldots$, не требующие вычислений.
2. Если нужно предусмотреть какие-то дополнительные действия на переходе, которые автомат должен выполнять помимо определения выходного символа и смены состояния, то их очень удобно запрограммировать прямо в соответствующей ветви оператора **case**.
3. В зависимости от типа автомата схема должна быть дополнена средствами задания начального состояния и получения очередного символа (или события).
4. Общий внешний цикл, обрамляющий выполнение одного шага, определяется назначением автомата. Если это автомат-распознаватель или преобразователь, то применяют цикл **for** по входному слову. Если же это управляющий автомат, то применяют цикл **while**, управляемый достижением заключительного состояния или иным условием.

TODO Нужны простые, но убедительные примеры.

2.6.2. Представление автомата в программном коде (3/3)

Сравнительные характеристики двух подходов к реализации автоматов собраны в следующую таблицу.

Характеристика	Интерпретация	Компиляция
Объем памяти для хранения данных	Значительный. Линейно зависит от размера автомата	Незначительный. Не зависит от размера автомата
Объем исполняемого кода	Незначительный. Не зависит от размера автомата	Значительный. Линейно зависит от размера автомата
Скорость работы	Замедленная. Выборка из массивов	Максимальная. Присваивание констант
Встраивание дополнительных процедур	Затруднено	Очень просто

2.6.3. Задача поиска вхождений (1/2)

Рассмотрим применение техники конечных автоматов для решения одной распространённой задачи программирования, тесно связанной с синтаксическим анализом — поиск вхождений образца в текст. Задан текст T : **array** $[1..n]$ **of** A и образец P : **array** $[1..m]$ **of** A, $m \leqslant n$. Требуется найти все вхождения P в T, то есть определить все такие значения s, что $\forall\, i \in 1..m \;\; (T[s+i] = P[i])$. Если P не входит в T, то множество ответов пусто. Следующий простой алгоритм, очевидно, решает данную задачу.

Вход: Текст T : **array** $[1..n]$ **of char**, образец P : **array** $[1..m]$ **of char**.

Выход: Множество позиций, с которых начинаются вхождения P в T.

```
for s from 1 to n − m + 1 do
    for i from 1 to m do
        if P[i] ≠ T[s + i] then
            next for s  // нет совпадения — переходим к следующей позиции
        end if
    end for
    yield s  // нашли вхождение
end for
```

2.6.3. Задача поиска вхождений (2/2)

Замечание. Если P не входит в T, то алгоритм ничего не возвращает.

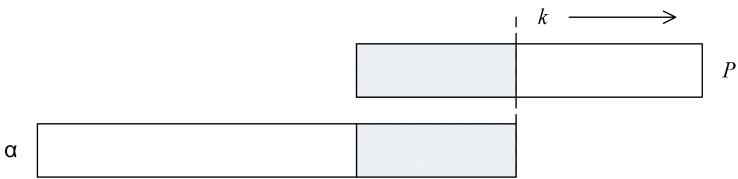
Отступление. Эта задача кажется очень простой, однако это не так. Приведённый алгоритм поиска вхождений «в лоб» имеет неприемлемо высокую трудоемкость $O(mn)$. Дело в том, что задача поиска вхождений находится в центре многих самых насущных приложений. В частности, в задаче *полнотекстового поиска* в Интернете объёмы текстов, в которых надобно искать, измеряются терабайтами, а количество образцов (поисковых запросов) исчисляется миллионами. Не проще и задачи в *биоинформатике*, где постоянно приходится искать вхождения достаточно длинных фрагментов в очень длинные последовательности, описывающие белки и другие органические полимеры. В символическом искусственном интеллекте поиск вхождений является основой *унификации формул*. Хотя формулы в логическом программировании и не так длинны, как тексты в Интернете и молекулы в биоинформатике, но зато из-за *комбинаторного взрыва* при полном переборе искать вхождения приходится миллионы и миллиарды раз.

Известно несколько весьма остроумных способов более эффективного решения этой задачи. В последующих параграфах рассматривается способ, основанный на применении понятия конечного автомата.

2.6.4. Функция постфикса (1/3)

Определим *функцию постфикса* $\sigma_P\colon A^* \to 1..m$ для данного образца P как длину максимального постфикса слова α, который является одновременно префиксом образца P (см. рисунок):

$$\sigma_P(\alpha) := \max k, \text{ где } \forall\, i \in 1..k \ \ (P[i] = \alpha[|\alpha| - k + i])\,.$$



Пример. Если $P = ab$, то $\sigma_P(\varepsilon) = 0$; $\sigma_P(ccaca) = 1$; $\sigma_P(ccab) = 2$.

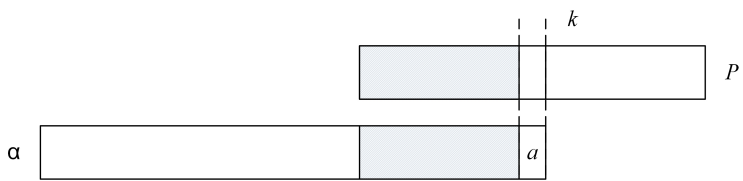
Очевидно, что $\sigma_P(\alpha) = m$ тогда и только тогда, когда P является постфиксом α. Если y является постфиксом x, то $\sigma_P(x) \geqslant \sigma_P(y)$. Если образец P подразумевается, то индекс P в обозначении σ_P можно опустить.

2.6.4. Функция постфикса (2/3)

Лемма. [*Неравенство для функции постфикса]*

$$\forall\, \alpha\ \ (\forall\, a\ \ (\sigma(\alpha a) \leqslant \sigma(\alpha) + 1))\,.$$

Доказательство. От противного. Пусть $\sigma(\alpha a) > \sigma(\alpha) + 1$, тогда отбросим последний символ a от наибольшего постфикса αa, который является одновременно префиксом P. Получаем, что $\sigma(\alpha) > \sigma(\alpha)$ — противоречие (см. рисунок). □



2.6.4. Функция постфикса (3/3)

Лемма. [*Реккурентная формула для функции постфикса]*

$$\forall\, \alpha\ \ (\forall\, a\ \ (\sigma(\alpha a) = \sigma(P[1..\sigma(\alpha)]a)))\,.$$

Доказательство. Пусть $k := \sigma(\alpha)$. По предыдущей лемме $\sigma(\alpha a) \leqslant k+1$, поэтому $\sigma(\alpha a) = \sigma(\alpha[|\alpha| - k + 1..|\alpha|]a)$. Но $\alpha[|\alpha| - k + 1..|\alpha|] = P[1..k]$ по определению σ. □

Замечание. Запись $X[a..b]$ означает последовательность элементов массива X с индесами от a до b. Такая операция называется *вырезкой* из массива.

2.6.5. Автомат поиска вхождений

Построим конечный автомат-распознаватель $S = \langle A, Q, \delta, q_0, F \rangle$, соответствующий образцу P следующим образом: $Q : 0..m$; $q_0 := 0$; $F : \{m\}$; $\delta(q, a) := \sigma_P(P[1..q]a)$. Другими словами, если автомат находится в состоянии q и входным символом является a, то автомат переходит в состояние с номером, который определяется значением функции постфикса для строки, составленной из первых q букв образца P и буквы a.

Автомат-распознаватель работает следующим образом.

Вход: Текст T : **array** $[1..n]$ **of char**, образец P : **array** $[1..m]$ **of char**

Выход: Множество позиций, с которых начинаются вхождения P в T. Если P не входит в T, то алгоритм ничего не возвращает

$q := 0$ // начальное состояние
for i **from 1 to** n **do**
  $q := \delta(q, T[i])$ // следующее состояние
  if $q = m$ **then**
    yield $i - m$ // нашли вхождение
  end if
end for

2.6.6. Построение функции переходов (1/3)

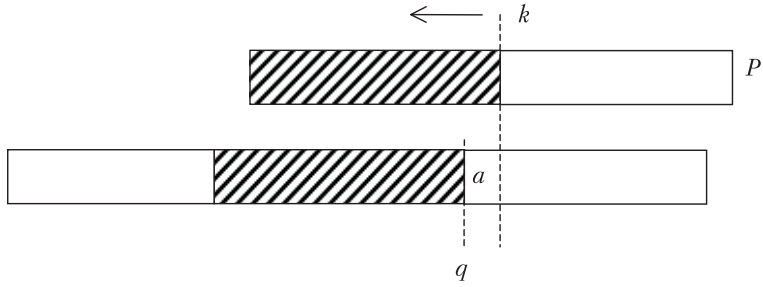
Функцию переходов δ, соответствующую образцу P, нетрудно вычислить, например, с помощью следующего алгоритма.

Вход: Образец P : **array** $[1..m]$ **of** A
Выход: Функция переходов δ : **array** $[0..m, A]$ **of** $0..m$

```
for q from 0 to m do
    for a ∈ A do
        k := min(m + 1, q + 2)  // наибольшее возможное k
        repeat
            k := k − 1  // пытаемся уменьшить k
        until P[1..k] = P[1..q] + a ∨ k = 0
        δ[q, a] := k
    end for
end for
```

2.6.6. Построение функции переходов (2/3)

Обоснование. В двух вложенных циклах перебираются все пары q, a и вычисляется число $k = \sigma(P[1..q]a)$, как это требуется для построения автомата. Данное вычисление выполняется «в лоб»: начинаем с максимально возможного k и уменьшаем его, пока префикс $P[1..k]$ не совпадет с постфиксом слова $P[1..q]a$ (см. рисунок).



□

Внутренний цикл выполняется не более $m + 1$ раз и на каждом шаге требует не более $m + 1$ сравнения для проверки условия окончания. Таким образом, общая трудоемкость данного алгоритма построения функции переходов составляет $O(m^3|A|)$.

2.6.6. Построение функции переходов (3/3)

Существует значительно более эффективные алгоритмы, которые строят функцию переходов за $O(m|A|)$.

TODO Развить эту тему. См., например, Кормен. Сократить число параграфов. Тимофеев Александр

2.6.7. Автоматное программирование (1/9)

TODO Шалыто и т.д. Примеры. ≈ 30 атомов. Рыженко

TODO Коррекция плана на пару параграфов

2.6.8. Взаимодействие автоматных объектов (1/9)

TODO Язык CIAO и примеры. Авторы языка: Новиков, Тихонова, Афанасьева.

TODO Коррекция плана на пару параграфов

3. Языки предметных областей

В этой главе рассматриваются приёмы, методы и технологии разработки *прикладного* программного обеспечения в конкретных предметных областях.

Отправной точкой рассмотрений являются следующие положения:

1) результат решения прикладной задачи с помощью программы намного важнее самой программы;
2) решить типовую прикладную задачу намного труднее, чем запрограммировать готовое решение;
3) учёт специфики предметной области намного эффективнее универсальных методов.

Среди многообразия методов прикладного программирования практической эффективностью выделяются методы, получившие название *языково-ориентированного программирования*.

3.1 Языково-ориентированное программирование

В популярных исторических обзорах утверждается, что термин «языково-ориентированное программирование» (Language Oriented Programming, LOP) предложил Вард[10] в 1994 году, а термин «язык предметной области» (Domain Specific Language, DSL) ввёл Фаулер[11] в 1997 году.

Однако с нашей точки зрения, языковая ориентация присуща программированию с середины прошлого века, начиная с дифференциации языков FORTRAN и COBOL.

[10] Martin Ward, p. ЧЧЧЧ
[11] Мартин Фаулер, p. 1963

3.1.1. Алгоритмизация предметных областей (1/5)

Алгоритмизацией предметной области называется разработка методов и программ, которые позволяют прикладным специалистам (*пользователям*) решать на компьютере все или большинство *типовых* задач данной предметной области.

Замечание. Иногда используют термин «инженерия предметной области» (domain engineering). В таком случае термин «инженер предметной области» был бы более точен, но термин «пользователь» (user) короче и привычнее.

Предметная область — специфическая область человеческой деятельности — здесь трактуется как *предмет* алгоритмизации: история, социальная значимость предметной области и тому подобное — всё остаётся «за кадром». Алгоритмизация предметной области — очень широкое понятие, включающее различные процессы и принимающее разнообразные формы. *Объектно-ориентированные анализ и проектирование*, *моделирование и построение баз данных*, *технология разработки прикладного программного обеспечения* — всё это и многое другое, относится к алгоритмизации предметных областей.

3.1.1. Алгоритмизация предметных областей (2/5)

Таким образом, методы алгоритмизации предметных областей практически необозримы в своих частных проявлениях, поэтому необходимо конкретизировать предмет исследования. Здесь понятию алгоритмизации дается следующее конструктивное истолкование. Алгоритмизация предметной области — это три перечисленных ниже взаимосвязанных процесса, каждый из которых в результате дает определённый набор информационных объектов:

1) определение предметно-ориентированных *структур данных*, представляющих объекты предметной области;
2) реализация программных *процедур обработки* этих структур — действий, или операций предметной области;
3) разработка методов и алгоритмов решения *типовых задач* предметной области —последовательностей действий, доставляющих значимый для пользователя результат.

При этом считается, что предметная область достаточно четко очерчена, а множества объектов A, операций F и типовых задач Q надёжно идентифицированы (перечислены) независимо от алгоритмизации предметной области.

3.1.1. Алгоритмизация предметных областей (3/5)

Примеры. В предметной области «бухгалтерия»: часто используемый объект — «счёт», операция — «проводка», типовая задача — «начисление зарплаты». В предметной области «астрономия»: объект — «орбита», операция — «вычисление эфемериды»,[12] типовая задача — «уточнение параметров орбиты из наблюдений».

Традиционные методы алгоритмизации состоят в том, что объекты предметной области представляются средствами систем управления базами данных (СУБД), операции программируются на языках программирования общего назначения в форме пакетов прикладных программ (ППП), а методы решения типовых задач, частично остаются в форме математических моделей вне компьютера, а частично встраиваются в СУБД и ППП. Традиционные методы алгоритмизации хороши тем, что опираются на универсальные средства, и могут быть применены в любом случае, в котором алгоритмизация вообще возможна. Недостатки традиционных методов также давно известны: процесс алгоритмизации является трудоемким, дорогим и результаты алгоритмизации трудно использовать повторно в других предметных областях.

[12] Эфемерида (от древнегреческого эфемерис — годный на один день) — в астрономии предвычисленное положение небесного тела на определённый момент времени.

3.1.1. Алгоритмизация предметных областей (4/5)

В настоящее время, наряду с традиционными методами, распространение получили и другие методы алгоритмизации, и, прежде всего, методы, основанные на применении языков предметной области и моделей предметных областей.

Замечание. Наряду с термином «язык предметной области» используются термины «предметно- и проблемно-ориентированный язык». Предметно-ориентированный язык ориентирован на данные конкретной предметной области, а проблемно-ориентированный язык ориентирован на методы обработки данных в ряде предметных областей. В большинстве случаев все три термина можно использовать как синонимы.

В данном контексте языки предметной области противопоставляются языкам программирования общего назначения по следующему признаку. Языки общего назначения *алгоритмически полны*, то есть универсальны, область их применения не ограничена, и в силу своей универсальности такие языки не используют абстракции, характерные для предметных областей и далекие от машинной реализации.

3.1.1. Алгоритмизация предметных областей (5/5)

Абсолютно жесткой границы между языками предметных областей и языками общего назначения провести невозможно, да это и не нужно. Специфические понятия предметных областей должны быть выражены в языке программирования общего назначения, то есть *запрограммированы*. От языков предметных областей, напротив, не требуется универсальности, поэтому в них применяются абстракции высокого уровня, прямо соответствующие понятиям предметной области, и затраты на программирование сокращаются, хотя, возможно, возрастают затраты на реализацию языка. В то же время выбранные абстракции конкретного языка, разработанного для одной предметной области, могут оказаться неприменимыми в другой предметной области. Поэтому область применения каждого конкретного языка предметной области *ограничена*.

3.1.2. Продуктивность прикладного программирования (1/9)

Технология программирования — это совокупность методов и средств, позволяющая наладить *производственный* процесс создания программного обеспечения.

Замечание. Сейчас иногда употребляют термин «программная инженерия», полученный переводом английского термина software engineering. Отечественный термин «технология программирования» появился существенно раньше и точнее передаёт суть дела.

Технология программирования имеет четко выделенный объект изучения — процессы разработки и сопровождения программного обеспечения, но, в настоящее время, не имеет единого метода и общепринятого способа построения. Технология программирования не является строгой математической дисциплиной, которую можно изложить последовательно, начиная с основополагающих понятий и применяя дедуктивные доказательства. Напротив, технология программирования является собранием разнородных и часто несогласованных друг с другом, но на практике доказавших свою полезность моделей, методик и средств. Поэтому методы разработки *прикладного* программного обеспечения в конкретных предметных областях уместно называть *технологией прикладного программирования*.

3.1.2. Продуктивность прикладного программирования (2/9)

При обсуждении технологии прикладного программирования необходимо учитывать три обстоятельства.

Независимость предметной области. Каждая конкретная предметная область существует независимо от прикладного программного обеспечения в этой области. Предметная область первична, а программное обеспечение вторично. Объективные характеристики предметной области определяют требования к программному обеспечению. Для алгоритмизации одной и той же предметной области может использоваться различное прикладное программное обеспечение, полученное из разных источников, в том числе и совместно.

Потребность в решении типовых задач. Существует объективно обусловленная потребность в решении типовых задач, причем решать их необходимо многократно. Кроме того, типовые задачи достаточно сложны и трудоемки, для того, чтобы алгоритмизация предметной области была экономически оправдана. У прикладного программного обеспечения заведомо существуют пользователи — специалисты по решению типовых задач данной предметной области.

3.1.2. Продуктивность прикладного программирования (3/9)

Продолжающаяся разработка и постоянное сопровождение. Каждая предметная область живет и развивается, изменяется состав и природа объектов предметной области, изменяются старые типовые задачи и возникают новые. Поэтому прикладное программное обеспечение непрерывно дорабатывается, причём, возможно, разными разработчиками, но с сохранением преемственности.

Отступление. Несмотря на то, что технология программирования начала развиваться одновременно с программированием, данная область знаний находится все ещё скорее в стадии становления, нежели в стадии разработанной инженерной дисциплины, поэтому даже ее структура далеко не устоялась. Различные исследователи, так же как и различные стандарты, принятые в этой области, используют несколько отличающиеся друг от друга структуры описания технологии программирования, что оставляет нам известную свободу в изложении материала. В настоящее время наиболее популярными являются модели CMM/CMMI Института технологии программирования (SEI) при университете Карнеги-Меллон (http://sei.cmu.edu).

3.1.2. Продуктивность прикладного программирования (4/9)

Технология прикладного программирования суть совокупность трех аспектов.

- *Модель процесса*, то есть порядок проведения типового проекта по разработке программного обеспечения. Сюда относятся понятия жизненного цикла программного обеспечения, определение модели процесса — выделение в нём фаз, вех, потоков работ и других составляющих. Характерным для данного аспекта является рассмотрение на уровне программирующей организации в целом.
- *Модель команды*, то есть отношения между людьми в процессе разработки. Сюда относятся определение обязанностей работников — участников процесса, регламенты их взаимодействия, рабочие процедуры, выделяемые роли и т. п. Характерным для данного аспекта является рассмотрение на уровне команды или проекта.
- *Дисциплина программирования*, то есть методы создания конкретных артефактов, входящих в состав программного обеспечения. Сюда относятся описание и применение образцов проектирования, стандарты кодирования, методы тестирования и отладки и т. д. Характерным для данного аспекта является рассмотрение на уровне отдельного работника.

3.1.2. Продуктивность прикладного программирования (5/9)

Все идеи, приемы и методы технологии программирования так или иначе направлены на решение следующих задач (одной или сразу нескольких):

- повышение управляемости команды,
- повышение качества программного продукта,
- повышение продуктивности процесса разработки.

Управляемость команды — это совокупность показателей, характеризующих способность команды выполнять работу в срок, управлять рисками, обеспечивать информационную безопасность и т. д.

Качество программного продукта — это степень соответствия программного продукта требованиям. Включает все характеристики продукта: функциональность, надежность, удобство использования и т. д.

Продуктивность процесса разработки — это объём произведенного программного продукта на одного работника в единицу времени, с учетом трудозатрат на тестирование, исправление ошибок и т. д.

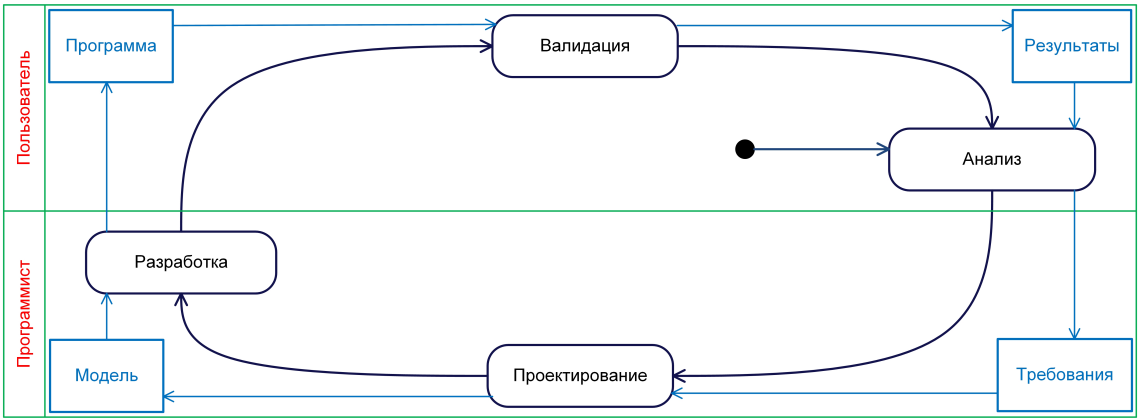
3.1.2. Продуктивность прикладного программирования (6/9)

Существует необозримое множество конкретных процессов разработки: практически в каждой программирующей организации, а иногда и в каждом проекте используется свой процесс разработки. В настоящее время можно выделить три класса моделей процесса разработки.

- *Линейные модели*. Требования определены в начале разработки, и выпуск программного обеспечения производится один раз в конце разработки.
- *Итерационные модели*. Требования также определены в начале разработки, но выпуск версий программного обеспечения производится многократно по ходу разработки.
- *Эволюционные модели*. Выпуск версий программного обеспечения производится многократно по ходу разработки, но требования не определены в начале разработки, они определяются по ходу разработки.

3.1.2. Продуктивность прикладного программирования (7/9)

В настоящее время модели процессов линейного класса применяются реже, предпочтение отдается итерационным и эволюционным моделям. Здесь приведена модель процесса разработки для случая простого итеративного процесса. При этом тестирование, как вид верификации, отнесено на фазу разработки.



3.1.2. Продуктивность прикладного программирования (8/9)

Как известно, два фактора решающим образом влияют на продуктивность прикладного программирования:

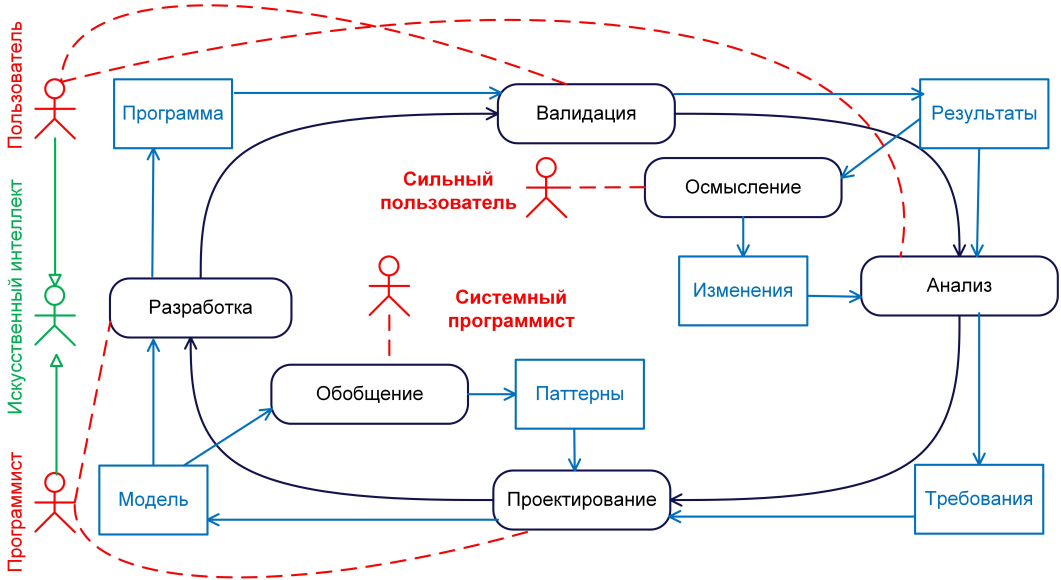
- сокращение объёма *внеплановых изменений* артефактов;
- увеличение объёма *повторно использованных* артефактов.

Внеплановые изменения возникают при исправлении ошибок. Чем позже обнаружена ошибка, тем дороже ее исправление. Ключевой фактор повышения продуктивности — своевременные *изменения* требований. Плодотворные изменения получаются при *осмыслении* результатов *сильным пользователем* (Power User).

Повторное использование артефактов (компонентов, модулей, классов) опирается на использование *образцов проектирования* (*паттернов*). Полезные паттерны появляются из опыта квалифицированных *системных программистов* (System Programmer).

Ясно, что скорость движения артефактов в *циклах повышения продуктивности* (см. следующий слайд), определяется языком предметной области, одинаково удобным как для постановки и решения типовой задачи, так и для спецификации свойств полученного решения.

3.1.2 Продуктивность прикладного программирования (9/9)



3.1.3. Языковая классификация предметных областей (1/2)

Для алгоритмизации предметной области необходимо решить три задачи:

1) определить *представление* объектов предметной области;
2) реализовать *операции* с этими объектами;
3) разработать методы решения *типовых задач*, как систем правил применения операций к объектам для получения значимого результата.

Здесь необходимо подчеркнуть одно тонкое, но важное при всякой алгоритмизации обстоятельство, которое часто упускается из виду. Поскольку предметная область первична, а её алгоритмизация вторична, выделение объектов, операций и типовых задач является *субъективным* и *неоднозначным*, а потому может оказаться как удачным, так и неудачным. Следовательно, конструирование языков предметных областей является деятельностью сугубо творческой и зависит как от *предметной области*, так и от *способа использования* языка предметной области.

Замечание. Сказанное относится только к прикладному программированию. Другие виды программирования, такие как системное (операционные системы), вредоносное (вирусы), развлекательное (компьютерные игры), спортивное программирование и так далее здесь не рассматриваются, хотя имеют свои специфические приёмы.

3.1.3. Языковая классификация предметных областей (2/2)

С точки зрения *требований* к языку предметной области необходимо принимать во внимание несколько независимых градаций предметных областей.

Степень ответственности и уверенности в программном обеспечении. Лестница: однократный запуск → демонстрация примеров → исчерпывающее тестирование → верификация на модели (model checking) → математическое доказательство.

Пример. Игровая приставка флайт-симулятор → профессиональный тренажёр → программа управления автономного БПЛА → авионика пассажирского авиалайнера.

Время жизни и тиражность программного обеспечения. Лестница: одноразовая программа → для себя на некоторый период → внутренний инструмент команды проекта → рыночный программный продукт → общественно-значимое программное обеспечение.

Пример. Проверка платёжки за электроэнергию → ассистент «Мои финансы» → программа расчёта зарплаты → платежная система «МИР».

Мотивировация и квалификация пользователей языка. Лестница: свой личный язык → команда проекта → профессиональное сообщество → международный стандарт.

Пример. Набор макросов в документе → Стилевой файл журнала → TeX.

3.1.4. Предметная классификация языков (1/4)

Решение первых двух задач алгоритмизации в настоящее время обеспечено технологиями систем управления базами данных (СУБД) и пакетов прикладных программ (ППП). Технологии представления объектов предметной области в СУБД и операций в библиотеках хорошо определены, устоялись и не здесь обсуждаются. Нерешённые вопросы связаны с представлением знаний о методах решения типовых задач. Заметим, что, в любом случае, решение типовых задач предметной области опирается на готовые программы и данные, которым пользователь *доверяет*.

Совокупность взаимосвязанных программ и данных, для которых проведена валидация, то есть которой доверяют пользователи, и на основе которой выполняются дальнейшие шаги по алгоритмизации, называется *доверительным программным фондом* (ДПФ). Если ДПФ имеется в наличии, то нет причин не использовать этот ресурс при конструировании языка предметной области.

3.1.4. Предметная классификация языков (2/4)

Для достижения целей алгоритмизации очень важно определить подходящую категорию языка предметной области. Выделяются три категории языков по типу взаимодействия с пользователем.

- *Командные предметно-ориентированные языки*. Языки этой категории, как правило, опираются на развитый ДПФ. Характерным для этой категории является то, что пользователь последовательно, по одной, дает системе команды на выполнение тех или иных действий, причём состав и порядок действий, которые требуются для решения некоторой задачи, пользователь определяет сам по ходу дела, используя как результаты выполнения предыдущих команд, так и другие трудно формализуемые факторы, и не сообщает системе заранее в виде «программы». Каждая команда имеет строго определенное функциональное назначение и условия применения.
  Пример. Команды меню графического интерфейса пользователя любого офисного приложения образуют типичный командный язык.

3.1.4. Предметная классификация языков (3/4)

- *Императивные (в том числе графические) языки*. Языки этой категории примечательны тем, что (сильный) пользователь описывает решаемую задачу в целом, предъявляя совокупность требуемых шагов не по отдельности, а во взаимосвязи, часто в виде графической схемы или диаграммы. Система осуществляет решение задачи, опираясь на ДПФ. Также как и в языках первой категории, способ решения задачи определяет пользователь, но в этом случае он в явном виде предъявляет «программу» системе.

  Пример. Язык описания бизнес процессов (BPMN).

- *Декларативные языки предметной области*. Языки этой категории появляются в том случае, когда алгоритмизация предметной области достигает уровня, при котором возможно (автоматическое) построение алгоритма решения типовых задач предметной области и внесение этого алгоритма в систему. Пользователь только ставит задачу, а система сама строит программу её решения, подбирает подходящие шаги и сама выполняет их.

  Пример. Надстройка «Поиск решения» в Excel.

3.1.4. Предметная классификация языков (4/4)

Языки первой категории не увеличивают степень алгоритмизации предметной области по сравнению с используемым ДПФ. Командные языки должны опираться на развитый ДПФ, в противном случае команды оказываются слишком мелкими и их становится слишком много.

Языки второй категории повышают степень алгоритмизации за счет наглядности и автоматизации процесса выполнения последовательности модулей. Императивные языки хороши в тех случаях, когда типовые задачи предметной области допускают явное алгоритмическое решение.

Языки третьей категории дают наибольшую степень алгоритмизации за счет своей выразительной силы. Для построения декларативного языка необходима *модель предметной области* — явное описание сущностей предметной области и отношений между ними, как правило в форме *онтологии* или *графа знаний* или *семантической сети* или чего-то подобного.

3.1.5. Модель использования и архитектура (1/8)

Любой язык программирования, в частности, язык предметной области, возможно использовать различными способами, причём разнообразие *вариантов использования* может быть значительным. Среди них базовым и основополагающим является вариант использования *решить типовую задачу*.

Чтобы решить задачу, необходимо иметь *программу* решения задачи, которую можно *выполнить*. Программа — это не обязательно текст на языке Си, это может быть и последовательность нажатий кнопок в графическом интерфейса пользователя, и некоторое шевеление пальцами над экраном смартфона, и голосовые команды для ИИ.

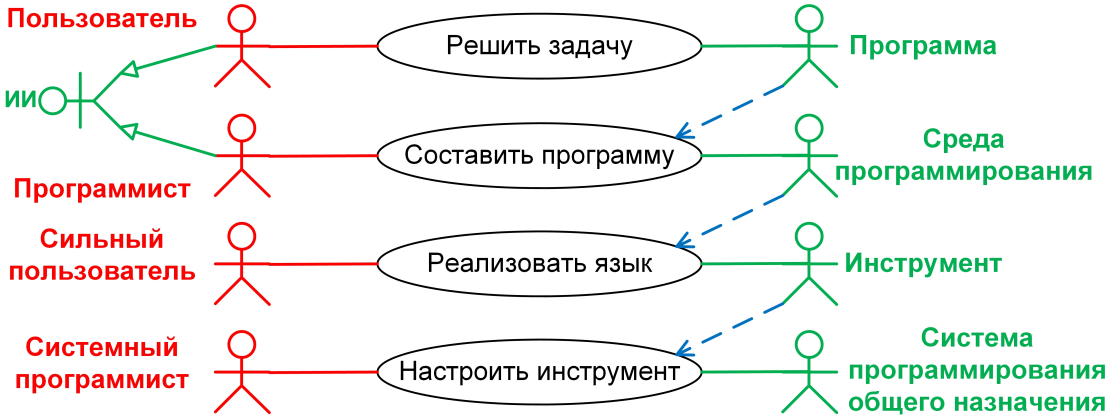
Программы составляются с помощью *среды программирования*, как правило, специфичной для каждого вида программ и для каждого языка предметной области.

Среда программирования на каком-либо языке с необходимостью требует детальной *реализации* этого языка. Задача определения и реализации языка не является простой (см. разделы 3.2 и 3.3) и требует наличия адекватного *инструмента*.

Эта последовательность уточнений не является бесконечной, к счастью — рано или поздно мы доходим до того, что уже есть в наличии, например, до реализованного языка программирования общего назначения.

3.1.5. Модель использования и архитектура (2/8)

Варианты использования, связанные с решением типовой задачи.

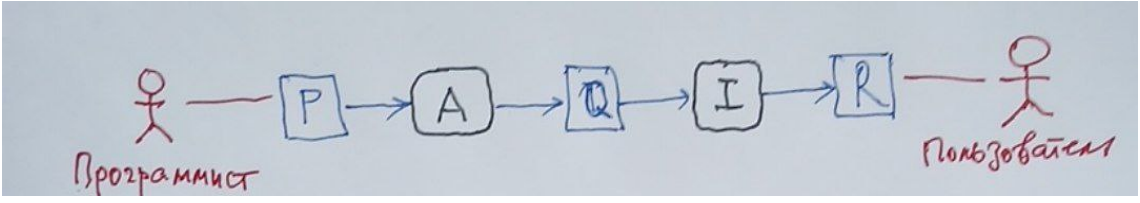


Варианты использования косвенно зависят друг от друга, но при этом могут быть реализованы в рамках различных программных систем, не связанных прямо во время выполнения.

3.1.5. Модель использования и архитектура (3/8)

Язык предметной области уместно называть *реализованным*, если хотя бы *некоторые* программы P на языке возможно *выполнить* и получить результат R.

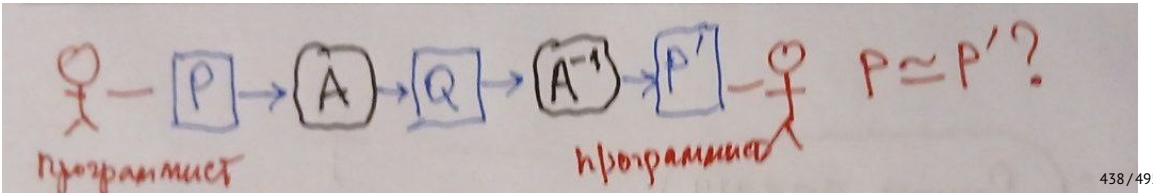
В простейших случаях программы удаётся выполнять постепенно, «по шагам», не анализируя всю программу сразу. Так, например, устроены калькуляторы. Но в более сложных случаях, чтобы выполнить программу, необходимо сначала проанализировать всю программу (или её существенную часть), после чего уже возможно выполнение. Так получается типовая схема работы основного варианта использования реализованного языка: анализатор A разбирает программу P и строит некоторое *внутреннее* или *интерпретируемое* представление Q, а затем интерпретатор I по внутреннему представлению вычисляет результат R.



3.1.5. Модель использования и архитектура (4/8)

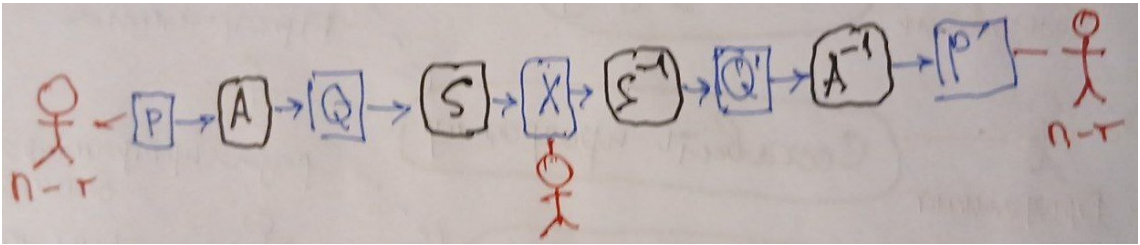
Существует и используется на практике великое множество вариаций приведённой простейшей схемы, отличающиеся тем, как выбрать внутреннее представление, какие выполняемые функции и какие объекты включены в схему, как распределить выполняемые функции между *временем компиляции* и *временем выполнения*, на сколько этапов разбито выполнение каждой функции, возможна или нет сериализация промежуточных результатов.

Пример. Одним из самых полезных приёмов отладки описания синтаксиса любого текстового языка является *ретранслятор* (recompiler). Идея состоит в том, чтобы, наряду с проверяемым анализатором A, написать программу-ретранслятор A^{-1}, который переводит внутреннее представление снова в текст программы P'. Если окажется, что $P \sim P'$, то анализатор, скорее всего, работает правильно.



3.1.5. Модель использования и архитектура (5/8)

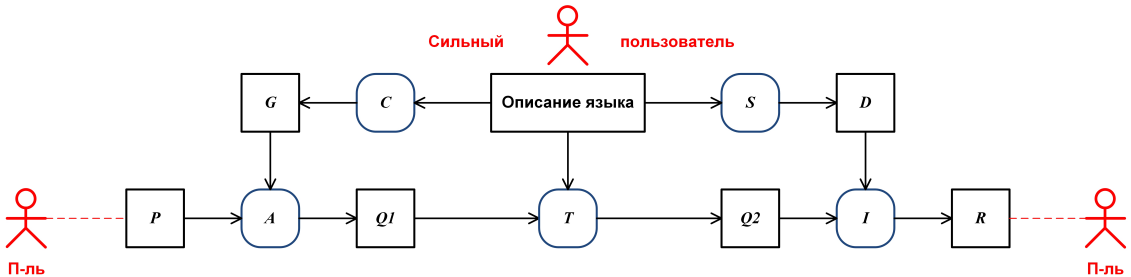
Пример. Можно также использовать сериализацию S в какой-либо удобный человеко-читаемый формат X наподобие JSON или XML. Тогда можно проконтролировать правильность внутреннего представления вручную и затем с помощью десериализатора S^{-1} проверить работу ретранслятора A^{-1}.



Очень важно обратить внимание на следующее обстоятельство. Если в рассматриваемых схемах участвующие алгоритмы *однопроходные*, то есть последовательно читают свой вход и строят свой выход, никогда не возвращаясь назад, то можно использовать *конвейерную архитектуру* (pipe), запуская алгоритмы параллельно в разных потоках, что позволяет кратно наращивать производительность, если это необходимо.

3.1.5. Модель использования и архитектура (6/8)

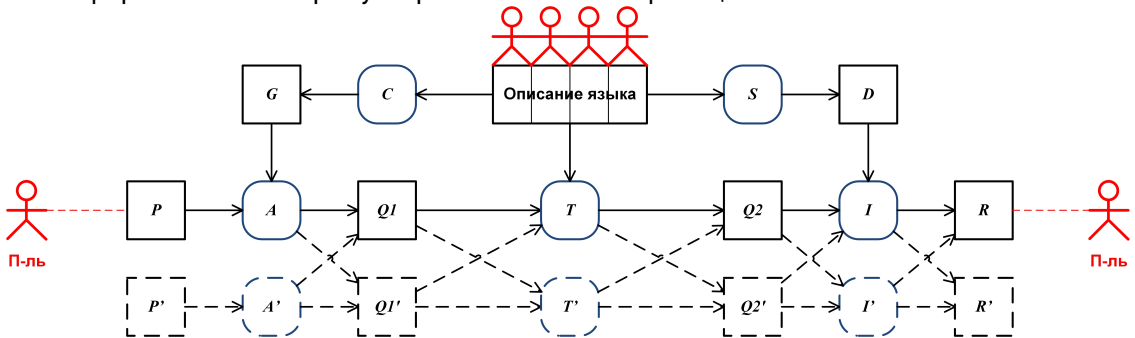
В современных реализациях анализатор и интерпретатор обычно не являются жёстко закодированными программами, а представляют собой более или менее универсальные процессоры, использующие при работе подаваемые извне описания синтаксиса и семантики языка.



Практика показывает, что всевозможные трансформации, например, оптимизации, удобнее проводить на уровне внутреннего представления, а не на уровне входного кода и не на уровне результатов. Так возникают несколько представлений и трансформация T .

3.1.5. Модель использования и архитектура (7/8)

Описание языка неоднородно. Различные части описания предполагают использование разных формализмов и требуют различной квалификации.



Очень часто один и тот же язык включает разные диалекты, ориентированные на различные аспекты предметной области, и предполагает получение разнородных результатов при решении различных типовых задач. Поэтому реализация языка предметной области включает несколько специализированных модулей для гибкого комбинирования возможностей.

3.1.5. Модель использования и архитектура (8/8)

Пример. Наиболее популярный язык запросов SQL (Structured Query Language) для работы с реляционными базами данных включает в себя несколько подъязыков. Как минимум, это язык определения данных (Data Definition Language, DDL) и язык манипулирования данными (Data Manipulation Language, DML), а кроме того ещё несколько возможных расширений. Более того, почти каждый крупный производитель СУБД поддерживает свой собственный диалект SQL, и все эти языки постоянно обновляются. При этом SQL стандартизован ANSI (American National Standards Institute) и ISO (International Organization for Standardization).

Отступление. Поясним сказанное аналогией. Допустим, есть желание собрать некоторую, возможно, игрушечную, конструкцию из пластика (= реализовать некоторый язык предметной области). Тогда можно поступить двояко. Можно напечатать конструкцию на 3-D принтере (= закодировать монолитную программу на языке программирования), или можно собрать примерно то, что хочется, из кубиков LEGO (= сконструировать свой конвейер из стандартных алгоритмов). В первом случае конструкция будет крепче и точнее отвечать требованиям. Но если требования изменятся, то напечатанную конструкцию придётся выкинуть целиком, а конструкцию из кубиков LEGO можно разобрать и собрать по-другому.

3.2. Методы описания языков

Мы рассматриваем исключительно описание *языков предметной области*, применяемых в компьютерных технологиях.

Прежде всего, заметим, что языки программирования, в отличии от естественных языков, создаются конкретными людьми с определённой целью, и авторы языков могут придать им нужные свойства. Поэтому для описания *синтаксиса* языков программирования не применяются грамматики типов $\mathfrak{T}_0$ и $\mathfrak{T}_1$ (все алгоритмы работы с такими грамматиками слишком неэффективны), а применяются более простые грамматики типов $\mathfrak{T}_2$ и $\mathfrak{T}_3$. Но и в этих простых случаях выделяются ещё более простые подклассы, чтобы облегчить компьютерную обработку описаний таких языков.

Далее, заметим, что хотя языки предметных областей являются формальными языками, но сами предметные области редко имеют готовую формализацию. Поэтому формальные методы определения *семантики* часто оказываются неприменимы и тогда приходится использовать описания ad hoc.

Наконец, *прагматика*, будучи важнейшей частью описания языка с практической точки зрения, вовсе не имеет развитых формальных методов описания.

3.2.1. Структура описания языка программирования (1/4)

В лингвистике для естественных языков используется довольно сложная иерархия описаний. Даже если ограничиться только письменным языком (текстом), оставив в стороне устную речь, мы видим, что правила для образования слов (морфология), совсем не похожи на правила составления предложений (синтаксис). Если же принять во внимание иероглифическую письменность, или правила произношения, интонации и прочее, то видимая способность лингвистов справиться с задачей вызывает уважение.

В программировании при описании раличных языков используются самые разнообразные методы. Здесь принято выделять шесть частей описания языка: алфавит, лексика, синтаксис, контекстные условия, семантика и прагматика. Это деление субъективно, но оно хорошо работает в простых случаях и оставляет место для применения других методов в более сложных случаях.

[1.] *Алфавит* — описание множества допустимых символов языка. Как правило, это просто конечное множество символов, которые можно явно перечислить, но иногда встречаются и особенности. В некоторых языках программирования и разметки текста встречаются конструкции, в которых могут участвовать *любые* символы, а не только те, которые перечислены как основные.

3.2.1. Структура описания языка программирования (2/4)

Пример. Таковы комментарии и строковые константы в некоторых языках или конструкция CDATA в XML.

Отступление. В современном языкотворчестве наблюдается тенденция к усложнению алфавитов и повышению их роли в описании языка. В частности, кодировки семества Unicode позволяют использовать намного более представительные множетсва символов, нежели коды ASCII. Другое направление — *атрибутированные символы*. Действительно, такие атрибуты, как цвет, размер шрифта, начертание, гарнитура и т. д. несут определённую информацию и могут использоваться как части языка.

[2.] *Лексика* — описание множества элементарных конструкций, называемых *лексическими единицами* (или *лексемами*). Отличительной особенностью лексем является то, что они не содержат в своем составе других лексем, т. е. являются «атомами», то есть терминалами с точки зрения синтаксиса. Для описания лексики в настоящее время обычно используются автоматные грамматики (см. п. 1.1.6) и регулярные выражения (что эквивалентно, см. п. 1.2.2).

Пример. Изображения чисел, строковые константы, знаки операций, имена и ключевые слова — типичные лексемы современных языков программирования.

3.2.1. Структура описания языка программирования (3/4)

[3.] *Синтаксис* — описание правил, по которым из лексем строятся предложения (или *операторы*) языка, а из операторов строятся программы. Как правило, для описания синтаксиса языков программирования используются контекстно-свободные порождающие грамматики в одной из описанных выше форм. При этом некоторые языки устроены так, что даже отдельные операторы допускают немедленное выполнение, такие языки называются *интерпретируемыми* (или *скриптовыми*). Другие же языки требуют наличия определенной конструкции для того, чтобы программа могла считаться законченной.

Пример. Текст программы на языке VBScript может состоять из одного оператора присваивания или вывода сообщения. Программа на языке Си должна содержать описание функции с именем main для того, чтобы быть выполнимой.

[4.] *Контекстные условия* — описание синтаксических правил, которые невозможно или неудобно выразить средствами контекстно-свободной грамматики. Некоторые способы описания контекстных условий обсуждаются в п. 1.5.4 и п. 3.2.5.

Пример. Фактические аргументы в операторе вызова процедуры должны соответствовать по количеству, порядку и типам формальным параметрам в описании этой процедуры.

3.2.1. Структура описания языка программирования (4/4)

[5.] *Семантика* — описание правил приписывания смысла синтаксически правильным конструкциям языка. Для языков программирования смыслом программ является их выполнение. В настоящее время формальные методы описания семантики языков программирования разработаны недостаточно, хотя постоянные усилия в этом направлении предпринимаются. Как правило, семантика описывается либо на естественном языке, либо в терминах работы какой-то абстрактной вычислительной машины.

Пример. Семантика языка Java описана в терминах абстрактной машины — JVM (Java Virtual Machine).

[6.] *Прагматика* — описание практически значимых особенностей языка, не выраженных средствами описания синтаксиса и семантики. Yапример: производительность, лаконичность, наличие сообщества носителей языка, наличие стандартов и тому подобное. Для разных языков специфичны различные аспекты прагматики.

Пример. Для языка Forth объём памяти для хранения программы во внутреннем представлении (*шитый код*) меньше, чем объём самой компактной эквивалентной программы на машинном языке. Это прагматическое свойство очень ценно для встроенных и бортовых систем.

3.2.2. Средства описания лексики (1/3)

3.2.3. Средства описания синтаксиса (1/3)

3.2.3. Средства описания контестных условий (1/3)

3.2.2. Нормальные формы Бэкуса–Наура (1/3)

Почти одновременно с появлением и развитием теории формальных грамматик для практических целей был предложен целый ряд других средств. Исторически самым заметным явилось описание языка программирования Алгол-60 с помощью *нормальных форм Бэкуса-Наура* (или *металингвистических формул*).

Описание языка с помощью нормальных форм Бэкуса–Наура состоит из набора *основных символов* языка, набора названий конструкций (или *металингвистических переменных*) и набора *металингвистических формул*. Каждая металингвистическая формула состоит из левой части, в которой стоит металингвистическая переменная, *металингвистической связки* $::=$ (читается «это есть») и правой части, в которой указывается один или несколько вариантов построения конструкции в левой части. Каждый вариант — это последовательность металингвистических переменных и основных символов. Варианты разделяются символом $|$ (читается «или»). Металингвистические переменные заключаются в угловые скобки, чтобы не путать их с основными символами. Считается, что метасимволы $<, >, |$ и $::=$ не входят в число основных символов.

3.2.2. Нормальные формы Бэкуса–Наура (2/3)

Пример. Язык целых десятичных чисел со знаком в обозначениях Бэкуса-Наура можно описать следующими металингвистическими формулами:

<Целое со знаком> ::= <Целое без знака> | $+$ <Целое без знака> | $-$ <Целое без знака>
<Целое без знака> ::= 0 | <Цифра не ноль> <Последовательность цифр>
<Последовательность цифр> ::= | <Последовательность цифр> <Цифра>
<Цифра> ::= 0 | <Цифра не ноль>
<Цифра не ноль> ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Совершенно ясно, что нормальные формы Бэкуса-Наура — это один из способов записать контекстно-свободную грамматику, то есть грамматику типа 2. Действительно, основные символы — это алфавит терминальных символов T, металингвистические переменные — это алфавит нетерминальных символов N, металингвистические формулы — это множество правил вывода R, в которых вместо символа $\to$ использована связка ::=, а аксиома грамматики обычно ясна из контекста.

Замечание. Форма Бэкуса–Наура была предназначена, по замыслу авторов, прежде всего для точного и в то же время удобочитаемого описания синтаксиса языков программирования, поэтому используются развёрнутые названия конструкций.

3.2.2. Нормальные формы Бэкуса–Наура (3/3)

Отступление. Нормальные формы Бэкуса–Наура оказали большое влияние на развитие техники определения искусственных языков, в частности, языков программирования. Некоторое время эта форма описания синтаксиса признавалась в качестве стандарта и использовалась как авторами языков, так и разработчиками трансляторов. Если бы дело ограничилось сравнительно немногочисленными языками программирования общего назначения, то металингвистических формул было бы достаточно. Но для специализированных языков предметных областей формы Бэкуса–Наура оказались недостаточно удобны.

Требуются более лаконичные, более наглядные, более современные средства, поскольку языков предметных областей в сотни раз больше, время их жизни в десятки раз меньше, а пользователями описания языка являются специалисты предметной области, не знающие и не нуждающиеся в знаниях тонкостей синтаксического анализа и компиляции.

Некоторые из таких средств рассматриваются в следующих параграфах.

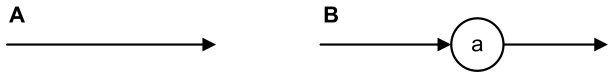
TODO Разные примеры и замечания, есть заготовки! 1) Федоров Лебедь, шахматная нотация. 2) Раскрутка = Лавриченко / Кононов = Плохо сделано Смирнова Дарья ???

3.2.3. Синтаксические диаграммы Вирта (1/5)

Синтаксические диаграммы — это особый вид орграфов, предназначенный для записи контекстно-свободных грамматик в графической форме. Синтаксические диаграммы были предложены Никлаусом Виртом при определении языка Паскаль.

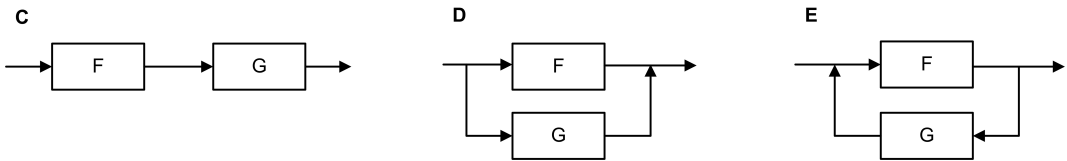
Фактически отдельная синтаксическая диаграмма — это сеть с одним источником и одним стоком, которые называются вход и выход, соответственно. По традиции вход и выход не изображают на диаграмме как узлы, а рисуют приходящую «из ниоткуда» входную стрелку и уходящую «в никуда» выходную стрелку. Узлы на синтаксической диаграмме изображают двух типов: с прямыми углами (они соответствуют нетерминалам) и со скругленными углами (они соответствуют терминалам). Диаграммы имеют имя, оно обычно пишется в левом верхнем углу диаграммы.

Пример. На рисунке приведены две диаграммы, **A** и **B**. Диаграмма **A** — это *тривиальная* диаграмма, она имеет только вход, выход и соединяющую их дугу, которую мы видим на диаграмме. Диаграмма **B** — это *элементарная* диаграмма, она имеет один узел, помеченный буквой «a» и входную и выходную дуги.



3.2.3. Синтаксические диаграммы Вирта (2/5)

Над синтаксическими диаграммами определены операции последовательного соединения (рисунок ниже, диаграмма **C**), параллельного соединения (диаграмма **D**) и итерации (диаграмма **E**), которые позволяют строить более сложные диаграммы из более простых. На рисунке F и G — это *любые* диаграммы, в том числе тривиальные.



Язык в целом описывается совокупностью синтаксических диаграмм. Каждая конструкция языка (нетерминальный символ, металингвистическая переменная) определяется на отдельной диаграмме (и записывается в качестве названия диаграммы). При этом диаграмма определяет нетерминал в следующем смысле. Каждый путь на синтаксической диаграмме, ведущий от входа к выходу — это цепочка, выводимая из данного нетерминала.

3.2.3. Синтаксические диаграммы Вирта (3/5)

TODO 1) решить вопрос о нотации – начало конец???
2) Денисов / Туников – текст, картинки и ТЕОРЕМА
3) ссылка на графы-источники – кто???
4) Жуков / Сергеев – Шахматная нотация

Пример. На рис. **??** представлена пара синтаксических диаграмм, определяющих целые числа со знаком.

Рис.: Синтаксические диаграммы для целых со знаком

Говорят, что совокупность синтаксических диаграмм *правильно построена*, если для любого нетерминала есть ровно одна синтаксическая диаграмма и все синтаксические диаграммы построены из тривиальных и элементарных с помощью операций последовательного и параллельного соединения и итерации.

3.2.3. Синтаксические диаграммы Вирта (4/5)

Теорема. *Для любой правильно построенной совокупности синтаксических диаграмм существует эквивалентная контекстно-свободная грамматика.*

Доказательство. Рассмотрим синтаксическую диаграмму для некоторого нетерминала A. Построим совокупность контекстно-свободных правил, так что каждому пути в диаграмме будет соответствовать вывод из A по данным правилам и обратно. Для этого введем необходимое количество новых нетерминалов, соответствующих структурным элементам диаграммы, построенной по правилам, приведённым на предыдущих слайдах, и добавим правила в соответствии со следующей таблицей.

Диаграмма	Рисунок	Правила
Тривиальная	A	$A \to \varepsilon$
Элементарная с терминалом a	B	$B \to a$
Элементарная с нетерминалом A'	нет	$B \to A'$
Последовательное соединение D_1 и D_2	C	$C \to D_1 D_2$
Параллельное соединение D_1 и D_2	D	$D \to D_1 \mid D_2$
Итерация D_1 и D_2	E	$E \to D_1 \mid E D_2 D_1$

Правила в правом столбце контекстно-свободные.

3.2.3. Синтаксические диаграммы Вирта (5/5)

Доказательство. (Окончание) Далее, по всякому пути на диаграмме можно построить вывод цепочки, прочитываемой вдоль пути. Вывод строится в обратном порядке: сначала мы имеем путь и заменяем в нем прохождение пустых ребер на нетерминалы по ε-правилам (см. первую строку таблицы), затем заменяем элементарные диаграммы на нетерминалы, затем пары нетерминалов в соответствии с операциями и так далее, пока не получим один нетерминал — название диаграммы.

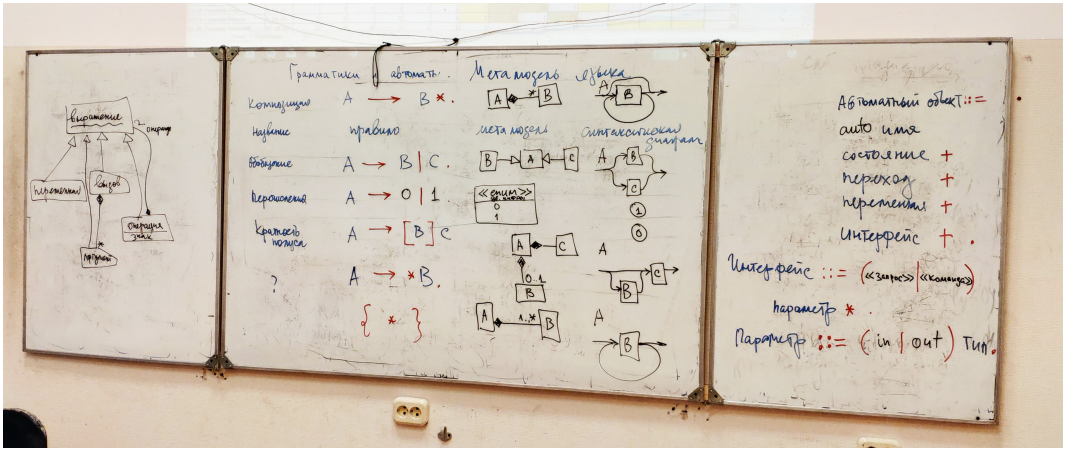
Обратно, по всякому выводу в построенном множестве правил, который заканчивается цепочкой, состоящей из терминалов и нетерминалов, присутствующих на диаграмме в виде узлов, можно указать путь, соответствующий данному выводу.

Построение правил (введение дополнительных нетерминалов) производится для каждой диаграммы независимо (все новые нетерминалы различны). Поэтому выводу в грамматике в целом соответствует путь в иерархии диаграмм и обратно. □

Отступление. Синтаксические диаграммы полезно сопоставить с источниками (см. п. 1.2.3). И в том и в другом случае для описания языка используется граф специального вида, а язык определяется как множество некоторых путей в таком графе. Но в источниках «лингвистическая» нагрузка приходится на ребра, а в синтаксических диаграммах — на вершины.

3.2.4. Метамодель языка

TODO Используемые средства UML. Кто??? Где???



TODO Денисов / Туников пересказ автоматного метода Новиков / Тихонова трудно, но ОЧЕНЬ ОЧЕНЬ полезно. КТО ???

3.2.5. Контекстные условия (1/3)

Реальные языки программирования не являются контекстно-свободными. Многие важные правила реальных языков не могут быть описаны в форме контекстно-свободных правил (хотя могут быть описаны в форме контекстно-зависимых правил).

Пример. Во многих современных языках различаются конструкции, в которых имена определяются (они называются описаниями или объявлениями), и конструкции, в которых имена используются (например, операторы присваивания). Вхождение имени (идентификатора) в конструкцию первого рода называется *определяющим*, в конструкцию второго рода — *использующим*. Требование, чтобы каждому использующему вхождению имени в текст программы однозначно соответствовало определяющее вхождение, является контекстным условием.

Какие правила языков программирования следует отнести к контекстным условиям?

Поскольку контекстные условия относятся к синтаксису, хотелось бы иметь возможность проверять их *статически*, по тексту программы, не выполняя её. Однако некоторые условия статически проверить очень трудно, или невозможно, в то время как *динамически*, во время выполнения программы, они проверяются элементарно.

3.2.5. Контекстные условия (2/3)

Пример. Статически, без выполнения программы, невозможно проверить соответствие фактических аргументов и формальных параметров процедуры или функции, если само определение процедуры или функции выполняется динамически, в процессе выполнения программы и потому неизвестно до выполнения программы. В то же время динамически, в момент вызова, такая проверка не вызывает затруднений, поскольку в этот момент определение процедуры уже известно.

С этой точки зрения свойства программ на языке программирования принято делить на синтаксические и семантические. *Синтаксические* свойства программы связаны с её формой, то есть со статическим текстом программы. *Семантические* свойства программы связаны с её содержанием, то есть с динамическим процессом выполнения программы.

Пример. Хорошо известно различение синтаксических и семантических ошибок в программах. Программы часто бываю синтаксически безупречны, но семантически бессмысленны.

Общепринятое однозначное деление свойств любого языка программирования на синтаксические и семантические вряд ли возможно. Слишком многое зависит от конкретных деталей каждого языка.

3.2.5. Контекстные условия (3/3)

По традиции к контекстным условиям относят, в частности, *проверку типов* и *проверку области действия*.

Примеры. Типичные контекстные условия современных языков программирования:

1. Тип значения в правой части оператора присваивания должен быть совместимым с типом переменной в левой части оператора присваивания.
2. Закрытый метод класса может быть вызван только в теле метода того же класса.

TODO Статические и динамические проверки, ошибки времени анализа, ошибки времени компиляции, ошибки времени выполнения. Административная система времени выполнения для языка программирования. Антонов Алексей

3.2.6. Грамматики ван Вейнгаардена (1/10)

TODO Слишком много слайдов — надобно поделить. Кто???

При определении языка Algol-68 был предложен метод, который позволяет формально специфицировать контекстные условия указанных видов.

Грамматика ван Вейнгаардена $G_V = \langle G_M, G_S \rangle$ — это *две* взаимосвязанных грамматики, *метаграмматика* (или *грамматика метаязыка*) $G_M = \langle N_M, T_M, R_M \rangle$ и *строгая грамматика* (или *грамматика строгого языка*) $G_S = \langle F, T_S, R_S, I \rangle$. Здесь

N_M — конечный алфавит нетерминальных символов метаграмматики,

T_M — конечный алфавит терминальных символов метаграмматики,

R_M — конечное множество контекстно-свободных правил метаграмматики, начальный символ метаграмматики не задается.

Таким образом, метаграмматика описывает $|N_M|$ контекстно-свободных грамматик, которые получаются, если в качестве аксиомы брать различные символы из N_M. Далее

F — конечное множество непустых цепочек терминальных и нетерминальных символов метаграмматики, которые называются *формами*, $F \subset (N_M \cup T_M)^*$, $\varepsilon \notin F$,

T_S — конечный алфавит терминальных символов строгой грамматики,

3.2.6. Грамматики ван Вейнгаардена (2/10)

R_S — конечное множество *схем правил* строгой грамматики, каждая схема имеет вид

$$\beta \to \beta_1, \ldots, \beta_n,$$

где β — некоторая форма, $\beta \in F$, β_i — форма или терминальный символ, $\beta_i \in F \vee \beta_i \in T_S$, а запятые в правой части правила являются метасимволами, разделяющими формы и терминальные символы строгой грамматики, и, наконец, I — некоторая форма, не содержащая нетерминальных символов метаграмматики.

Смысл введенных понятий состоит в следующем. Каждая форма является обобщенным представлением некоторого подмножества (возможно, бесконечного) нетерминалов строгой грамматики. Сами нетерминалы строгой грамматики записываются в виде непустых цепочек терминалов метаграмматики. Форма описывает множество нетерминалов строгой грамматики, которые можно вывести из формы, используя правила метаграмматики, причём все вхождения каждого нетерминального символа в форму нужно заменять на одно и то же терминальное порождение. Если форма не содержит нетерминалов из N_M, то она определяет только один нетерминал строгой грамматики (как I).

3.2.6. Грамматики ван Вейнгаардена (3/10)

Пример. Пусть $A \in N_M$, $b, c, d \in T_M$, $A \Rightarrow^*_{G_M} \alpha_1$ и $A \Rightarrow^*_{G_M} \alpha_2$, где $\alpha_1, \alpha_2 \in T^*_M$. Пусть форма $\beta \in F$ содержит два вхождения нетерминала A, $\beta = aAbAc$. Тогда цепочки $a\alpha_1 b\alpha_1 c$ и $a\alpha_2 b\alpha_2 c$ являются нетерминальными символами строгой грамматики, а цепочки $a\alpha_1 b\alpha_2 c$ и $a\alpha_2 b\alpha_1 c$ — не являются.

Далее, правила строгой грамматики получаются из схем правил подстановкой вместо форм нетерминальных символов строгой грамматики, причём нетерминальные символы должны быть *согласованными*, то есть вместо нетерминальных символов метаграмматики должны быть подставлены одни и те же терминальные порождения.

Пример. Пусть $A \in N_M$, $b, c, d, e \in T_M$, $A \Rightarrow^*_{G_M} \alpha_1$ и $A \Rightarrow^*_{G_M} \alpha_2$, где $\alpha_1, \alpha_2 \in T^*_M$. Пусть в некоторой схеме правил используются две формы: aAb и cAd. Тогда нетерминальные символы $a\alpha_1 b$ и $c\alpha_1 d$ являются согласованными, а символы $a\alpha_1 b$ и $c\alpha_2 d$ — не являются.

Язык, порождаемый грамматикой ван Вейнгаардена — это множество терминальных цепочек строгой грамматики, выводимых из начальной формы по правилам, полученным из схем правил.

3.2.6. Грамматики ван Вейнгаардена (4/10)

Пример. Язык $a^n b^n c^n$ порождается грамматикой ван Вейнгаардена $G_V = \langle G_M, G_S \rangle$, где метаграмматика $G_M = \langle N_M, T_M, R_M \rangle$, $N_M = \{A\}$, $T_M = \{\bar{a}, \bar{b}, \bar{c}, \bar{d}, \bar{e}\}$, $R_M = \{A \to \bar{d}A, A \to \varepsilon\}$. Таким образом, метаграмматика порождает цепочки вида $\bar{d}^n$. Строгая грамматика $G_S = \langle F, T_S, R_S, I \rangle$, $F = \{\bar{e}, \bar{a}, \bar{b}, \bar{c}, \bar{a}A, \bar{b}A, \bar{c}A, \bar{a}A\bar{d}, \bar{b}A\bar{d}, \bar{c}A\bar{d}\}$, $T_S = \{a, b, c\}$, $I = \bar{e}$, в R_S входят следующие схемы: 1) $\bar{e} \to \bar{a}A, \bar{b}A, \bar{c}A$; 2)$\bar{a}A\bar{d} \to \bar{a}A, a$; 3) $\bar{b}A\bar{d} \to \bar{b}A, b$; 4) $\bar{c} \to \bar{c}A, c$; 5) $\bar{a} \to \varepsilon$; 6) $\bar{b} \to \varepsilon$; 7) $\bar{c} \to \varepsilon$. Таким образом, из схемы 1 можно получить (учитывая согласованность!) правило вида

$$\bar{e} \to \bar{a}\bar{d}^n\bar{b}\bar{d}^n\bar{c}\bar{d}^n,$$

применением которого будет начинаться всякий вывод в строгой грамматике, а из схем 2–4 получаются правила вида

$$\bar{a}\bar{d}^n\bar{d} \to \bar{a}\bar{d}^n a, \quad \bar{b}\bar{d}^n\bar{d} \to \bar{b}\bar{d}^n b, \quad \bar{c}\bar{d}^n\bar{d} \to \bar{c}\bar{d}^n c,$$

которые породят нужное количество терминалов, «отщепляя» по одному символу $\bar{d}$ и, наконец, с помощью ε-правил 5–7 вывод будет закончен.

3.2.6. Грамматики ван Вейнгаардена (5/10)

Сравнение приведённого примера с примером **??** индуцирует естественный вопрос: какова же порождающая мощность грамматик ван Вейнгаардена? Ответ оказывается довольно неожиданным: хотя весь «механизм» грамматик ван Вейнгаардена контекстно-свободный, их порождающая мощность предельно велика, то есть они позволяют описать любое перечислимое множество слов.

Теорема. *Для любой порождающей грамматики существует эквивалентная грамматика ван Вейнгаардена.*

Доказательство. Пусть задана грамматика $G = \langle N, T, R, S\rangle$. Для терминальных и нетерминальных символов этой грамматики введем множества «двойников», $\bar{N}, \bar{T}$, элементы которых также будем надчеркивать. Построим грамматику ван Вейнгаардена $G_V = \langle G_M, G_S\rangle$ следующим образом. Метаграмматика $G_M = \langle N_M, T_M, R_M\rangle$, $N_M := \{A, B, C\}$, $T_M := \bar{N} \cup \bar{T}$. Правила вывода R_M метаграмматики позволяют из A и B вывести любые цепочки в алфавите $\bar{N} \cup \bar{T}$, а из C позволяют вывести любые цепочки в алфавите $\bar{T}$: 1) $\forall\, x \in T_M \ \ (A \to xA \in R_M)$; 2) $\forall\, x \in T_M \ \ (B \to xB \in R_M)$; 3) $\forall\, x \in \bar{T} \ \ (C \to xC \in R_M)$; 4) $A \to \varepsilon$; 5) $B \to \varepsilon$; 6) $C \to \varepsilon$.

3.2.6. Грамматики ван Вейнгаардена (6/10)

Доказательство. (Продолжение) В строгой грамматике $G_S = \langle F, T, R_S, \bar{I} \rangle$ множество форм F — это множество всех форм, которые встречаются в схемах правил вывода, множество терминальных символов совпадает, естественно, с множеством терминальных символов исходной грамматики, начальный символ — надчеркнутый начальный символ исходной грамматики, а множество схем правил вывода R_S образовано следующим образом: $R_S := R_1 \cup R_2$, где

$$R_1 := \left\{ A\bar{\alpha}B \to A\bar{\beta}B \mid \alpha \to \beta \in R \right\}, \quad R_2 := \left\{ C\bar{a} \to Ca \mid a \in T \right\}.$$

Покажем, что грамматики эквивалентны. Пусть $S \Rightarrow_G^* \omega$. Рассмотрим вывод $\sigma_0 = S, \ldots, \sigma_n \omega$. Перестроим этот вывод в вывод в строгой грамматике. Рассмотрим пару соседних цепочек вывода, $\sigma_i = \gamma\alpha\delta$ и $\sigma_{i+1} = \gamma\beta\delta$, где $\alpha \to \beta \in R$. Из нетерминальных символов метаграмматики A и B можно вывести любые (надчеркнутые) цепочки, в частности, $A \Rightarrow_{G_M}^* \bar{\gamma}$ и $B \Rightarrow_{G_M}^* \bar{\delta}$, а из соответствующей схемы правила $A\bar{\alpha}B \to A\bar{\beta}B$ из R_1 можно получить правило $\bar{\gamma}\bar{\alpha}\bar{\delta} \to \bar{\gamma}\bar{\beta}\bar{\delta}$. Таким образом, можно вывод в исходной грамматике перестроить в «надчеркнутый» вывод в строгой грамматике, $\bar{I} \Rightarrow_{G_S}^* \bar{\omega}$. □

3.2.6. Грамматики ван Вейнгаардена (7/10)

Доказательство. (Продолжение) Если $\bar{\omega} = \varepsilon$ (то есть целевая цепочка пустая), то перестройка вывода закончена. Если же $|\bar{\omega}| > 0$, то есть $\bar{\omega} = \bar{\omega}'\bar{a}$, то, поскольку $\Rightarrow^*_{G_M} \bar{\omega}'$, из схемы $C\bar{a} \to Ca$ можно получить правило $\bar{\omega}'\bar{a} \to \bar{\omega}'a$. Применяя подобные правила («снимая надчеркивания» по одному справа) получаем вывод цепочки ω в строгой грамматике.

Пусть теперь $\sigma_0 = \bar{I}, \ldots, \sigma_n = \omega$ — вывод в строгой грамматике. Заметим, что ω — терминальная цепочка и не содержит надчеркнутых символов. Правила вывода, полученные из схем множества R_1 выводят из надчеркнутых цепочек снова надчеркнутые, а правила, полученные из схем множества R_2 «снимают» надчеркивание, и это уже необратимо: символ без надчеркивания остаётся в последующих цепочках вывода без изменения. Таким образом, весь вывод можно разбить на чередующиеся участки применения правил из множеств R_1 и R_2. Сначала к символу $\bar{I}$ применяется одно или несколько правил первой группы, и выводится надчеркнутая цепочка $\bar{\mu}_1$ ($\bar{\mu}_1 = \sigma_i$ при некотором $i > 0$), затем все или часть надчеркиваний справа «снимается» правилами второй группы и получается цепочка $\bar{\mu}_2\nu_1$, где ν_1 — терминальная цепочка. □

3.2.6. Грамматики ван Вейнгаардена (8/10)

Доказательство. (Продолжение) Очевидно, что каждый участок первого типа можно однозначно преобразовать в участок вывода в исходной грамматике, если в качестве правил брать «прототипы» фактически использованных схем, то есть $S \Rightarrow^*_G \mu_1$, $\mu_1 \Rightarrow^*_G \mu_2\nu_1$ и так далее. Продолжая этот процесс, получаем, что $S \Rightarrow^*_G \omega$. □

Отступление. Грамматики ван Вейнгаардена являются поучительной иллюстрацией идеи самоприменимости. Фактически здесь само описание формальной грамматики рассматривается как текст, который порождается другой грамматикой (метаграмматикой). Такой прием с успехом используется и в других ситуациях.

Рассмотрим использование грамматик ван Вейнгаардена для формального задания контекстных условий языка программирования. Пусть в описываемом языке имеется оператор присваивания, синтаксис которого задается следующей металингвистической формулой:

<присваивание> ::= <переменная> := <выражение>

Пусть требуется наложить следующее контекстное условие: переменная в левой части должна иметь тот же тип, что и выражение в правой части оператора присваивания.

3.2.6. Грамматики ван Вейнгаардена (9/10)

Если в языке конечное множество типов (например, строки и логические значения), то можно просто «размножить» металингвистическую формулу:

<присваивание> ::= <строковое присваивание> | <логическое присваивание>
<строковое присваивание> ::= <строковая переменная> := <строковое выражение>
<логическое присваивание> ::= <логическая переменная> := <логическое выражение>
<присваивание> ::= <переменная> := <выражение>

Однако в современных языках множество типов бесконечно (поскольку программист может определять свои типы или классы). Пусть, например, в описываемом языке, кроме простых типов, используются ссылки: целое значение, ссылка на целое, ссылка на ссылку на целое и т. д. Размножением нетерминалов и правил отделаться не удаётся. Грамматика ван Вейнгаардена позволяет справиться с этой трудностью. Пусть (в соответствии с традицией, возникшей после описания языка Algol-68) нетерминалы метаграмматики записываются словами из прописных букв, терминалы метаграмматики (и нетерминалы строгой грамматики) записываются словами из строчных букв, а терминалы грамматики также записываются словами из строчных букв, но обязательно начинаются со слова «символ».

3.2.6. Грамматики ван Вейнгаардена (10/10)

Металингвистическая связка ::= сокращается до простого двоеточия, альтернативы разделяются точкой с запятой, а запятая разделяет формы. В таком случае схема правила для присваивания записывается следующим образом:
присваивание типа ТИП : переменная типа ТИП, символ присвоить, выражение типа ТИП
В метаграмматике должны быть правила для определения множества названий типа данного языка:
ТИП : строковый ; логический ; ссылка на ТИП

TODO Тщательная проверка, насыщение примерами, дописывание практических выводов. Кто???

TODO Неграмматические способы проверки контекстных условий. Кто???

3.2.X. ??? Нерассмотренные вопросы

TODO В этом разделе (или в другом???) Надобно рассмотреть:
нетекстовую лексику, в том числе жесты на смартфонах, голосовой ввод, графические грамматики, ...
абстрактный синтаксис и AST,
неграмматические средства задания синтаксиса.
Кто???

3.3. Методы реализации языков

TODO Общий обзор — исполняемые языки и языки публикаций, декларативные и императивные языки, компилируемые и интерпретируемые языки, скриптовые языки и языки разметки. Нужен план раздела.

3.3.1. Определение семантики языка программирования (1/2)

В настоящее время методы описания синтаксиса языков программирования развиты и исследованы настолько, что вполне удовлетворяют практические потребности, поэтому описания синтаксиса различных языков используют один и тот же аппарат и следуют общей схеме, различаясь лишь в деталях нотации. Методы описания семантики языков программирования начали развиваться параллельно с методами описания синтаксиса, но пока не приняли столь же законченного и унифицированного выражения, как формальные грамматики для описания синтаксиса. Поэтому для описания семантики применяются существенно различные подходы, и нет общего подхода для всех случаев. К сожалению, пока формальные методы для практических языков оказываются не вполне удовлетворительными и применяются не вместо, а наряду с неформальными описаниями на естественном языке. Наиболее характерными методами формального описания семантики являются следующие.

TODO Операционная, денотационная, деривационая семантика, доказательное программирование Деркаченко Анна ???

3.3.1. Определение семантики языка программирования (2/2)

1. *Операционная семантика*. Семантика конструкций языка программирования описывается в терминах некоторой абстрактной машины. Число различных вариаций здесь также весьма велико. Например, написание «канонического» интерпретатора определяемого языка, часто на том же самом языке. Другой вариант: описание перевода конструкций определяемого языка в команды абстрактной машины.
2. *Деривационная семантика*. Семантика описывается заданием аксиом, постулирующих свойства конструкций, и правил вывода, позволяющих доказывать утверждения о свойствах программ и их фрагментов, то есть строится некоторое *исчисление программ* (подраздел **??**).
3. *Денотационная семантика*. Семантика программы (или её фрагмента) описывается как отношение, связывающее начальное и заключительное состояния программ (подраздел **??**).

В следующих параграфах изложены фрагменты методов описания семантики, опирающиеся на методы дискретной математики.

TODO Венский метод – интерпретирующие автоматы.

3.3.2. Исчисление программ (1/11)

TODO Верифицировать и поделить параграф. Кто???

Здесь рассматривается фрагмент одного из вариантов исчисления программ, иллюстрирующий основную идею метода деривационного, или *аксиоматического* метода описания семантики.

Формулы исчисления программ строятся из подформул двух сортов. Подформулы первого типа — это формулы прикладного исчисления предикатов, в которых предметными переменными являются переменные программы, а функциональными буквами и предикатами — функции и отношения описываемого языка программирования. Такие формулы являются утверждениями о *состоянии программы*, то есть утверждениями о текущих значениях переменных.

Замечание. Иногда набор функциональных символов и предикатов приходится расширять, но, как правило, оказывается достаточно тех, которые определены в языке программирования.

Подформулы второго типа — это фрагменты программы, для которых можно определить семантику. В типичном современном языке программирования, подобном используемому нами в этом учебнике, таковыми оказываются операторы (и описания).

3.3.2. Исчисление программ (2/11)

Формула исчисления программ имеет вид

$$\{P\}\, S\, \{Q\},$$

где S — фрагмент программы (оператор), то есть подформула второго типа, P — подформула первого типа, которая называется *предусловием*, Q — подформула первого типа, которая называется *постусловием*.

Формула $\{P\}\, S\, \{Q\}$ имеет следующий содержательный смысл: если до выполнения оператора S программа находилась в состоянии w, для которого выполнено предусловие $P(w)$, то после выполнения оператора S программа перейдет в состояние $w' = S(w)$, в котором выполнено постусловие $Q(w')$:

$$\{P\}\, S\, \{Q\} \stackrel{\text{Def}}{=} \forall\, w\ \ (P(w) \Longrightarrow \exists\, w'\ \ (w' = S(w)\ \&\ Q(w')))\,.$$

Замечание. В данном случае считается, что выполнение любого оператора всегда завершается. Такую формулу исчисления программ называют *сильной*. В более полных вариантах исчисления программ наряду с сильными формулами, рассматривают *слабые* формулы, которые имеют следующий смысл: $\forall\, w\ \ (P(w) \Longrightarrow \forall\, w'\ \ (w' = S(w) \Longrightarrow Q(w')))\,.$

3.3.2. Исчисление программ (3/9)

Семантика элементарных операторов задается формулами исчисления программ, которые принимаются как аксиомы. Состав и форма аксиом зависят от используемого языка программирования. Например, выполнение оператора возврата значения **yield** не меняет состояния программы

$$\{P\}\,\textbf{yield}\ e\,\{P\},$$

а для оператора присваивания $v := e$, в котором выражение e не содержит переменной v, можно использовать следующую простую форму аксиомы:

$$\{P\}\,v := e\,\{P\ \&\ v = e\}.$$

Семантика составных операторов задается правилами вывода (см. также **??**). В частности, последовательность операторов определяется следующим правилом:

$$\frac{\{P\}\,S_1\,\{Q\} \quad \{Q\}\,S_2\,\{R\}}{\{P\}\,S_1; S_2\,\{R\}}\,1.$$

3.3.2. Исчисление программ (4/11)

Условный оператор имеет следующую семантику (в полной и в неполной форме):

$$\frac{\{P \;\&\; B\}\, S_1\, \{Q\} \quad \{P \;\&\; \neg B\}\, S_2\, \{Q\}}{\{P\}\ \textbf{if}\ B\ \textbf{then}\ S_1\ \textbf{else}\ S_2\ \textbf{end if}\ \{Q\}}\ 2, \qquad \frac{\{P \;\&\; B\}\, S\, \{Q\} \quad P \;\&\; \neg B \Longrightarrow Q}{\{P\}\ \textbf{if}\ B\ \textbf{then}\ S\ \textbf{end if}\ \{Q\}}\ 2'.$$

Оператор цикла можно определить так:

$$\frac{\{P \;\&\; B\}\, S\, \{P\}}{\{P\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{end while}\ \{P \;\&\; \neg B\}}\ 3.$$

Замечание. В правиле (3) для цикла условие P называется *инвариантом*, поскольку требуется, чтобы P выполнялось до начала и после окончания работы цикла. Всегда существует тривиальный инвариант **true**, но найти самое слабое предусловие для конкретных S и B бывает затруднительно.

3.3.2. Исчисление программ (6/11)

Помимо правил, определяющих семантику операторов конкретного языка программирования, полезно ввести «общелогические правила», позволяющие делать эквивалентные преобразования условий, как обоих сразу

$$\frac{P \iff P_1 \quad \{P_1\}\, S\, \{Q_1\} \quad Q_1 \iff Q}{\{P\}\, S\, \{Q\}}\, 4,$$

так и по одному

$$\frac{P \iff P_1 \quad \{P_1\}\, S\, \{Q\}}{\{P\}\, S\, \{Q\}}\, 4', \qquad \frac{\{P\}\, S\, \{Q_1\} \quad Q_1 \iff Q}{\{P\}\, S\, \{Q\}}\, 4''.$$

Кроме того, введем правила, позволяющие усиливать предусловие и ослаблять постусловие, одновременно

$$\frac{P \Longrightarrow P_1 \quad \{P_1\}\, S\, \{Q_1\} \quad Q_1 \Longrightarrow Q}{\{P\}\, S\, \{Q\}}\, 5,$$

и порознь

$$\frac{P \Longrightarrow P_1 \quad \{P_1\}\, S\, \{Q\}}{\{P\}\, S\, \{Q\}}\, 5', \qquad \frac{\{P\}\, S\, \{Q_1\} \quad Q_1 \Longrightarrow Q}{\{P\}\, S\, \{Q\}}\, 5''.$$

3.3.2. Исчисление программ (7/11)

Как обычно, вывод в исчислении программ — это последовательность формул, в которой каждая формула либо аксиома, либо получена из предшествующих по одному из правил вывода. Используя исчисление программ, построенное для конкретного языка программирования, можно доказывать некоторые свойства программ.

Пример. Рассмотрим язык, в котором есть специальная форма оператора присваивания, $x :=: y$, которую можно назвать *оператором обмена значениями*. Семантика этого оператора такова:

$$\{P(x,y)\}\, x :=: y\, \{P(y,x)\}.$$

Рассмотрим следующий фрагмент программы:

if $x > y$ **then** $x :=: y$ **endif**
if $y > z$ **then** $y :=: z$ **endif**
if $x > y$ **then** $x :=: y$ **endif**

Покажем, что этот фрагмент сортирует значения переменных x, y, z в порядке возрастания, то есть что постусловием фрагмента является утверждение $x \leqslant y \leqslant z$.

3.3.2. Исчисление программ (9/11)

Заметим, что операторы в программе очень похожи, поэтому введем процедуру

$$S2(a,b) := \textbf{if } a > b \textbf{ then } a :=: b \textbf{ endif}$$

и перепишем исходный фрагмент в более краткой форме: $S2(x,y); S2(y,z); S2(x,y)$. Покажем теперь, что введенная процедура $S2$ (фактически, новый оператор языка) сортирует два значения, то есть

$$\{P(a,b)\}\, S2(a,b)\, \{(P(a,b) \vee P(b,a)) \,\&\, a \leqslant b\}.$$

Действительно,

1. $\{P(a,b) \,\&\, a > b\}\, a :=: b\, \{P(b,a) \,\&\, b > a\}$	аксиома
2. $P(b,a) \,\&\, b > a \Longrightarrow (P(a,b) \vee P(b,a)) \,\&\, a \leqslant b$	общезначимая формула
3. $\{P(a,b) \,\&\, a > b\}\, a :=: b\, \{(P(a,b) \vee P(b,a)) \,\&\, a \leqslant b\}$	правило $5''; 1, 2$
4. $P(a,b) \,\&\, \neg(a > b) \Longrightarrow (P(a,b) \vee P(b,a)) \,\&\, a \leqslant b$	общезначимая формула
5. $\{P(a,b)\}$ **if** $a > b$ **then** $a :=: b\, \{(P(a,b) \vee P(b,a)) \,\&\, a \leqslant b\}$	правило $2'; 3, 4$

3.3.2. Исчисление программ (10/11)

Используя доказанное свойство процедуры $S2$, имеем

1. $\{\}\ S2(x,y)\ \{x \leqslant y\}$ — по доказ
2. $\{x \leqslant y\}\ S2(y,z)\ \{(x \leqslant y \vee x \leqslant z)\ \&\ y \leqslant z\}$ — по доказ
3. $(x \leqslant y \vee x \leqslant z)\ \&\ y \leqslant z \Longrightarrow x \leqslant z\ \&\ y \leqslant z$ — общезна
4. $\{x \leqslant y\}\ S2(y,z)\ \{x \leqslant z\ \&\ y \leqslant z\}$ — правило
5. $\{\}\ S2(x,y); S2(y,z)\ \{x \leqslant z\ \&\ y \leqslant z\}$ — правило
6. $\{x \leqslant z\ \&\ y \leqslant z\}\ S2(x,y)\ \{((x \leqslant z\ \&\ y \leqslant z) \vee (x \leqslant z\ \&\ y \leqslant z))\ \&\ x \leqslant y\}$ — по доказ
7. $((x \leqslant z\ \&\ y \leqslant z) \vee (x \leqslant z\ \&\ y \leqslant z))\ \&\ x \leqslant y \Longrightarrow x \leqslant y\ \&\ y \leqslant z$ — общезна
8. $\{x \leqslant z\ \&\ y \leqslant z\}\ S2(x,y)\ \{x \leqslant y\ \&\ y \leqslant z\}$ — правило
9. $\{\}\ S2(x,y); S2(y,z); S2(x,y)\ \{x \leqslant y\ \&\ y \leqslant z\}$ — правило

3.3.2. Исчисление программ (11/11)

Отступление. Приведённый пример производит двойственное впечатление. С одной стороны, доказательство правильности очень простой программы (это частный случай метода сортировки пузырьком для случая $n = 3$) на основе формальной семантики, заданной исчислением программ, оказывается достаточно громоздким. Заметим также, что в примере нет циклов, наличие которых порождает дополнительные и принципиальные трудности, связанные с инвариантами. В любом случае, формальное доказательство свойства программы всегда оказывается во много раз длиннее и сложнее текста самой программы. С другой стороны, как подмечено в [**?**], «аппарат исчисления предикатов первого порядка — это своего рода машинный язык для математиков». Как программисты пользуются языками значительно более высокого уровня, нежели машинный язык, так и математики используют языки более высокого уровня и менее формальные по сравнению с исчислением предикатов (частным случаем которого является исчисление программ). Неформально доказывать свойства программ вполне возможно и чрезвычайно полезно, для большинства программ в данном учебнике приведены такие неформальные доказательства в форме обоснований.

3.3.2. Исчисление программ (11/11)

Отступление. Основу как формального доказательства, так и неформального обоснования составляют утверждения (более или менее формальные) о значениях переменных при выполнении программы, то есть утверждения о состоянии программы. Очень удобно вставлять такие утверждения прямо в текст программ в форме комментариев, которые следует понимать как постусловие для предшествующего оператора и предусловие для последующего. Такой стиль называется *аннотированием* программ. Чтобы подчеркнуть связь неформального аннотирования программ с формальным исчислением программ, мы изменили общепринятые обозначения: в исчислении программ обычно операторы заключают в скобки, а формулы исчисления предикатов оставляют без скобок, а здесь предусловия и постусловия записаны в тех же фигурных скобках, какие используются для комментариев в нашем псевдокоде.

3.3.4. Денотационная семантика

TODO КТО???

3.3.5. Венский метод Оллонгрена

TODO КТО???

3.3.6. Автоматный метод Новиков / Тихонова

TODO КТО??? Кто спрашивает ...

3.4.

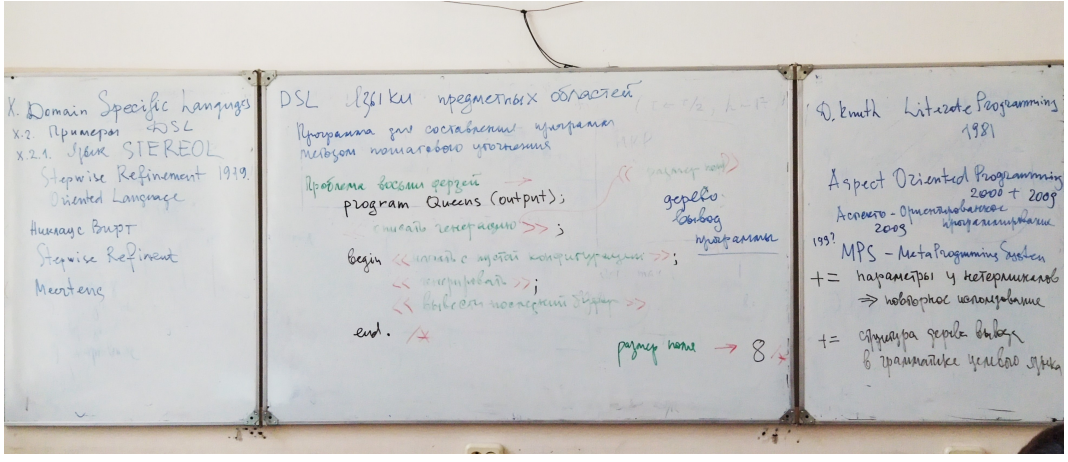
TODO Примеры и обобщения

3.4.1 Классификация языков предметных областей

TODO Примеры и обобщения

3.4.2. Языки трансформации

TODO Синтексически управляемый перевод, макросы, препропроцессоры, пример = STERE литеральное программирование Кнута. Кто???



3.4.3. Командные интерпретаторы

TODO Графический интерфейс как командный язык, примеры = GUIDL, AMPLE. Кто???

3.4.4. Порождающее программирование

TODO Автоматический структурный синтез программ, концептуальное программирование, пример = СПОРА, ПРИЗ. Кто???

3.4.5. Автоматное программирование

TODO Шалыто, пример = UNIMOD, CIAO. Кто???